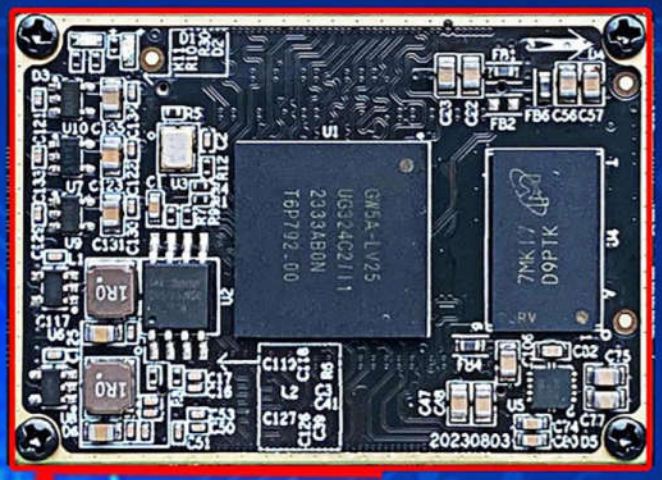
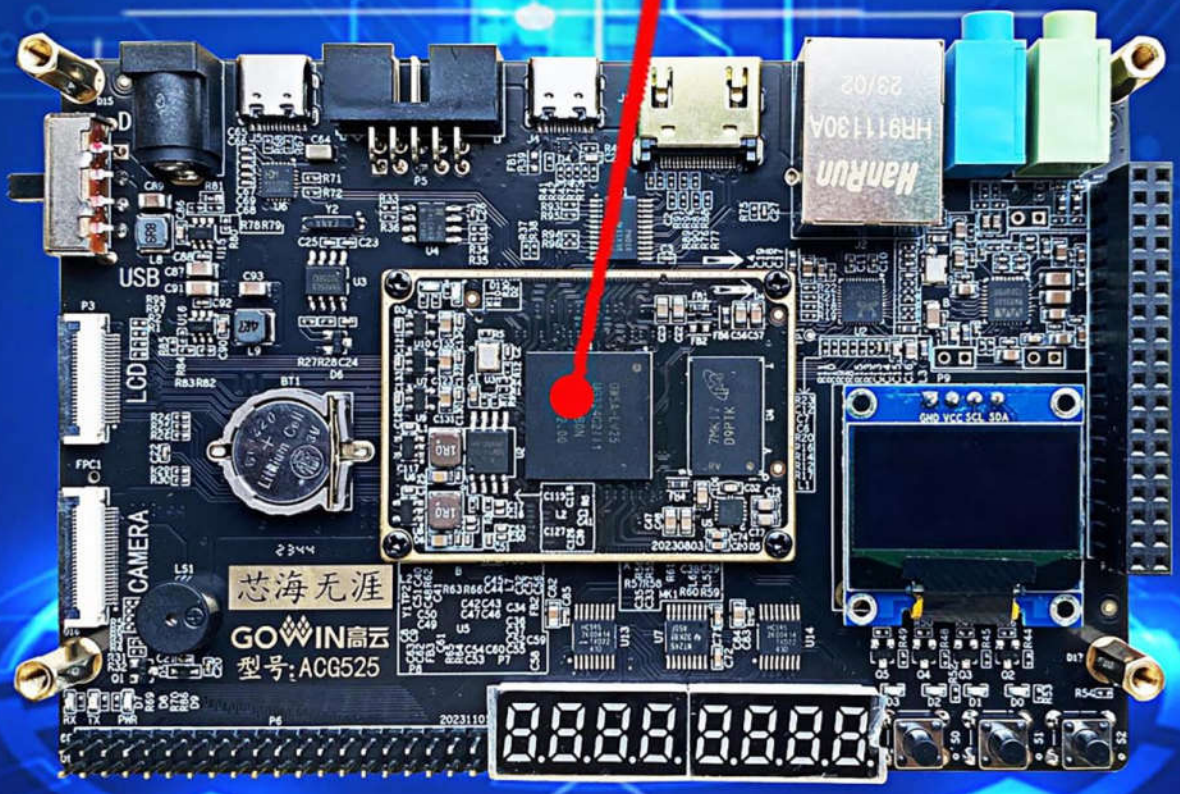
ACG525开发板

High Quality China FPGA Chip

- PHẦN CỨNG LINH HOẠT (CORE BOARD + CARIER BOARD)
- TÀI LIỆU HƯỚNG DẪN ĐỒ SỘ (PDF 1,000+ TRANG)
- HỖ TRỢ ĐÀO TẠO (NHIỀU VÍ DỤ CƠ BẢN)
- ỨNG DỤNG THỰC TẾ (CASE STUDY DOANH NGHIỆP)





LỜI MỞ ĐẦU

Field Programing Gate Array, hoặc FPGA, đã tồn tại gần bốn thập kỷ. Trong 40 năm qua, FPGA đã phát triển từ chỉ 64 đơn vị logic có thể lập trình lên hơn 10 triệu đơn vị logic lập trình ngày nay, từ chức năng liên kết logic ban đầu, đến truyền thông kỹ thuật số, điều khiển công nghiệp, xử lý Hình ảnh và bây giờ là trung tâm dữ liệu, ứng dụng trí tuệ nhân tạo. Từ đơn vị logic lập trình đơn giản trong những ngày đầu, đến việc bổ sung bộ nhớ và đơn vị DSP ở phía sau, đến việc tích hợp bộ điều khiển bộ nhớ hiệu suất cao, tích hợp bộ thu phát tốc độ cao và bây giờ là các hệ thống xử lý hiệu suất cao. FPGA hiện đại không còn là mảng cổng có thể lập trình trường thuần túy, mà là các thiết bị kiến trúc lai tích hợp cao.

Ở Trung Quốc, Các nhà sản xuất FPGA bắt đầu phát triển và sản xuất các thiết bị FPGA muộn, chủ yếu là sau năm 2010, nhưng cũng là trường hợp các nhà sản xuất trong nước có thể có đủ công nghệ và kinh nghiệm tham khảo khi tiến hành nghiên cứu và phát triển thiết bị FPGA, đồng thời có thể học hỏi đầy đủ kinh nghiệm R & D và sản xuất của các nhà sản xuất nước ngoài trong quá trình nghiên cứu và phát triển, đồng thời làm cho sản phẩm gần gũi hơn với môi trường ứng dụng địa phương kết hợp với các liên kết ứng dụng trong nước.

Hiện tại ở Trung Quốc có các nhà sản xuất chip FPGA đã phát hành và sản xuất hàng loạt nhiều loại chip FPGA bao gồm **GOWIN** Semiconductor, Anlu Technology, Unisplendour Tongchuang, Zhiduoji, Yilingsi, Fudan Micro và Jingwei Qili. Các nhà sản xuất này có quyền sở hữu trí tuệ độc lập trong công nghệ FPGA và cung cấp nghiên cứu và phát triển các công cụ phần mềm và phần cứng xác minh và phát triển FPGA toàn quy trình của riêng họ.

Tuy nhiên, một số vấn đề thực tế mà FPGA trong nước phải đối mặt không thể bỏ qua, trong đó quan trọng nhất là hệ sinh thái ứng dụng. Do FPGA trong nước bắt đầu muộn, cùng với sự tích lũy công nghệ hàng đầu và hệ sinh thái ứng dụng của chip FPGA nước ngoài, FPGA trong nước khó có thể được quảng bá hiệu quả trên thị trường chung. Do đó, FPGA trong nước thường được sử dụng trong một hoặc hai khách hàng hoặc ngành công nghiệp lớn trong quá khứ. Trên thị trường chung, ngay cả khi người dùng vừa và nhỏ sẵn sàng hiểu và thử, do hệ sinh thái ứng dụng còn non nớt và thiếu tài liệu tham khảo, tiến độ dự án của khách hàng rất khó được thúc đẩy hiệu quả.

Kể từ cuối năm 2020, làn sóng thiếu chip đã càn quét thế giới, và một số lượng lớn chip ban đầu rất rẻ và linh hoạt bắt đầu khan hiếm và tang giá. Tình trạng thiếu hụt thiết bị FPGA và tăng giá thậm chí còn đáng báo động hơn, và toàn bộ làn sóng tăng giá thiếu IP Core đã kéo dài hơn 2 năm. Trong trường hợp không mua được Chip nước ngoài, sự chú ý của công chúng một lần nữa đổ dồn vào các chip trong nước, mọi người bắt đầu bình tĩnh để nghiên cứu và sử dụng FPGA trong nước. Chỉ khi đó tôi mới nhận thấy FPGA trong nước không chỉ bắt kịp các thiết bị FPGA cấp thấp của nước ngoài về chức năng và hiệu năng mà còn khiến mọi người ngạc nhiên bởi sự đổi mới chức năng của nó đối với môi trường ứng dụng trong nước. Ví dụ, trong lĩnh vực thu thập hình ảnh và dữ liệu, FPGA trong nước thường được đóng gói với bộ nhớ SDRAM và DDR2, do đó chúng ta không còn cần thiết kế chip bộ nhớ ngoài trên bảng mạch khi sử dụng chúng, điều này có thể làm giảm đáng kể khó khăn trong việc phát triển phần cứng của chúng ta. Ngoài ra, thiết bị SOC FPGA tích hợp MCU Hard core Cortex-M1, Cortex-M3, 8051 và RISC-V cũng cho phép hệ thống ban đầu yêu cầu hai chip được thực hiện với FPGA trong nước. Những thiết kế sáng tạo này phù hợp hơn với nhu cầu của khách hàng công nghiệp trong nước.

Trong hoạt động dịch chuyển ứng dụng dùng FPGA trong nước này, chúng tôi tham gia sâu vào dự án cung ứng nhiều chip FPGA trong nước từ nhiều khách hàng. Trong quá trình sử dụng FPGA

trong nước, cảm nhận lớn nhất của chúng tôi là phần cứng FPGA trong nước cũng mạnh mẽ, nhưng có rất ít tư liệu để phát triển và các trường hợp ứng dụng khan hiếm. Thực sự khó khăn hơn cho người mới bắt đầu mà không có tài liệu giảng dạy và tài liệu tham khảo. Ngoài ra, trong nhóm khách hàng của chúng tôi, thường có những khách hàng gặp phải những vấn đề khó giải quyết khi nghiên cứu FPGA trong nước, và kêu gọi chúng tôi ra mắt tài liệu giảng dạy cho FPGA trong nước. Do đó, chúng tôi quyết định làm một bộ 'bản phát triển giảng dạy FPGA' trong nước, một bộ bộ dụng cụ học tập FPGA trong nước có thể được sử dụng làm tài liệu tham khảo cho người mới trong giai đoạn nâng cao nhập môn.

Tại thời điểm này, Gowin Semiconductor đã mời chúng tôi tham gia Chương trình Đại học Gowin FPGA với tư cách là đối tác chính thức để phát triển 'bản phát triển giảng dạy' dựa trên chip FPGA dòng Arora V 22nm mới của Gowin. Sau khi đánh giá đầy đủ hiệu suất thiết bị, chức năng và phần mềm EDA của Gowin Semiconductor, chúng tôi nhận thấy đây là một sự kết hợp rất phù hợp để người Trung Quốc học FPGA. Phần mềm phong phú và đầy đủ, hoạt động ngắn gọn và rõ ràng, tài nguyên IP dồi dào và tài liệu chính thức chi tiết. Vì vậy, chúng tôi đã ăn ý và bắt đầu dự án ngay lập tức.

Sau 9 tháng, với sự hợp tác của gần 10 người từ FAE ban đầu của Gaoyun Semiconductor, đại lý FAE và Xiaomeige FPGA, bản phát triển giảng dạy ACG525 đã ra mắt thành công.

Bản phát triển ACG525 có tài nguyên phần cứng dồi dào, tài liệu giảng dạy hoàn chỉnh và mã nguồn thông thường toàn diện. Và các tài liệu phần mềm và phần cứng đều là mã nguồn mở. Đó là sản phẩm công nghiệp tiền nghiên cứu và phát triển dự án nghiên cứu khoa học của bạn. Tài liệu chất lượng cho việc học cơ bản về FPGA.

Tiếp theo, hãy đi vào hành trình nghiên cứu của Gowin FPGA.

GOWIN VIETNAM

MỤC LỤC – PHẦN CƠ BẢN- CHƯƠNG 1-13

Chương 1: Hướng dẫn cài đặt phần mềm Gowin 10
1.1 Tải phần mềm Gowin Designer EDA 10
1.2 Các bước cài đặt 10
1.3 Xin cấp bản quyền License 16
Chương 2: Quá Trình Phát Triển 1 Dự Án 17
2.1 Khởi chạy phần mềm Gowin 17
2.2 Tạo dự án 17
2.3 Thêm tệp nguồn 19
2.4 Mô tả và phân tích RTL 23
2.4.1 Giới thiệu về các chức năng mô tả và phân tích RTL 23
2.4.2 RTL mô tả và phân tích quá trình 23
2.5 Phân tích và tổng hợp 23
2.6 Các ràng buộc về vật lý 25
2.7 Place & Route 27
2.8 Xác thực chương trình trên Board mạch 27
2.9 Lưu trữ chương trình trên các bộ nhớ FPGA 29
2.10 Tổng kết chương 30
Chương 3: Gowin Kết Hợp Mô Hình Mô Phỏng Modelsim 31
3.1 Modelsim biên dịch thư viện Gowin 31
3.1.1 Tạo mới thư viện Modelsim 31
3.1.2 Biên dịch tệp thư viện FPGA của Gowin để dùng với Modelsim 33
3.1.3 Thêm thư viện vào danh sách thư viện mặc định của ModelSim 35
3.2 Quy trình mô phỏng xác minh dùng Modelsim 36
3.2.1 Viết mã cho thiết kế mô phỏng Verilog 36
3.2.2 Xây dựng dự án Modelsim và thêm tệp giả lập 37
3.2.3 Biên dịch tệp mô phỏng 39
3.2.4 Cấu Hình môi trường mô phỏng 40
3.2.5 Quan sát dạng sóng mô phỏng 42
3.2.6 Lưu File dạng sóng 44
3.2.7 Mở lại dự án mô phỏng 45

GOWIN VIETNAM

3.3 Tổng kết chương 46
Chương 4: Logic Analyzer trực tuyến của Gowin GAO 47
4.1 Giới thiệu về Logic Analyzer trực tuyến của Gowin 47
4.2 Quy trình sử dụng GAO cơ bản 47
4.2.1 Tạo một File Gao mới 47
4.2.2 Mở cửa sổ cấu Hình GAO 49
4.2.3 Cấu Hình cho GAO ở Standard Mode 50
4.3 Định vị và định tuyến (Place & Route) 57
4.4 Tải file fs 57
4.5 Bắt sóng dạng tín hiệu (Waveform Capture) 58
4.6 Tổng kết chương 60
Chương 5: Thiết kế mạch logic tuần tự – Bộ đếm 61
5.1 Nguyên lý hoạt động của bộ đếm 61
5.2 Hiện thực bộ đếm bằng Verilog 62
5.2.1 Viết code Verilog cho project 63
5.3 Thiết kế Testbench và chạy mô phỏng 65
5.3.1 Tạo project mô phỏng dùng Model Sim 66
5.3.2 Định cấu Hình môi trường mô phỏng 66
5.3.3 Chạy mô phỏng và xem sóng tín hiệu 67
5.4 Gỡ lỗi và xác thực cấp bo mạch 69
5.4.1 Thêm ràng buộc I/O 69
5.4.2 Thêm ràng buộc thời gian 70
5.5 Place & Route 74
5.5.1 Kết nối phần cứng 74
5.5.2 Tải file fs 74
5.6 Tổng kết chương 75
Chương 6: Thiết kế module truyền data qua cổng nối tiếp 76
6.1 Nguyên lý truyền nối tiếp và thiết kế mạch 76
6.1.1 Chuẩn giao thức RS232 76
6.1.2 Tham số chính của UART và biểu đồ thời gian 77
6.1.3 Thiết kế mạch giao tiếp RS232 78

GOWIN VIETNAM

6.1.4 Cài đặt UART 79
6.2 Thiết kế và triển khai module giao tiếp nối tiếp không đồng bộ 81
6.2.1 Thiết kế module tại tín hiệu xung clock 82
6.2.2 Thiết kế modun đầu ra dữ liệu 85
6.2.3 Mô-đun điều khiển trạng thái truyền dữ liệu 86
6.3 Tạo testbench và mô phỏng 86
6.4 Module Kiểm tra UART 87
6.5 Xác thực chương trình trên phần cứng 89
6.5.1 Thêm ràng buộc I/O 89
6.5.2 Place & Route 89
6.5.3 Kết nối phần cứng 90
6.5.4 Cấu Hình Port kết nối cho bộ debugger 90
6.5.5 Tải Fle fs 91
6.6 Tổng kết chương 92
Chương 7: Thiết kế module nhận dữ liệu qua cổng nối tiếp 93
7.1 Phân tích nguyên lý nhận data qua UART 93
7.2 Thiết kế và triển khai module nhận data qua UART 94
7.2.1 Sơ đồ khối module nhận data UART 94
7.2.2 Thiết kế đồng bộ tín hiệu UART đơn BIT 95
7.2.3 Thiết kế phát hiện cạnh 95
7.2.4 Thiết kế module tạo xung clk cho khối 97
7.2.5 Thiết kế module nhận data 98
7.2.6 Module xác định trạng thái dữ liệu 100
7.3 Thiết kế testbench và mô phỏng 101
7.4 Module kiểm tra nhận data qua UART 102
7.5 Xác thực chương trình trên phần cứng 103
7.5.1 Place & Route 104
7.5.2 Kết nối phần cứng 104
7.5.3 Cấu hình debuger port nối tiếp 105
7.5.4 Tải file fs 106
7.6 Tổng kết chương 106

GOWIN VIETNAM

Chương 8: ĐIỀU KHIỂN LED QUA UART 107
8.1 Phân tích thiết kế 107
8.2 Định nghĩa khung giao thức truyền UART 108
8.3 Nguyên lý và ý tưởng điều khiển LED qua UART 108
8.3.1 Thiết kế module chuẩn đổi lệnh qua UART（uart_cmd） 109
8.3.2 Thiết kế bộ đếm điều khiển led（counter_led_4） 110
8.3.3 Tạo top module 112
8.3.4 Tạo testbench và kiểm tra mô phỏng 113
8.4 Kiểm chứng trên hardware 115
8.4.1 Ràng buộc I/O 116
8.4.2 Hệ thống phần cứng 116
8.4.3 Kết nối phần cứng 116
8.5 Tổng kết chương 118
Chương 9: Thiết kế và kiểm chứng module truyền nhận UART nhiều byte 119
9.1 Nguyên lý và thiết kế mô-đun truyền data UART nhiều byte. 119
9.1.1 Nguyên lý truyền data nhiều byte qua UART 119
9.1.2 Thiết kế module truyền dữ liệu nhiều byte qua UART 120
9.1.3 Tạo file testbench và mô phỏng 123
9.2 Nguyên lý và thiết kế module nhận đa byte 125
9.2.1 Nguyên lý nhận đa byte UART 125
9.2.2 Thiết module nhận đa byte qua UART 126
9.3 Tạo top module và thử nghiệm trên phần cứng 131
9.4 Kiểm chứng trên bo mạch 133
9.4.1 Phần cứng yêu cầu của hệ thống 133
9.4.2 Kết nối phần cứng 133
9.5 Tổng kết chương 134
Chương 10: Thiết kế FSM (Finite State Machine) 135
10.1 Nguyên lý hoạt động của FSM 135
10.2 Khiển khai FSM phát hiện chuỗi 137
10.3 Thiết kế Testbench và mô phỏng 140
10.4 Kiểm tra chương trình phát hiện chuỗi trên board mạch 142

GOWIN VIETNAM

10.5 Tổng kết chương 144
Chương 11: Thiết kế và xác thực chống rung cho nút nhấn độc lập 145
11.1 Cấu trúc vật lý và thiết kế mạch nút bấm 145
11.2 Cài đặt bộ máy trạng thái để lọc phím nhấn 149
11.3 Thiết kế giao diện mô đun 149
11.4 Thiết kế chuyển trạng thái và điều kiện của FSM 149
11.5 Thiết kế phát hiện cạnh tín hiệu nhập phím 150
11.5.1 Thiết kế bộ đếm 150
11.5.2 Thiết kế Máy trạng thái (FSM) 151
11.6 Thiết kế testbench và mô phỏng 152
11.6.1 Thiết kế mô hình nút nhấn key_model 153
11.6.2 Sử dụng task 153
11.7 Tổng kết chương 155
Chương 12: Module hóa thiết kế bộ đếm lên & xuống 156
12.1 Phân hóa chức năng của module 156
12.1.1 Thiết kế chức năng module 156
12.2 Tạo testbench và mô phỏng 158
12.3 Xác thực trên hardware 160
12.3.1 Ràng buộc I/O 160
12.3.2 Place & Route 161
12.3.3 Tải file fs và quan sát hiện tượng 161
12.4 Thuyết minh các vấn đề thường gặp 162
12.5 Tổng kết chương 162
Chương 13: Thiết kế xác minh điều khiển Led 7 đoạn 163
13.1 Nguyên lý điều khiển màn hình số 163
13.2 Thiết kế điều khiển quét động cho màn Hình số 165
13.2.1 Sơ đồ khối mô đun và phân chia chức năng 165
13.2.2 Thiết kế module Clk quét 166
13.2.3 Thiết kế module chọn vị trí Led 7 đoạn 166
13.2.4 Thiết kế hiển thị dữ liệu cho Led 7 đoạn 167
13.2.5 Thiết kế tín hiệu kích hoạt cho module 168

GOWIN VIETNAM

13.3 Thiết kế Module điều khiển 74HC595 168
13.4 Mô đun hiển thị số giả lập thử nghiệm 170
13.5 Xác thực trên bo mạch phần cứng 171
13.6 Giải thích các vấn đề thường gặp 172
13.7 Tổng kết chương 172

GOWIN VIETNAM

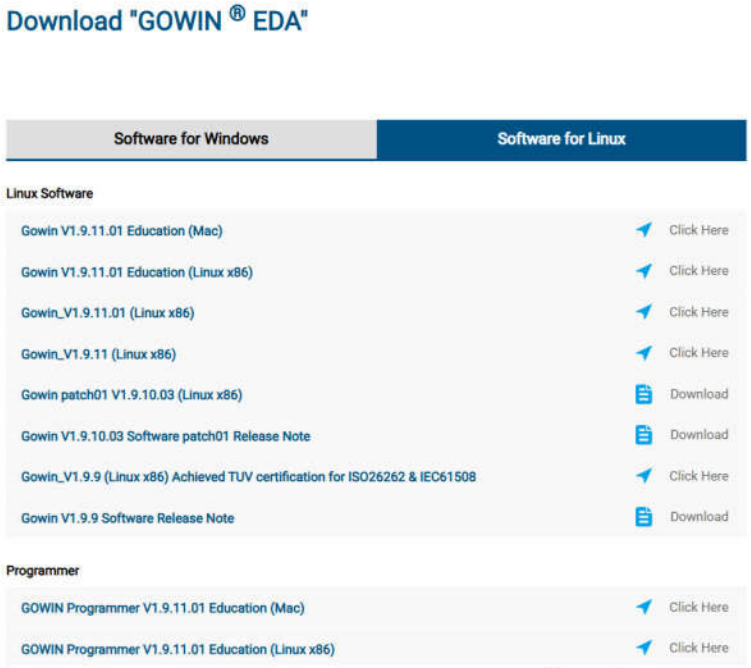
Chương 1: Hướng dẫn cài đặt phần mềm Gowin

1.1 Tải phần mềm Gowin Designer EDA

EDA của Gowin tên là Gowin Designer, người dùng có thể thông qua nhiều kênh khác nhau để tải phần mềm EDA của Gowin, bởi trang chính thức của hãng và các đối tác thương mại, link chính thức tải Gowin EDA GOWIN EDA | GOWIN Semiconductor

Sau khi vào Web, cần tạo 1 tài khoản, chỉ mất vài phút và bạn đăng nhập vào tài khoản của mình, có thể thấy được có nhiều phiên bản phần mềm: Giáo dục, thương mại cho các hệ điều hành Window, Linux & Mac OS.

Hình 1-1– Nhấp chuột để tải



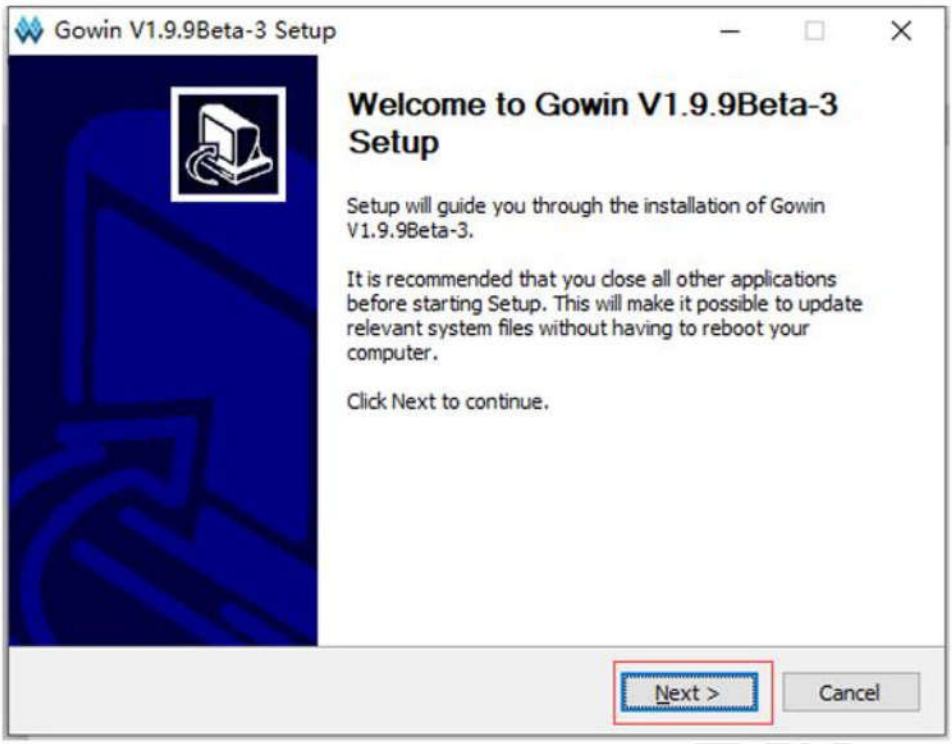
Hình 1-2– Lựa chọn phiên bản phần mềm

1.2 Các bước cài đặt

Khi cài đặt, cần giải nén như tệp trong Hình dưới đây:

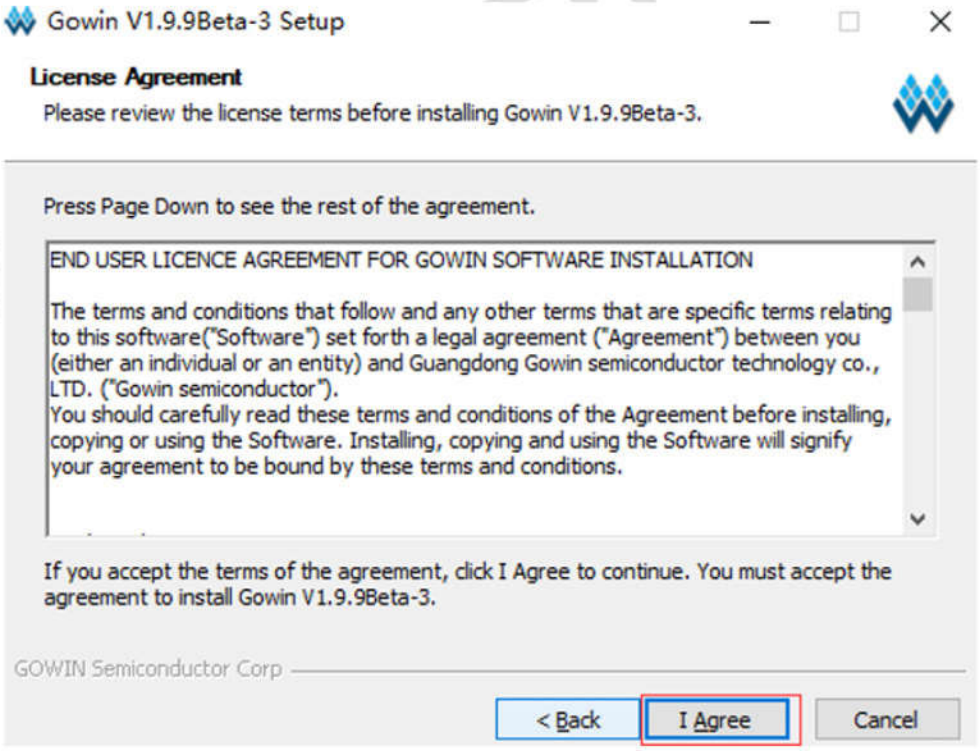
Hình 1-3 – Gói file cài đặt

Nhấp đúp vào file.exe, sẽ xuất hiện giao diện cài đặt như Hình 1-4 bên dưới, nhấp tiếp theo để tiếp tục cài đặt



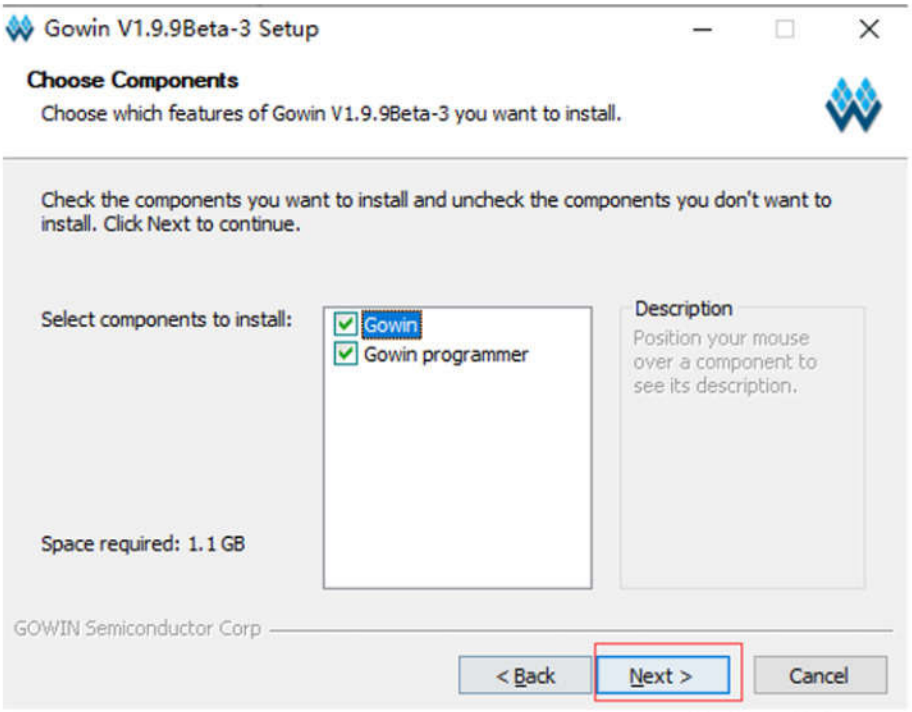
Hình 1-4 – Giao diện khởi động cài đặt

Sau khi nhấp 'Next'' sẽ hiển thị giao diện như Hình 1-5, tiếp tục đồng ý để cài, nhấp "I Agree"



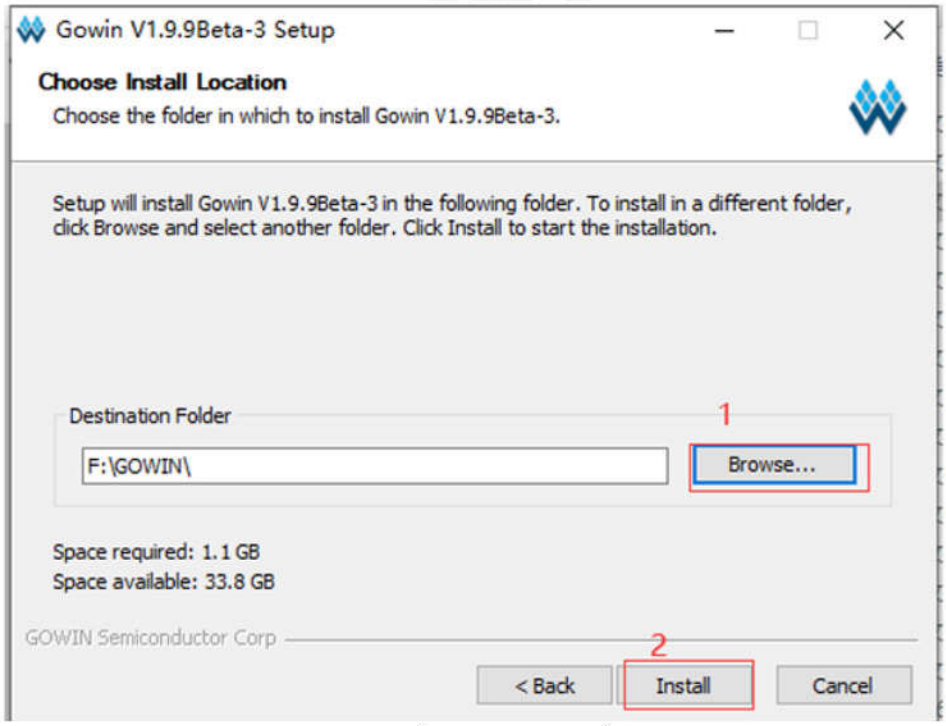
Hình 1-5 – Cho phép cài đặt

Sau khi đồng ý cài đặt thì sẽ xuất hiện cửa sổ như Hình 1-6, nhấp Next để tiếp tục



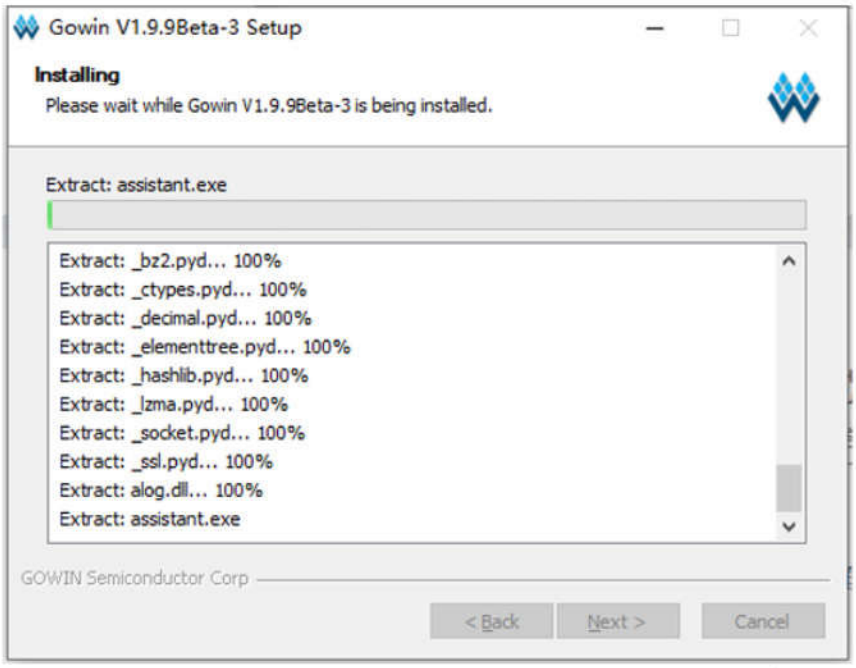
Hình 1-6 – lựa chọn thành phần cài đặt

Sau đó sẽ xuất hiện cửa sổ Hình 1-7 để lựa chọn đường dẫn và thư mục cài đặt trên ổ cứng PC. Chọn xong nhấp Install



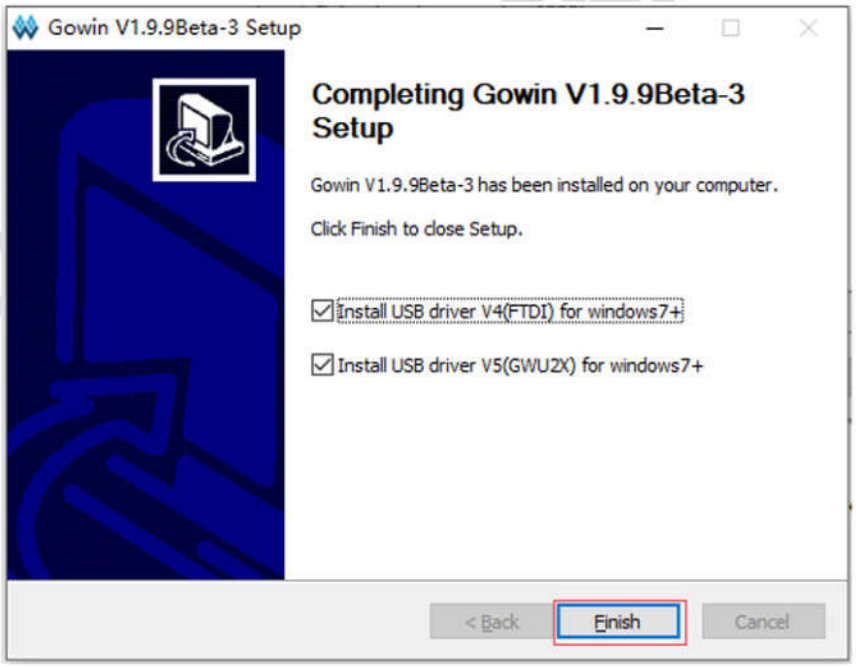
Hình 1-7– Thiết lập đường dẫn cài đặt

Nhấp vào Install sẽ xuất hiện cửa sổ Hình 1-8. Đợi đến khi cài đặt hoàn thành.



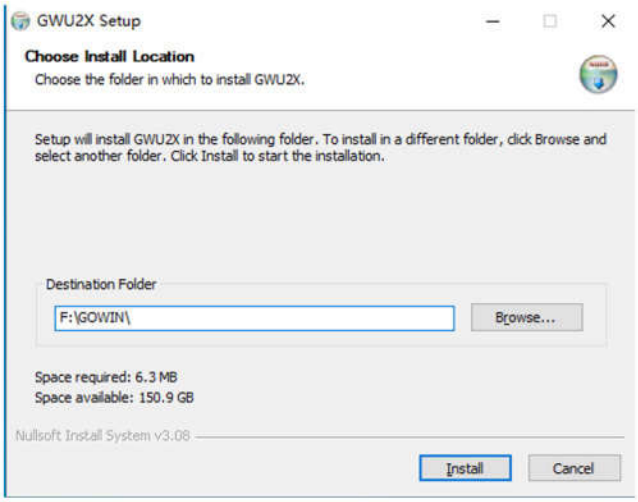
Hình 1-8 – Quá trình cài đặt

Sau khi cài đặt xong, giao diện hiển thị trong Hình 1-9 bên dưới sẽ bật lên và nhấp vào "Hoàn tất" để hoàn tất cài đặt phần mềm GOWIN. Bấm "Finish"



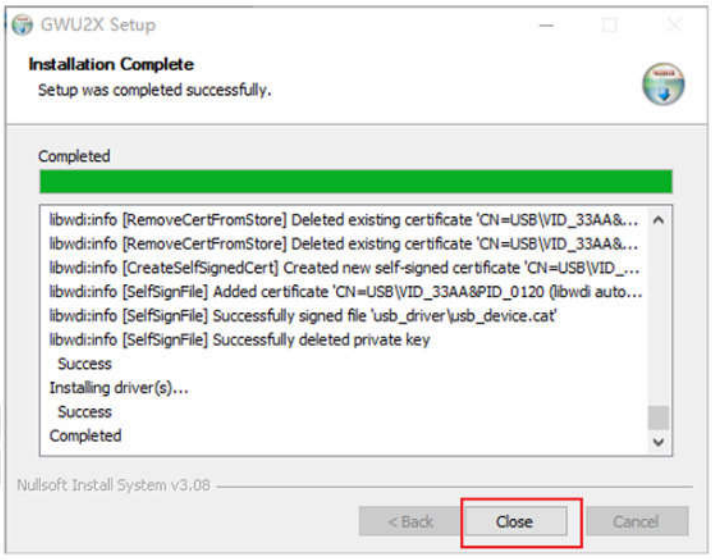
Hình 1-9 – Phần mềm cài đặt xong

Cài xong phần mềm EDA sẽ xuất hiện cửa sổ cài đặt driver cho GWU2X, chọn đường dẫn giống với EDA tool và nhấp Install để cài đặt.



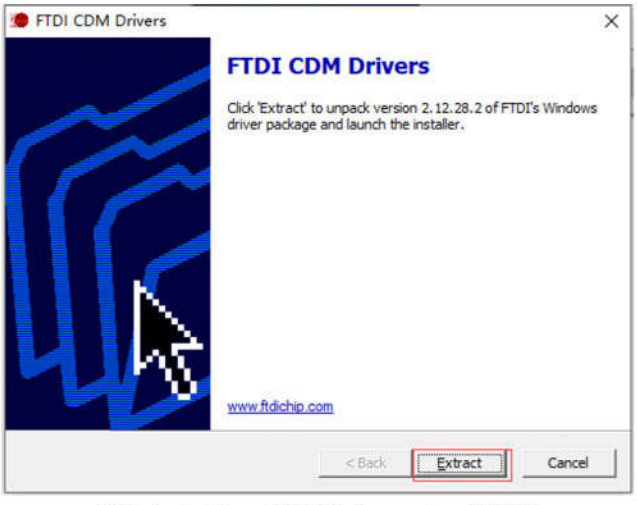
Hình 1-10 – Cài đặt GWU2X driver

Đợi cài đặt hoàn thành rồi bấm close như bình bên dưới:



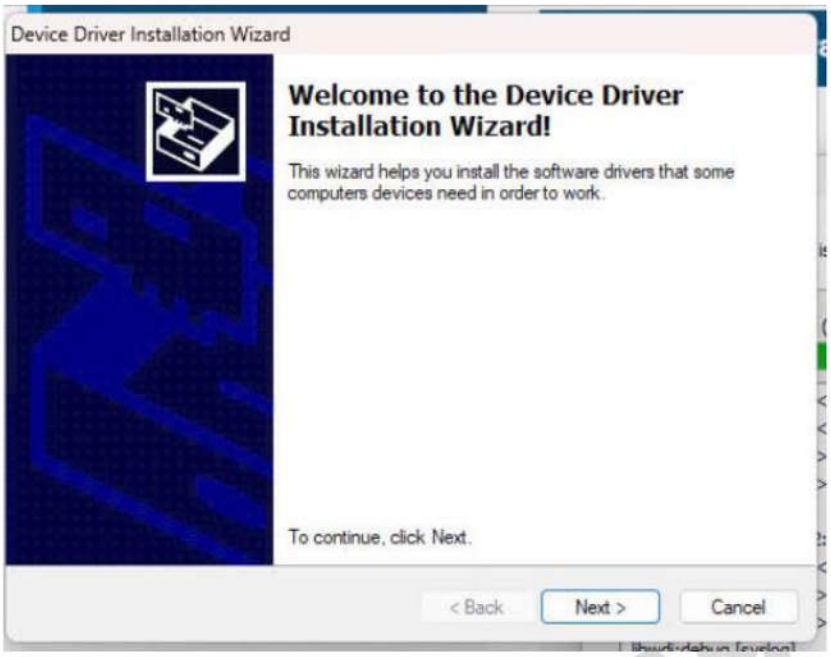
Hình 1-11– Cài đặt GWU2X hoàn thành

Sau đó chúng ta sẽ thấy cửa sổ như Hình 1-12 bên dưới, cài đặt Driver cho FTDI kết nối USB. Nhấp Extract để tiếp tục



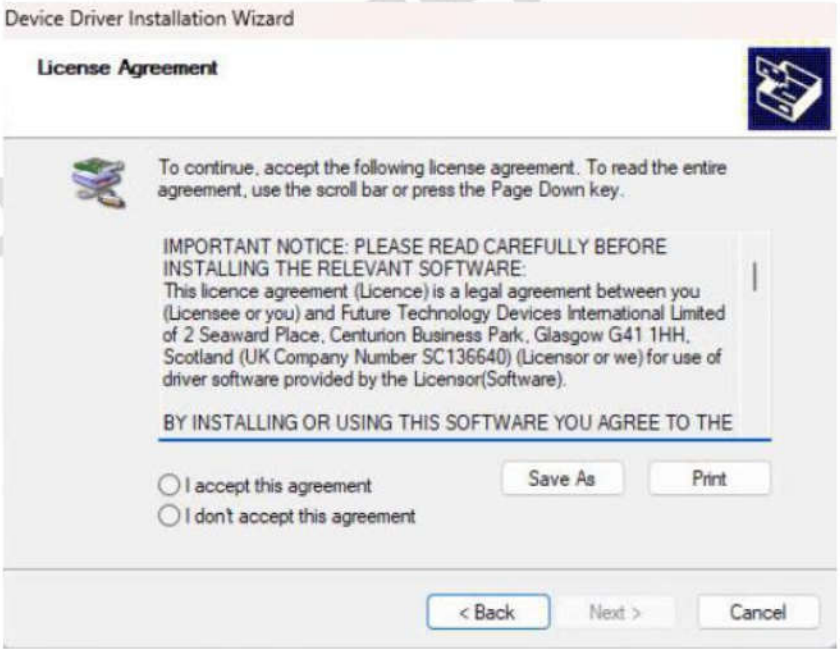
Hình 1-12 – Cài Driver cho FTDI

Sau đó, trình hướng dẫn cài đặt trình điều khiển sẽ bật lên, như trong Hình 1-13 bên dưới, nhấp vào "next"



Hình 1-13 – Giao diện cài đặt driver

Sau đó, giao diện thỏa thuận cho phép cài đặt sẽ bật lên, như trong Hình 1-14, nhấp vào "I accept this agreement" - > "Next"



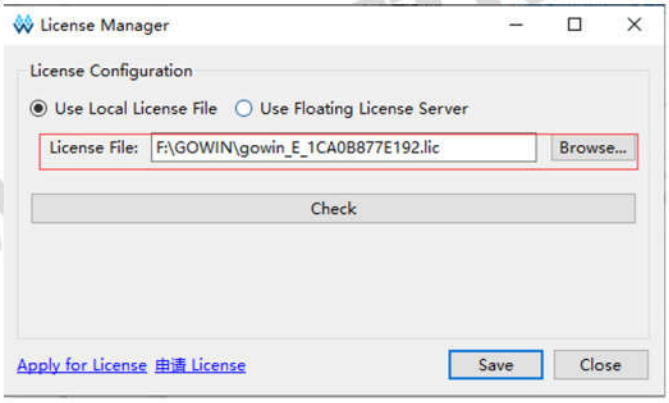
Hình 1-14 – Cho phép cài đặt

Sau đó đợi quá trình cài đặt hoàn tất, như trong Hình 1-15 bên dưới. Nhấp chuột vào Finish để hoàn tất.

Hình 1-15 – Quá trình cài đặt trình điều khiển đã hoàn tất

1.3 Xin cấp bản quyền License

Sau khi cài đặt xong phần mềm GOWIN EDA, sẽ có một Icon đến phần mềm Gowin trên màn Hình, sau khi nhấp đúp, giao diện hiển thị trong Hình 1-16 bên dưới sẽ bật lên và bạn cần tìm đường dẫn của License của chúng tôi và liên kết nó.



Hình 1-16 – Hộp thoại đường dẫn license

Lựa chọn đường dẫn đến file License rồi nhấp vào check, nếu không có vấn đề gì thì nhấp vào Save để lưu rồi có thể mở phần mềm lên sử dụng được.

Chương 2: Quá Trình Phát Triển 1 Dự Án

Mã nguồn bài thí nghiệm	|----ch2_led_flash

Giới thiệu chương

Trong chương này, chúng tôi sẽ giúp bạn làm quen với quy trình phát triển project cơ bản bằng phần mềm Gowin, bao gồm: thiết lập kỹ thuật, phân tích và tổng hợp, các ràng buộc vật lý, Place & Route, thực nghiệm xác minh cấp bo mạch và xử lý chương trình.

2.1 Khởi chạy phần mềm Gowin

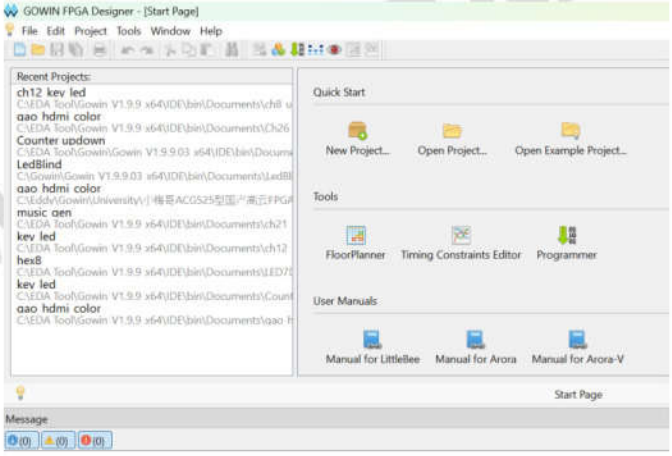
Sau khi cài đặt thì phần mềm sẽ tạo ra 1 icon trên laptop như Hình bên dưới. Nhấp đúp vào icon để khởi chạy chương trình

Hình 2-1– Icon Gowin EDA trên Desktop

2.2 Tạo dự án

Các bước tạo project như sau:

1. Sau khi khởi động chương trình sẽ mở ra trang khởi động phần mềm Gowin như bên dưới



Hình 2-2 màn Hình khởi chạy Gowin Designer

Màn Hình khởi động chia làm 4 phần:

(1) Recent project: các dự án gần đây đã được mở

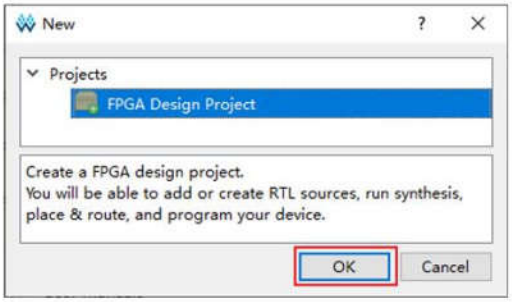
(2) Quick start: Create new project/Mở một project có sẵn/mở 1 project mẫu có sẵn.

(3) Tool: Các công cụ bao gồm: Floor Planner, Timing Constraints Editor, Programmer.

(4) User Manual: Hướng dẫn sử dụng, nhấp chuột vào sẽ liên kết đến Website của Gowin

Nhấp chuột vào new project trong để tạo project mới

2. Trình hướng dẫn để tạo ra project mới như Hình 2-3

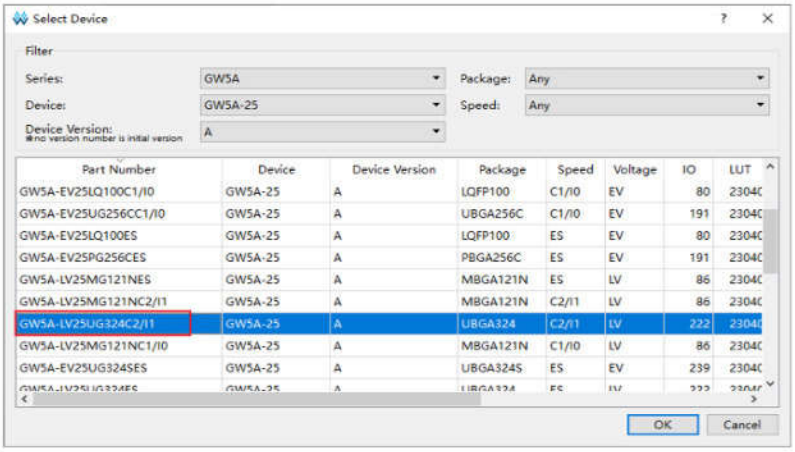


Hình 2-3 – Giao diện tạo project mới

3. Vào hộp thoại Project Name, điền tên dự án và đường dẫn dự án, cần lưu ý rằng tên dự án không được có ký tự tiếng Trung và dấu cách, nếu không sẽ có lỗi trong quá trình tiếp theo. Đường dẫn tệp không được quá dài, độ dài của đường dẫn hệ thống Windows được giới hạn ở 260 ký tự và hệ thống Linx được giới hạn ở 4096 ký tự. Xác định xem có chọn sử dụng làm vị trí dự án mặc định theo tình Hình thực tế hay không, sau khi kiểm tra, lần tiếp theo bạn tạo dự án, đường dẫn dự án sẽ được đặt theo mặc định. Sau khi hoàn tất, hãy nhấp vào Tiếp theo.

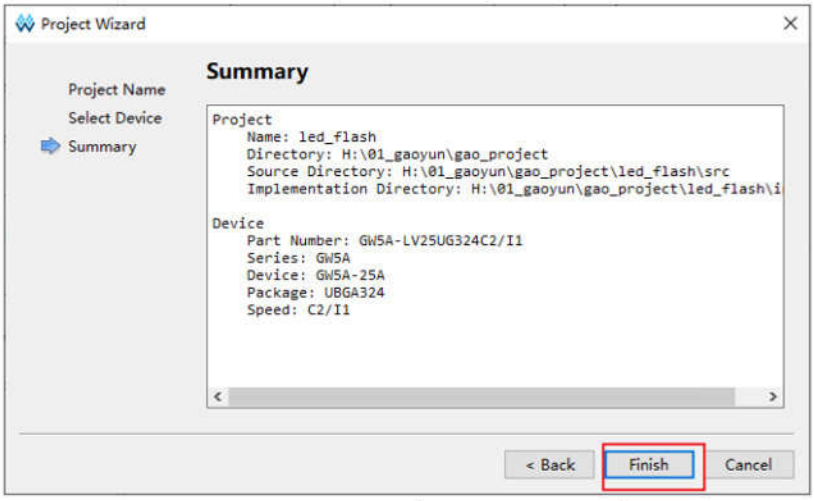
Hình 2-4– Giao diện tạo dự án

4. Hộp thoại Select Device: như Hình 2-5, Ở đây chúng ta cần chọn mẫu chip FPGA cụ thể được sử dụng trong thiết kế và đánh giá nó theo dòng, gói, mức tốc độ, v.v. của FPGA được sử dụng trong bo mạch và người dùng cũng có thể đánh giá theo các từ trên chip FPGA. Sau khi hoàn tất, hãy nhấp vào Tiếp theo.



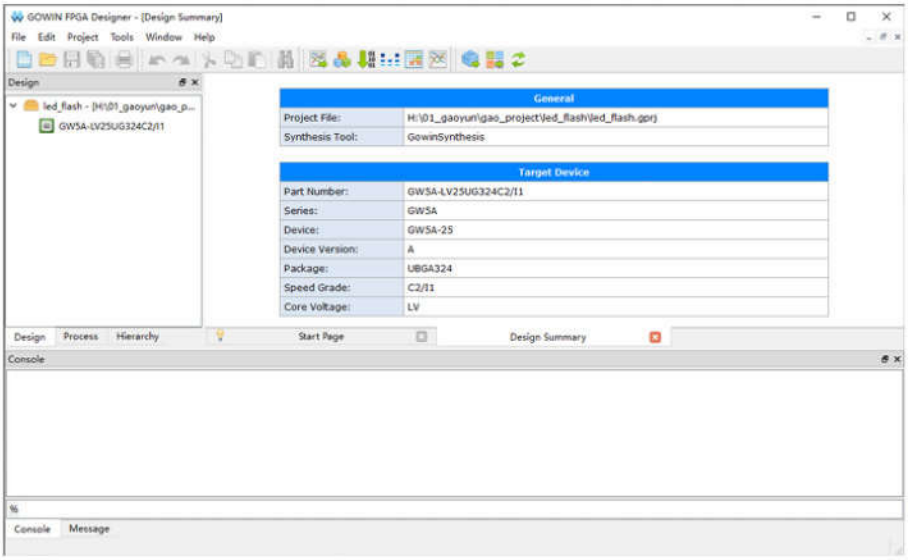
Hình 2-5 – Chọn chip cho dự án thiết kế

5. Summary: Sau cùng là giao diện Summary tổng hợp thông tin cơ bản như Hình 2-6



Hình 2-6 – Giao diện tổng hợp thông số dự án

6. Tiếp theo, chúng ta sẽ vào giao diện thiết kế Gowin, như trong Hình 2-7 bên dưới. Giao diện chính của thiết kế chủ yếu bao gồm các module như Design, Proses, Hierachi và Consol. Sau đây sẽ dẫn bạn làm quen với giao diện thiết kế công cụ Gowin.

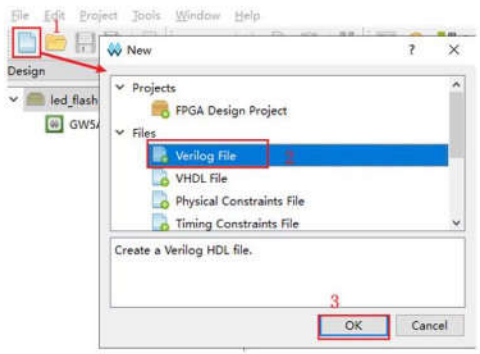


Hình 2-7 – Giao diện công cụ thiết kế

2.3 Thêm tệp nguồn

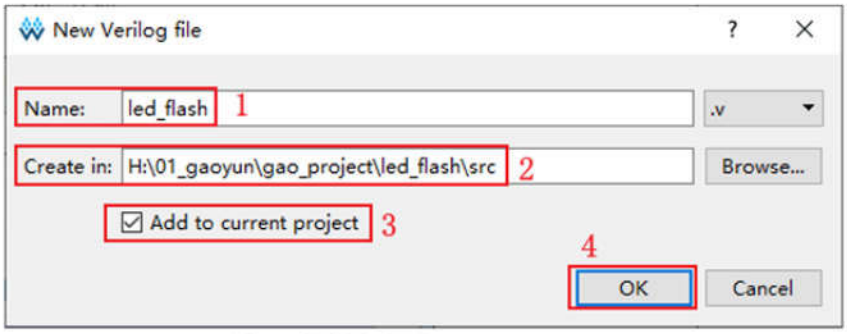
Sau khi tạo một dự án bằng công cụ Gowin, chúng ta có thể thêm các tệp nguồn vào dự án trong phần mềm Gowin. Tiếp theo, chúng ta sẽ trình bày quy trình bằng cách tạo một tệp nguồn Verilog.

1. Bằng cách nhấp vào thanh công cụ hoặc lần lượt nhấp vào thanh menu, File > New, Sau nhấp vào như Hình bên dưới, giao diện Chọn Loại tệp xuất hiện, ở đây ta chọn loại Verilog file, sau đó nhấp vào Ok.



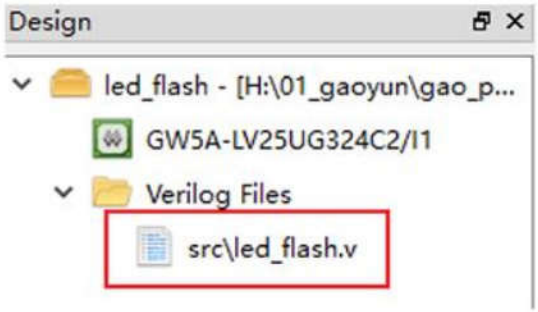
Hình 2-8 – Thêm tệp nguồn Verilog

2. Sau đó, giao diện New Verilog file xuất hiện, đặt tên cho file trong hộp thoại này và chọn file đường dẫn lưu, theo mặc định, nó được lưu trong thư mục src của thư mục dự án. Sau đó 'tick' vào 'Add to Current Project' (thêm vào thư mục dự án hiện tại), như trong Hình 2-9 bên dưới. Và nhấp OK.



Hình 2-9 – Giao diện bày trí New Verilog file

3. Sau khi các tệp nguồn được thêm vào, chúng ta có thể thấy rằng một thư mục mới Verilog Files xuất hiện trong hộp khung Design và dưới thư mục này chính là tệp nguồn của Verilog như Hình 2-10



Hình 2-10 – Hiển thị tệp nguồn được thêm vào

4. Nhấp vào tệp led_flash.v để viết mã kỹ thuật của thí nghiệm này, chức năng của thí nghiệm này là lật trạng thái của đèn LED và thực hiện nó bằng cách thiết kế bộ đếm, như sau:

```
module led_flash(
        input clk,
        input rst_n,
        output reg led
);
```

```
    reg [25:0] cnt;
    always @ (posedge clk or negedge rst_n)
    begin
        if (!rst_n)
          cnt <= 26'd0;
      else if (cnt < 26'd49_999_999)
            cnt <= cnt + 1'b1;
      else
            cnt <= 26'd0;
    end
    always @ (posedge clk or negedge rst_n)
    begin
       if (!rst_n)
            led <= 1'b0;
        else if (cnt == 26'd49_999_999)
              led <= ~led;
        else
              led <= led;
        end
endmodule
```

verilog Copy Edit

```
module led_flash(
    input clk,       // Xung nhịp (Clock)
    input rst_n,     // Tín hiệu reset (Active Low)
    output reg led   // Tín hiệu điều khiển LED
);
```

- `clk` : Tín hiệu xung nhịp (clock), thường là **50MHz** hoặc **100MHz** tùy vào phần cứng FPGA.
- `rst_n` : **Reset chủ động mức thấp** (nếu `rst_n = 0`, hệ thống reset).
- `led` : Biến output kiểu **reg**, dùng để điều khiển đèn LED.

2. Bộ đếm `cnt` để tạo độ trễ

verilog Copy Edit

```
reg [25:0] cnt; // Thanh ghi đếm 26 bit
```

- `cnt` có 26 bit, có thể đếm từ `0` đến `2^26 - 1 = 67,108,863`.
- Giá trị `49_999_999` được dùng để tạo độ trễ dựa trên tần số `clk`.

verilog Copy Edit

```
always @ (posedge clk or negedge rst_n)
begin
    if (!rst_n)
        cnt <= 26'd0; // Reset bộ đếm khi rst_n = 0
    else if (cnt < 26'd49_999_999)
        cnt <= cnt + 1'b1; // Tăng bộ đếm
    else
        cnt <= 26'd0; // Reset bộ đếm khi đạt giá trị 49_999_999
end
```

- Bộ đếm `cnt` tăng lên mỗi chu kỳ clock.
- Khi đạt đến **49_999_999**, bộ đếm được reset về 0.

⏳ **Mục đích:** Nếu `clk = 50MHz`, thì:

$$T = \frac{49,999,999 + 1}{50,000,000} = 1 \text{ giây}$$

→ Cứ **1 giây**, bộ đếm sẽ lặp lại.

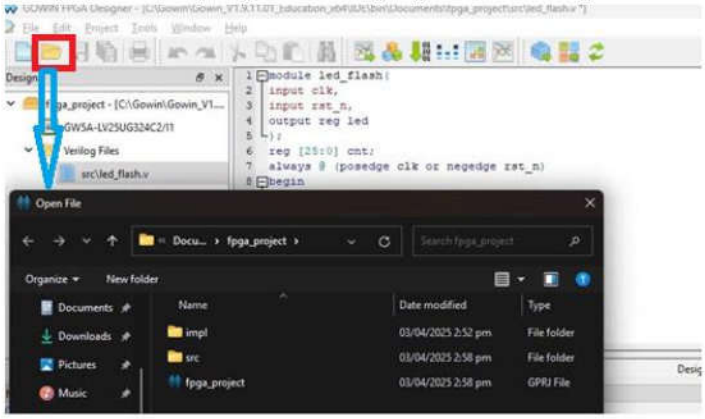
verilog Copy Edit

```
always @ (posedge clk or negedge rst_n)
begin
    if (!rst_n)
        led <= 1'b0; // Reset LED khi rst_n = 0
    else if (cnt == 26'd49_999_999)
        led <= ~led; // Đảo trạng thái LED mỗi khi bộ đếm đạt 49_999_999
    else
        led <= led; // Giữ nguyên trạng thái LED
end
```

- Khi `cnt == 49_999_999`, **LED sẽ đổi trạng thái** (`led <= ~led`).
- Vì `cnt` đạt giá trị này sau **mỗi 1 giây**, LED sẽ nhấp nháy với **chu kỳ 2 giây** (1 giây sáng, 1 giây tắt).

5. Sau khi chỉnh sửa xong, Ctrl + S lưu tệp nguồn. Lúc này, tệp nguồn đã được thêm thành công.

Với mô tả trên, một tệp nguồn mới được tạo. Nếu bạn đã có tệp nguồn Verilog, bạn có thể thêm tệp đó bằng cách nhấp vào thanh công cụ open file or project rồi đến đường dẫn chọn tập tin trong thư mục dự án để thêm vào. Chú ý, phải chép tập tin hoặc project vào trong thư mục dự án.



Hình 2-11– Mở file nguồn lưu sẵn

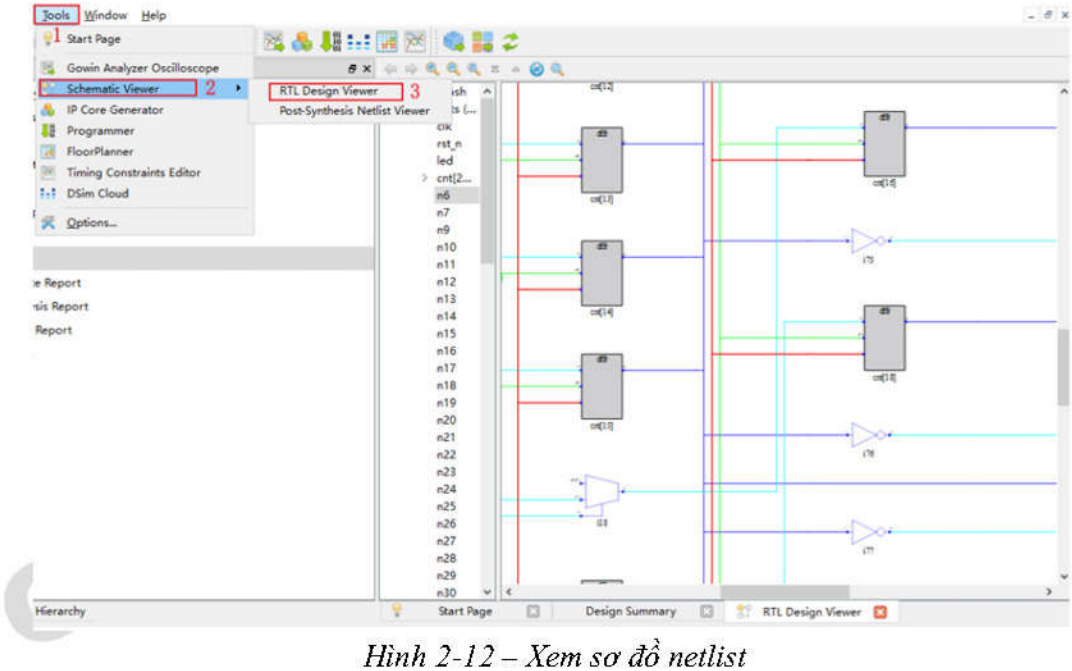
2.4 Mô tả và phân tích RTL

2.4.1 Giới thiệu về các chức năng mô tả và phân tích RTL

Khi chúng ta sử dụng Verilog để mô tả một mạch nhất định, có thể có lỗi cú pháp hoặc logic. Lỗi cú pháp sẽ được phần mềm Gowin nhắc trong quá trình biên dịch, nhưng phần mềm Gowin không thể phát hiện lỗi logic, chẳng hạn như đi dây không chính xác giữa các mô-đun. Chức năng mô tả và phân tích RTL cho phép bạn xem cấu trúc và cú pháp RTL của dự án, sau đó phân tích và sửa lỗi logic.

2.4.2 RTL mô tả và phân tích quá trình

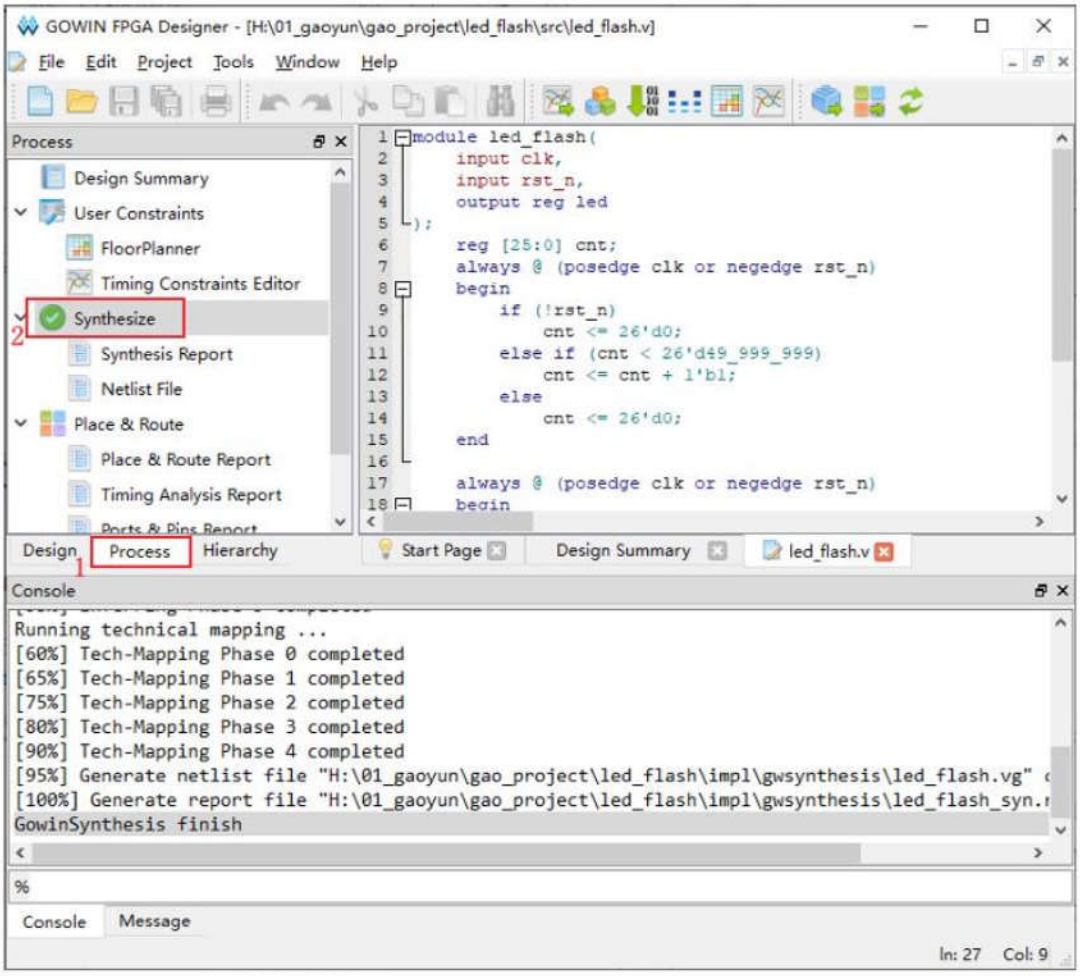
Trong phần mềm Gowin, người dùng được cung cấp chức năng xem phân tích thiết kế RTL, nhấp vào Tools->Schematic Viewer->RTL Design Viewer, có thể xem sơ đồ sơ đồ Netlist sau giai đoạn RTL. Như thể hiện trong Hình 2-12 bên dưới, netlist được thể hiện bằng các ký hiệu logic không liên quan đến các phần cơ bản của FPGA. Người đọc có thể xem sơ đồ sau giai đoạn tổng hợp, trong đó các phần tử logic được xây dựng từ các thành phần cơ bản trong FPGA.



Hình 2-12 – Xem sơ đồ netlist

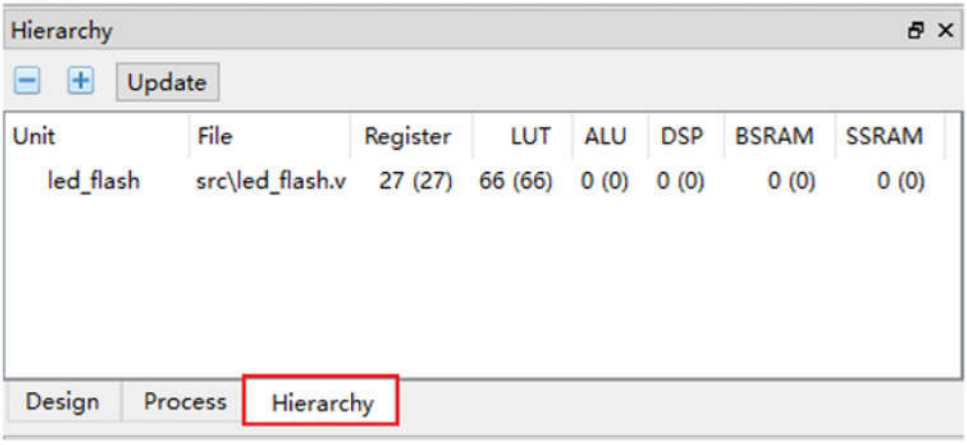
2.5 Phân tích và tổng hợp

Chúng ta có thể lần lượt nhấp vào Process->Synthesize để phân tích và tổng hợp, hoặc chúng ta có thể nhấp trực tiếp vào Analysis and Synthesize trên thanh công cụ, và sau khi phân tích và tổng hợp, danh sách mạng mạch dựa trên các tài nguyên cơ bản của FPGA được tạo ra, như trong Hình 2-13 bên dưới, nếu không có lỗi có nghĩa là việc tạo thành công, nếu có lỗi, người dùng nên sửa đổi theo lời nhắc.



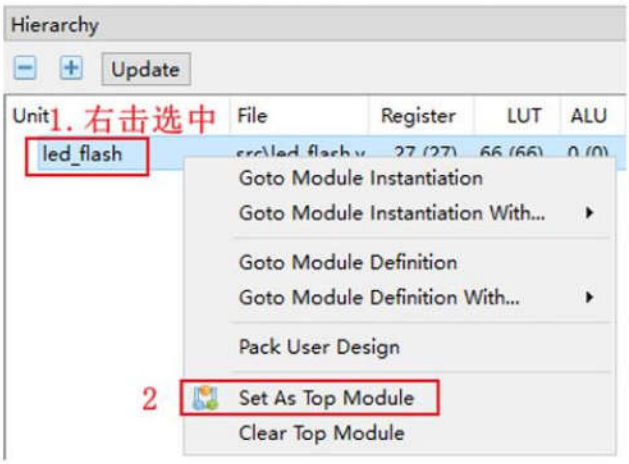
Hình 2-13 – Synthesis

Sau khi tổng hợp xong, cửa sổ Hierachi tự động hiển thị thông tin tài nguyên trong thiết kế dự án hiện tại, như trong Hình 2-14 bên dưới.



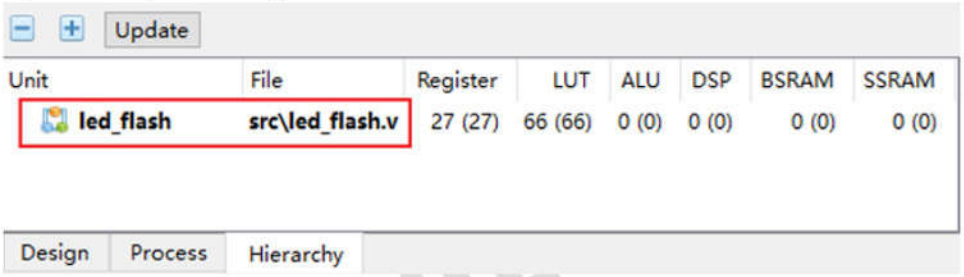
Hình 2-14 – Cửa sổ hiển thị thông tin tài nguyên

Trong cửa sổ Hierachi, chúng ta đặt chuột qua tệp kích thước Ledlash và nhấp chuột phải vào tệp để đặt tệp làm tệp cấp cao nhất, như trong Hình 2-15 bên dưới.



Hình 2-15 – Thiết lập Top Module

Sau khi cài đặt hoàn tất, file tên sẽ được in đậm, sau đó tổng hợp sẽ được phân tích lại và cửa sổ Hierarchy sẽ được hiển thị như trong Hình 2-16 bên dưới.

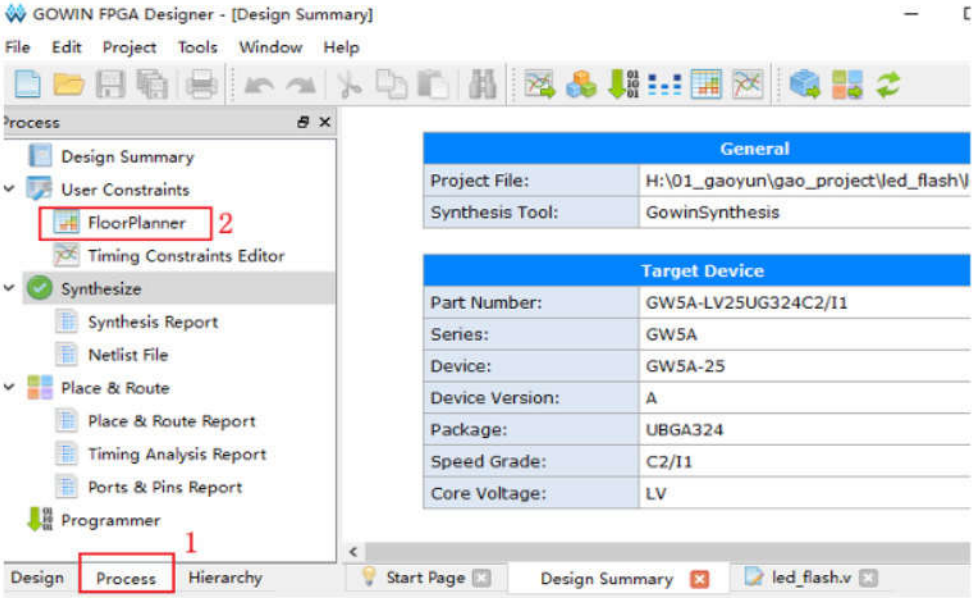


Hình 2-16 – Thiết lập Top Module thành công

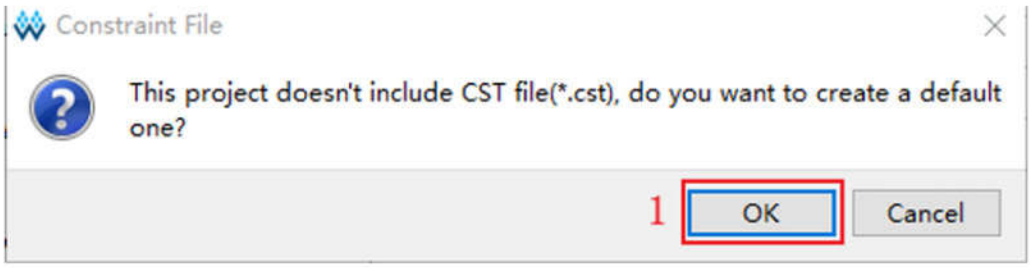
2.6 Các ràng buộc về vật lý

Chúng ta có thể thực hiện ràng buộc vật lý bằng cách nhấp vào Process-> Floorplan, hoặc chúng ta có thể nhấp trực tiếp vào thanh công cụ, như trong Hình 2-17 bên dưới.

Sau đó, hộp thoại hiển thị trong Hình 2-18 tiếp theo bên dưới bật lên, vì chúng ta không tạo một tệp .cst mới để lưu trữ các ràng buộc vật lý, một hộp thoại sẽ bật lên, hỏi chúng ta có cần tạo tệp .cst hay không, ở đây chúng ta có thể trực tiếp nhấp vào OK.

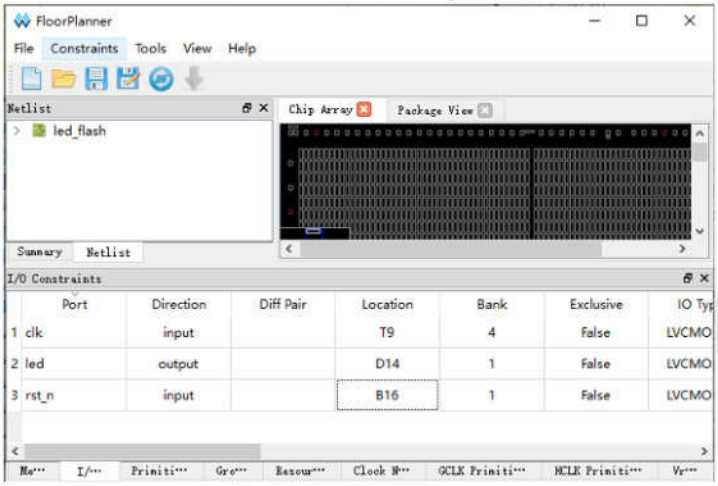


Hình 2-17 – Cách thức hoạt động của các ràng buộc vật lý



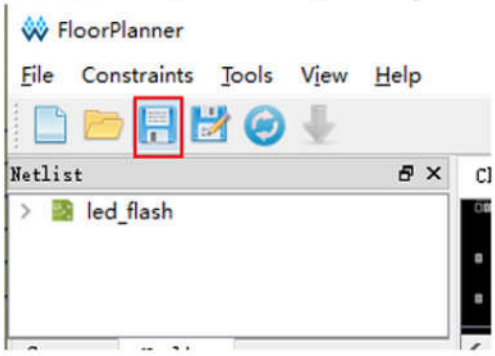
Hình 2-18 – xác nhận tạo CTS file (.cts)*

Chuyển đến giao diện Floorplan nhấp vào I /O Constraints, thực hiện các ràng buộc I/O, sau đó gán các chân và tiêu chuẩn mức theo bảng của riêng bạn, như thể hiện trong Hình 2-19 bên dưới



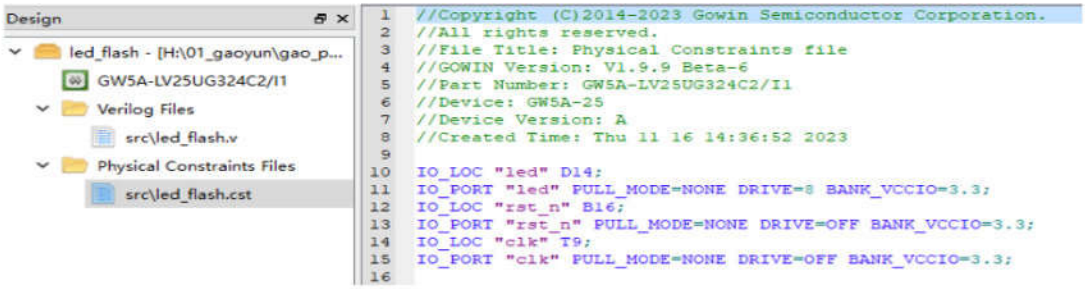
Hình 2-19 – Giao diện ràng buộc

Sau khi các ràng buộc hoàn tất, hãy lưu tệp, như trong Hình 2-20 bên dưới.



Hình 2-20 – Lưu tệp ràng buộc

Sau khi ràng buộc hoàn tất, chúng ta có thể nhấp vào Design->led_flash.cts. để xem nội dung của tệp, như trong Hình 2-21 bên dưới.

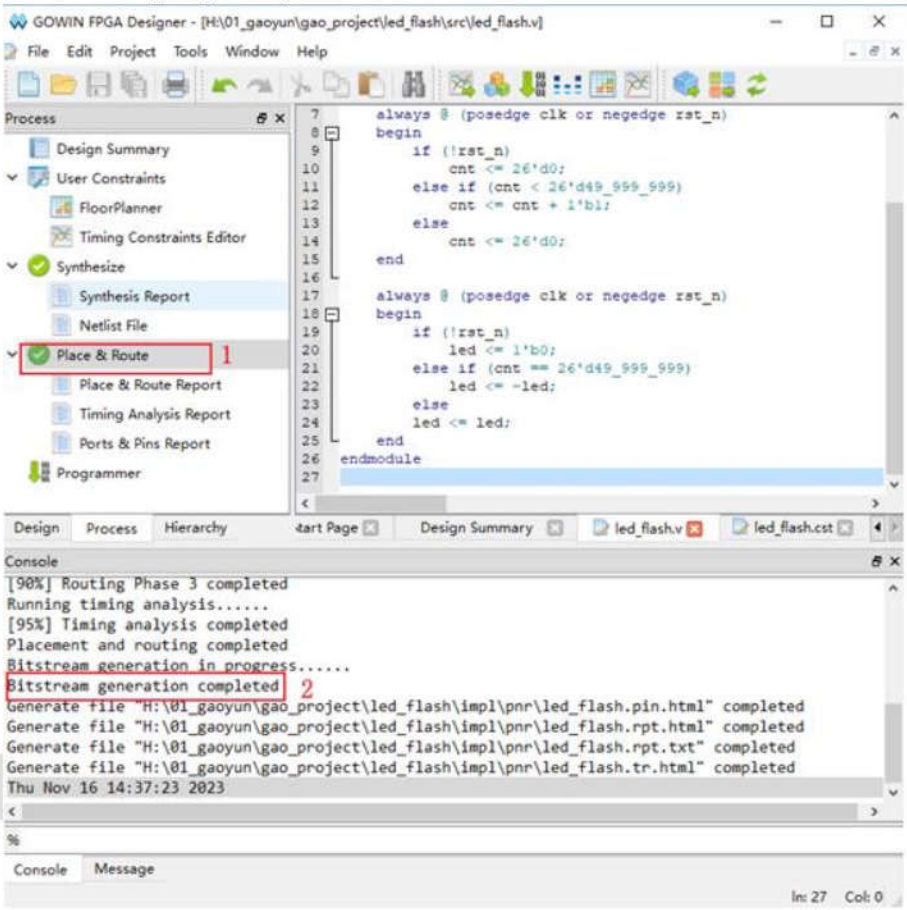


Hình 2-21– Nội dung ràng buộc vật lý

Như bạn có thể thấy từ Hình trên, nội dung của tệp giống như những gì chúng tôi thiết lập trong **Floorplan**, cho thấy rằng các ràng buộc vật lý của chúng tôi đã thành công.

2.7 Place & Route

Chúng ta nhấp vào **Process - > Place & Route** để đặt và định tuyến, sau khi bố cục và tuyến đường thành công, chúng ta có thể thấy **"Bitstream Generate Complete",** có nghĩa là tệp File được tạo thành công, như trong Hình 2-22 bên dưới, nếu nó không thành công, vui lòng sửa đổi nó theo thông tin nhắc nhở được cung cấp trong Console



Hình 2-22 – Place & Route thành công

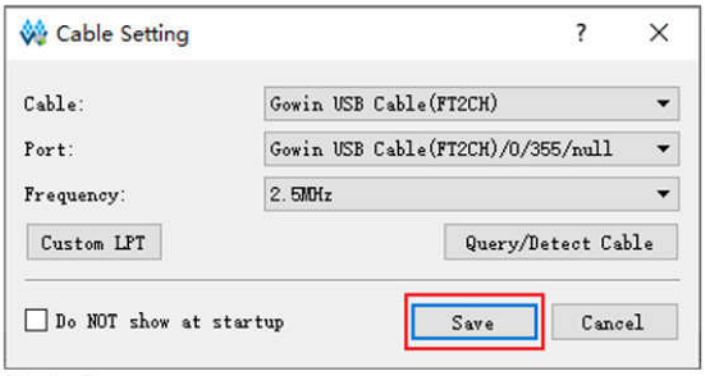
2.8 Xác thực chương trình trên Board mạch

Chúng ta đã tạo tệp fs cần thiết cho thử nghiệm thông qua các bước trước và sau đó chúng ta cần đặt tệp fs xuống FPGA,

1. Kết nối trình tải xuống và nguồn điện với bo mạch, như trong Hình 2-23 bên dưới.

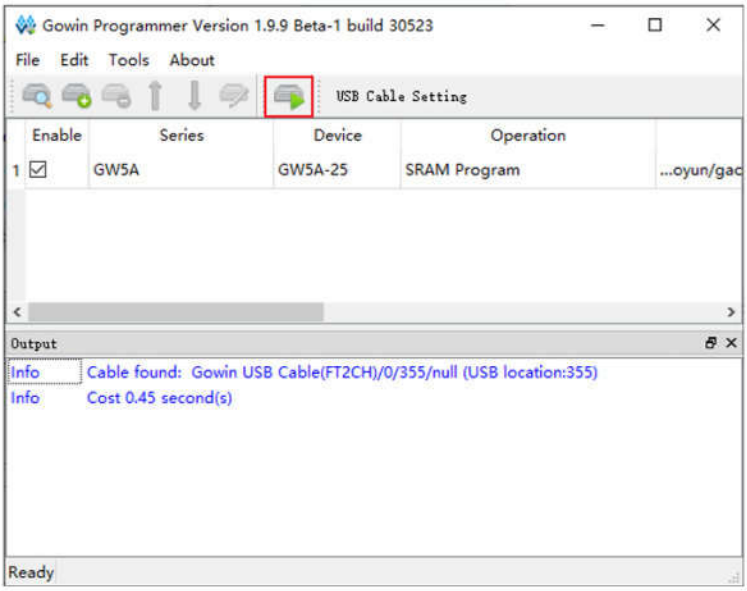
Hình 2-23 – Sơ đồ kết nối phần cứng

2. Sau đó bật nguồn bo mạch phát triển, nhấp vào Process->Program Device, và giao diện như trong Hình 2-24 bên dưới sẽ bật lên, cho biết thiết bị đã được phát hiện, sau đó nhấp vào Save. Nếu No USB Cable Connection được hiển thị, hãy kiểm tra kết nối phần cứng của bạn.

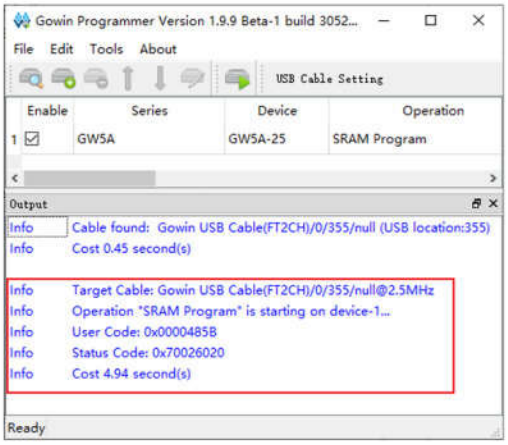


Hình 2-24 – Giao diện kết nối mạch nạp và cấu Hình

3. Nhấp vào Gowin Programmer để tải tệp Files xuống chip FPGA, như trong Hình 2-25 bên dưới. Sau khi tải xuống thành công, một thông báo sẽ được in, như trong Hình 2-26 bên dưới.

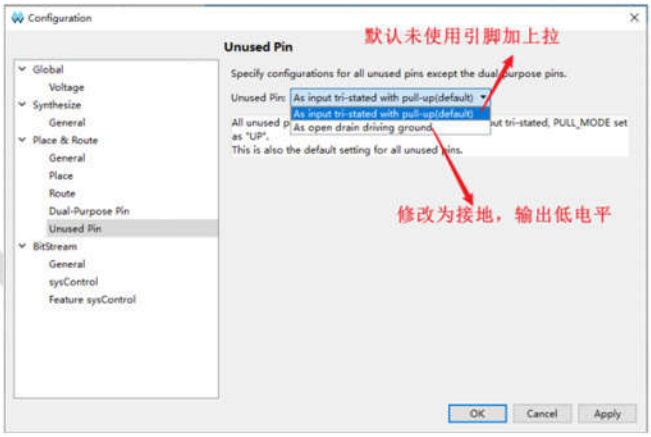


Hình 2-25– Tải file fs



Hình 2-26 – Tải tệp thành công

4. Hiện tượng quan sát: Lúc này, chúng ta có thể thấy rằng đèn LED0 tương ứng trên bảng phát triển đang nhấp nháy, có nghĩa là thí nghiệm của chúng ta đã thành công. Lúc này, chúng ta có thể thấy rằng sau khi chương trình được tải xuống, cả 4 đèn LED đều sáng, nhưng chúng ta chỉ điều khiển một trong các đèn, điều này là do trong phần mềm Gowin, chân không kiểm soát mặc định được thêm vào kéo lên, tức là nó sẽ xuất ra mức cao, khiến đèn LED không kiểm soát bật sáng, điều này có thể được đặt trong phần mềm, như Hình dưới đây.

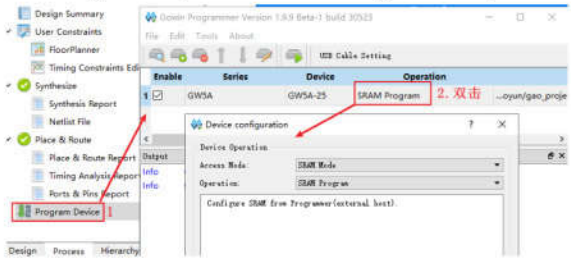


Hình 2- 27– Giao diện cấu Hình cho device

2.9 Lưu trữ chương trình trên các bộ nhớ FPGA

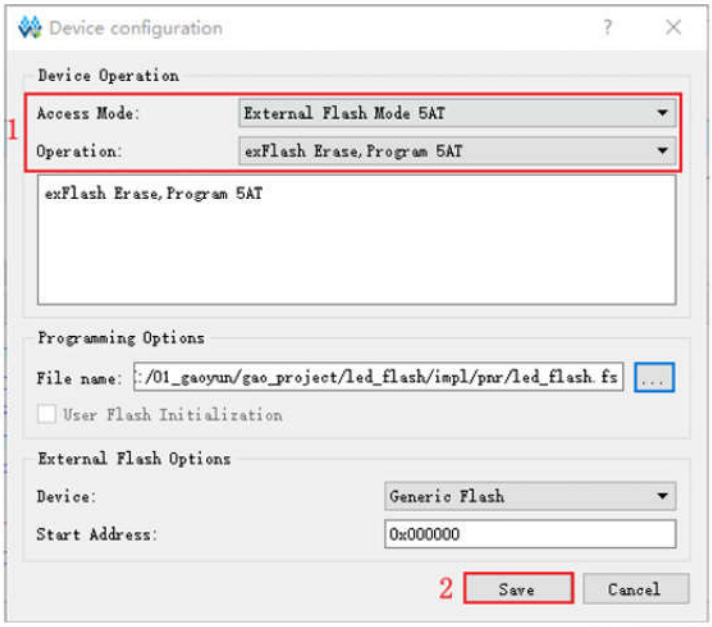
Chúng tôi đã tải xuống chương trình qua phần trước, sau khi bo mạch phát triển được bật lại, chương trình sẽ bị mất, để đạt được sự cố mất điện mà không bị mất, phần tiếp theo sẽ hướng dẫn bạn cách lưu chương trình trên FPGA.

1. Nhấp chuột vào Program Device sẽ vào trong giao diện Gowin Programme, ở cột Operation nhấp đôi vào SRAM Program sẽ vào giao diện Device configuration, như Hình 2-28 dưới đây:



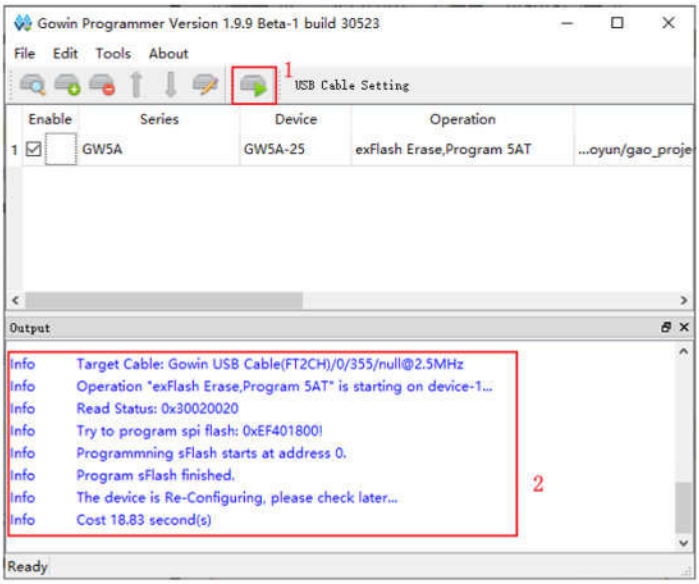
Hình 2- 28 – Giao diện cấu Hình cho device

2. Tại giao diện Device configuration, định Access Mode set cấu Hình External Flash Mode 5A, Phần Operation thì set là exFlash Erase, Program 5A, sau đó nhấp Save, như Hình 2-29 dưới đây



Hình 2-29 – Giao diện set lưu trữ

3. Nhấp để tải xuống chương trình và sau khi tải xuống thành công, thông báo sau sẽ được in, như trong Hình 2-30 bên dưới.



Hình 2-30 – Tải chương trình thành công

4. Sau đó, bật lại bảng phát triển và bạn có thể thấy rằng bảng phát triển đang chạy chương trình đã được củng cố trong thử nghiệm này

2.10 Tổng kết chương

Chương này sẽ đưa bạn qua quá trình phát triển cơ bản của phần mềm Gowin và mô tả các chi tiết cần chú ý trong từng quy trình và một số thiếu sót sẽ được bổ sung trong các chương sau. Là chương đầu tiên của thử nghiệm, chương này giới thiệu toàn bộ quy trình thiết kế bằng tay, người dùng phải tuân theo quy trình và làm quen với toàn bộ quy trình, biết mục đích và chức năng của từng bước, tránh lỗi kỹ thuật do thiếu bước, hoặc thậm chí toàn bộ dự án bị đảo ngược.

Chương 3: Gowin Kết Hợp Mô Hình Mô Phỏng Modelsim

Mã nguồn bài thí nghiệm	|----Bài thí nghiệm số 2 |----ch3_led_flash_sim

Dẫn nhập chương

Chương này sẽ hướng dẫn bạn tìm hiểu và thành thạo mô phỏng Modelsim, cách biên dịch Gowin trong Modelsim và thực hiện mô phỏng chung Gowin và Modelsim.

3.1 Modelsim biên dịch thư viện Gowin

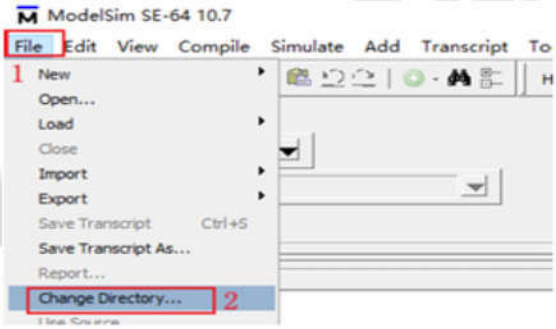
"IP CORE hoặc nguyên lý của Gowin chỉ có thể được sử dụng trong nền tảng phát triển tương ứng, công cụ bên thứ ba như Modelsim không thể truy cập vào kết quả logic hoạt động bên trong IP CORE của Gowin, do đó không thể thực hiện mô phỏng trực tiếp. Cần phải tiến hành biên dịch thư viện Gowin. Chương này sẽ hướng dẫn các bạn cách biên dịch thư viện Gowin, bước này cần thực hiện khi lần đầu tiên sử dụng FPGA Gowin, và thư viện đã biên dịch có thể được sử dụng bất kỳ lúc nào sau đó mà không cần biên dịch lại trước mỗi lần mô phỏng."

Nếu bạn chưa cài đặt Moelsim, bạn có thể truy cập diễn đàn của chúng tôi để tải xuống và cài đặt nó và liên kết tải xuống như sau:

https://pan.baidu.com/s/1a0exXp1salxQINumsARs3Q?pwd=dziw

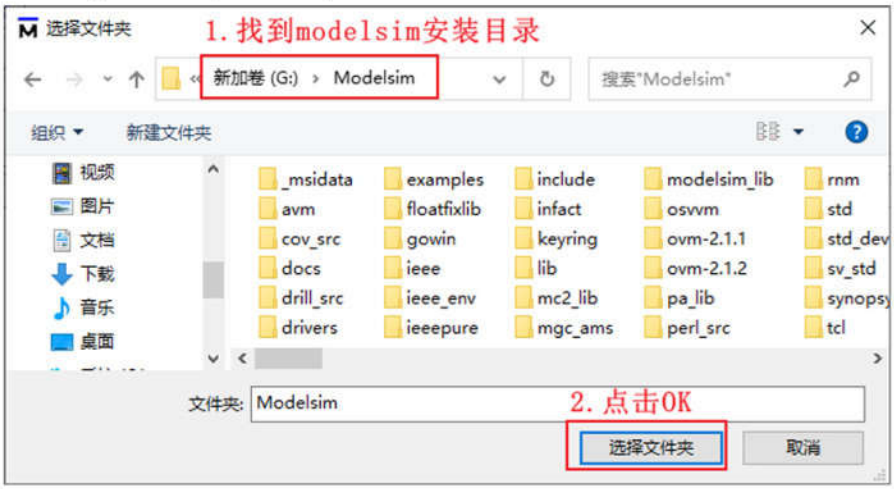
3.1.1 Tạo mới thư viện Modelsim

a. Mở phần mềm Modersim và nhấp vào các tùy **chọn [File] - > [Change Directory]** trong thanh menu để vào trang chuyển đổi đường dẫn, như trong Hình 3-1 bên dưới.



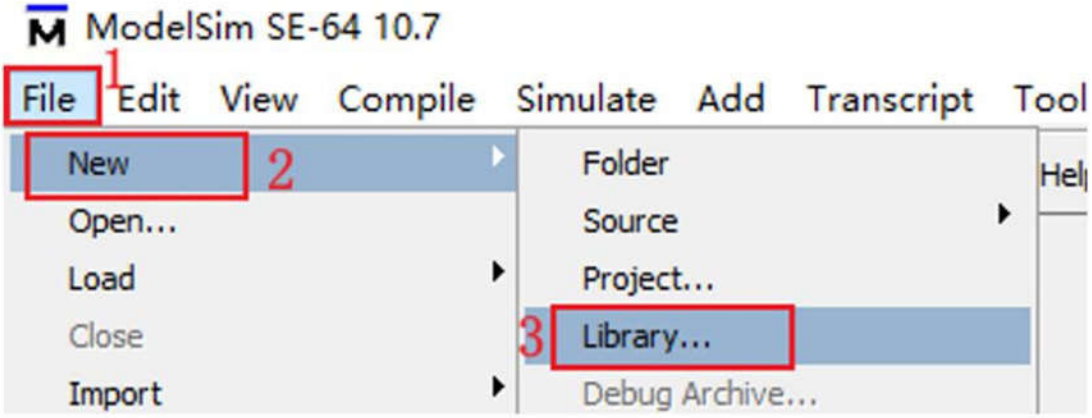
Hình 3-1– Truy cập Change Directory

b. Trên trang Chọn thư mục mở ra, hãy tìm thư mục cài đặt phần mềm Modelsim, như trong Hình 3-2 bên dưới, sau đó nhấp vào Chọn thư mục



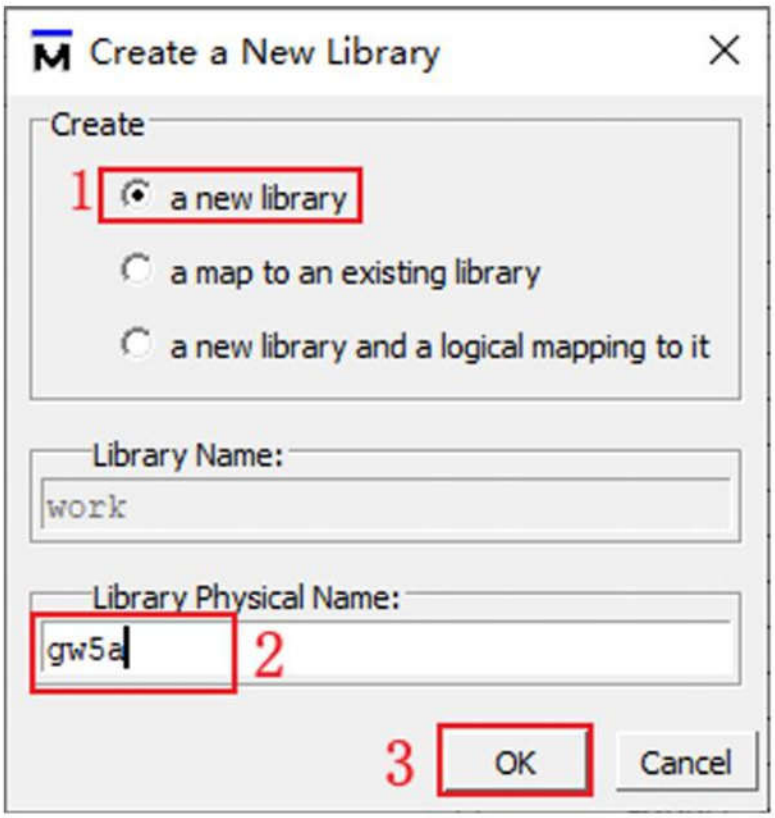
Hình 3-2 – Lựa chọn cài đặt Modelsim

c. Nhấp vào thanh Menu [File]→ [New] → [library], mở trang tạo thư viện như Hình 3-3 bên dưới:



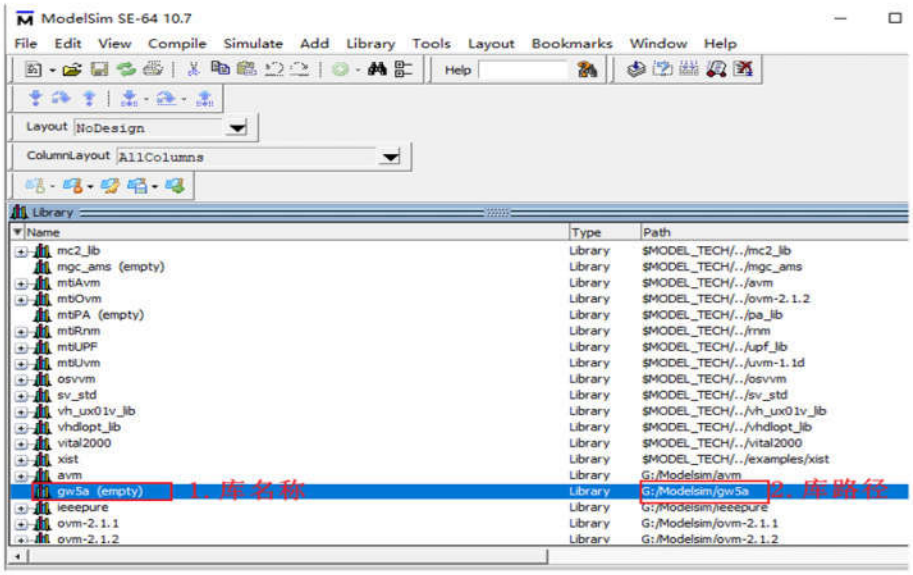
Hình 3-3 – Mở giao diện tạo thư viện

d. Trong giao diện mở ra, chọn mục đầu tiên "a new Library", "Library Physical Name" và nhập tên thư viện, ví dụ: chúng tôi đang biên dịch một thiết bị thuộc dòng Gowin 5A, đặt tên là "gw5a", sau đó nhấp vào Ok, như Hình dưới đây



Hình 3-4 – Tạo lập thư viện mới

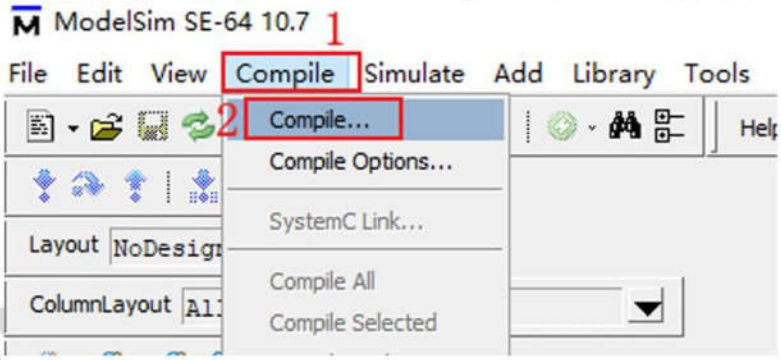
e. Nếu trong danh sách thư viện xuất hiện thư viện **gw5a (empty)** và đường dẫn đúng với vị trí đã thiết lập trước đó (trong thư mục gốc của dự án), thì điều đó cho thấy việc tạo thư viện mới đã hoàn tất, như minh họa trong Hình 3-5 dưới đây."



Hình 3-5 – Tạo thư viện mới hoàn thành

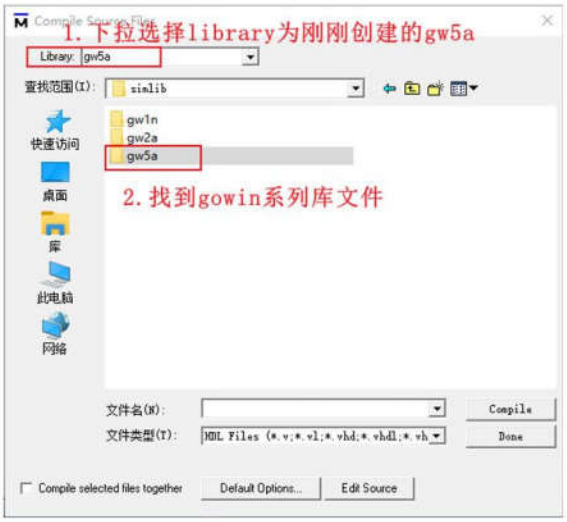
3.1.2 Biên dịch tệp thư viện FPGA của Gowin để dùng với Modelsim

a. Nhấp chuột vào [compile] trên thanh Menu để mở giao diện biên dịch thư viện như Hình 3-6



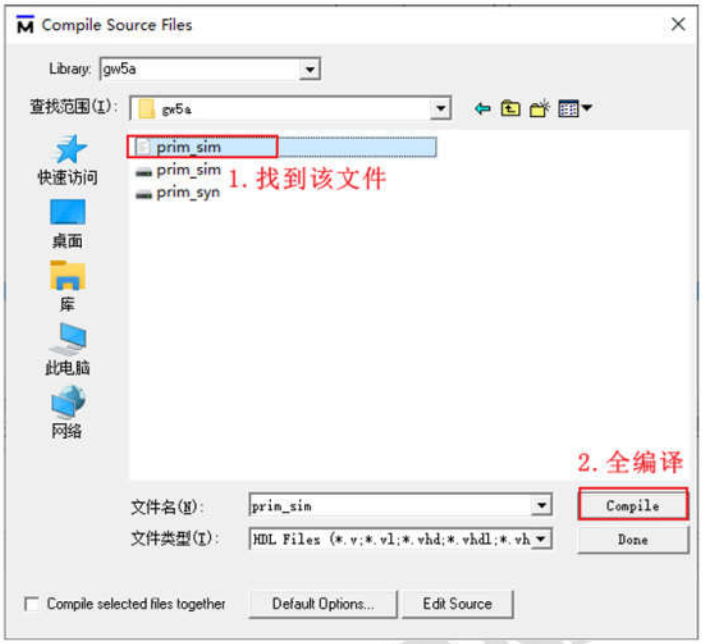
Hình 3-6 – Mở giao diện biên dịch thư viện

b. Trong giao diện mở ra, sẽ sổ xuống danh mục thư viện gw5a vừa tạo ở trên, sau đó sử đổi đường dẫn thư viện FPGA dòng gw5a của phần mềm Gowin, cụ thể: "\G owin\Gowin_V1.9.9Beta-1\IDE\simlib\gw5a" như Hình 3-7 bên dưới:



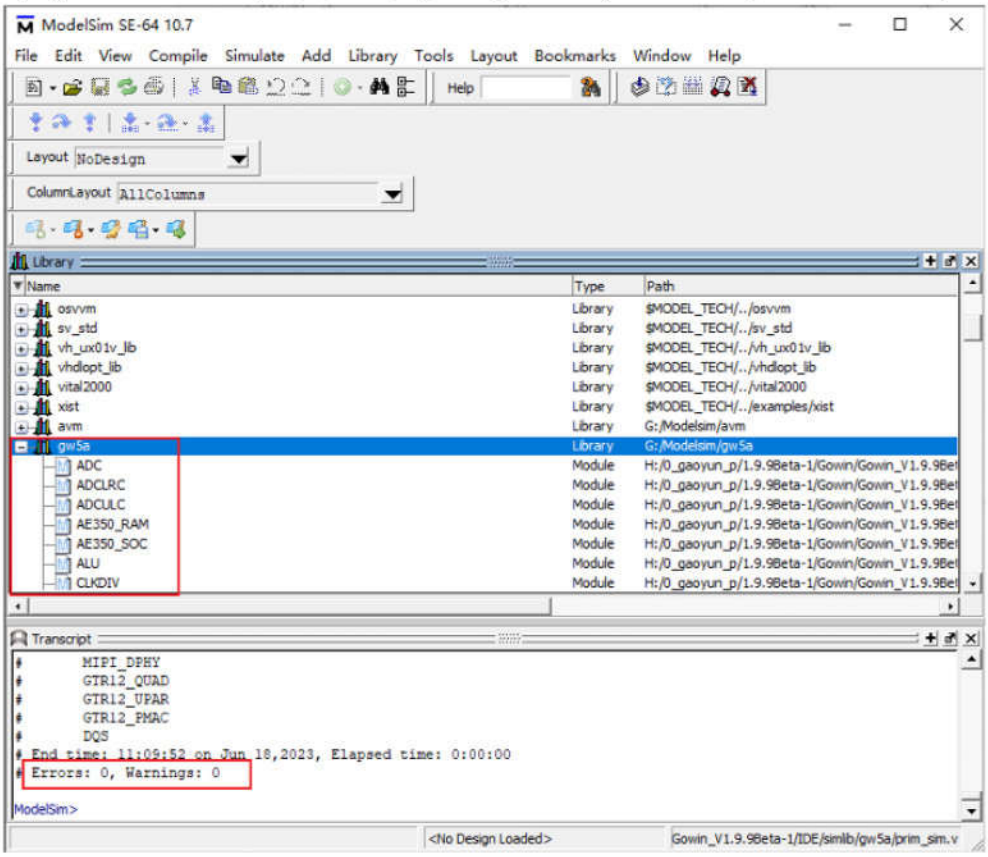
Hình 3-7 – Tìm tệp thư viện Gowin FPGA

c. "Tại đường dẫn đó, tìm tệp **prim_sim.v**, sau đó nhấp vào nút Compile để bắt đầu quá trình biên dịch, như minh họa trong Hình 3-8. Lúc này, cửa sổ Trnscript của ModelSim sẽ liên tục cập nhật các thông tin liên quan đến quá trình biên dịch. Khi biên dịch hoàn tất, nhấp vào nút Done để kết thúc quá trình."



Hình 3-8 – tìm tệp thư viện và tiến hành biên dịch

d. Sau khi biên dịch hoàn thành, giao diện Transcript sẽ hiển thị 0 lỗi, 0 cảnh báo và đồng thời bên dưới thư viện gw5a sẽ xuất hiện các thành phần (Module) khác nhau, như Hình 3-9,

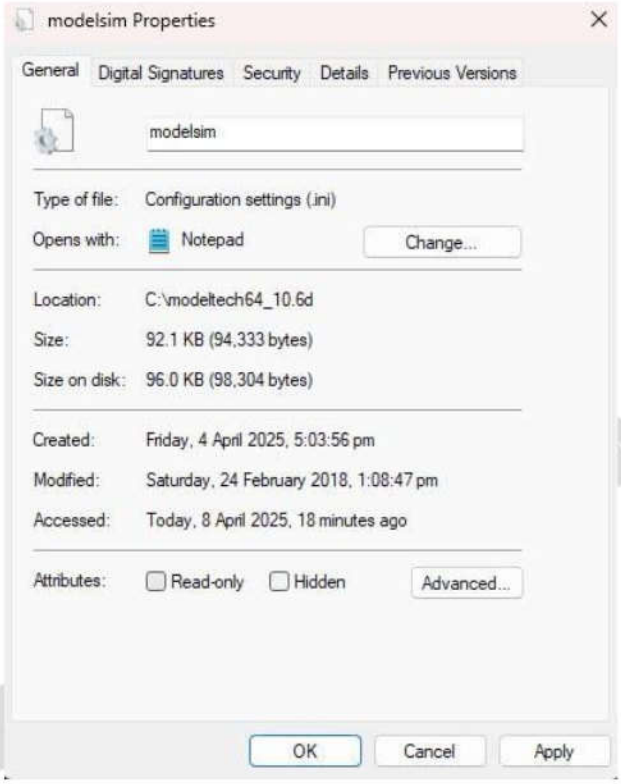


Hình 3-9 – Giao diện thư viện đã biên dịch

3.1.3 Thêm thư viện vào danh sách thư viện mặc định của ModelSim

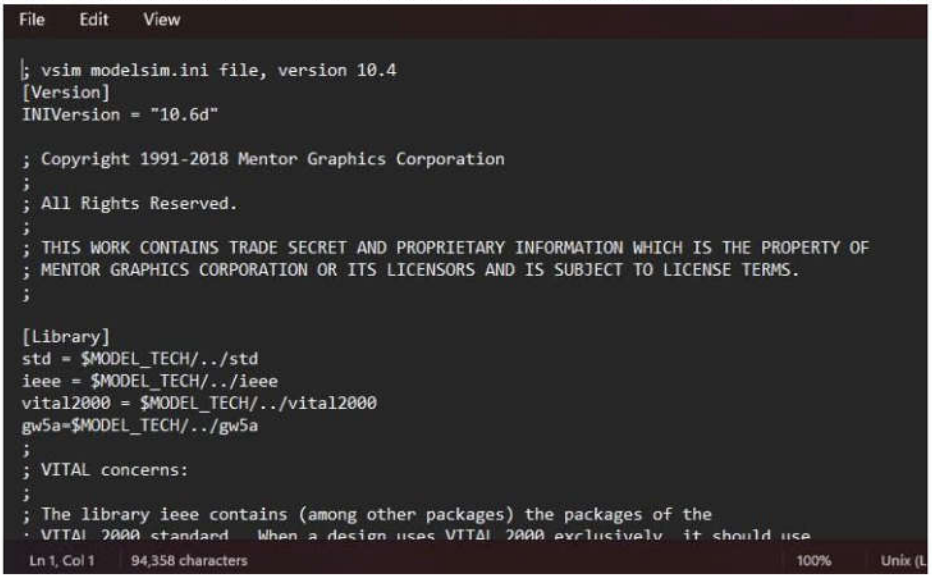
Để ModelSim có thể tự động nhận diện và sử dụng thư viện vừa biên dịch ở các bước sau, cần thêm thư viện đó vào danh sách thư viện mặc định của ModelSim. Cách thêm rất đơn giản, như sau:"

1. Tìm đến thư mục cài đặt ModelSim — ví dụ, trên máy của tác giả là ổ G:\Modelsim. Trong thư mục này có một tệp tên là modelsim.ini, đây là một tệp chỉ đọc. Nhấp chuột phải vào tệp modelsim.ini, chọn *Thuộc tính* (*Properties*), sau đó bỏ chọn thuộc tính 'Chỉ đọc' (*Read-only*) trong cửa sổ thuộc tính, như minh họa ở Hình 3-10



Hình 3-10 – Bỏ đặc tính read only của file modelsim.ini

2. Sau đó, mở tệp này bằng Notepad hoặc Notepad++ rồi, tại khoảng dòng thứ 17, thêm dòng lệnh khai báo thư viện gw5a: gw5a = $MODEL_TECH/../gw5a, rồi lưu lại, như minh họa dưới đây



Hình 3-11– Thêm gw5a vào thư viện trong file modelsim.ini

3. Khôi phục đặc tính chỉ đọc của file modelsim.ini:

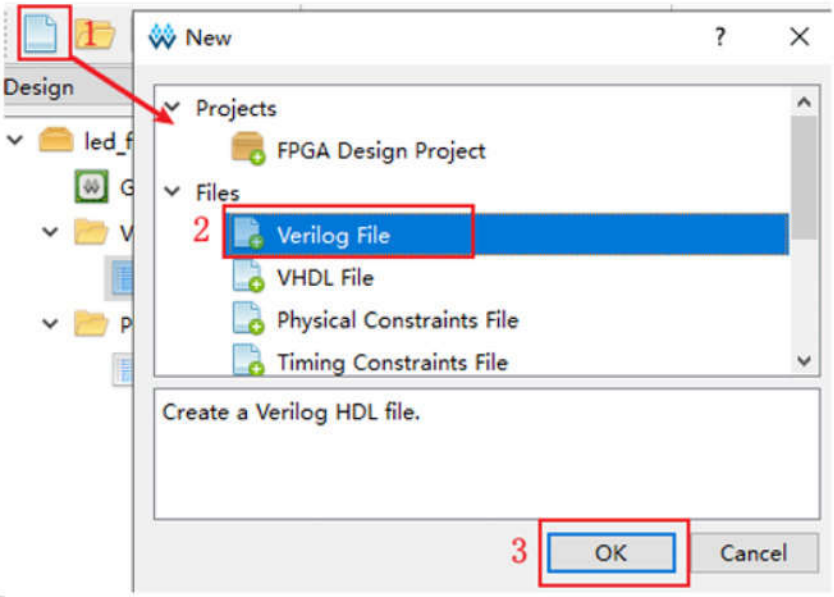
Đến đây, phần hướng dẫn về cách sử dụng ModelSim để biên dịch thư viện của dòng FPGA Gowin 5A đã kết thúc. Đối với các dòng thiết bị 1N và 2A, bạn cũng có thể thực hiện biên dịch theo phương pháp tương tự. Ở bước tiếp theo, chúng ta sẽ tiến hành tạo dự án mô phỏng trong ModelSim và thực hiện mô phỏng cho thiết kế.

3.2 Quy trình mô phỏng xác minh dùng Modelsim

Trong chương 2 "Quy trình phát triển cơ bản của phần mềm Gowin", chúng tôi giải thích quy trình phát triển cơ bản của Gowin FPGA và thắp sáng đèn Led để làm cho nó nhấp nháy, và phần này sẽ giải thích quy trình xác minh mô phỏng của Gowin Fpga và Modelsim trên cơ sở dự án **led_flash**.

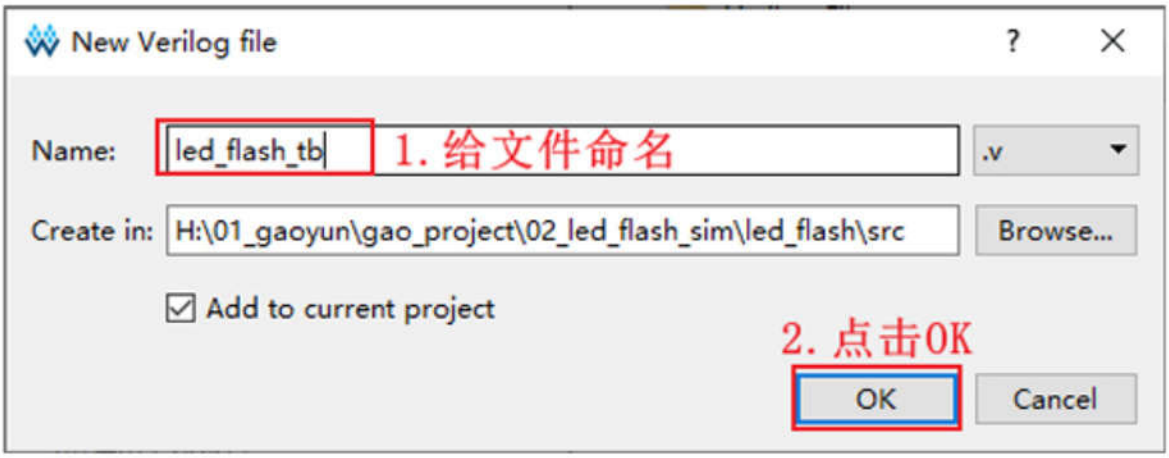
3.2.1 Viết mã cho thiết kế mô phỏng Verilog

1. Trước tiên, mở dự án, sau đó nhấp vào 'Tạo một tệp Verilog mới' hoặc sử dụng phím tắt Ctrl+N, như minh họa trong Hình 3-12



Hình 3-12 – Tạo File Verilog mới

2. Sau đó chọn New Verilog File, xuất hiện hộp thoại New Verilog File điện vào phần tên **led_flas_tb** (tb= test bench) và đường dẫn mặc định không thay đổi như Hình 3-13 bên dưới



Hình 3-13 – Đặt tên cho Testbench

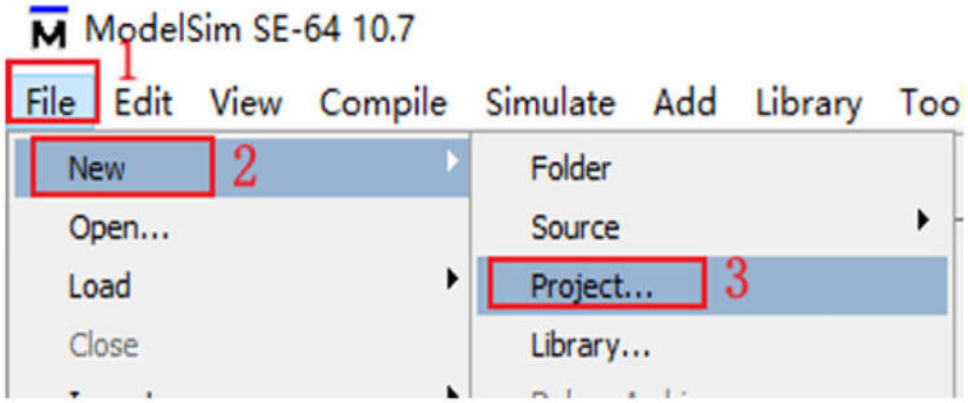
3. Module mô phỏng sẽ xuất hiện trong khung Design, chúng ta nhấp chuột vào để viết code.

```
`timescale 1ns/1ns
module led_flash_tb();

    reg Clk;
    reg Rst_n;
    wire Led;

    led_flash led_flash(
        .clk(Clk),
        .rst_n(Rst_n),
        .led(Led)
    );

    initial Clk = 1;
    always#10 Clk = ~Clk;

    initial begin
        Rst_n = 0;
        #201;
        Rst_n = 1;
        #200;
        #100000000;
    end

endmodule
```

4. Sau khi lưu tệp **led_flash_tb**, tiến hành phân tích và tổng hợp để kiểm tra xem có lỗi cú pháp nào không. Khi tệp mô phỏng đã được thiết kế hoàn chỉnh, chúng ta có thể sử dụng ModelSim để thực hiện kiểm tra chức năng của dự án thông qua mô phỏng."

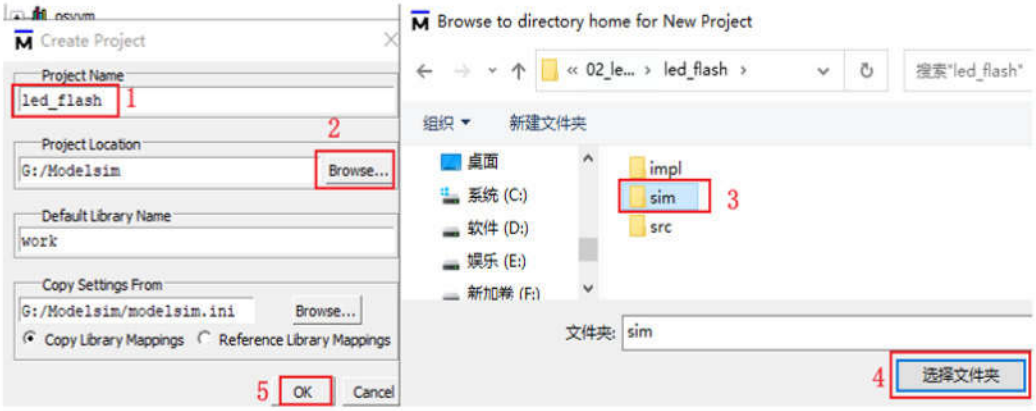
3.2.2 Xây dựng dự án Modelsim và thêm tệp giả lập

1. Mở phần mềm Modelsim, rồi tạo 1 project mới File>New->project như Hình 3-14 bên dưới:



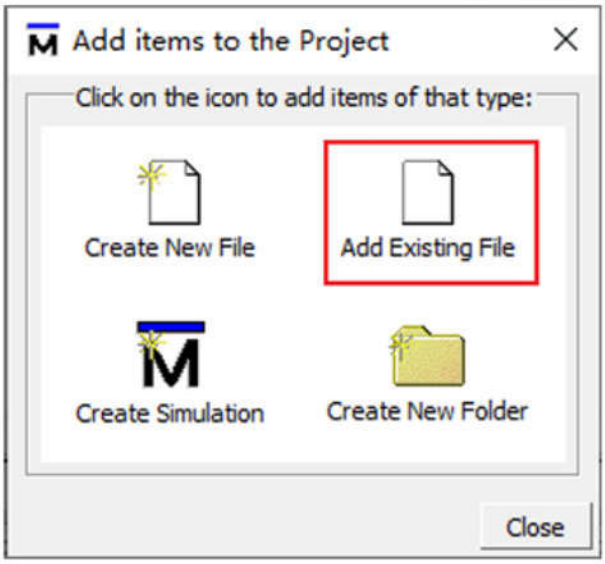
Hình 3-14 – Thiết lập project trong Modelsim

2. Phần mềm sẽ xuất hiện giao diện tạo project mới và tại phần "Project Name" ta đặt tên cho project "led_flash". Chọn đường dẫn đến thư mục trong các file chứa chương trình led_flash trong phần mềm Gowin.



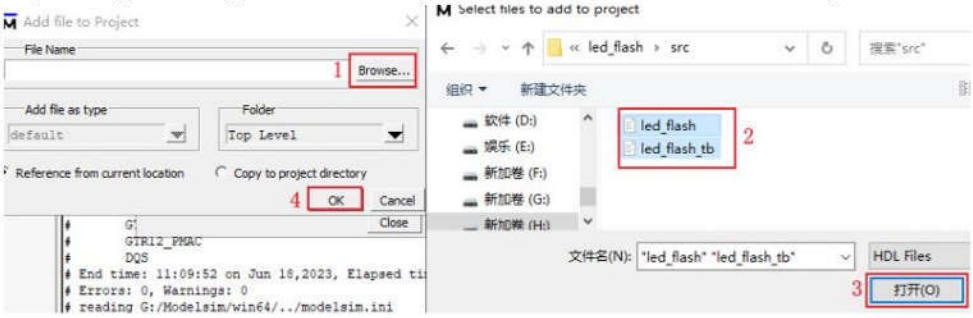
Hình 3-15 – Tạo prcject modelsim

3. Sau khi nhấn OK, giao diện như Hình 3-16 sẽ xuất hiện. Như có thể thấy trong Hình, có bốn tùy chọn thao tác: **'Create New File'** (Tạo tệp mới), **'Add Existing File'** (Thêm tệp có sẵn), 'Create Simulation' (Tạo mô phỏng) và **'Create New Folder'** (Tạo thư mục mới). Ở đây, chúng ta chọn **'Add Existing File'.**



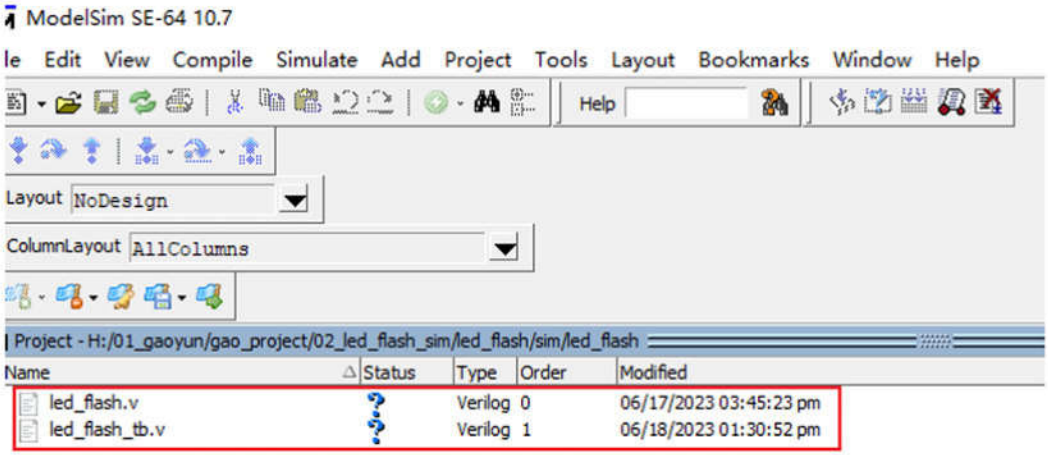
Hình 3-16 – Thêm file dự án

4. Tiếp theo, trong hộp thoại hiện ra, nhấn 'Browse' để chọn các tệp thiết kế cần thêm vào cho bài thực hành này, bao gồm 'led_flash.v' và 'led_flash_tb.v', như minh họa trong Hình 3-17



Hình 3-17– Chọn tệp để thêm tệp

Sau khi thêm xong, chúng ta có thể thấy các tệp thiết kế đã được thêm vào trong phần mềm ModelSim, như minh họa trong Hình 3-18



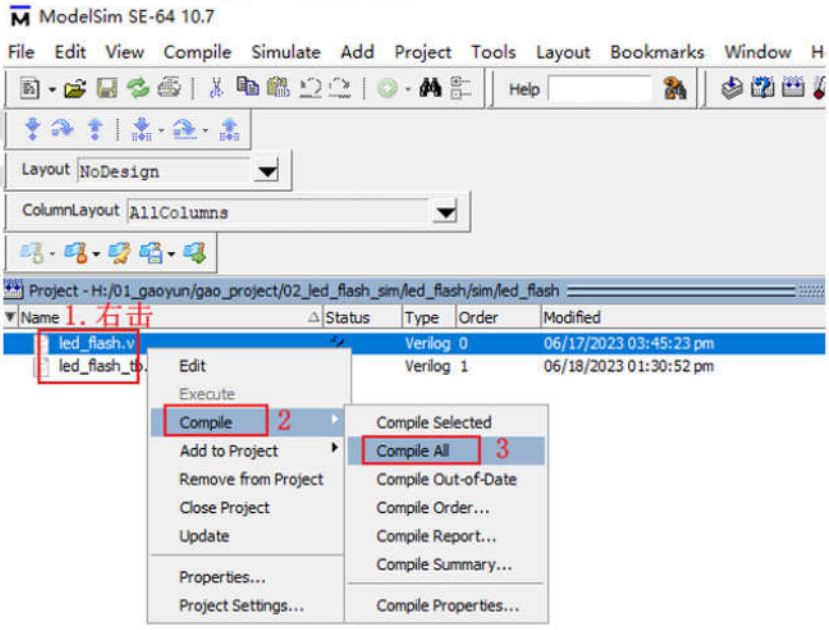
Hình 3-18 – Giao diện sau khi thêm tệp mô phỏng

3.2.3 Biên dịch tệp mô phỏng

Có hai cách biên dịch trong ModelSim:

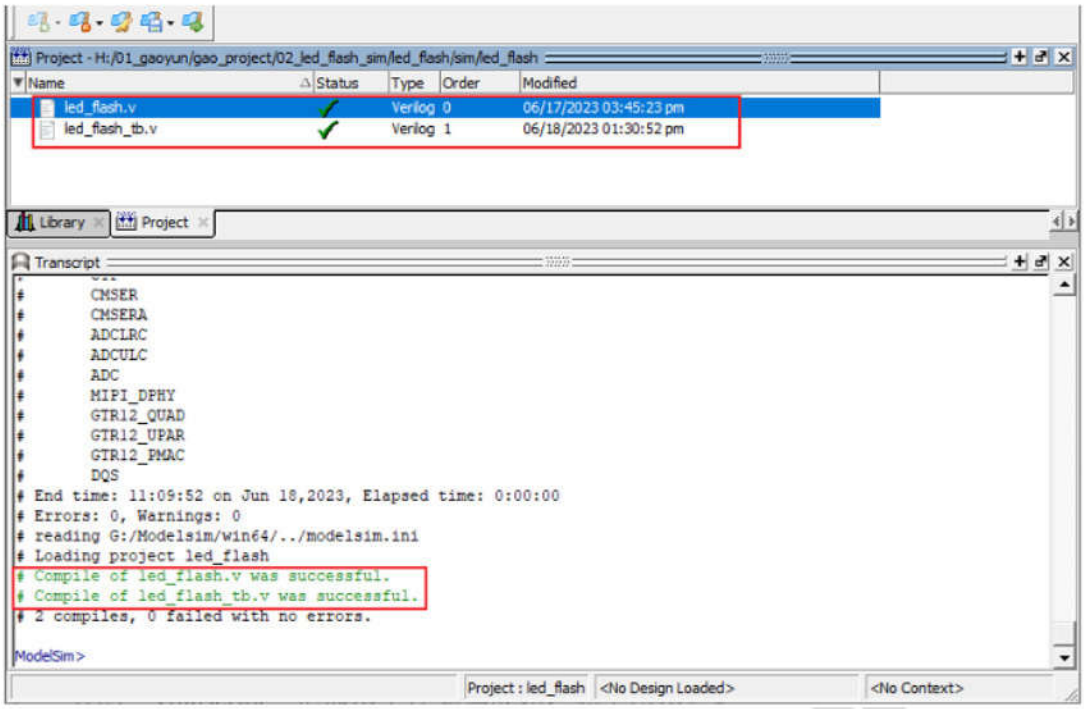
- **Compile Selected** (Biên dịch các tệp được chọn): cách này yêu cầu bạn phải chọn trước một hoặc nhiều tệp, sau đó thực hiện lệnh để biên dịch riêng các tệp đã chọn.
- **Compile All** (Biên dịch tất cả tệp): lệnh này sẽ biên dịch toàn bộ các tệp trong dự án theo đúng thứ tự biên dịch đã thiết lập.

Ở đây, chúng ta chọn cách biên dịch toàn bộ các tệp bằng cách vào menu Compile → Compile All, như minh họa trong Hình 3-19



Hình 3-19 – Biên dịch các tệp

Sau khi biên dịch hoàn tất, trạng thái phía sau tên tệp sẽ chuyển từ dấu '?' thành dấu '√', cho biết quá trình biên dịch đã thành công. Ngoài ra, cửa sổ 'Transcript' phía dưới cũng hiển thị thông báo xác nhận biên dịch thành công, như minh họa trong Hình 3-20



Hình 3-20 – Biên dịch các tệp

Cần lưu ý rằng trong hai thiết kế này, có hai trạng thái không mong muốn có thể xuất hiện:

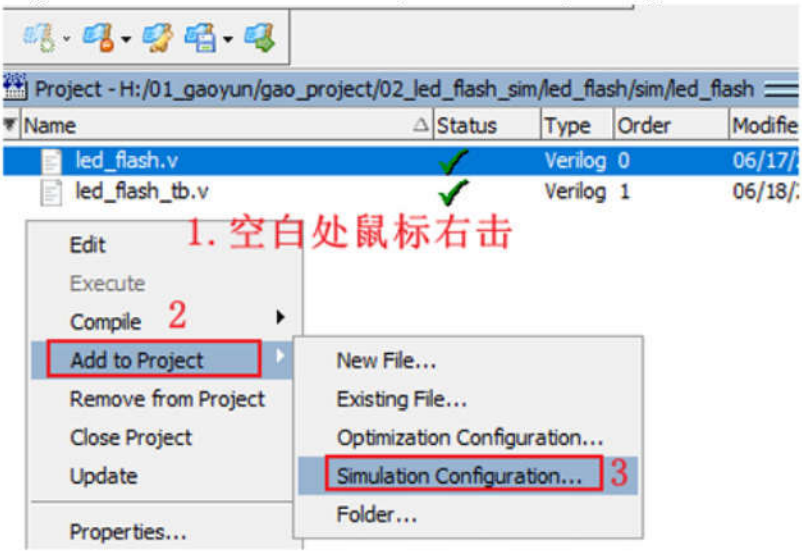
- **Lỗi biên dịch** (hiển thị dấu "×" màu đỏ),
- **Biên dịch thành công nhưng có cảnh báo** (dấu "√" kèm theo biểu tượng tam giác màu vàng).

Lỗi biên dịch có nghĩa là ModelSim không thể hoàn tất quá trình biên dịch tệp. Thông thường, lỗi này xảy ra do tệp đang biên dịch chứa lỗi cú pháp rõ ràng. ModelSim sẽ phát hiện các lỗi cú pháp này và hiển thị thông báo chi tiết, giúp người dùng xác định và chỉnh sửa lỗi dựa trên gợi ý từ ModelSim."

3.2.4 Cấu Hình môi trường mô phỏng

Sau khi biên dịch hoàn thành, chúng ta tiến hành cầu Hình môi trường mô phỏng

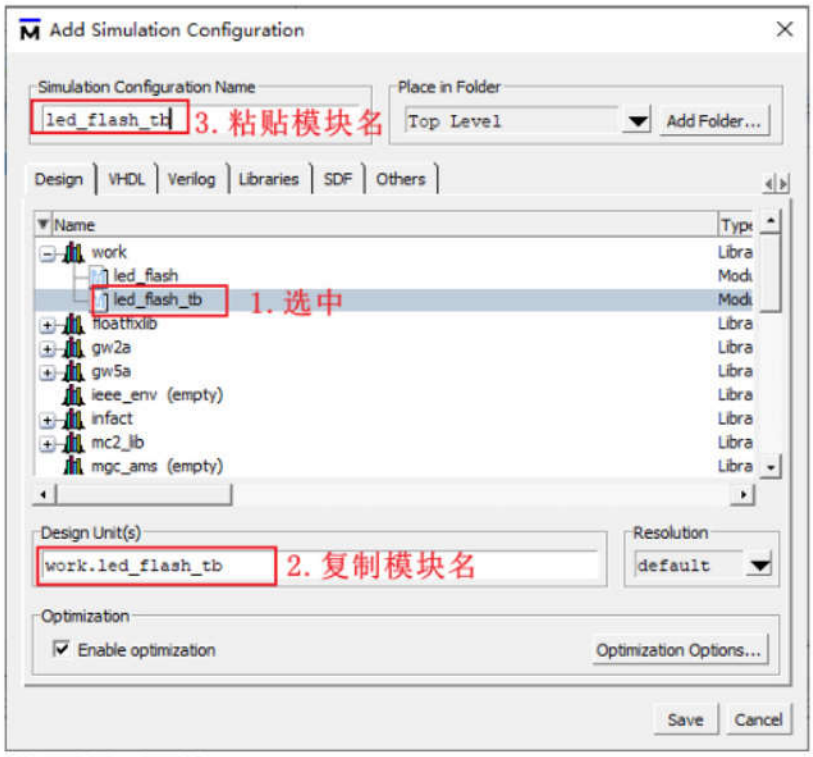
1. Trong thanh trạng thái của project, chúng ta nhấp chuột phải, sau đó chọn "**Add to Project**" → "**Simulation Configuration...**" và nhấn vào đó, như minh họa trong Hình 3-21.



Hình 3-21– Vào cấu Hình môi trường mô phỏng

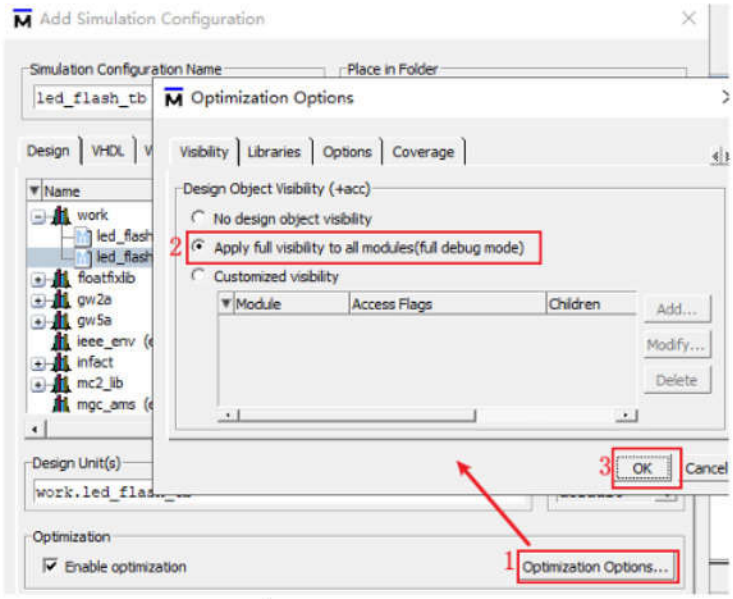
2. Trong cửa sổ "**Add Simulation Configuration**", chúng ta chuyển sang thẻ "**Design**", sau đó chọn module "**led_flash_tb**" trong thư viện gw5a làm tầng thiết kế trên cùng (top-level). Tiếp theo, nhấn nút để sao chép tên module này làm tên cho mục "**Simulation Configuration Name**", đảm bảo tên được đặt trùng khớp. Như minh họa trong Hình 3-22.

Trong các thiết kế phức tạp, chúng ta có thể tạo nhiều cấu Hình mô phỏng khác nhau, mỗi cấu Hình có một tầng thiết kế mô phỏng riêng để phục vụ việc kiểm thử toàn bộ hệ thống



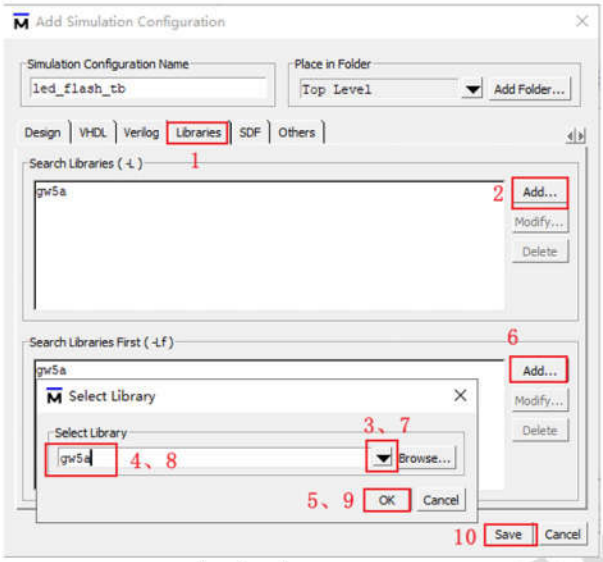
Hình 3-22 – Cấu Hình môi trường mô phỏng 1

3. Nhấn vào "**Optimization Options**...", trong bảng thiết lập "**Optimization Options**..." chọn mục "**Apply full visibility to all modules (full debug mode)**", sau đó nhấn **"OK"**, như minh họa trong Hình 3-23



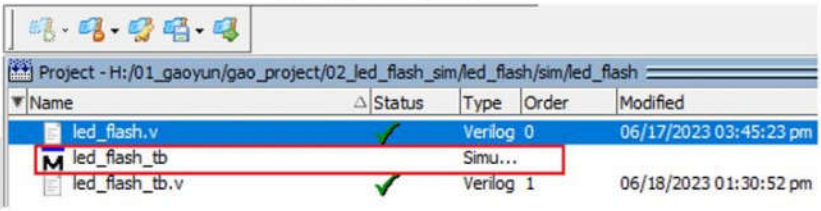
Hình 3-23 – Cấu Hình môi trường mô phỏng 2

4. Chuyển sang thẻ thiết lập "**Libraries**", tại mục "**Search libraries** (-L)", nhấn "**Add**..." để thêm thư viện Gowin mà chúng ta đã tạo trước đó, có tên là "gw5a".Tại mục "**Search Libraries First** (-Lf)", cũng chọn thư viện "gw5a". Cuối cùng nhấn "**Save**" để lưu lại các thiết lập, như minh họa trong Hình 3-24



Hình 3-24 – Cấu hình môi trường mô phỏng 3

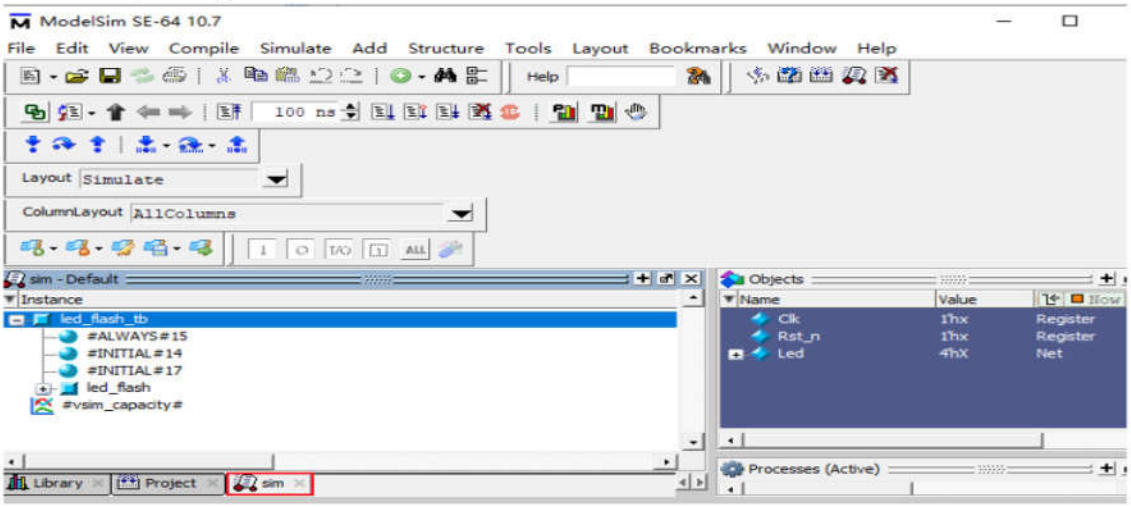
Sau khi hoàn tất các bước cấu Hình môi trường mô phỏng như trên và lưu lại cấu Hình, chúng ta có thể thấy trong thanh "Project" xuất hiện tệp cấu hình mô phỏng có tên "led_flash_tb", như minh họa trong Hình 3-25.



Hình 3-25

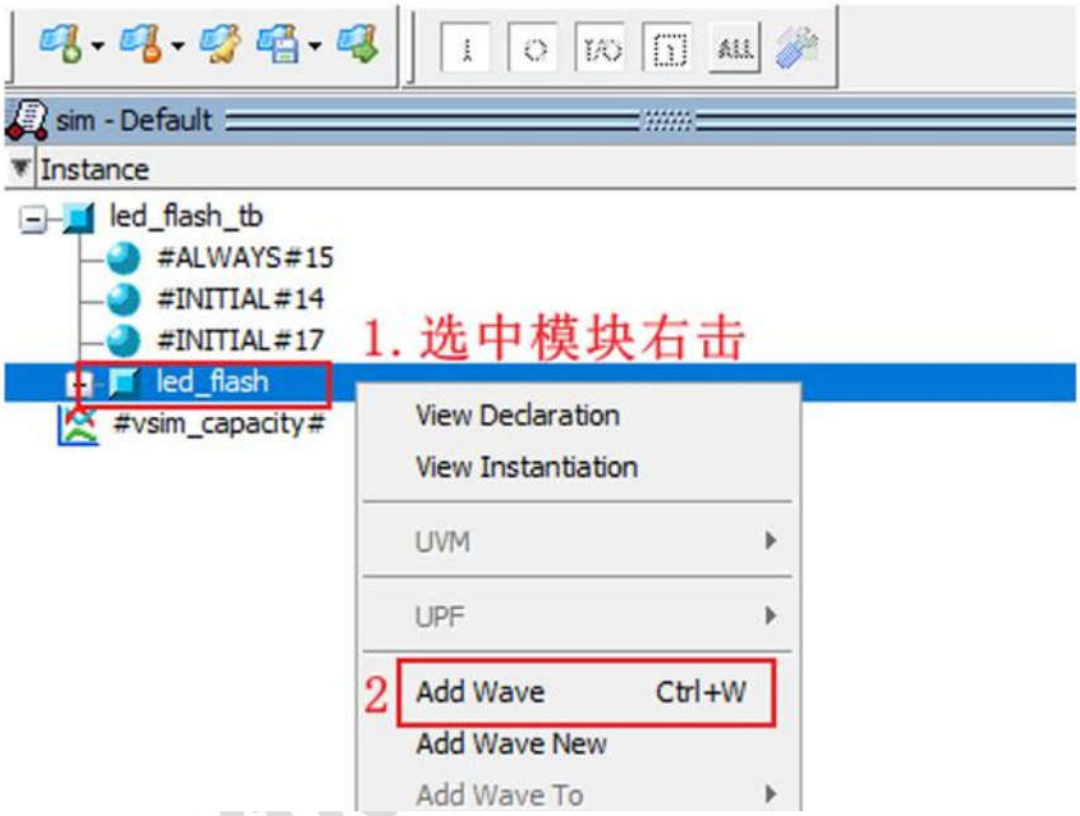
3.2.5 Quan sát dạng sóng mô phỏng

1. Trước tiên, chúng ta cần nhấp đúp vào tệp mô phỏng "led_flash_tb" vừa tạo, để vào giao diện "sim", như minh họa trong Hình 3-26.



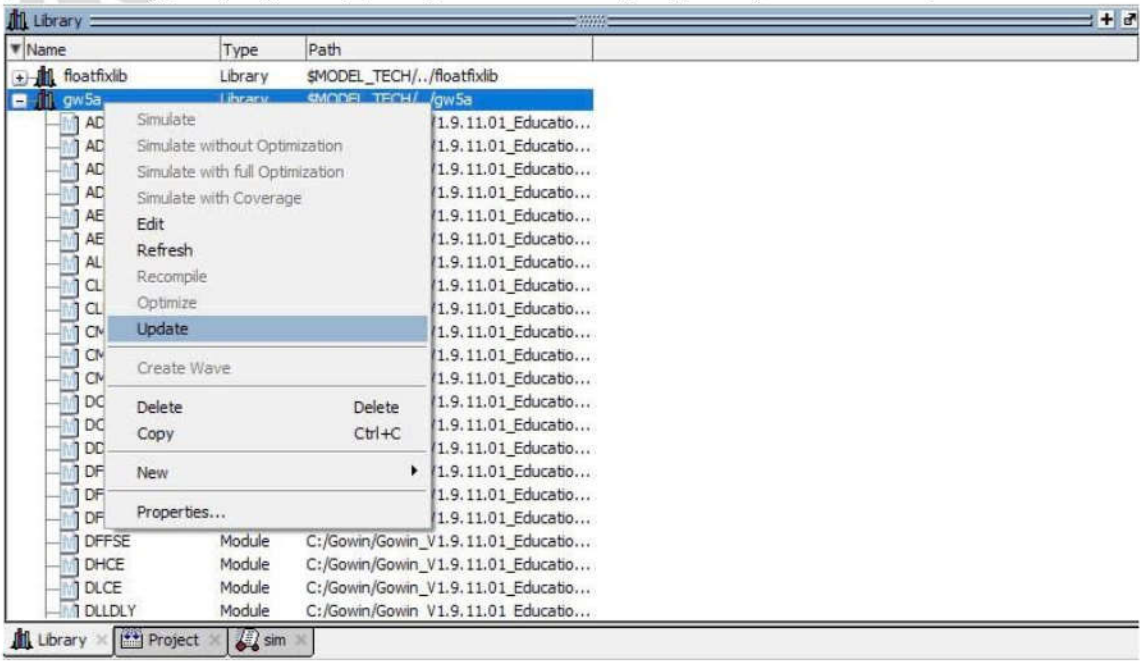
Hình 3-26 – Vào giao diện mô phỏng

2. Trong giao diện "sim", chúng ta có thể thêm các tín hiệu của module mà mình muốn quan sát dạng sóng. Chọn module cần xem, sau đó nhấp chuột phải và chọn "**Add Wave**", như minh họa trong Hình 3-27.



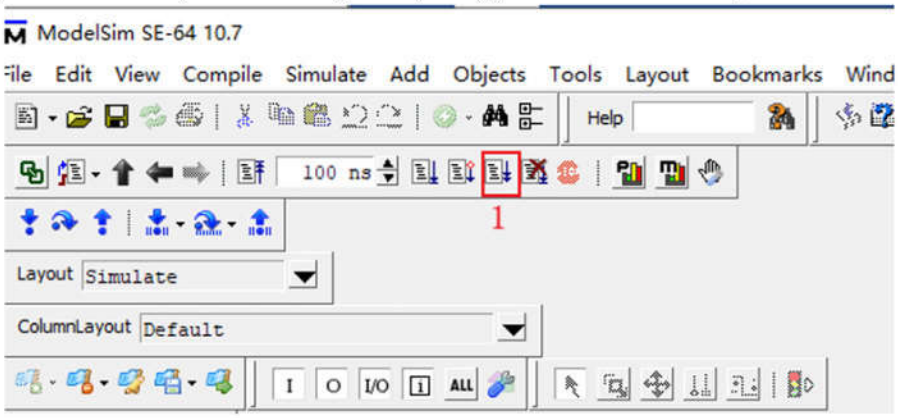
Hình 3-27 – Thêm tín hiệu quan sát

3. Sau khi thêm xong các tín hiệu sóng, quay lại thẻ "Library", nhấp chuột phải vào "gw5a" và chọn "Update" để cập nhật tệp "led_flash_tb" vào thư viện "gw5a", như minh họa bên dưới.



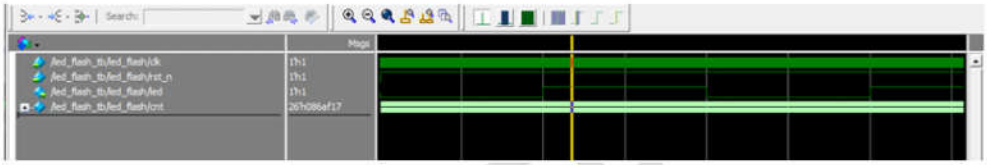
Hình 3-28 – Cập nhật tín hiệu quan sát

4. Nhấp chuột vào **Run all,** bắt đầu chạy mô phỏng, màn Hình hiển thị như Hình 3-29



Hình 3-29 – Khởi động mô phỏng

5. Sau đó sẽ tự động chuyển sang cửa sổ Wave. Đợi chương trình chạy trong chốc lát, chúng ta có thể quan sát được sự thay đổi của dạng sóng, như minh họa trong Hình 3-30. Từ Hình có thể thấy rằng, dựa vào giá trị đếm của cnt, trạng thái của led đã xảy ra thay đổi (đảo trạng thái).



Hình 3-30 – Quan sát dạng song

Nếu muốn dừng mô phỏng, hãy nhấn vào nút trên thanh công cụ phía trên cửa sổ **Wave**.

Để chạy lại từ đầu, nhấn vào nút .

Lúc này, hãy nhấn vào nút **"Zoom Full"** để nhanh chóng quan sát toàn bộ dạng sóng.

Nếu bạn muốn hiểu chức năng của từng nút trên thanh công cụ, hãy **di chuyển chuột đến nút đó**, sẽ có lời giải thích hiện ra.

Ví dụ như nút để **tiếp tục chạy mô phỏng** , như Hình 3-31 bên dưới minh họa.

Hình 3-31– Nút nhấn chức năng trên thanh công cụ

Trong quá trình phát triển, sau khi chỉnh sửa các tập tin thiết kế, hãy **lưu lại** và **kiểm tra cú pháp không có lỗi**, sau đó **chạy lại chương trình**.

Trong **Modelsim**, chúng ta vào giao diện **"Library"**, sau đó trong thư mục **"Work"**, hãy **biên dịch lại (Recompile)** các mô-đun **"led_flash_tb"** và **"led_flash"**, rồi **tải lại dạng sóng mô phỏng** để tiếp tục kiểm tra.

3.2.6 Lưu File dạng sóng

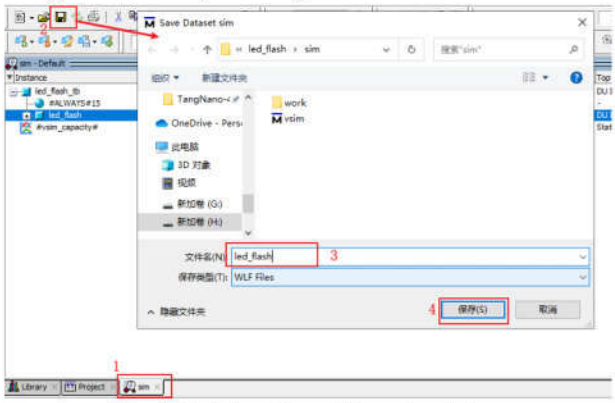
Khi sử dụng ModelSim để mô phỏng, đối với những phiên mô phỏng mất nhiều thời gian, bạn có thể lưu lại kết quả dạng sóng để lần sau có thể mở lại và xem trực tiếp, không cần mô phỏng lại từ đầu.

Bước đầu tiên: lưu tập tin dataset sim, mở cửa sổ sim, sau đó:

- Nhấn vào menu File → Save Dataset As...

- Hoặc trong giao diện sim, nhấn vào nút lưu file .wlf
- Sau đó đặt tên cho file, lưu lại để sử dụng về sau

Thao tác như minh họa trong hình dưới (nếu có).

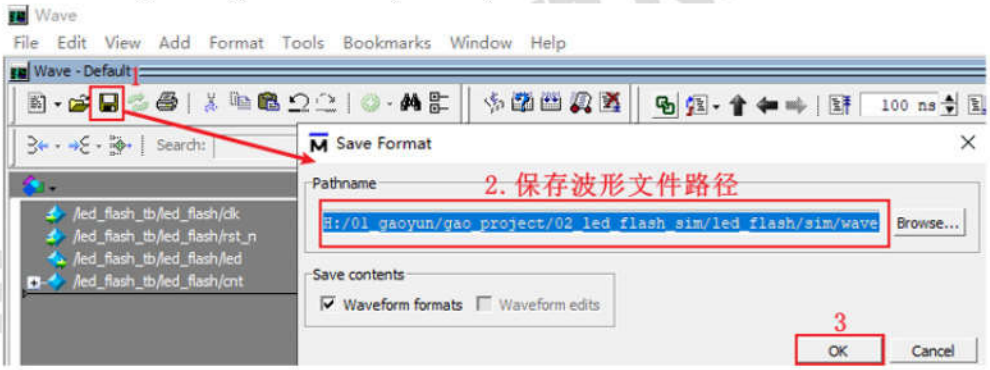


Hình 3-32 – Lưu file mô phỏng

Bước thứ hai: Lưu file dạng sóng trong giao diện Wave, trong cửa sổ Wave, bạn thực hiện thao tác lưu dạng sóng như sau:

- Nhấn vào biểu tượng lưu (Save icon) trên thanh công cụ.
- Sau khi nhấn, file dạng sóng có tên wave.do sẽ được lưu mặc định vào thư mục sim của dự án mô phỏng.

Thao tác như minh họa trong Hình 3-33 (nếu có).



Hình 3-33 – Lưu file dạng sóng

3.2.7 Mở lại dự án mô phỏng

Sau khi **đóng phiên mô phỏng**, nếu người dùng muốn **mở lại dự án mô phỏng, chỉ cần mở tệp .mpf**, như minh họa trong Hình 3-34.

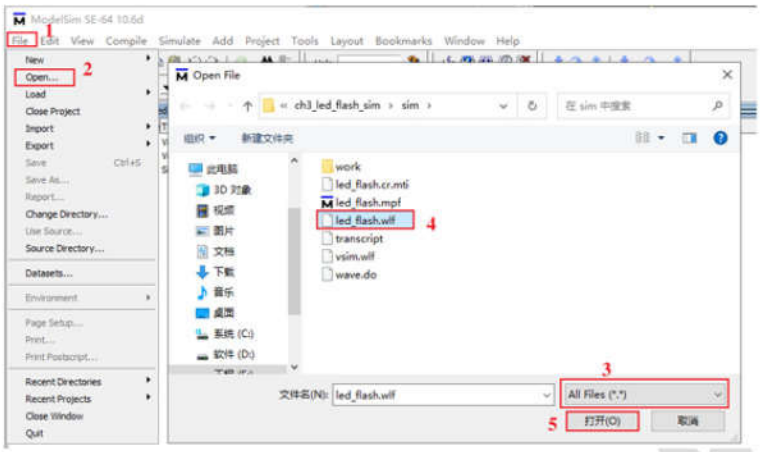
Hình 3-34 – Mở file .mpf

Lưu ý: Khi mở tệp .mpf lần đầu tiên, hệ thống **sẽ không tự động chọn Modelsim** làm phần mềm mở mặc định. Người dùng cần **thiết lập thủ công** để mở tệp này bằng Modelsim.

Với những máy đã **cài nhiều phiên bản Modelsim**, cần **chọn đúng phiên bản** tương ứng, ví dụ trong bài là: modelsim-win64-10.7-se.

Sau khi mở phần mềm Modelsim:

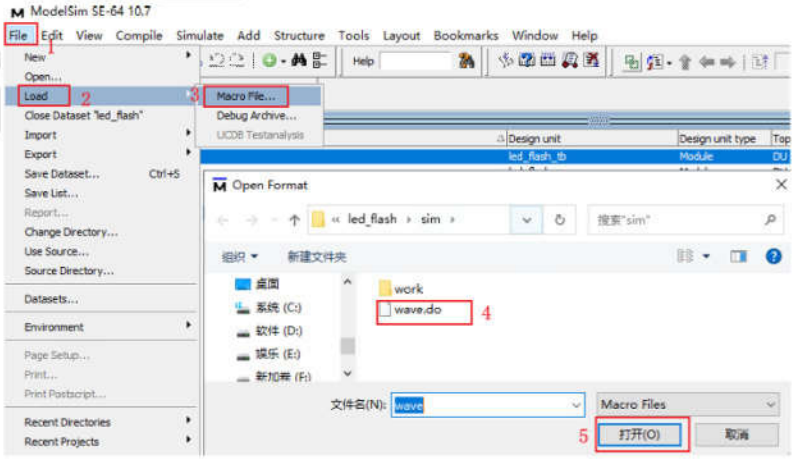
- Vào menu File -> All Files
- Chọn và mở **tệp sóng đã lưu trước đó**, định dạng .wlf,
- Thao tác này được minh họa trong Hình 3-35.



Hình 3-35 – Mở file .wlf

Tiếp theo, nhấp vào **File -> Load -> Macro File**, sau đó **mở tệp .do** mà bạn đã lưu trước đó, như minh họa trong **Hình 3-36**.

Sau khi mở xong, bạn sẽ thấy **tập tin dạng sóng (waveform)** mà bạn đã lưu **trước đó được hiển thị lại** trên cửa sổ sóng.



Hình 3-36 – Mở file .do

3.3 Tổng kết chương

Trong chương này, bài thực hành đã hướng dẫn mọi người cách sử dụng **Modelsim** để **biên dịch thư viện thiết bị của FPGA Gowin**, cũng như cách **thực hiện mô phỏng bằng Modelsim**.

Trong phần thực hành mô phỏng, **mã nguồn LED** được thiết kế để **thay đổi trạng thái sau mỗi 1 giây**, do đó **sóng mô phỏng sẽ cần chờ một lúc mới có thể quan sát được sự thay đổi**.

Nếu bạn muốn **quan sát sự thay đổi trên dạng sóng một cách nhanh hơn**, có thể **giảm giá trị đếm của bộ đếm cnt**, như vậy quá trình thay đổi trạng thái LED sẽ xảy ra nhanh hơn và dễ thấy hơn trên waveform.

Chương 4: Logic Analyzer trực tuyến của Gowin GAO

Mã nguồn bài thí nghiệm	|----Bài thí nghiệm số 2 |----ch4_led_flash_gao

Dẫn nhập chương

Trong chương này, dựa trên các thí nghiệm trước đó, chúng ta sẽ cùng tìm hiểu cách sử dụng Logic Analyzer trực tuyến của Gowin. Phần này chỉ giới thiệu quy trình sử dụng cơ bản của công cụ, còn các chức năng nâng cao hơn sẽ được trình bày khi cần thiết trong các thí nghiệm sau.

4.1 Giới thiệu về Logic Analyzer trực tuyến của Gowin

Logic Analyzer trực tuyến của công ty bán dẫn Gowin (Gowin Analyzer Oscilloscope, viết tắt là **GAO**) giúp người dùng phân tích dễ dàng hơn mối quan hệ thời gian giữa các tín hiệu trong thiết kế, từ đó nhanh chóng thực hiện phân tích hệ thống và định vị lỗi, nâng cao hiệu quả thiết kế.

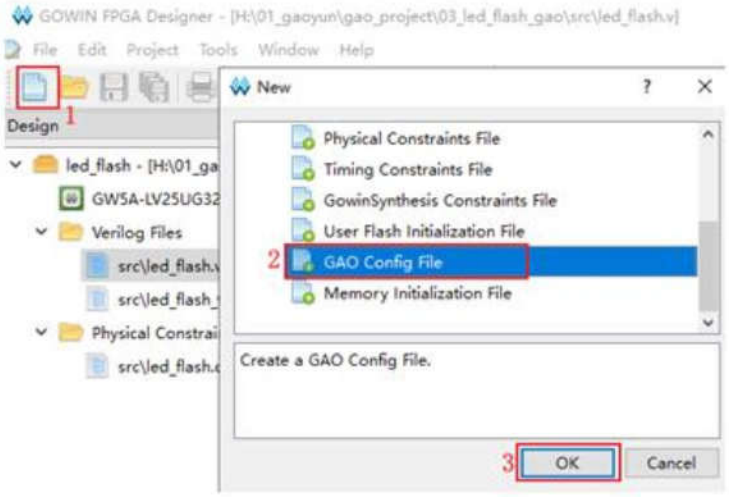
Nguyên lý hoạt động của GAO: Khi FPGA hoạt động, GAO sẽ tận dụng các tài nguyên bộ nhớ chưa sử dụng trong device. Theo điều kiện kích hoạt do người dùng device, các tín hiệu sẽ được lưu trữ theo thời gian thực vào bộ nhớ. Sau đó, thông qua giao diện JTAG, trạng thái của các tín hiệu sẽ được đọc và hiển thị trực tiếp trên phần mềm Gowin IDE. GAO bao gồm hai phần chính: **cửa sổ cấu hình tín hiệu** và **cửa sổ hiển thị dạng sóng**.

- **Cửa sổ cấu hình tín hiệu**: dùng để chèn thông tin định vị vào thiết kế. Các thông tin này dựa trên xung nhịp lấy mẫu, phần tử kích hoạt và biểu thức kích hoạt.
- **Cửa sổ hiển thị dạng sóng**: kết nối phần mềm Gowin với phần cứng thông qua JTAG để hiển thị trực quan dữ liệu của các tín hiệu được cấu hình.
- GAO hỗ trợ bắt tín hiệu ở mức **RTL** (thiết kế ở mức register) và **mức netlist sau khi tổng hợp**. GAO cung cấp hai phiên bản:
- **Standard GAO**: hỗ trợ tối đa 16 lõi chức năng, mỗi lõi có thể cấu hình một hoặc nhiều cổng kích hoạt, hỗ trợ biểu thức kích hoạt tĩnh hoặc động nhiều tầng.
- **Lite GAO**: cấu hình đơn giản, không cần thiết lập điều kiện kích hoạt, có thể bắt được giá trị khởi tạo của tín hiệu — rất tiện lợi để phân tích trạng thái hoạt động khi thiết bị vừa cấp nguồn.

4.2 Quy trình sử dụng GAO cơ bản

4.2.1 Tạo một File GAO mới

Mở phần mềm Gowin, kích chuột vào **New** , sẽ xuất hiện hộp thoại **New**, chọn **GAO Config File**, như Hình 4-1, nhấp **OK**



Hình 4-1– Tạo tệp GAO

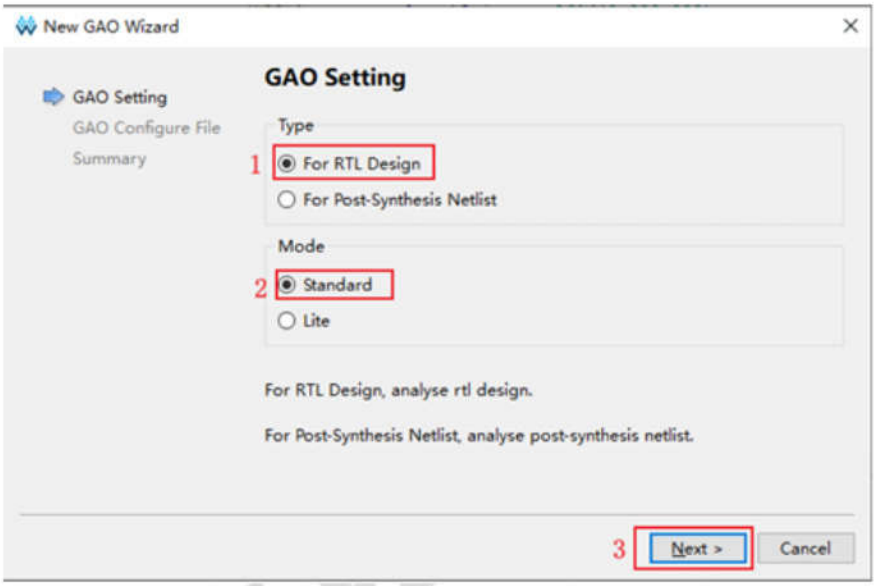
GOWIN VIETNAM

Sau khi chọn **New → GAO File**, một hộp thoại có tên **New GAO Wizard** sẽ xuất hiện.

Trong phần **Type**, có hai tùy chọn cấu Hình:

- **For RTL Design**: Dùng để bắt tín hiệu RTL (chưa qua tổng hợp), tệp cấu Hình sẽ có phần mở rộng là **.rao**.
- **For Post-Synthesis Netlist**: Dùng để bắt tín hiệu sau khi đã tổng hợp (netlist), tệp cấu Hình có phần mở rộng là **.gao**.

Tại đây, chúng ta chọn loại **"For RTL Design"**, và cấu Hình theo chế độ **Standard GAO**, như minh họa trong Hình 4-2.

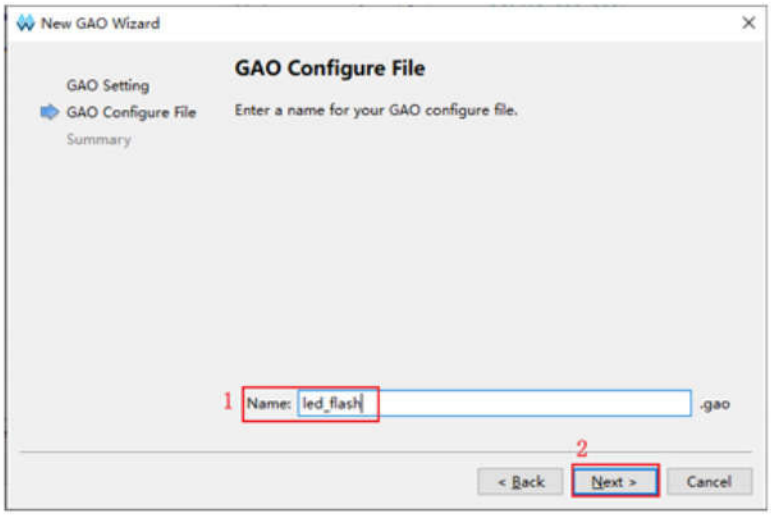


Hình 4-2 – Cấu Hình cho GAO

Sau đó, trong cửa sổ vừa hiển thị, tại ô nhập liệu **"Name"**, bạn hãy nhập tên cho tệp cấu Hình GAO.

Theo mặc định, tên tệp sẽ giống với tên của dự án hiện tại.

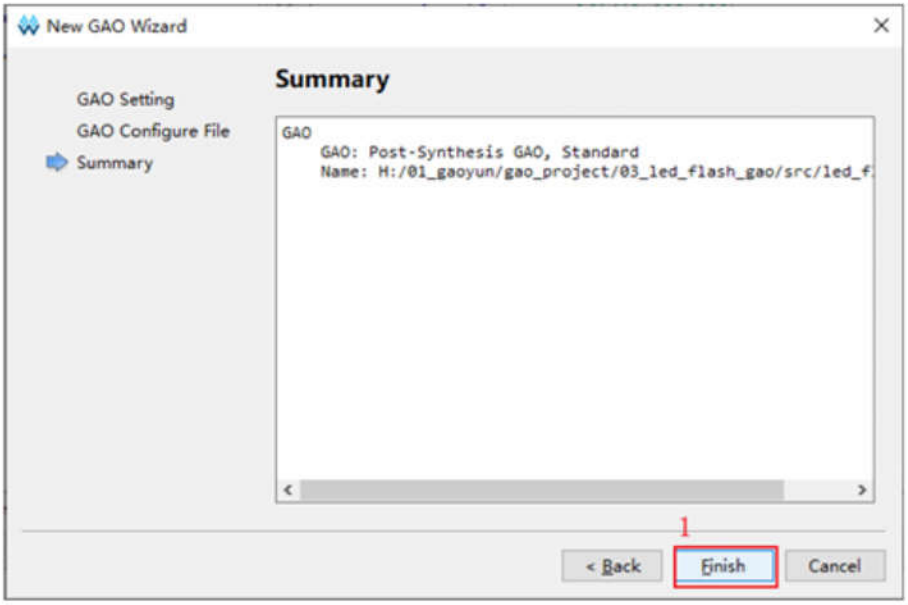
Ở đây, chúng ta giữ nguyên tên mặc định, sau đó nhấn nút **"Next"** để tiếp tục, như minh họa dưới đây.



Hình 4-3 – Đặt tên cho GAO

Cuối cùng, hộp thoại **Summary** sẽ xuất hiện, hiển thị thông tin tóm tắt về cấu Hình vừa tạo.

Nhấn **"Finish"** để hoàn tất việc tạo tệp GAO



Hình 4-4 – Hoàn tất cài đặt cho GAO

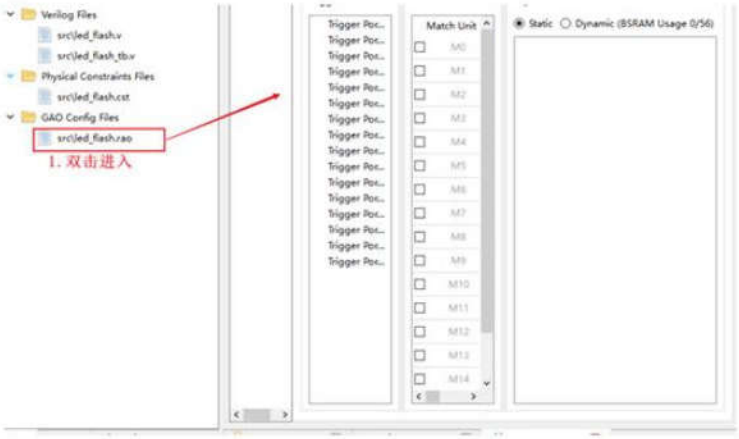
4.2.2 Mở cửa sổ cấu Hình GAO

Sau khi tạo xong tệp GAO, trong cửa sổ **Design** sẽ xuất hiện một mục mới có tên **GAO Config Files.**

Bên dưới mục này, bạn sẽ thấy tệp GAO mà mình vừa tạo. Hãy nhấp đúp vào tệp đó để mở cửa sổ cấu Hình GAO, như minh họa trong Hình 4-5.

Giao diện cấu Hình GAO bao gồm:

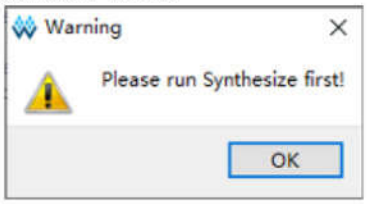
- Số lượng lõi phân tích **(AO Core)** mà bạn muốn cấu Hình
- **Khung cấu Hình tín hiệu tương ứng với từng lõi (Core).**
- Trong phần cấu Hình tín hiệu của mỗi lõi, có hai khu vực chính:
- **Trigger Options**: Cài đặt điều kiện kích hoạt tín hiệu
- **Capture Options**: Thiết lập điều kiện lấy mẫu tín hiệu
- *Giải nghĩa thêm về AO Core, mỗi AO Core là một "khối chức năng" bên trong FPGA được Gowin sử dụng để:*
- *Ghi lại tín hiệu được chỉ định*
- *Áp dụng điều kiện kích hoạt (trigger)*
- *Lưu dữ liệu dạng sóng vào bộ nhớ FPGA*



Hình 4-5 – Cửa sổ GAO Config Files và giao diện cấu Hình tín hiệu

Lưu ý: Nếu khi nhấp đúp vào tệp .gao, xuất hiện **cảnh báo như Hình 4-6**, điều đó có nghĩa là bạn **chưa thực hiện phân tích và tổng hợp (Synthesize)** cho thiết kế.

Trong trường hợp này, bạn cần thực hiện thao tác **phân tích và tổng hợp** trước, sau đó mới có thể nhấp đúp vào tệp .gao để mở cửa sổ cấu Hình GAO.



Hình 4-6 – Cảnh báo chưa tổng hợp code

4.2.3 Cấu Hình cho GAO ở Standard Mode

Dùng để thiết lập các thông số như:

- Số lượng lõi phân tích (AO Core)
- Điều kiện kích hoạt tín hiệu (Trigger)
- Điều kiện lấy mẫu tín hiệu (Sampling)

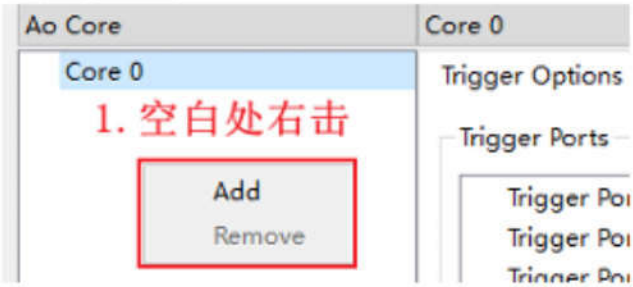
1. Cấu Hình số lượng lõi phân tích (AO Core)

Khung **AO Core View** được sử dụng để hiển thị và quản lý số lượng lõi phân tích hiện đang dùng trong dự án.

- Theo mặc định, hệ thống sẽ khởi tạo **Core 0.**
- GAO hỗ trợ tối đa **16 Core** trong một thiết kế

Bạn có thể nhấn **"Add"** để thêm lõi hoặc **"Remove"** để xoá lõi tại bất kỳ vị trí nào trong khung **AO Core View**, như minh họa trong Hình 4-7.

*Trong ví dụ này, chúng ta sử dụng **mặc định là Core 0**



Hình 4-7 – Cấu Hình số lượng AO Core

2. Cấu Hình cổng kích hoạt (Trigger Ports)

Trong khung **Trigger Ports** của **Core 0**, bạn có thể cấu Hình các cổng kích hoạt cho lõi phân tích.

- Tổng cộng có **16 cổng kích hoạt** (Trigger Ports) khả dụng
- Mỗi cổng kích hoạt có thể được cấu Hình với **độ rộng từ 1 đến 64 bit**

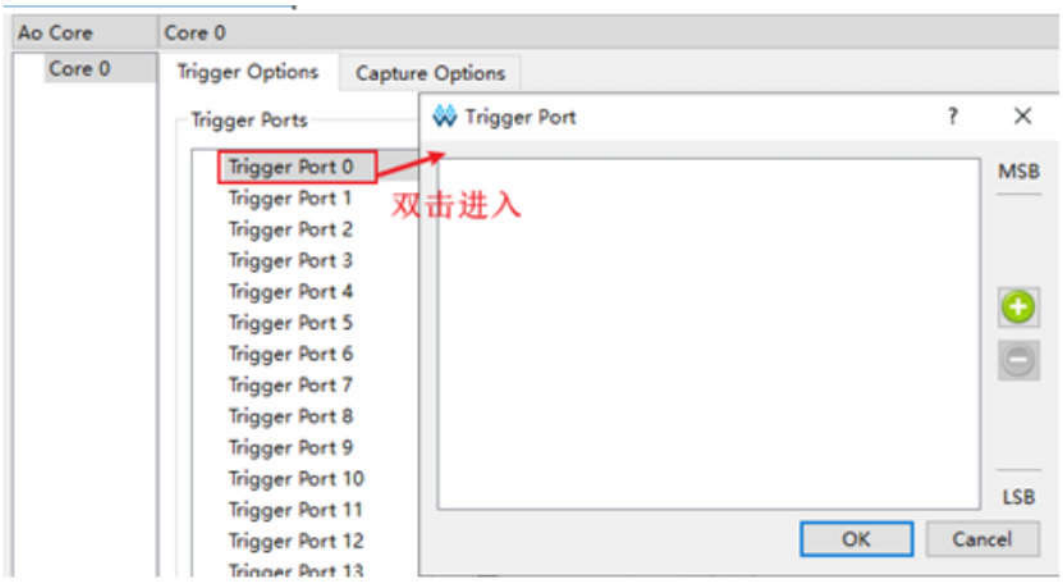
Trong ví dụ này, chúng ta sử dụng **cổng số 0**.

Khi **nhấp đúp vào Trigger Port 0**, cửa sổ cấu Hình cổng sẽ mở ra – có tên là **Trigger Port**, như minh họa trong Hình 4-8.

Trong giao diện đó:

- **MSB** biểu thị bit có trọng số cao nhất (Most Significant Bit)
- **LSB** biểu thị bit có trọng số thấp nhất (Least Significant Bit)

GOWIN VIETNAM

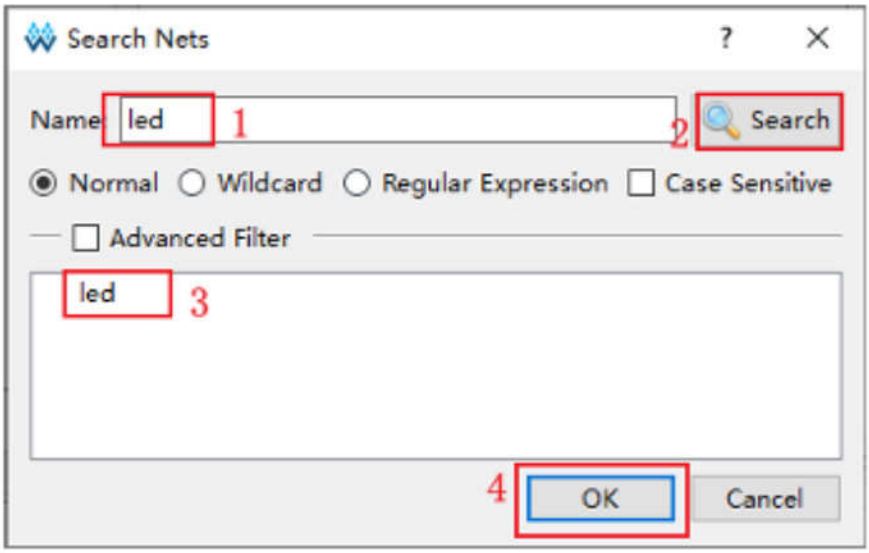


Hình 4-8 – Giao diện cấu Hình Trigger Port của Core 0

Tiếp theo, nhấn vào nút **"+" (Add)** để mở hộp thoại **Search Nets**.Tại ô **Name**, nhập tên tín hiệu mà bạn muốn thêm làm điều kiện kích hoạt.

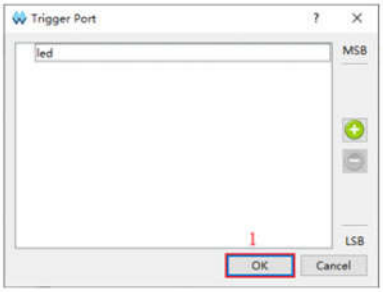
Ví dụ trong bài thực hành này, chúng ta muốn quan sát tín hiệu trạng thái của đèn LED, nên sẽ nhập **led** vào ô tìm kiếm.

Sau đó nhấn nút **Search** để tìm kiếm tín hiệu, như minh họa trong Hình 4-9.



Hình 4-9 – Tìm kiếm và thêm tín hiệu LED trong Search Nets

Sau đó trở về Trigger Port nhấp OK, như Hình dưới đây



Hình 4-10 – Thêm tín hiệu kích hoạt

3. Cấu Hình cho Match Unit

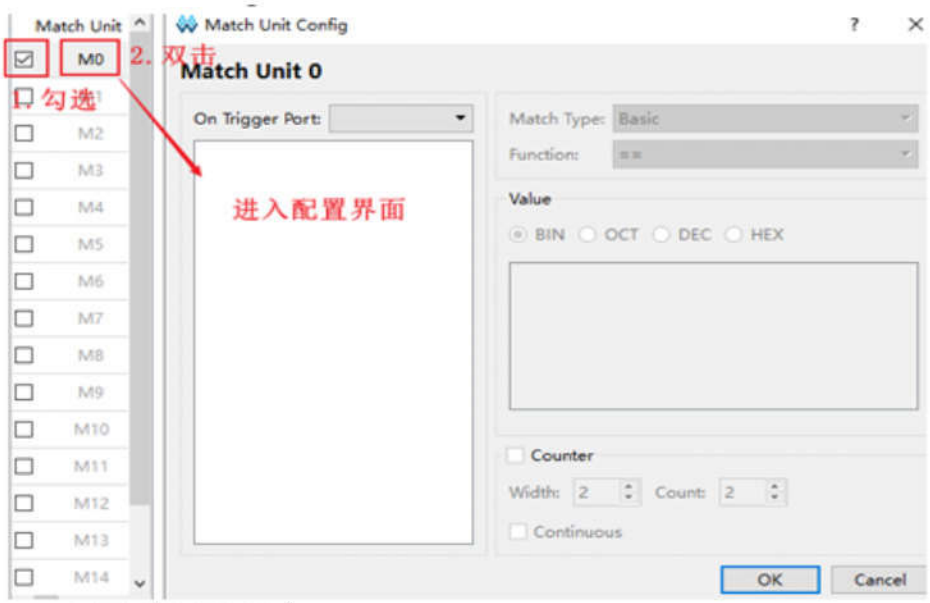
Khung **Match Units** được sử dụng để cấu Hình các **Match Unit** của cổng kích hoạt. GAO hỗ trợ tối đa **16 Match Unit**, tương ứng với các đơn vị **M0 ~ M15**.

Mỗi **Match Unit** là đơn vị nhỏ nhất trong lõi phân tích GAO dùng để xác định điều kiện kích hoạt.

Khi tín hiệu trên cổng kích hoạt đáp ứng điều kiện đã cấu Hình trong Match Unit, **quá trình kích hoạt sẽ được thực hiện**.

Trong ví dụ này, chúng ta chọn **Match Unit M0**.

Nhấp đúp vào M0 để mở hộp thoại cấu Hình **Match Unit Config**, nơi bạn có thể thiết lập điều kiện kích hoạt, như minh họa trong Hình 4-11.



Hình 4-11– Cấu Hình điều kiện kích hoạt trong Match Unit M0

Trong hộp thoại **"Match Unit Config"** vừa mở, hãy nhấp vào **trường thả xuống "On Trigger Port"**, sau đó chọn **Trigger Port 0** từ danh sách.

Ngoài ra, hộp thoại còn bao gồm các mục cấu Hình sau: **Match Type (Loại so khớp), Function (Hàm so sánh), Value (Giá trị bit cần thiết lập)**

Ý nghĩa của từng mục được trình bày cụ thể trong **Bảng 4-1** dưới đây.

Chú thích: Trong phần *Bit Values*:

Type	Bit Value	Match Functions	Descriptions
Basic	0, 1, X	==, !=	Dùng cho so sánh tín hiệu thông thường, là loại so sánh đơn giản nhất.
Basic w/edges)	0, 1, X, R, F, B, N	==, !=, kiểm tra biên độ (transition)	Dùng trong các trường hợp cần xét đến sự thay đổi của tín hiệu điều khiển.
Extended	0, 1, X	==, !=, >, >=, <, <=	Dùng khi cần xét giá trị địa chỉ hoặc dữ liệu tín hiệu.

Extended w/edges	0, 1, X, R, F, B, N	==, !=, >, >=, <, <=, kiểm tra biên độ	Dùng khi xét cả giá trị và sự thay đổi của tín hiệu địa chỉ hoặc dữ liệu.
Range	0, 1, X	>, >=, <, <=, kiểm tra trong/ngoài phạm vi	Dùng khi cần xét tín hiệu địa chỉ hoặc dữ liệu trong phạm vi nhất định.
Range w/edges	0, 1, X, R, F, B, N	>, >=, <, <=, kiểm tra phạm vi, kiểm tra biên độ	Dùng khi cần xét cả giá trị và sự thay đổi của tín hiệu trong một phạm vi cụ thể.

- "0" biểu thị mức logic thấp (0)
- "1" biểu thị mức logic cao (1)
- "X" biểu thị bất kỳ mức logic nào đều được
- "R" biểu thị cạnh lên (chuyển từ 0 → 1)
- "F" biểu thị cạnh xuống (chuyển từ 1 → 0)
- "B" biểu thị cả cạnh lên và cạnh xuống đều được
- "N" biểu thị không có sự chuyển đổi mức logic nào

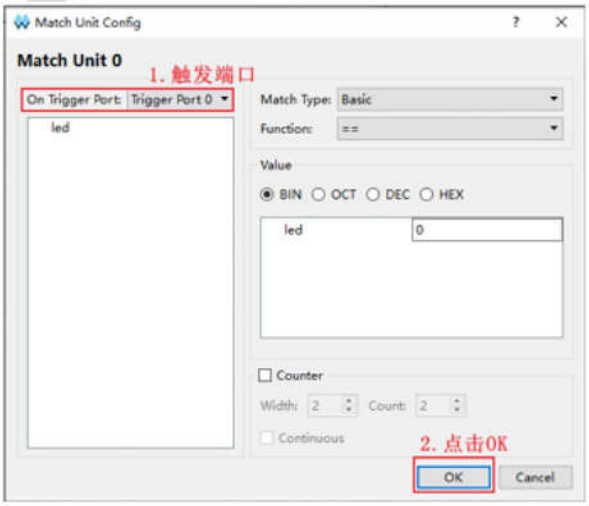
Bảng 4-1 – Giải thích các trường cấu Hình trong Match Unit

Mỗi **đơn vị so khớp (Match Unit)** đều có một bộ đếm (counter), dùng để thiết lập số lần điều kiện kích hoạt cần được thỏa mãn **trước khi bắt đầu thu thập dữ liệu**.

Giá trị **N** chính là số lần được đặt trong bộ đếm.

Việc sử dụng bộ đếm này phụ thuộc vào việc **ô chọn "Counter"** có được đánh dấu hay không:

- Nếu **ô "Counter" được chọn**, bạn có thể đặt số lần kích hoạt mong muốn.
- Nếu **không chọn "Counter"**, mặc định sẽ bắt đầu thu thập dữ liệu **sau khi điều kiện kích hoạt xảy ra đúng 1 lần**.



Hình 4-12 – Cấu Hình Match Unit Config

4. Cấu Hình biểu thức kích hoạt

Khung **Expressions** được dùng để thiết lập **biểu thức kích hoạt**.

- Mỗi **AO Core** hỗ trợ tối đa **16 biểu thức kích hoạt**.
- Mỗi biểu thức có thể kết hợp nhiều **Match Unit** bằng các toán tử logic như &, |, !, v.v.

Để thêm biểu thức mới, **nhấp chuột phải vào bất kỳ vị trí nào trong khung Expressions**, chọn **"Add"**.

GOWIN VIETNAM

Lúc này, một hộp thoại có tên **Expression** sẽ xuất hiện để bạn nhập biểu thức logic, như minh họa trong Hình 4-13.

Hình 4-13 – Hộp thoại thêm Trigger Expression

Trong ví dụ này, chúng ta chỉ sử dụng **một Match Unit là M0**.

Vì vậy, trong hộp thoại **Expression**, chỉ cần nhấp vào **M0**, sau đó nhấn **OK** để xác nhận, như minh họa trong Hình 4-14.

Hình 4-14 – Chọn M0 làm biểu thức kích hoạt

Sau khi thêm xong biểu thức, giao diện sẽ hiển thị như minh họa trong **Hình 4-15**.

Người dùng có thể **nhấp đúp vào biểu thức** nếu muốn chỉnh sửa nội dung sau này.

Hình 4-15 – Biểu thức kích hoạt sau khi thêm vào

5. Cấu Hình tín hiệu lấy mẫu (Capture Signals)

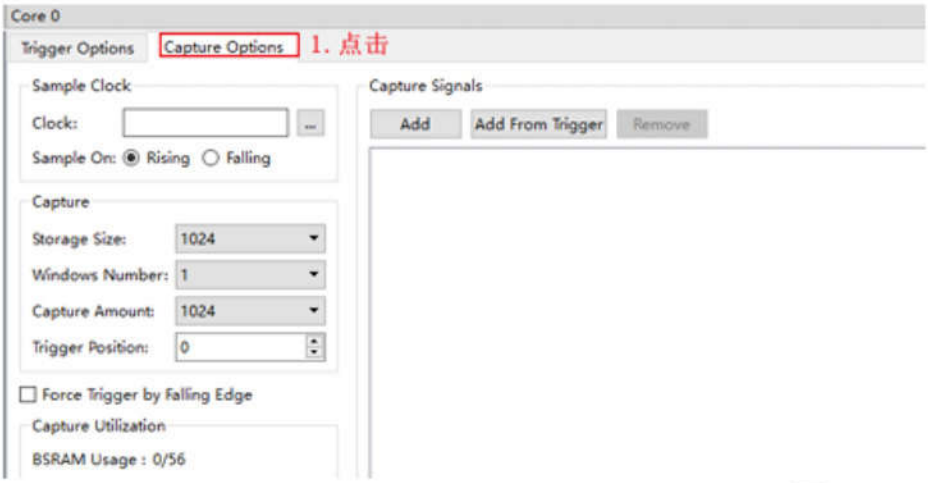
Nhấp vào mục **"Capture Options"** để vào **giao diện cấu Hình tín hiệu lấy mẫu**.

Giao diện **Capture Options** dùng để thiết lập các thông tin quan trọng cho quá trình lấy mẫu, bao gồm:

- Xung nhịp lấy mẫu (Sampling Clock)

- Độ sâu bộ nhớ (Memory Depth)
- Tín hiệu dữ liệu cần thu thập (Capture Signals)
- Thông tin về phương thức lấy mẫu của từng tín hiệu

Ngoài ra, giao diện này cũng sẽ hiển thị **lượng tài nguyên BSRAM** đang được sử dụng bởi các Capture Signals trong AO Core hiện tại.



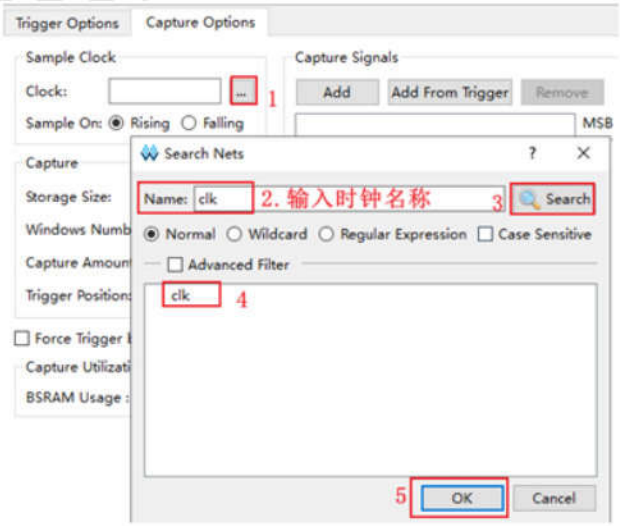
Hình 4-16 – Capture Options – Cấu Hình tín hiệu lấy mẫu

Trước tiên, chúng ta cần cấu Hình **xung nhịp lấy mẫu (Sampling Clock)**.

Nhấn vào nút **"Clock"**, sau đó hệ thống sẽ mở cửa sổ **Search Nets**.

Tại ô nhập liệu trong hộp thoại này, gõ trực tiếp tên của tín hiệu đồng hồ, ví dụ **clk**, rồi nhấn **Search** để tìm kiếm.

Khi tín hiệu được tìm thấy, hãy chọn và nhấn **Add** để thêm vào cấu Hình, như minh họa trong **Hình 4-17**.



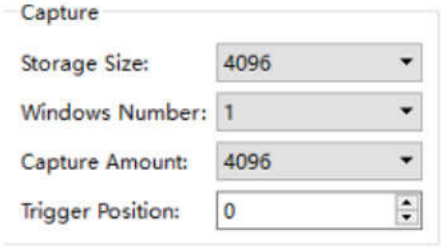
Hình 4-17– Thêm tín hiệu xung nhịp clk

Tiếp theo, thiết lập các tham số liên quan đến quá trình lấy mẫu:

- **Storage Size (Độ sâu bộ nhớ)**: 4096
- **Số lượng cửa sổ ghi mẫu (Capture Window Count)**: 1
- **Capture Amount (Chiều dài dữ liệu cần lấy mẫu)**: 4096
- **Trigger Position (Vị trí điểm kích hoạt)**: 0

Như minh họa trong **Hình 4-18**

GOWIN VIETNAM



Hình 4-18 – Dung lượng lưu trữ và chiều dài dữ liệu lấy mẫu

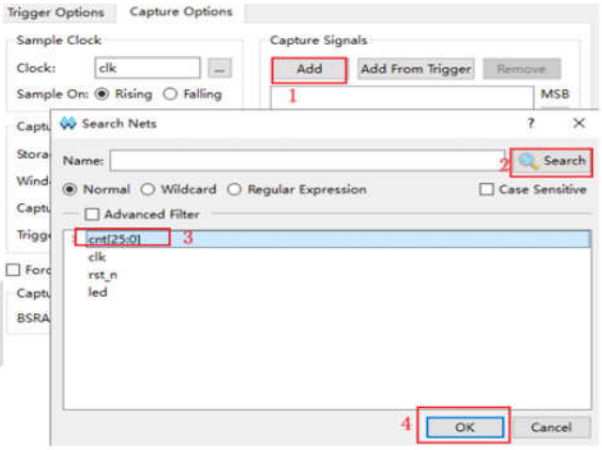
Chọn các **tín hiệu dữ liệu cần thu thập** trong quá trình lấy mẫu.

Nhấn vào nút **"Add"**, sau đó chọn các tín hiệu mà bạn muốn **lõi phân tích (AO Core)** thu thập và lưu trữ dưới dạng sóng.

Lúc này, hộp thoại **Search Nets** sẽ hiện ra. Tại đây, bạn có thể tìm kiếm và chọn các tín hiệu mong muốn, sau đó nhấn **"OK"** để hoàn tất quá trình thêm tín hiệu.

📌 Ngoài các tín hiệu đơn, bạn cũng có thể thêm **tín hiệu dạng bus (bus signal)**.

Trong ví dụ này, chúng ta thêm tín hiệu bus **cnt[25:0]** để quan sát, như minh họa trong **Hình 4-19**.



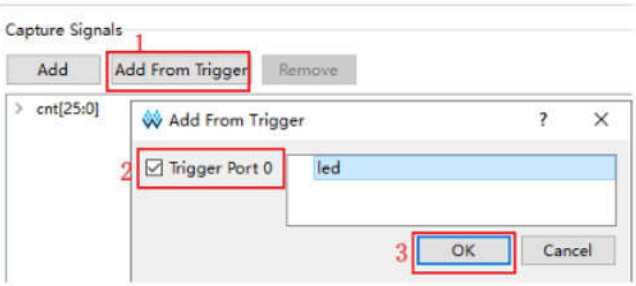
Hình 4-19 –Thêm tín hiệu bus cnt[25:0] vào Capture Signals

Trong hình trên còn có một nút là **"Add From Trigger"**.

Nút này cho phép bạn **trực tiếp sử dụng các tín hiệu từ cổng kích hoạt (Trigger Ports)** làm **tín hiệu dữ liệu cần lấy mẫu (Capture Signals)**.

Khi nhấn vào **"Add From Trigger"**, phía dưới sẽ hiển thị danh sách các Trigger Ports đã được cấu Hình.

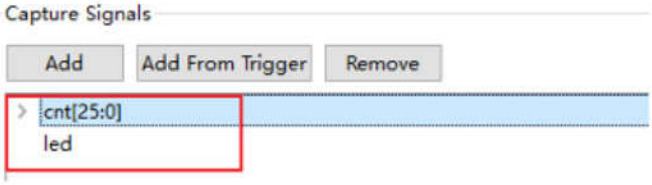
Bạn có thể **chọn một hoặc nhiều Trigger Port** từ danh sách này, để sử dụng các tín hiệu đã ánh xạ làm **tín hiệu lấy mẫu**.



Hình 4-20– Thêm Capture Signals từ các Trigger Ports đã có

Sau khi thêm tín hiệu thành công, giao diện sẽ hiển thị như minh họa trong **Hình 4-21**.

Nếu muốn xóa một tín hiệu đã thêm, bạn chỉ cần **chọn tín hiệu đó**, sau đó nhấn nút **"Remove"** để loại bỏ khỏi danh sách các tín hiệu cần lấy mẫu.



Hình 4-21– Xóa tín hiệu đã thêm trong Capture Signals

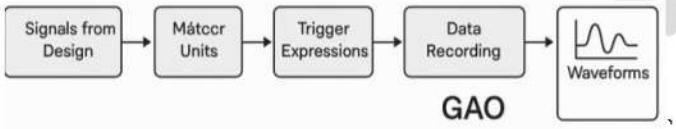
Đến đây, việc cấu Hình tệp **GAO** đã hoàn tất.

Nhấn tổ hợp phím **Ctrl + S** để **lưu lại cấu Hình**.

Nếu trong quá trình lưu **không xuất hiện thông báo lỗi**, điều đó cho thấy cấu Hình đã được thực hiện chính xác.

Tuy nhiên, nếu có lỗi xảy ra, người dùng nên **kiểm tra lại toàn bộ các bước cấu Hình**, đảm bảo rằng **không bỏ sót thao tác nào**, và các thông số đã được thiết lập đúng theo yêu cầu.

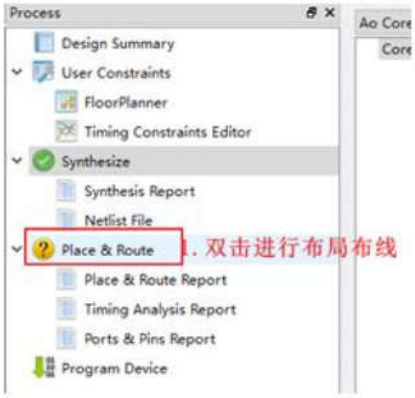
📌 **Lưu ý:** Các lỗi cấu Hình thường do thiếu tín hiệu, sai biểu thức trigger, hoặc chưa đặt clock lấy mẫu.



4.3 Định vị và định tuyến (Place & Route)

Sau mỗi lần chỉnh sửa tệp GAO, bạn cần nhấn vào nút "**Place & Route**" để thực hiện lại quá trình bố trí và định tuyến mạch.

Điều này đảm bảo rằng các phần tử GAO mới được thêm vào sẽ được ánh xạ và kết nối đúng trong thiết kế FPGA.Thao tác này được minh họa trong Hình 4-22.

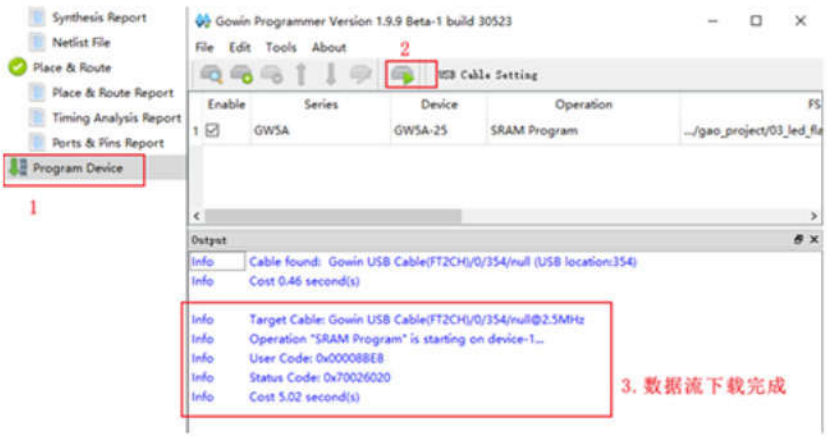


Hình 4-22 – Nhấn Place & Route sau khi chỉnh sửa GAO

4.4 Tải file fs

Nhấn vào nút **Program Device tải xuống tệp .fs**, thực hiện thao tác **nạp cấu Hình vào FPGA**.

Cách thao tác cụ thể được minh họa như trong **Hình dưới đây**.



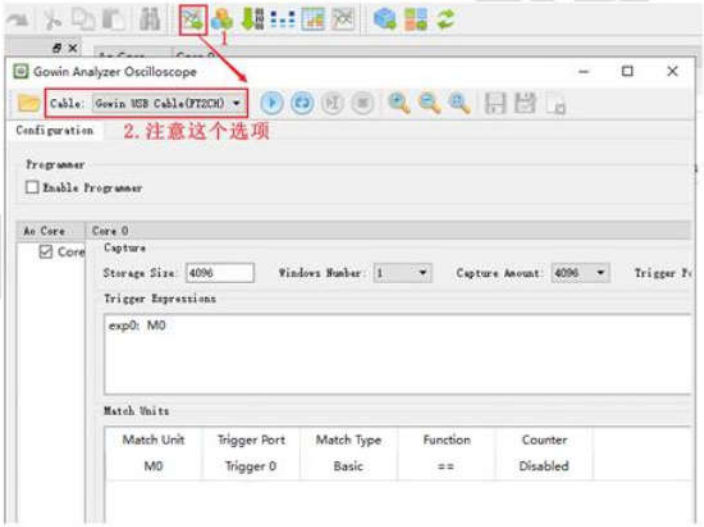
Hình 4-23 – Thao tác tải tệp .fs xuống thiết bị FPGA

4.5 Bắt sóng dạng tín hiệu (Waveform Capture)

Nhấn vào **nút trên thanh công cụ** để **mở giao diện phân tích logic trực tuyến (GAO) của Gowin**, như minh họa trong **Hình 4-24**.

📌 **Lưu ý quan trọng:**

Ở phần trên cùng của giao diện, mục **Cable phải được chọn là "Gowin USB Cable (FT2CH)"**. Nếu không chọn đúng loại cáp này, hệ thống sẽ **không thể nhận tín hiệu kích hoạt**, dẫn đến **dạng sóng không bao giờ hiển thị** dù cấu Hình hoàn toàn chính xác.

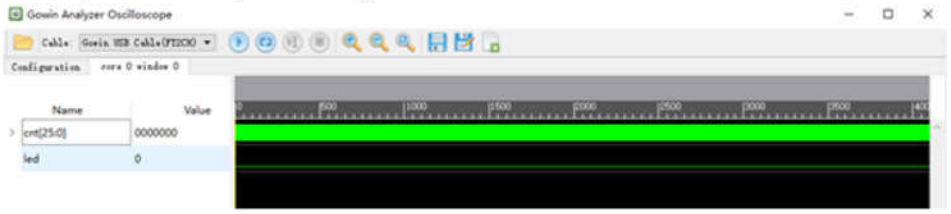


Hình 4-24 – Giao diện GAO – đảm bảo chọn đúng loại cáp

Từ giao diện trên, chúng ta có thể thấy rằng **người dùng vẫn có thể tiếp tục chỉnh sửa cấu Hình lấy mẫu của Core 0** nếu cần.

Tuy nhiên, trong ví dụ này, **chúng ta không thực hiện thay đổi nào**, mà sẽ **nhấn trực tiếp vào nút "Quan sát dạng sóng"** ở phía trên để bắt đầu hiển thị dữ liệu.

Dạng sóng thu được sẽ hiển thị như trong **Hình 4-25**.



Hình 4-25 – Dạng sóng thu được sau khi quan sát tín hiệu

Lúc này, hãy nhấn vào **nút ở phía trên** để **quan sát sự thay đổi của tín hiệu cnt**, như minh họa trong **Hình 4-26**.

Từ Hình có thể thấy rằng **cnt liên tục tăng dần**, trong khi **led không có sự thay đổi nào**.

👉 Nguyên nhân là do **giá trị đếm của cnt quá lớn**, khiến cho **sự thay đổi trạng thái của led diễn ra rất chậm**, và **khó quan sát trong khoảng thời gian lấy mẫu hiện tại**.

Hình 4-26 – Quan sát sự thay đổi giá trị của cnt

Nếu người dùng muốn **quan sát được sự thay đổi của tín hiệu led**, có thể **giảm giá trị đếm của cnt**,

ví dụ: đặt giá trị đếm là **299**, như minh họa trong **Hình 4-27**.

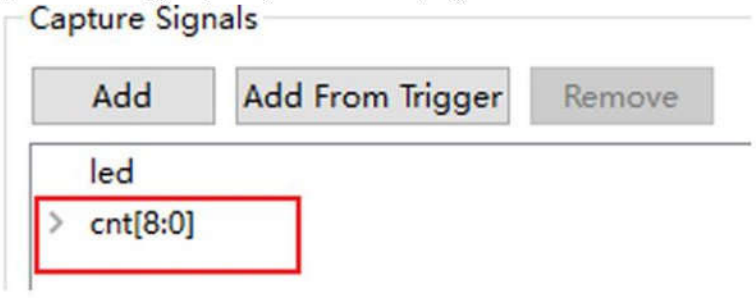
Việc giảm giá trị này giúp cnt **tăng nhanh hơn đến ngưỡng thay đổi của led**, từ đó **có thể quan sát được xung thay đổi rõ ràng trong khoảng thời gian lấy mẫu**.

```
reg [25:0] cnt;
always @ (posedge clk or negedge rst_n)
begin
    if (!rst_n)
        cnt <= 26'd0;
    else if (cnt < 26'd299)
        cnt <= cnt + 1'b1;
    else
        cnt <= 26'd0;
end

always @ (posedge clk or negedge rst_n)
begin
    if (!rst_n)
        led <= 1'b0;
    else if (cnt == 26'd299)
        led <= ~led;
    else
    led <= led;
end
```

Hình 4-27 – Giảm giá trị đếm cnt để quan sát sự thay đổi của led

Sau khi chỉnh sửa xong đoạn mã, bạn cũng cần **chỉnh sửa lại tín hiệu lấy mẫu cnt trong tập tin GAO**, như hình dưới đây (Hình 4-28).

Việc chỉnh sửa này là do trong thực tế, ta **chỉ sử dụng từ bit thứ 0 đến bit thứ 8 của cnt**.



Hình 4-28 – Giảm giá trị đếm cnt để quan sát sự thay đổi của led

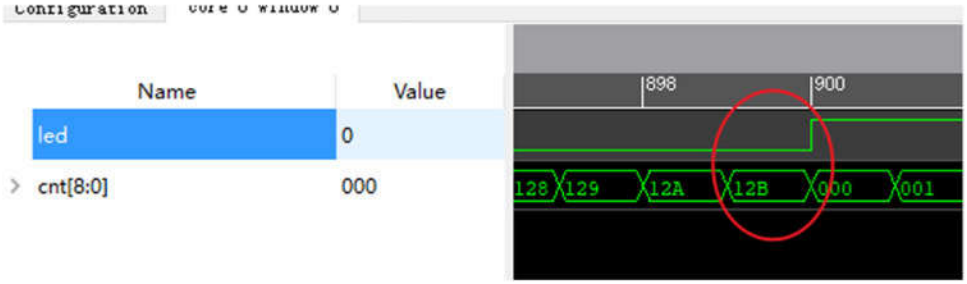
Biên dịch lại và tải xuống tệp fs, lúc này chúng ta **không thể quan sát sự thay đổi của đèn LED bằng mắt thường**, nhưng có thể **bắt dạng sóng thông qua bộ phân tích logic Gowin**, như được hiển thị trong **Hình 4-29** dưới đây.

Từ biểu đồ có thể thấy **trạng thái của đèn LED luôn thay đổi**.

Hình 4-29 – Quan sát dạng sóng sau thay đổi

Chúng ta phóng to một đoạn có sự thay đổi cạnh như Hình dưới đây. Từ biểu đồ có thể thấy rằng **khi cnt đếm đến giá trị 12B, đèn LED bắt đầu thay đổi**.

Chuyển sang hệ **thập phân**, giá trị đó là **299, hoàn toàn khớp với thiết kế trong mã nguồn của chúng ta**.



Hình 4-30 – Dạng sóng tín hiệu Led chuyển 0-1

Đến đây, **quy trình sử dụng cơ bản của Logic Analyzer trực tuyến Gowin** đã được giới thiệu đầy đủ.

Tuy nhiên, **vẫn còn nhiều tính năng khác chưa được đề cập trong thí nghiệm lần này**. Những phần đó sẽ được **bổ sung thêm trong các thí nghiệm tiếp theo khi cần sử dụng đến**.

4.6 Tổng kết chương

Trong quá trình phát triển FPGA, **việc sử dụng bộ phân tích logic trực tuyến (Online Logic Analyzer)** là vô cùng phổ biến và hữu ích.

Thí nghiệm trong chương này đã giúp người học **làm quen với cách sử dụng Logic Analyzer trong phần mềm Gowin**, từ việc tạo file cấu Hình, thiết lập điều kiện kích hoạt, đến bắt và hiển thị dạng sóng tín hiệu thực tế.

Hy vọng người dùng sẽ **làm quen kỹ lưỡng với quy trình thao tác theo các bước đã hướng dẫn**, từ đó có thể **áp dụng hiệu quả vào các thiết kế của riêng mình trong tương lai**.

GOWIN VIETNAM

Chương 5: Thiết kế mạch logic tuần tự – Bộ đếm

Mã nguồn bài thí nghiệm	|----Bài thí nghiệm số 2 |----ch5_bin_counter

Dẫn nhập chương

Mạch logic tuần tự là loại mạch mà trạng thái ổn định đầu ra tại bất kỳ thời điểm nào không chỉ phụ thuộc vào đầu vào hiện tại, mà còn liên quan đến trạng thái được Hình thành bởi đầu vào ở thời điểm trước đó. Điều này trái ngược với mạch logic tổ hợp, nơi đầu ra chỉ là một hàm của đầu vào hiện tại. Nói cách khác, mạch logic tuần tự có phần tử lưu trữ để lưu giữ thông tin, trong khi mạch logic tổ hợp thì không.

Mạch logic tuần tự có nhiều loại, trong chương này chúng ta sẽ lấy **bộ đếm (counter)** – một loại mạch thông dụng nhất – làm ví dụ để học cách thiết kế mạch logic tuần tự đơn giản, đồng thời so sánh sự khác biệt với mạch logic tổ hợp.

Ở đây, chúng ta sẽ thiết kế một bộ đếm sao cho **trạng thái LED trên bo mạch phát triển ACG525 thay đổi (lật trạng thái) mỗi 500ms một lần**. Bo mạch ACG525 sử dụng một thạch anh dao động có tần số 50MHz, nghĩa là chu kỳ xung clock là 20ns. Từ đó có thể tính được:

500ms = 500.000.000ns, 500.000.000ns, 20ns=25.000.000500ms = 500.000.000ns,\quad \frac{500.000.000ns}{20ns} = 25.000.000 500ms = 500.000.000ns, 20ns500.000.000ns=25.000.000

Tức là cần **bộ đếm đếm đến 25.000.000 lần**, điều này yêu cầu bộ đếm phải có **ít nhất 25 bit** (vì 225>25.000.000>2242^{25} > 25.000.000 > 2^{24}225>25.000.000>224). Sau mỗi lần đếm đến giá trị này, bộ đếm sẽ phải **được xóa (reset) và bắt đầu đếm lại từ đầu**.

5.1 Nguyên lý hoạt động của bộ đếm

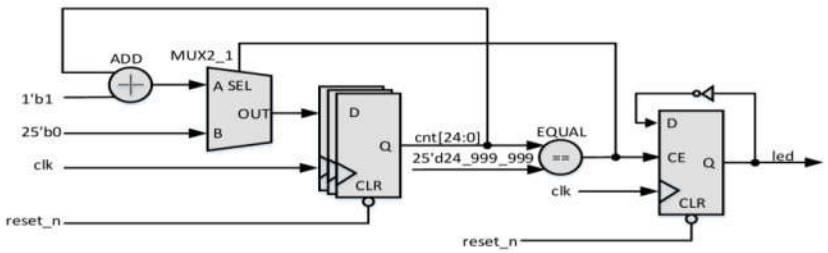
Thành phần cốt lõi của bộ đếm là **chốt (flip-flop)**, với chức năng cơ bản là **đếm các xung**. Số lượng xung lớn nhất mà bộ đếm có thể ghi nhớ được gọi là **modulo (giá trị đếm tối đa)** của bộ đếm đó. Bộ đếm thường được sử dụng trong các ứng dụng như **chia tần số** và **định thời**.

Có rất nhiều loại bộ đếm, được phân loại theo các tiêu chí khác nhau:

- **Theo hệ đếm**: gồm có bộ đếm nhị phân, bộ đếm thập phân, và bộ đếm với hệ đếm tùy ý.
- **Theo nguồn xung clock cấp cho các chốt**: gồm **bộ đếm đồng bộ** và **bộ đếm không đồng bộ.**
- **Theo chiều đếm**: gồm có bộ đếm **tăng (cộng)**, bộ đếm **giảm (trừ)**, và **bộ đếm hai chiều (có thể đảo chiều đếm)**.

Ngôn ngữ **Verilog HDL** được gọi là ngôn ngữ mô tả phần cứng là vì **chúng ta không lập trình theo cách thông thường như C**, mà là **mô tả và xây dựng một mạch phần cứng thực sự**.

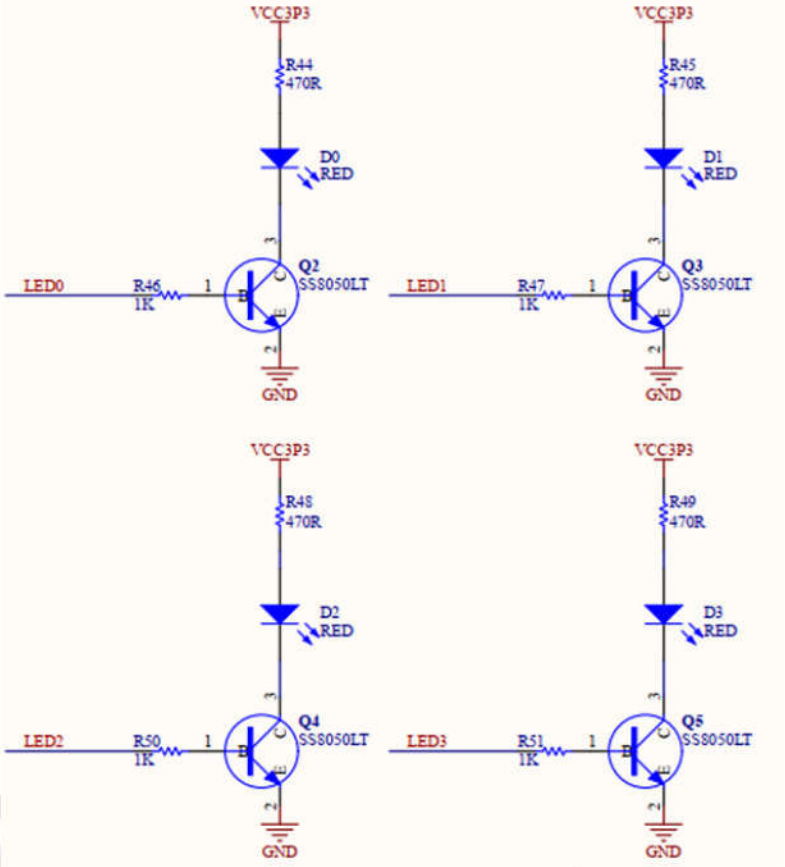
Tiếp theo, chúng ta sẽ thiết kế một bộ đếm để điều khiển **một đèn LED nhấp nháy**. Như đã nói ở trên, một bộ đếm thường được cấu thành từ **bộ cộng, bộ so sánh, thanh ghi và bộ chọn**, như được minh họa trong Hình 5-1 dưới đây:



Hình 5-1 – Cấu trúc cơ bản của bộ đếm

Mạch phần cứng của LED trên bo mạch phát triển được thể hiện trong **Hình 5-2** dưới đây. Có thể thấy rằng **khi chân điều khiển ở mức cao (logic 1)** thì **LED sáng**, còn khi **ở mức thấp (logic 0)** thì **LED tắt**.

Vì vậy, trong thiết kế này, **chỉ cần mỗi khi bộ đếm đếm đầy (đạt giá trị tối đa), ta đảo trạng thái đầu ra điều khiển LED**, như vậy là có thể điều khiển LED nhấp nháy theo yêu cầu.

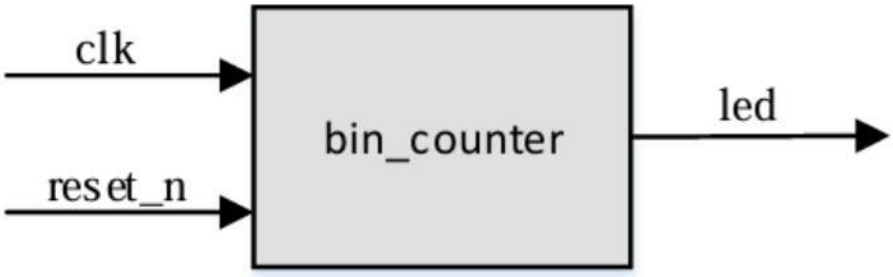


Hình 5-2 – Mạch nguyên lý trên phần cứng

5.2 Hiện thực bộ đếm bằng Verilog

Từ phân tích ở trên, ta có thể rút ra sơ đồ khối giao diện của module thiết kế này như thể hiện trong Hình 5-3 dưới đây.

Chức năng của từng cổng (port) trong module được mô tả chi tiết trong bảng 5-1.



Hình 5-3 – Sơ đồ Module mạch đếm

Bảng 5-1 – Mô tả chức năng các cổng trong module:

Tên cổng	I/O	Mô tả chức năng cổng
clk	I	Xung clock hoạt động của module, tần số 50MHz
reset_n	I	Tín hiệu reset cho module, **reset mức thấp**
led	O	Đầu ra điều khiển LED, **mức thấp thì LED sáng**

Tạo dự án mới

Dựa theo các phần đã trình bày ở trên, ta tiến hành **tạo một project mới có tên là bin_counter**. Mở phần mềm phát triển **Gowin IDE** bằng cách **nhấp đúp vào biểu tượng Gowin trên màn Hình desktop**, sau đó thực hiện các bước sau:

1. **Trang khởi động** – Nhấp vào **"New Project"** (Tạo dự án mới).
2. **Trang New** – Nhấp **"OK"** để tiếp tục.
3. **Giao diện đặt tên Project** – Nhập tên dự án là bin_counter, **chọn thư mục lưu trữ dự án**, sau đó nhấn **"Next"**.
4. **Giao diện chọn chip (Select Device)** – Chọn **mã chip FPGA tương ứng với bo mạch của bạn** (dựa trên thông tin ghi trên chip), rồi nhấn **"Next"**.
5. **Giao diện tổng kết (Summary)** – Nhấn **"Finish"** để hoàn tất việc tạo project.

Sau khi tạo xong sẽ xuất hiện thông tin tổng quan của dự án như Hình bên dưới:

General	
Project File:	H:\01_gaoyun\gao_project\05_bin_counter\bin_counter\bin_counter.gprj
Synthesis Tool:	GowinSynthesis

Target Device	
Part Number:	GW5A-LV25UG324ES
Series:	GW5A
Device:	GW5A-25
Package:	UBGA324
Speed Grade:	ES
Core Voltage:	LV

Hình 5-4 – Tạo xong prcject bin_counter

5.2.1 Viết code Verilog cho project

Dựa trên **sơ đồ khối của module bộ đếm** và **bảng mô tả chức năng các cổng của module**, ta có thể **xây dựng được danh sách các cổng (port list)** cho module thiết kế.

```
module bin_counter
(       clk,
        reset_n,
        led
);
        input clk;
        input reset_n;
        output led;
        reg led;
endmodule
```

Từ nguyên lý thí nghiệm có thể thấy rằng cần một bộ đếm. Do bộ đếm bắt đầu từ 0 chứ không phải từ 1, nên khi giá trị đếm đạt đến **25'd24_999_999** thì cần phải được xóa về 0, chứ không phải đợi đến **25'd25_000_000**. Trong thiết kế này, giá trị lớn nhất của bộ đếm được định nghĩa bằng cách sử dụng tham số (**parameter**).

Việc sử dụng **parameter** để định nghĩa tham số hóa mang lại lợi ích là: nếu sau này cần thay đổi giá trị lớn nhất của bộ đếm, chỉ cần sửa giá trị trong **parameter**, tất cả những nơi trong thiết kế có sử dụng tham số này sẽ tự động cập nhật theo, giúp cho mã dễ bảo trì và tái sử dụng hơn.n

"Khi bộ đếm đếm đến giá trị đã định trước thì sẽ đảo trạng thái của LED một lần, để đạt được mục đích nhấp nháy (bật/tắt luân phiên)."

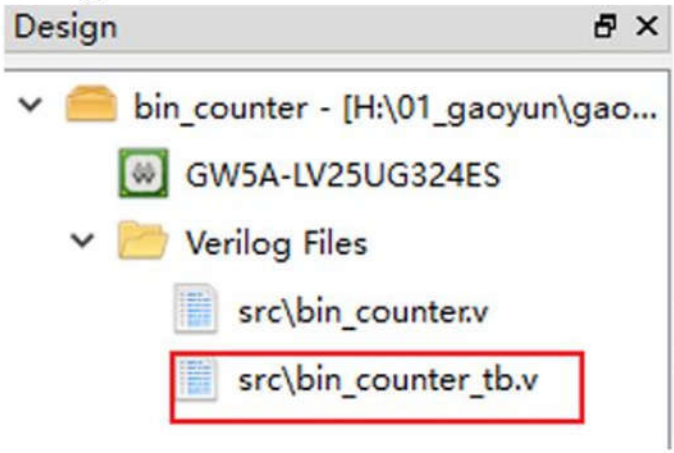
```
// Tiến trình điều khiển xuất ra Led
always@(posedge clk or posedge reset)
if(reset)
        led <= 1'b1;
else
        if(cnt == MCNT)
                led <= ~led;
        else
                cnt <= led;
```

```
parameter MCNT = 24_999_999;
reg [24:0]cnt; // Định nghĩa thanh ghi đếm (counter)
//quá trình bộ đếm
always@(posedge clk or posedge reset)
if(reset)
        cnt <= 25'd0;
else
        if(cnt == MCNT)
                cnt <= 25'd0;
        else
                cnt <= cnt + 1'b1;
```

5.3 Thiết kế Testbench và chạy mô phỏng

Phân tích và tổng hợp thiết kế trên cho đến khi không còn lỗi hoặc cảnh báo. Tạo mới tệp mô phỏng cho bộ đếm. Các bước cụ thể như sau:

1. Nhấn vào biểu tượng "Tạo mới tệp mô phỏng" trên thanh công cụ.
2. Trong hộp thoại "**New**" hiện ra, chọn loại tệp là Verilog File, sau đó nhấn **OK**.
3. Trong hộp thoại "**New Verilog File**", tại mục Name, đặt tên cho tệp là **bin_counter_tb**. Các mục khác giữ nguyên mặc định, nhấn **OK**, tệp mô phỏng sẽ được tạo thành công.
4. Như Hình 5-5 minh họa, trong **Design -> Verilog Files** sẽ xuất hiện tệp mô phỏng vừa tạo. Nhấp đúp chuột vào tên tệp để mở cửa sổ chỉnh sửa.



Hình 5-5 – Tạo xong file Testbench project bin_counter

Dưới đây là đoạn mô phỏng Verilog theo yêu cầu của bạn: **tạo xung clock 20ns (tương đương 50MHz), tín hiệu reset**, và **khởi tạo module cần mô phỏng bin_counter.v**

```
`timescale 1ns/1ns
`define CLOCK_PERIOD 20
module bin_counter_tb();
        reg clk;
        reg reset_n;
        wire led;
                bin_counter bin_counter_inst (
                .clk(clk),
                .reset_n(reset_n ),
                .led(led)
        );
        initial clk = 1;
        always #(`CLOCK_PERIOD/2) clk = ~clk;
        initial begin
                reset_n = 1'b0;
                #(`CLOCK_PERIOD *200 + 1);
                reset_n = 1'b1;
                #2000000000;
                $stop;
                end
endmodule
```

Sau khi thêm file mô phỏng xong, tiến hành các bước sau:

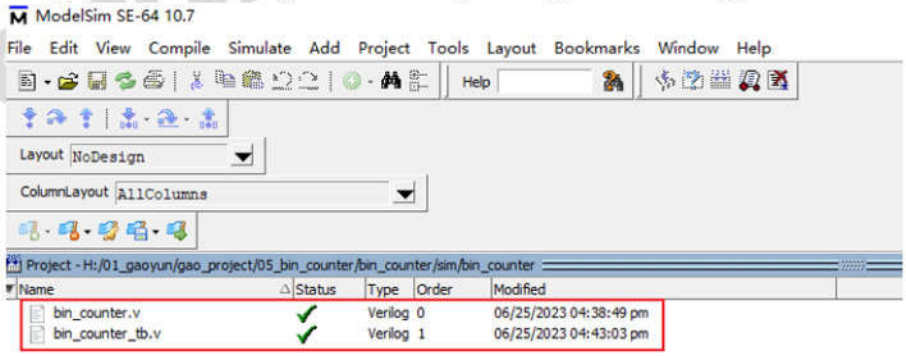
1. Phân tích và tổng hợp mã nguồn (Analyze & Synthesize)
   - Kiểm tra xem có lỗi cú pháp hay không.
   - Nếu có cảnh báo (warning), xem xét mức độ ảnh hưởng, nhưng lý tưởng là không có lỗi hay cảnh báo.
2. Kiểm tra toàn bộ thiết kế
   - Đảm bảo rằng tất cả các module đã được kết nối đúng, tín hiệu khai báo phù hợp.
3. Dùng phần mềm ModelSim để chạy mô phỏng và xác minh thiết kế.
   - Load testbench (bin_counter_tb.v) vào ModelSim.
   - Compile toàn bộ project.
   - Chạy mô phỏng (Run) và quan sát sóng tín hiệu (Wave) như: clk, reset_n, led.
   - Xác minh rằng LED nhấp nháy đúng theo thiết kế (mỗi khi đếm đủ sẽ đảo trạng thái LED).

5.3.1 Tạo project mô phỏng dùng Model Sim

1. Trong thư mục dự án, tạo thư mục con tên sim để chứa các file mô phỏng.
2. Mở phần mềm ModelSim.
3. Chọn File → New → Project để tạo project mới.
4. Trong cửa sổ Create Project, đặt tên là bin_counter, nhấn Browse chọn thư mục sim làm nơi lưu project, rồi nhấn OK.
5. Trong hộp thoại hiện ra, nhấn Add Existing File → Browse, chọn và thêm các file mô phỏng:
   - bin_counter.v (mạch thiết kế)
   - bin_counter_tb.v (testbench)

   → Nhấn OK.
6. Vào menu Compile → Compile All để biên dịch tất cả các file.

Nếu có lỗi biên dịch, hãy xem thông báo và sửa lại mã nguồn cho đúng.



Hình 5-6 – Biên dịch project bin_counter trong Modelsim

5.3.2 Định cấu Hình môi trường mô phỏng

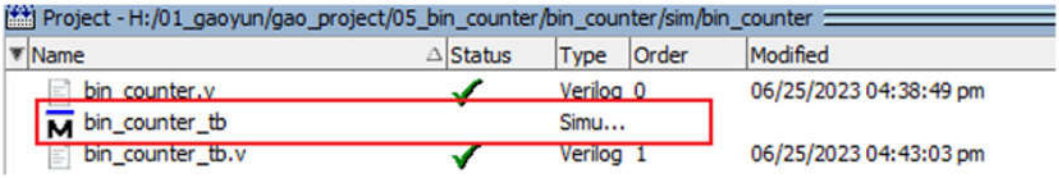
1. Trong thanh trạng thái **Project**, nhấp chuột phải → chọn **"Add to Project" → "Simulation Configuration…"**
2. Trong cửa sổ **Add Simulation Configuration**:
   - Tại tab **Design**, chọn module bin_counter_tb trong thư viện work làm **top module**.
   - Nhấn **Copy** để dùng tên bin_counter_tb làm tên cấu Hình mô phỏng (đảm bảo tên khớp).
3. Nhấn **"Optimization Options…"**, chọn: **"Apply full visibility to all modules (full debug mode)"** → nhấn **OK**.
4. Trong tab **Libraries**:

a. Tại mục **"Search libraries (-L)"**, nhấn **Add…** → thêm thư viện Gowin: gw5a

b. Ở mục **"Search Libraries First (-Lf)"** cũng chọn thư viện gw5a

c. Nhấn **Save** để lưu cấu Hình.

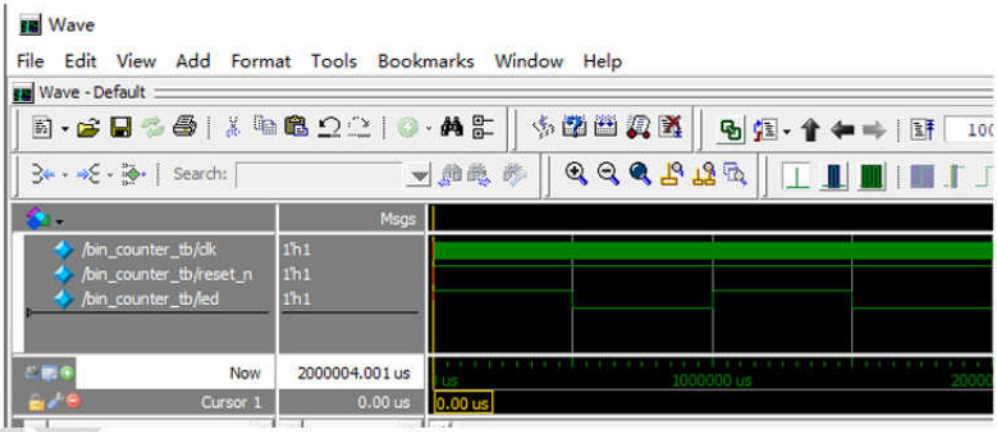
✅ Sau khi hoàn tất, trong thanh **Project** sẽ xuất hiện cấu Hình mô phỏng mới tên là bin_counter_tb, sẵn sàng để chạy mô phỏng.



Hình 5-7 – Tạo thành công file mô phỏng project bin_counter

5.3.3 Chạy mô phỏng và xem sóng tín hiệu

1. **Nhấp đúp vào file mô phỏng bin_counter_tb** vừa tạo để vào giao diện **Sim**.
2. Trong giao diện **Sim**, chọn module **bin_counter_tb**, **nhấp chuột phải → chọn "Add Wave"** để thêm các tín hiệu cần quan sát vào cửa sổ sóng (**Wave**).
3. Chuyển sang cửa sổ **Wave**, nhấn nút **Run All** trên thanh công cụ để chạy mô phỏng.
4. Sau khi mô phỏng dừng, nhấn **Stop** để dừng hẳn và quan sát sóng hiển thị như Hình minh họa.



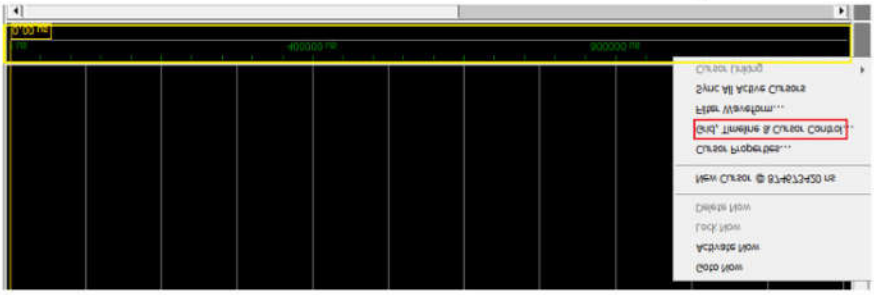
Hình 5-8 – Sơ đồ dạng sóng project bin_counter

Kết luận từ Hình mô phỏng: Thời gian thay đổi mức cao/thấp là 500 000 000ns (0,5s), đúng với yêu cầu thiết kế. Mô phỏng chức năng hoàn tất.

Đơn vị thời gian trong mô phỏng hiện là **µs**, nên để xác định thời gian LED ở mức cao/thấp, cần tự quy đổi — điều này **không trực quan**.

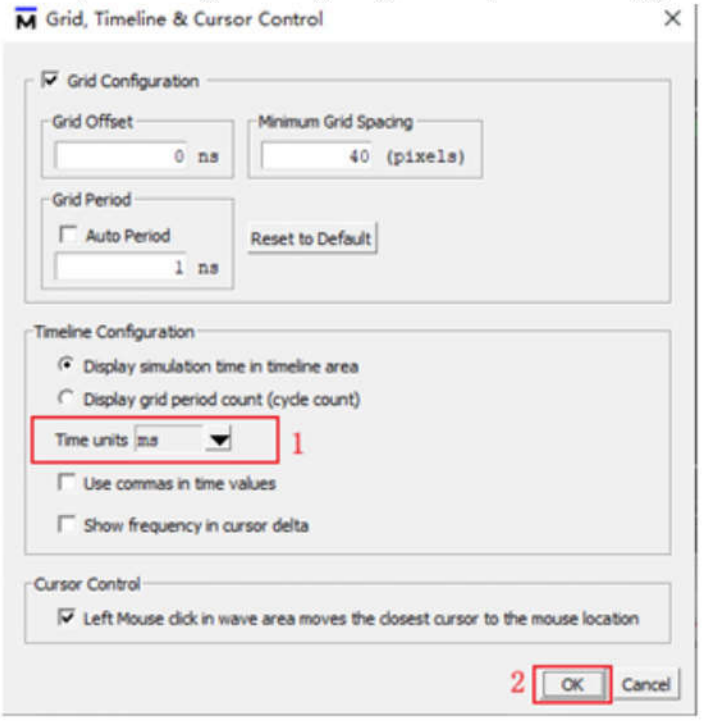
→ Có thể **thay đổi đơn vị hiển thị thời gian** để dễ quan sát hơn. Các bước thực hiện như sau:

(1) Như Hình 5-9 minh họa, **nhấp chuột phải vào vùng được khoanh vàng** trong cửa sổ sóng, sau đó chọn **"Grid, Timeline & Cursor Control..."** để mở cài đặt thời gian hiển thị.



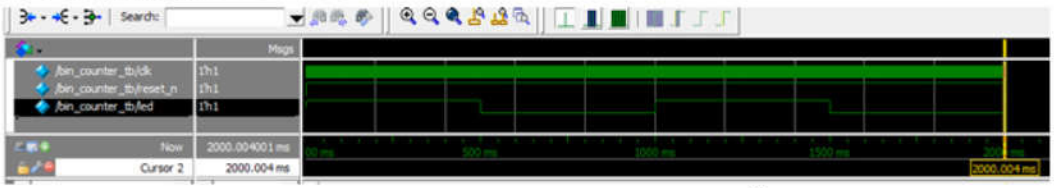
Hình 5-9 – Cài đặt thời gian hiển thị dạng sóng

(2) Một cửa sổ như Hình 5-10 sẽ xuất hiện, **vùng được khoanh đỏ** chính là nơi bạn có thể **cài đặt đơn vị hiển thị thời gian**. Sử dụng **menu thả xuống** để thay đổi đơn vị từ **us/ns** thành **ms**. (Chọn đơn vị thời gian phù hợp với độ dài thời gian mô phỏng để dễ quan sát sóng).



Hình 5-10 – Cài đặt đơn vị thời gian hiển thị dạng sóng

Sau khi thay đổi đơn vị thời gian hiển thị, bạn có thể quan sát sóng tín hiệu mức cao/thấp. Như Hình 5-11, đơn vị thời gian đã được thay đổi thành **ms**, và thời gian mức cao/thấp là **500ms**. Kết quả vẫn giống như trước, chỉ khác là đơn vị thời gian hiển thị đã thay đổi.



Hình 5-11 – Dạng sóng sau khi thay đổi đơn vị thời gian

Trong quá trình mô phỏng chức năng như trên, có thể nhận thấy thời gian chạy mô phỏng trên máy tính khá dài, điều này là do giá trị đếm của bộ đếm quá lớn. Để rút ngắn thời gian mô phỏng, bạn có thể thay đổi giá trị đếm tối đa của mô phỏng. Dưới đây là hai cách thay đổi, cách đầu tiên là chỉnh sửa giá trị đếm tối đa trong file thiết kế bin_counter.v bằng cách sử dụng parameter, thay giá trị tối đa thành 24_999. Cụ thể mã nguồn như sau:

```
parameter MCNT = 24_999;
```

Cách thứ hai là thay đổi giá trị đếm tối đa trong file mô phỏng bin_counter_tb.v thành 24_999. Cụ thể mã nguồn như sau:

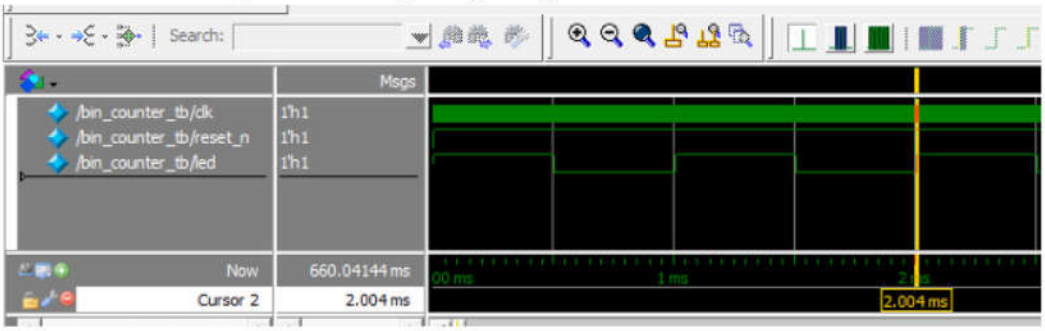
```
defparam bin_counter_inst.MCNT = 24_999;
```

Thông qua hai cách trên, bạn có thể **rút ngắn thời gian mô phỏng** mà vẫn đảm bảo xác minh tính đúng đắn của thiết kế. So với hai cách, **cách thứ hai** có vẻ tốt hơn, vì nó có thể **rút ngắn thời**

gian mô phỏng mà không cần thay đổi mã thiết kế, chỉ cần thay đổi trong mã mô phỏng. Dưới đây là **sóng mô phỏng sau khi đã sửa đổi** như Hình 5-12.

Trong Hình sóng, thời gian thay đổi mức cao/thấp đã được rút ngắn xuống **0.5ms**, so với **500ms** trước đó, thời gian đã được **rút ngắn 1000 lần**, điều này cũng chứng tỏ mô phỏng chức năng là chính xác.

Cần lưu ý rằng, sau khi **thay đổi mã nguồn**, bạn cần **biên dịch lại và chạy mô phỏng** trong **ModelSim** để đảm bảo các thay đổi được áp dụng đúng.



Hình 5-12 – Dạng sóng sau khi thu hẹp giá trị đếm

5.4 Gỡ lỗi và xác thực cấp bo mạch

Mục tiêu kiểm tra trên board trong thí nghiệm này:

1. Kiểm tra xem có thể tải chính xác file fs vào board phát triển hay không.
2. Sau khi tải xong, kiểm tra xem đèn LED trên board có nhấp nháy mỗi giây một lần hay không.

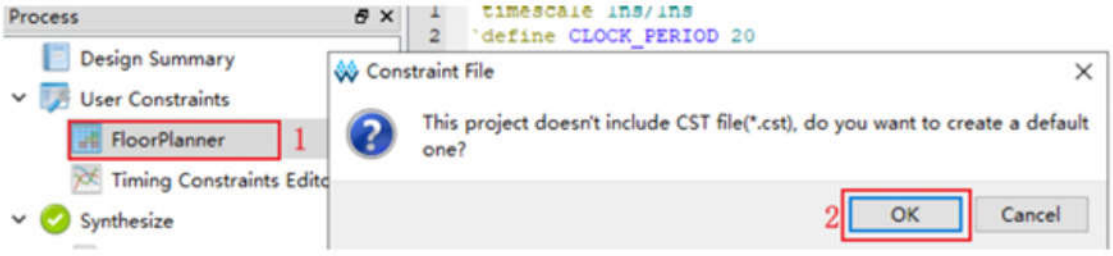
✅ Phần cứng yêu cầu cho hệ thống:

1. Board phát triển ACG525.
2. Bộ tải Gowin (Gowin downloader) và cáp tải.
3. Nguồn cung cấp điện 12V.
4. PC có đầy đủ chức năng phát triển và đáp ứng yêu cầu thí nghiệm.

5.4.1 Thêm ràng buộc I/O

Trong giao diện Gowin, ở khung Process bên phải, nhấp vào FloorPlanner để mở công cụ thiết lập sơ đồ chân (pin layout).

Sau đó tạo mới một file .cst dùng để lưu các ràng buộc I/O (I/O constraints), như minh họa ở Hình 5-13.



Hình 5-13 – Thêm ràng buộc I/O

Sau khi vào giao diện **FloorPlanner**, hãy nhấp vào mục **I/O Constraints** ở phía dưới để mở giao diện thiết lập ràng buộc chân I/O.

Dựa vào **sơ đồ nguyên lý (schematic)**, tiến hành **gán đúng các chân (pin) cho các tín hiệu**.

Sau khi hoàn tất việc gán chân, kết quả sẽ hiển thị như **Hình 5-14**.

GOWIN VIETNAM

I/O Constraints

	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type	Drive
1	clk	input		T9	4	False	LVCMOS33	OFF
2	led	output		D14	1	False	LVCMOS33	8
3	reset_n	input		B16	1	False	LVCMOS33	OFF

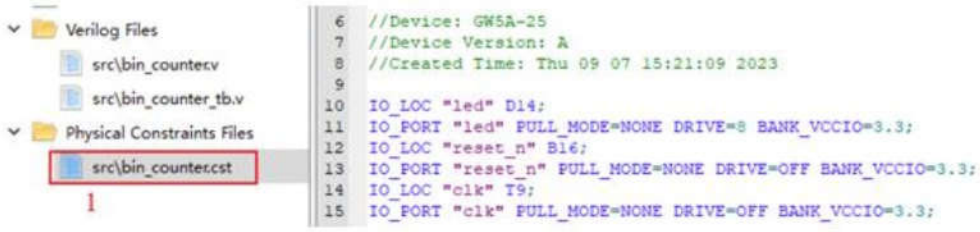
Hình 5-14 – Giao diện gán chân

Như vậy là **việc gán chân I/O đã hoàn tất**. Tuy nhiên, **các ràng buộc này hiện mới lưu tạm trong bộ nhớ**, chưa được ghi vào file.

→ Hãy **nhấn nút lưu trên thanh công cụ** hoặc đơn giản là bấm **Ctrl+S** để lưu lại.

Sau khi **đóng giao diện FloorPlanner**, nhấp vào tab **Design** ở bên phải, bạn sẽ thấy file .cst vừa được tạo.

Nhấp đúp để mở, có thể xem nội dung ràng buộc như minh họa trong **Hình 5-15**.



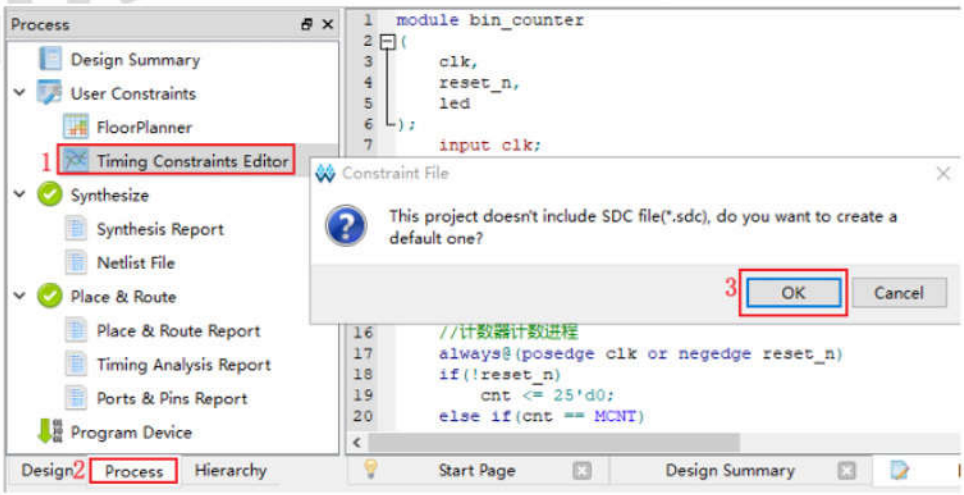
Hình 5-15 – Kết quả file ràng buộc I/O

5.4.2 Thêm ràng buộc thời gian

Ta cần nhấp vào **"Timing Constraints Editor"** trong khung **Process** để thiết lập **ràng buộc thời gian (timing constraints)**.

Lúc này sẽ xuất hiện một hộp thoại thông báo rằng **dự án hiện chưa có file ràng buộc**, hỏi bạn có muốn tạo file mới hay không.

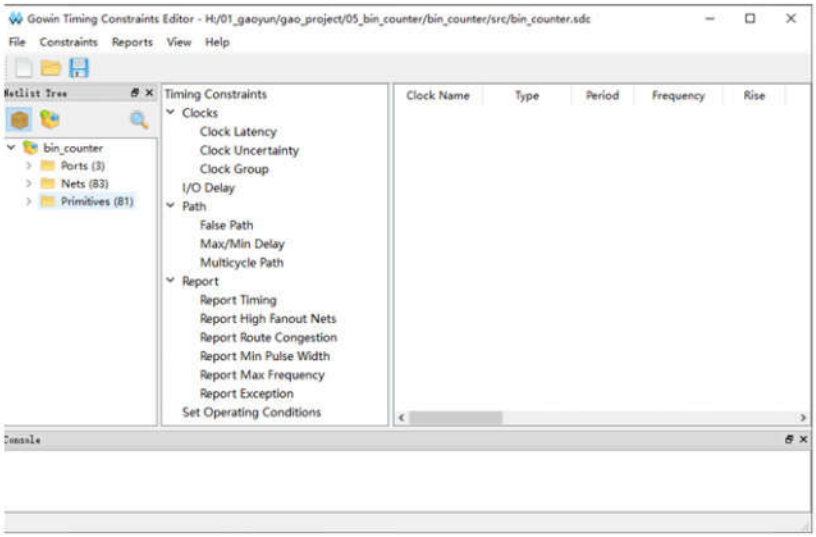
→ **Nhấn OK** để tạo, như minh họa trong **Hình 5-16**



Hình 5-16 – Tạo file ràng buộc thời gian

Sau đó, bạn sẽ được chuyển đến **giao diện thiết lập ràng buộc thời gian**, như minh họa trong **Hình 5-17**.

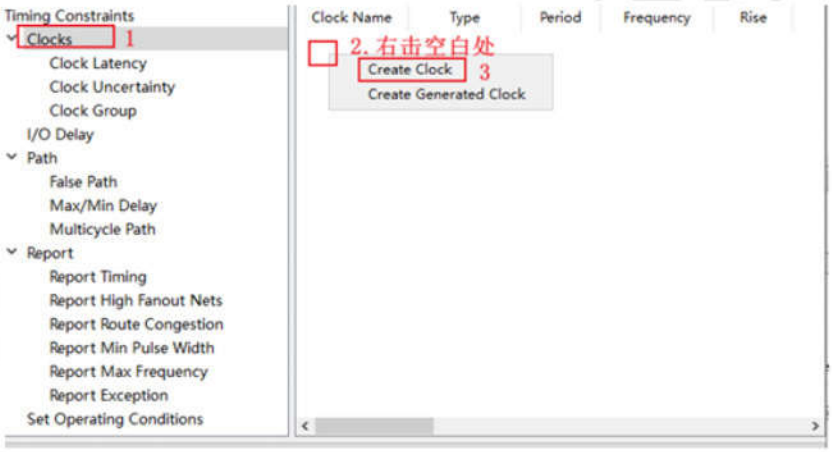
Tại đây, bạn có thể thêm các ràng buộc liên quan đến **tần số clock, độ trễ tín hiệu**, v.v... để đảm bảo thiết kế hoạt động đúng thời gian mong muốn



Hình 5-17 – Giao diện tạo thông số cho rang buộc thời gian

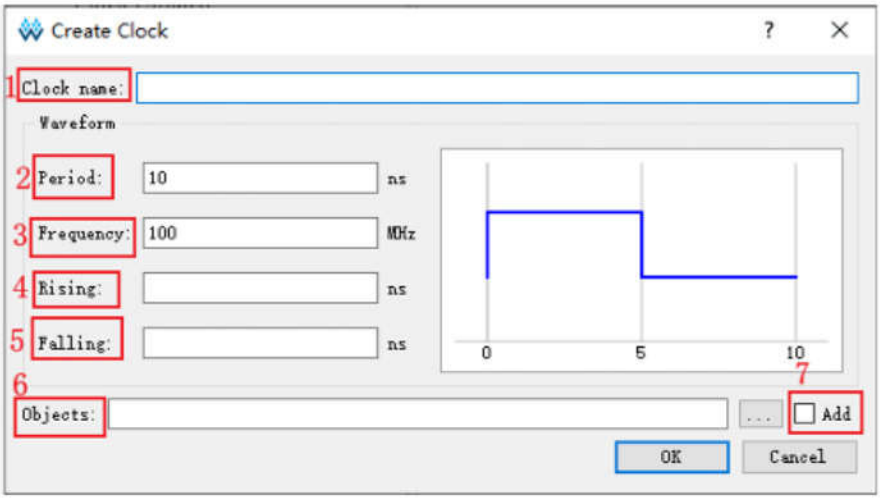
Trong mục **Timing Constraints** → chọn **Clocks**, sau đó di chuyển chuột đến phần trống ở bên phải.

→ **Nhấp chuột phải** và chọn **Create Clock** để tạo xung clock mới, như minh họa trong **Hình 5-18**.



Hình 5-18 – Tạo ràng buộc cho xung clk

Cửa sổ tạo xung clock hiển thị như Hình 5-19 bên dưới



Hình 5-19 – Giao diện Create Clock

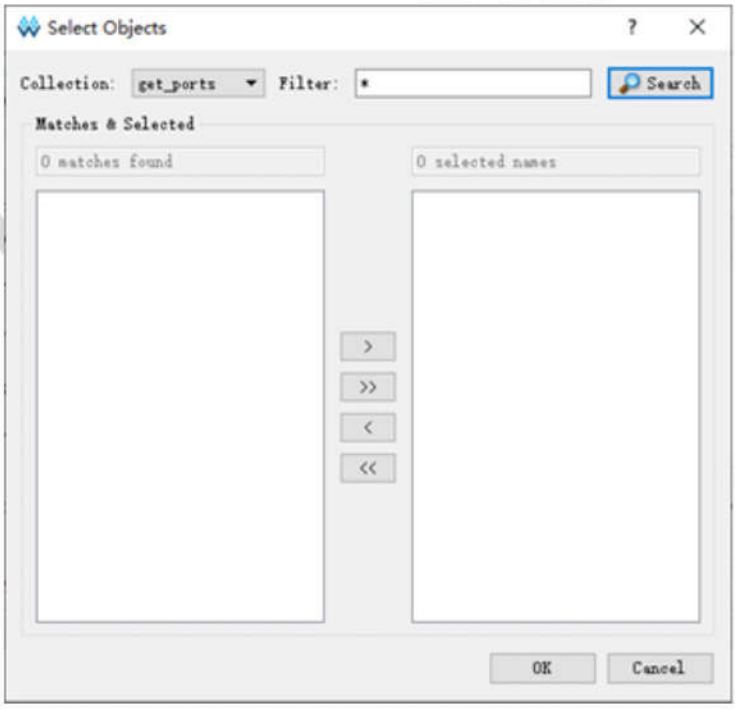
Dưới đây là phần giải thích ngắn gọn cho các tham số trong cửa sổ tạo xung clock như trong Hình:

1. **Clock Name**: Tên clock – phải bắt đầu bằng chữ cái hoặc dấu gạch dưới (_).
2. **Period**: Chu kỳ – đơn vị ns, mặc định là 10ns, hỗ trợ số thực với độ chính xác đến 3 chữ số thập phân.
3. **Frequency**: Tần số – đơn vị MHz, mặc định là 100MHz, cũng hỗ trợ số thực chính xác đến 3 chữ số thập phân.
4. **Rising**: Thời điểm cạnh lên (rising edge) – thời gian trong chu kỳ mà cạnh lên xảy ra.
5. **Falling**: Thời điểm cạnh xuống (falling edge) – thời gian trong chu kỳ mà cạnh xuống xảy ra.
6. **Objects**: Đối tượng áp dụng clock – thường là tên tín hiệu clock trong thiết kế, có thể chọn từ danh sách để tự động điền.
7. **Add**: Dùng khi cần **thêm nhiều clock trên cùng một nguồn** – nếu có, cần đánh dấu chọn.

Các thiết lập này dùng để **ràng buộc thời gian** cho xung nhịp trong thiết kế nhằm đảm bảo hoạt động chính xác của mạch số.

Khi chúng ta nhấn vào nút để **thiết lập đối tượng ràng buộc clock**, một cửa sổ sẽ bật lên như minh họa trong **Hình 5-20**.

Tại đây, bạn có thể **chọn tín hiệu clock cụ thể trong thiết kế** để gán ràng buộc thời gian cho nó một cách chính xác.



Hình 5-20 – Chọn đối tượng

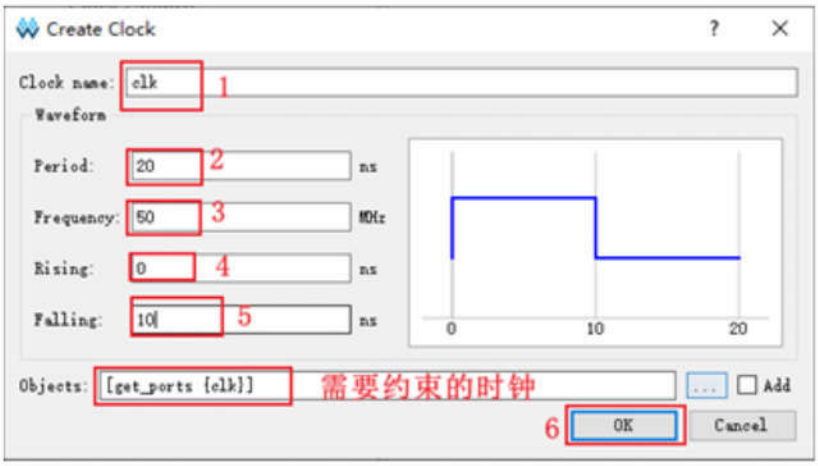
Trong trường hợp này, **xung clock cần ràng buộc là xung hệ thống 50MHz đến từ chân pin** (ngoại vi).

Thực hiện các bước sau để chọn đúng tín hiệu:

1. Nhấn **Search**
2. Gõ vào **clk** để tìm tín hiệu clock
3. Chọn tín hiệu phù hợp

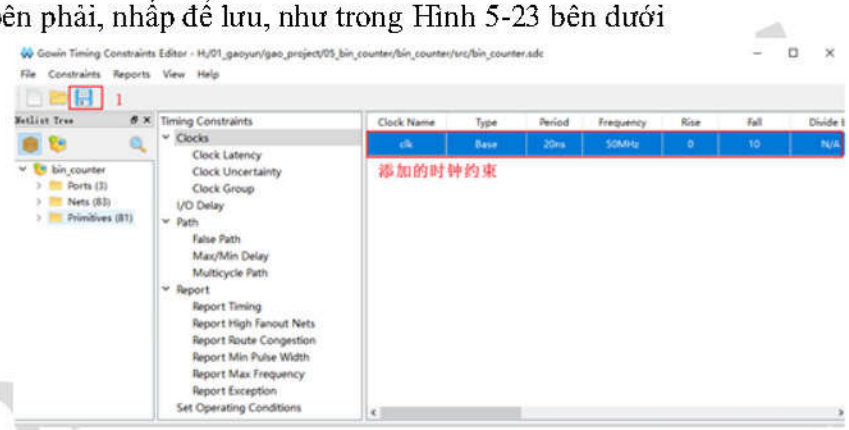
4. Nhấn **OK** xác nhận

Các thao tác này được minh họa trong **Hình 5-21**



Hình 5-22 – Cấu Hình cho xung clock

Sau khi quá trình cộng hoàn tất, bạn có thể thấy rằng có ràng buộc đồng hồ mà chúng ta vừa thêm vào khoảng trống bên phải, nhấp để lưu, như trong Hình 5-23 bên dưới

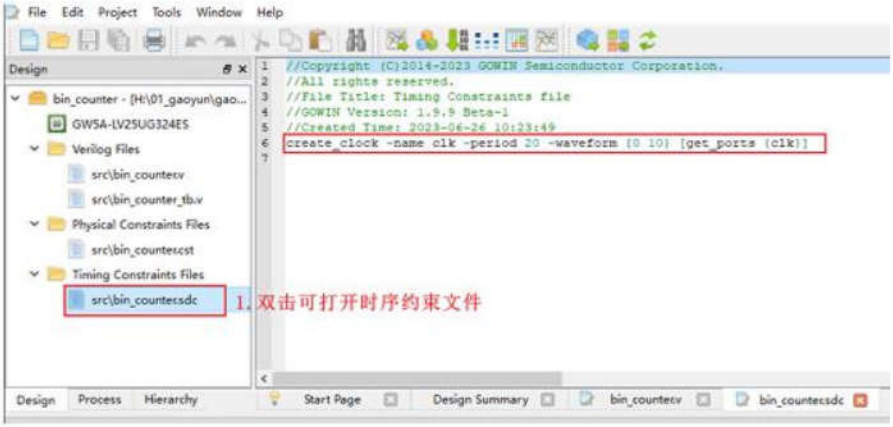


Hình 5-23 – Lưu ràng buộc thời gian

Thông qua các thao tác trên, ta đã **hoàn thành việc ràng buộc thời gian cho xung clock 50MHz**.

Sau đó, **đóng cửa sổ Gowin Timing Constraints Editor** lại. Lúc này, trong giao diện **Design**, bạn sẽ thấy **file ràng buộc thời gian bin_counter.sdc** đã được thêm vào.

→ **Nhấp đúp vào file bin_counter.sdc** để xem nội dung chi tiết của ràng buộc, như minh họa trong **Hình 5-24**.



Hình 5-24 – Xem phương thức tệp ràng buộc thời gian

5.5 Place & Route

Nhấp vào **Place & Route** trong khung **Process** để thực hiện bước **layout và routing** cho thiết kế.

Sau khi **layout và routing thành công**, hệ thống sẽ **tạo ra file fs**. Thông báo thành công sẽ xuất hiện trong **Console**, như minh họa trong **Hình 5-25**.

Hình 5-25 – Tạo file fs thành công

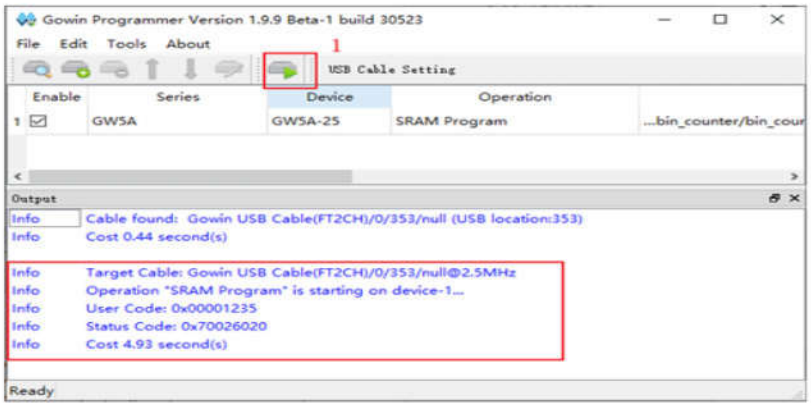
5.5.1 Kết nối phần cứng

Lần lượt kết nối trình tải xuống và nguồn điện với bo mạch phát triển và các kết nối phần cứng tổng thể được hiển thị trong Hình 5-26 bên dưới

Hình 5-26 – Sơ đồ kết nối phần cứng

5.5.2 Tải file fs

Nhấp vào **Program Device** để vào giao diện **tải chương trình** vào thiết bị. Khi đó, cửa sổ **Cable Setting** sẽ xuất hiện. Hãy nhấn **Save** để lưu các thiết lập kết nối. Sau đó, nhấn vào **Download file** để tải file đã biên dịch vào board phát triển. Sau khi quá trình tải hoàn tất, kết quả sẽ hiển thị như **Hình 5-27**.



Hình 5-27 – Tải chương trình thành công

Sau khi quá trình Program hoàn tất, ta có thể thấy LED0 nhấp nháy mỗi giây một lần, và kết quả thí nghiệm chứng tỏ rằng thiết kế đạt yêu cầu như mong đợi.

5.6 Tổng kết chương

Chương này đã giúp chúng ta học cách thiết kế **logic thời gian đơn giản** thông qua ví dụ về **bộ đếm (counter)**. Đồng thời, chương cũng **giới thiệu cách thiết lập ràng buộc thời gian (clock constraints)** trong quá trình thiết kế.

Dựa trên những kiến thức này, bạn có thể **thiết kế một mạch logic** điều khiển **đèn LED nhấp nháy với tần số khác nhau**, tiến hành **mô phỏng** và **kiểm tra trên board** để xác nhận tính đúng đắn của thiết kế.

Đây là nền tảng để phát triển thêm các ứng dụng điều khiển mạch số phức tạp hơn. Chúc bạn có thể áp dụng tốt vào thực tế!

GOWIN VIETNAM

Chương 6: Thiết kế module truyền data qua cổng nối tiếp

Mã nguồn bài thí nghiệm	|----Bài thí nghiệm số 2	|----ch6_uart_tx

Dẫn nhập chương

Trong các hệ thống điện tử ngày nay, thường cần phải thực hiện việc gửi và nhận dữ liệu giữa các bảng mạch, giữa các bảng mạch hoặc giữa thiết bị cấp dưới và thiết bị cấp trên. Điều này yêu cầu cả hai bên tuân theo một giao thức truyền thông nhất định để đảm bảo tính chính xác của việc truyền dữ liệu. Các giao thức thông dụng bao gồm **UART** (Bộ truyền nhận không đồng bộ tổng quát), **IIC** (Bus hai dây hai chiều), **SPI** (Bus ngoại vi nối tiếp), **USB2.0/3.0** (Bus nối tiếp tổng quát) và **Ethernet** (Mạng Ethernet) v.v. Trong số các giao thức này, giao thức cơ bản nhất là **UART**, vì cấu trúc mạch điện đơn giản và chi phí thấp, nên được sử dụng rất phổ biến trong trường hợp chú trọng đến hiệu quả chi phí.

Chương này sẽ học về nguyên lý giao tiếp **UART** và thiết kế mạch phần cứng của nó, đồng thời sử dụng FPGA để thực hiện thiết kế phần gửi dữ liệu trong giao tiếp **UART**.

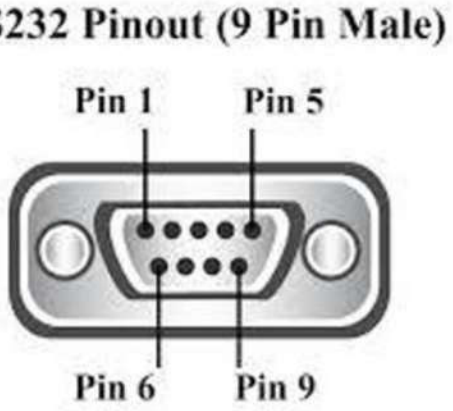
6.1 Nguyên lý truyền nối tiếp và thiết kế mạch

6.1.1 Chuẩn giao thức RS232

Bộ truyền nhận không đồng bộ tổng quát (Universal Asynchronous Receiver/Transmitter, UART) là một loại bộ truyền nhận không đồng bộ, trong đó khi gửi dữ liệu, nó chuyển đổi dữ liệu song song thành dữ liệu nối tiếp để truyền tải, và khi nhận dữ liệu, nó chuyển đổi dữ liệu nối tiếp nhận được thành dữ liệu song song, có thể thực hiện truyền và nhận toàn bộ hai chiều. Nó bao gồm các chuẩn giao thức bus như RS232, RS449, RS423, RS422 và RS485. Nói cách khác, UART là tên gọi chung cho giao tiếp nối tiếp không đồng bộ. Các chuẩn giao tiếp và tiêu chuẩn bus như RS232, RS449, RS423, RS422 và RS485 quy định các đặc tính điện, tốc độ truyền, đặc tính kết nối và đặc tính cơ học của giao tiếp.

RS-232 mà chương này sẽ tập trung học là tiêu chuẩn giao thiếp truyền thông dữ liệu nối tiếp do Hiệp hội Công nghiệp Điện tử Hoa Kỳ (EIA) quy định, mã số gốc đầy đủ là EIA-RS-232 (viết tắt là 232, RS232), được sử dụng rộng rãi trong việc kết nối các thiết bị ngoại vi qua giao diện nối tiếp của máy tính. Định nghĩa chân cắm của giao diện DB9 được thể hiện trong Hình 6-1 dưới đây, chức năng của các chân được trình bày trong bảng 6-1 dưới đây. Nếu hệ thống có nhiều giao diện UART, chúng có thể được gọi lần lượt là COM1, COM2, v.v.

Pin 1	DCD
Pin 2	RXD
Pin 3	TXD
Pin 4	DTR
Pin 5	GND
Pin 6	DSR
Pin 7	RTS
Pin 8	CTS
Pin 9	RI



Hình 6-1 – Định nghĩa chân RS232, DB9

Chân	Ký hiệu	Ý nghĩa	Diễn giải
Pin1	CD	Carrier Detect	Thông báo cho máy tính có tín hiệu được phát hiện
Pin2	RXD	Receiver	Nhận dữ liệu
Pin3	TXD	Transmit	Truyền dữ liệu
Pin4	DTR	Data Terminal Ready	Máy tính thông báo cho bộ điều khiển có thể tiến hành truyền
Pin5	GND	Ground	Nối đất
Pin6	DSR	Data Set Ready	Bộ điều khiển thông báo cho máy tính rằng mọi thứ đã sẵn sàng
Pin7	RTS	Request to send	Máy tính yêu cầu bộ điều khiển gửi dữ liệu
Pin8	CTS	Clear to Send	Bộ điều khiển thông báo cho máy tính rằng có thể truyền dữ liệu
Pin9	RI	Ring Indicator	Bộ điều khiển thông báo cho máy tính rằng có cuộc gọi đến

6.1.2 Tham số chính của UART và biểu đồ thời gian

Giao tiếp UART cần thực hiện nhiều thiết lập trước khi sử dụng, các thiết lập phổ biến nhất bao gồm số bit dữ liệu, tốc độ baud, loại kiểm tra chẵn lẻ và số bit dừng.

Số bit dữ liệu (Data bits): Tham số này xác định số bit dữ liệu được gửi trong một lần truyền từ bắt đầu đến dừng. Có thể chọn: 5, 6, 7 hoặc 8 (mặc định).

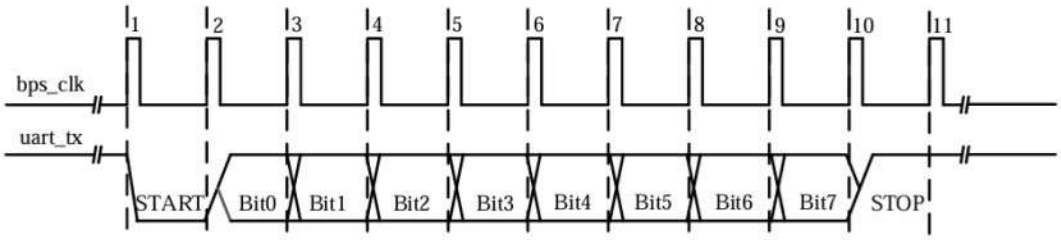
Tốc độ baud (Baud): Là tốc độ truyền dữ liệu từ một thiết bị đến thiết bị khác, tức là số bit dữ liệu có thể truyền mỗi giây. Các tốc độ baud điển Hình có **300, 1200, 2400, 9600, 19200, 115200, v.v.** Thông thường, hai thiết bị giao tiếp cần được thiết lập cùng một tốc độ baud, nhưng một số thiết bị cũng có thể được thiết lập để tự động phát hiện tốc độ baud.

Kiểm tra chẵn lẻ (Parity Type): Được sử dụng để xác minh tính chính xác của dữ liệu. Kiểm tra chẵn lẻ thường không được sử dụng, nếu có thì có thể thực hiện kiểm tra lẻ (Odd) hoặc kiểm tra chẵn (Even). Trong kiểm tra chẵn, vì bit kiểm tra sẽ được đặt thành 1 hoặc 0 (thường là bit cao nhất hoặc thấp nhất), nên dữ liệu sẽ bị thay đổi để đảm bảo rằng số lượng bit "1" trong tất cả các bit truyền (bao gồm các bit của ký tự và bit kiểm tra) là số chẵn; trong kiểm tra lẻ, số lượng bit "1" trong tất cả các bit truyền là số lẻ. Kiểm tra chẵn lẻ có thể được sử dụng để bên nhận kiểm tra xem truyền tải có bị lỗi hay không, nếu số lượng "1" trong một byte xảy ra lỗi, thì byte đó chắc chắn đã có lỗi trong quá trình truyền. Nếu kiểm tra chẵn lẻ là đúng, thì hoặc là không có lỗi xảy ra, hoặc có một số lỗi chẵn. Nếu người dùng chọn độ dài dữ liệu là 8 bit, thì vì không có bit dư nào có thể được sử dụng làm bit kiểm tra, nên gọi là "không kiểm tra chẵn lẻ (Non)".

Bit dừng (Stop bits): Sau khi hoàn thành việc gửi từng byte dữ liệu, gửi bit dừng để đánh dấu việc truyền dữ liệu hoàn tất, đồng thời giúp phần cứng bên nhận đồng bộ lại. Có thể chọn: 1 (mặc định), 1.5 hoặc 2 bit.

Trong tiêu chuẩn RS-232, cấu Hình thường được sử dụng nhất là 8N1 (tức là tám bit dữ liệu, không có kiểm tra chẵn lẻ, một bit dừng), biểu đồ thời gian khi gửi một byte như Hình 6-2 dưới đây. Theo một byte hoàn chỉnh bao gồm một bit bắt đầu, 8 bit dữ liệu, một bit dừng, tổng cộng là mười bit dữ liệu, để thực hiện việc gửi mười bit này hoàn chỉnh, cần 11 xung đồng hồ tốc độ baud, xung đầu tiên đánh dấu bắt đầu một lần truyền, xung thứ 11 đánh dấu kết thúc một lần truyền

GOWIN VIETNAM

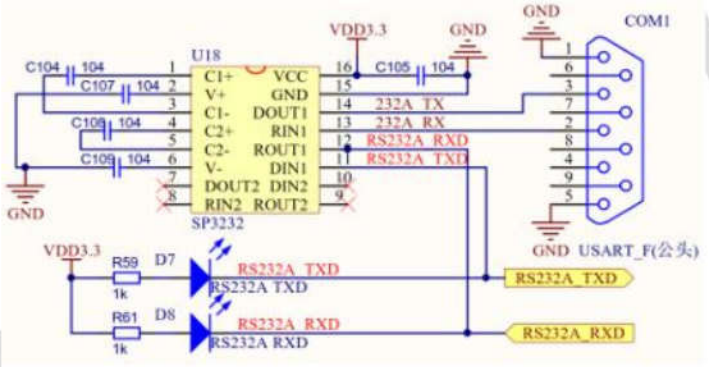


Hình 6-2 – Biểu đồ thời gian gửi 1 Byte của UART

Khi cạnh lên đầu tiên của tín hiệu bps_clk đến, mô-đun gửi byte bắt đầu gửi bit bắt đầu. Trong 2 đến 9 cạnh lên tiếp theo, gửi 8 bit dữ liệu, từ cạnh lên thứ 10 đến cạnh lên thứ 11 là việc gửi bit dừng.

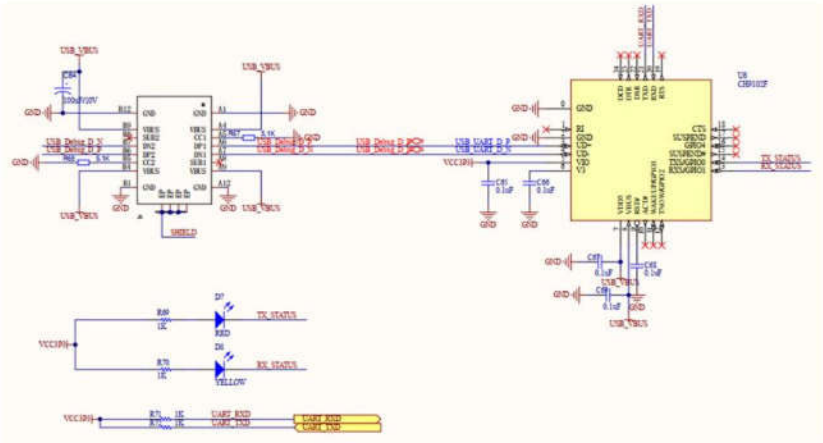
6.1.3 Thiết kế mạch giao tiếp RS232

Giao thức RS232 cần một số hỗ trợ phần cứng nhất định, trước đây hầu hết các giải pháp sử dụng là chuyển đổi RS232 sang TTL, lúc này cần các chip chuyển đổi mức như MAX232 hoặc SP3232 để thực hiện chuyển đổi dữ liệu. Mạch ngoại vi rất đơn giản, tối thiểu chỉ cần 4 tụ điện để hoạt động bình thường, sơ đồ mạch điển Hình như Hình dưới đây. Ở đây chỉ sử dụng một trong hai đường truyền, và thông qua việc thêm hai đèn LED D7, D8 có thể quan sát trạng thái dữ liệu tốt hơn.



Hình 6-3 – Mạch chuyển RS232 sang TTL

Hiện nay, độ tích hợp của hệ thống ngày càng cao, giao diện RS232 DB9 chiếm diện tích PCB quá lớn, hầu hết các hệ thống đã chuyển sang sử dụng USB chuyển đổi sang cổng nối tiếp, bo mạch phát triển ACG525 cũng sử dụng giải pháp USB chuyển đổi sang cổng nối tiếp, sơ đồ mạch của nó như Hình 6-4 dưới đây.



Hình 6-4 – Sơ đồ mạch cổng nối tiếp trên board ACG525

CH9102F là một chip chuyển đổi USB, thực hiện chuyển đổi USB sang cổng nối tiếp bất đồng bộ tốc độ cao. Nó cung cấp các tín hiệu liên lạc board mạch điều khiển thông dụng, dùng để mở rộng cổng nối tiếp bất đồng bộ cho máy tính, hoặc nâng cấp trực tiếp các thiết bị cổng nối tiếp thông thường hoặc MCU lên bus USB. Các đặc điểm của nó như sau:

1. Cổng thiết bị USB tốc độ cao, tương thích với USB V2.0.
2. Firmware tích hợp, mô phỏng cổng nối tiếp chuẩn, dùng để nâng cấp thiết bị ngoại vi cổng nối tiếp gốc, hoặc thêm cổng nối tiếp bổ sung qua USB.
3. Hỗ trợ hệ điều hành không cần cài đặt với driver CDC tích hợp sẵn hoặc driver VCP của nhà sản xuất với tốc độ cao đa chức năng.
4. Cổng nối tiếp phần cứng full-duplex, tích hợp bộ đệm gửi và nhận độc lập, hỗ trợ tốc độ truyền thông từ 50bps đến 4Mbps.
5. Cổng nối tiếp hỗ trợ 5, 6, 7 hoặc 8 bit dữ liệu, hỗ trợ kiểm tra chẵn, lẻ, không có và kiểm tra dấu hiệu.
6. Hỗ trợ các tín hiệu liên lạc MODEM thông dụng RTS, DTR, DCD, RI, DSR, CTS. Hỗ trợ điều khiển dòng tự động phần cứng CTS và RTS.
7. Hỗ trợ half-duplex, cung cấp trạng thái đang gửi TNOW hỗ trợ chuyển đổi RS485.
8. Thông qua các thiết bị chuyển đổi mức điện áp bổ sung, hỗ trợ giao tiếp RS232.
9. Cổng I/O CH9102F được cấp điện độc lập, hỗ trợ điện áp 5V, 3.3V, 2.5V và thậm chí 1.8V.
10. Cổng I/O CH9102X hỗ trợ tín hiệu 3.3V.
11. Tích hợp reset khi cấp điện, tích hợp đồng hồ, không cần dao động bên ngoài.
12. CH9102F tích hợp EEPROM, có thể cấu Hình các tham số như VID, PID, giá trị dòng tối đa, chuỗi thông tin nhà sản xuất và sản phẩm.
13. Chip tích hợp Unique ID (Số Serial USB).
14. Cung cấp kiểu chân QFN24 và QFN28 không chứa chì, tương thích với RoHS.

6.1.4 Cài đặt UART

Khi sử dụng cổng nối tiếp lần đầu tiên, chúng ta cần cài đặt driver trước, các bước thực hiện như sau:

1. Đầu tiên, truy cập vào trang web chính thức của công ty Nanjing Qinheng để tải và cài đặt phần mềm driver, trang web như sau: https://www.wch.cn/

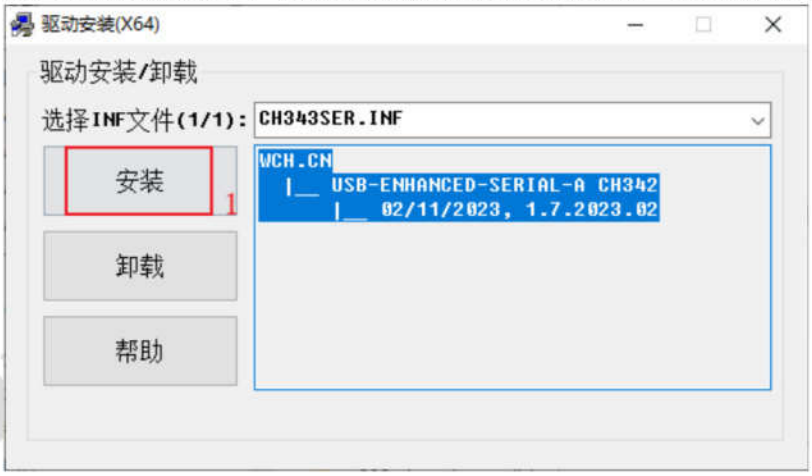
Sau khi vào trang web, tìm kiếm chip USB trên trang chủ, sau đó vào giao diện mới, lần lượt chọn USB chuyển đổi UART, CH9102. Nhấn vào đó để vào giao diện giới thiệu CH9102, cuộn xuống dưới, tìm CH343SER.EXE để tải về, toàn bộ các bước thực hiện như Hình 6-5 dưới đây.

Hình 6-5 – Tải driver cho giao tiếp nối tiếp

2. Nhấp đúp để mở phần mềm, nếu có hộp thoại cảnh báo an ninh xuất hiện, chỉ cần nhấn vào "Chạy" như Hình 6-6 bên dưới.

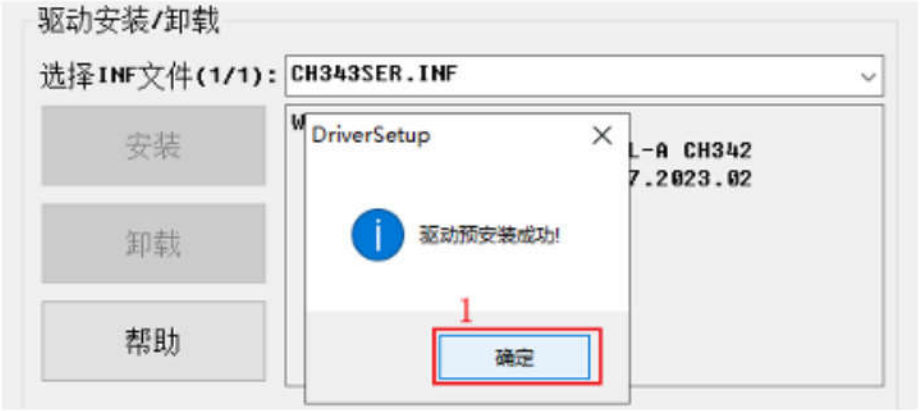
Hình 6-6 – Vận hành phần mềm

3. Sau khi mở, giao diện như Hình 6-7 bên dưới, chỉ cần nhấn vào "Cài đặt" là được.



Hình 6-7– Cài đặt driver

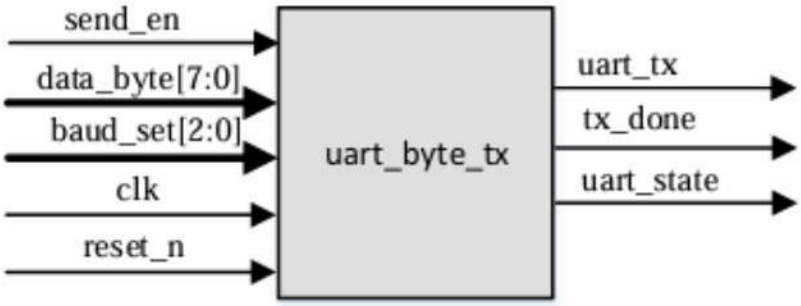
4. Chờ đợi quá trình cài đặt, sau khi cài đặt thành công, sẽ xuất hiện hộp thoại thông báo cài đặt thành công như Hình dưới đây. Lúc này, điều này có nghĩa là chúng ta đã cài đặt driver thành công, chỉ cần nhấn "Đồng ý" là được.



Hình 6-8 – Driver cài đặt thành công

6.2 Thiết kế và triển khai module giao tiếp nối tiếp không đồng bộ

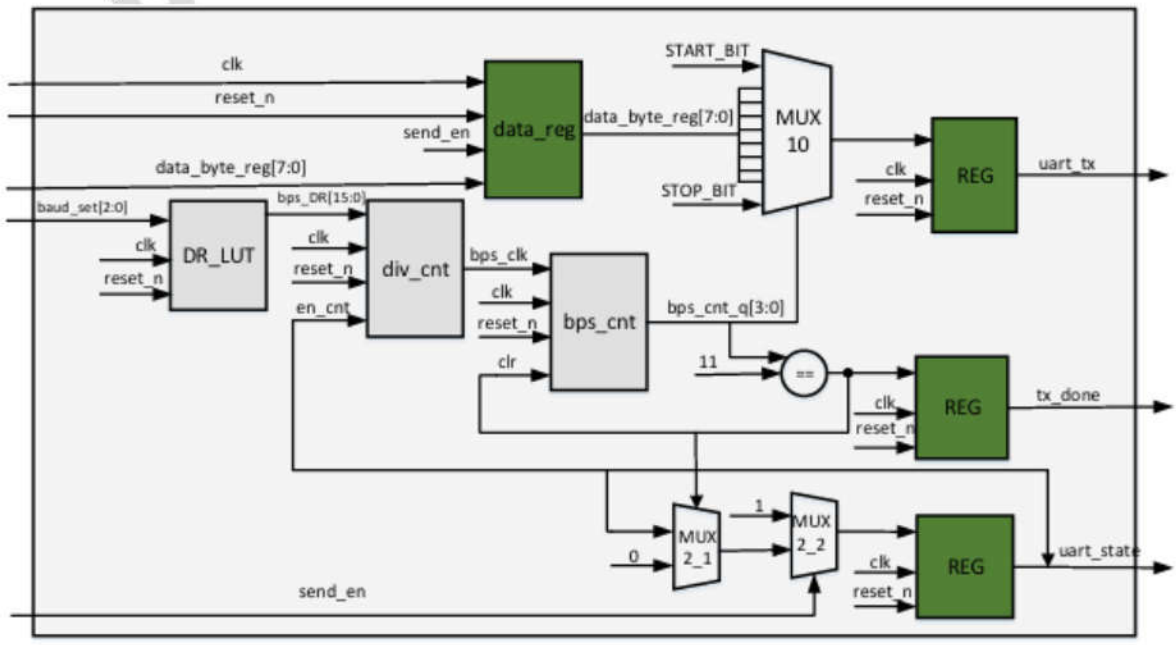
Dựa trên nguyên tắc trên, sơ đồ khối tổng thể của mô-đun truyền cổng nối tiếp sẽ được triển khai trong chương này được thể hiện trong Hình 6-9 bên dưới và danh sách giao diện được hiển thị trong Bảng 6-2 bên dưới



Hình 6-9 – Sơ đồ khối module UART

Tên tín hiệu	I/O	Mô tả chức năng
clk	I	Xung nhịp toàn cục của module (50MHz)
reset_n	I	Tín hiệu reset toàn cục của module (active low)
data_byte	I	Dữ liệu 8 bit cần truyền
send_en	I	Tín hiệu cho phép gửi dữ liệu
baud_set	I	Tín hiệu cài đặt tốc độ baud (tốc độ truyền)
uart_tx	O	Tín hiệu đầu ra truyền UART
tx_done	O	Tín hiệu báo hoàn thành gửi (mức cao trong 1 chu kỳ xung nhịp)
uart_state	O	Trạng thái truyền (mức 1 khi đang trong quá trình gửi)

Theo yêu cầu chức năng, mô-đun gửi qua cổng nối tiếp có thể được chi tiết hóa hơn nữa như Hình 6-10 dưới đây, trong đó vai trò của mỗi mô-đun con được trình bày trong bảng 6-3. Các khung màu xanh lá cây đại diện cho các thanh ghi của cấu trúc đơn, nhằm thực hiện việc nhập và xuất dữ liệu một cách ổn định.



Hình 6-10 – Sơ đồ cấu trúc dạng khối của mô đun truyền UART

Bảng 6-3 – Mô tả chức năng của từng khối trong module UART

Tên khối	Mô tả chức năng các khối
div_cnt	Tạo xung nhịp tốc độ baud (baud rate clock)
Bps_cnt	Đếm xung nhịp tốc độ baud
DR_LUT	Bảng lookup table, hỗ trợ các tốc độ baud khác nhau
MUX10	Chọn bit cần gửi dựa trên giá trị của bps_cnt
Toán tử "=="	Kiểm tra xem một lần truyền dữ liệu đã kết thúc hay chưa
MUX2	Tín hiệu kích hoạt (enable) của module

Giải thích nguyên lý hoạt động

a. Thiết lập tốc độ baud

- ***DR_LUT*** *nhận baud_set[2:0] → chọn giá trị bps_DR[15:0] tương ứng.*
- ***div_cnt*** *sử dụng bps_DR để chia tần số clk → tạo bps_clk (xung nhịp tốc độ baud).*

b. Quy trình truyền dữ liệu

1. ***Kích hoạt****: Khi send_en = 1, hệ thống bắt đầu truyền:*
- *Đặt START_BIT = 0 trên uart_tx.*
- *Reset bps_cnt về 0.*

2. ***Truyền từng bit****:*
- *Mỗi chu kỳ bps_clk, bps_cnt tăng 1.*
- ***MUX10*** *chọn bit cần gửi dựa trên bps_cnt:*
- *bps_cnt = 0: START_BIT (0).*
- *bps_cnt = 1–8: 8 bit dữ liệu từ data_byte_reg.*
- *bps_cnt = 9: STOP_BIT (1).*

3. ***Kết thúc truyền****:*
- *Khi bps_cnt = 10, đặt tx_done = 1 và trở về trạng thái idle.*

*Giả sử truyền giá trị 0x55 = 5*16 +5 +(01010101) = 85 Decimal*

1. ***Bit bắt đầu****: uart_tx = 0.*
2. ***8 bit dữ liệu****: Lần lượt là 1-0-1-0-1-0-1-0.*
3. ***Bit dừng****: uart_tx = 1.*

6.2.1 Thiết kê module tại tín hiệu xung clock

Từ phần nguyên lý có thể thấy, tốc độ baud là một trong những tham số cần thiết để thiết lập trong giao tiếp UART. Trong mô-đun tạo xung nhịp tốc độ baud, giá trị đếm cần thiết của bộ đếm và tốc độ baud có mối quan hệ như bảng 6-4 dưới đây, trong đó chu kỳ xung nhịp hệ thống là **System_clk_period**, ở đây là **20ns**. Nếu tần số xung nhịp kết nối vào mô-đun này là giá trị khác, cần phải điều chỉnh tham số này theo giá trị tần số cụ thể.

Bảng 6-4 – Tính toán baud rate

Baud_set	Baud rate	Baud period	Clock count calculation	Divisor Value
0	9600	104167ns	104167/System_clk_period	5208-1
1	19200	52083ns	52083/System_clk_period	2604-1
2	38400	26041ns	26041/ System_clk_period	1302-1
3	57600	17361ns	17361/ System_clk_period	868-1
4	115200	8680ns	8680/ System_clk_period	434-1

GOWIN VIETNAM

Giải thích chính:

1. *Tốc độ Baud: Tốc độ truyền thông nối tiếp (bit/giây)*
2. *Chu kỳ Baud: Thời gian cho một bit (1/baud_rate)*
3. *Tính số xung nhịp: Số chu kỳ xung hệ thống trong một chu kỳ baud*

- *Với xung 50MHz, Chu_kỳ_xung_hệ_thống = 20ns (1/50,000,000)*

4. *Giá trị chia: Giá trị lập trình vào phần cứng*

- *Công thức: (Số xung nhịp mỗi chu kỳ baud) - 1*
- *Ví dụ: 9600 baud: (104167ns / 20ns) - 1 = 5208-1*

Thiết kế của mô-đun này nhằm đảm bảo tính tái sử dụng của mô-đun. Khi cần các tốc độ baud khác nhau, chỉ cần thiết lập giá trị đếm của bộ đếm đồng hồ tốc độ baud khác nhau. Việc sử dụng bảng tra cứu có thể thực hiện được, mã thiết kế dưới đây chỉ bao gồm các thiết lập cho 5 tốc độ baud. Nếu cần các tốc độ baud khác, có thể sửa đổi cụ thể theo tình huống sử dụng thực tế.

Khái niệm về việc tạo ra tốc độ baud là sử dụng một bộ định thời để tạo ra tín hiệu đồng hồ có tần số tương đương với tần số đồng hồ của tốc độ baud tương ứng. Ví dụ, khi chúng ta sử dụng tốc độ baud 115200 bps, chúng ta cần tạo ra một tín hiệu đồng hồ có tần số 115200 Hz. Vậy làm thế nào để tạo ra tín hiệu đồng hồ 115200 Hz này? Đầu tiên, chúng ta sẽ tính chu kỳ của tín hiệu đồng hồ 115200 Hz. Một giây tương đương với 1.000.000.000 ns, vì vậy chu kỳ đồng hồ tốc độ baud Tb = 1000000000115200 = 8680.6*Tb*= 1152001000000000 = 8680.6ns, tức là một chu kỳ 115200bps là 8680.6ns. Chúng ta chỉ cần thiết lập thời gian định thời của bộ định thời là 8680.6 ns, mỗi khi thời gian định thời kết thúc, chúng ta sẽ tạo ra một xung cao có độ dài chu kỳ đồng hồ hệ thống. Tần số đồng hồ hệ thống là 50 MHz, tức là chu kỳ 20 ns, vì vậy chúng ta chỉ cần đếm 868020208680 chu kỳ đồng hồ hệ thống để có được thời gian 8680 ns. Công thức tính bps là bps=TbTclk−1=Tb·fclk−1=fclk115200−1*Ɩps=TclkTb*−1=*Tb·fclk*−1=115200*fclk*−1. Tương tự, các giá trị định thời cho các tốc độ baud khác cũng được tính toán như vậy.

Để có thể điều khiển tốc độ baud từ bên ngoài, trong thiết kế đã sử dụng một cổng chọn tốc độ baud 3 bit: baud_set. Bằng cách cung cấp các giá trị khác nhau cho cổng này, chúng ta có thể chọn tốc độ baud khác nhau. Nguyên lý điều khiển tốc độ baud của cổng này rất đơn giản, đó là một bộ chọn nhiều đường. Bộ chọn nhiều đường này thông qua việc chọn giá trị tối đa của bộ định thời khác nhau để thiết lập tần số đồng hồ bit khác nhau. Giá trị của baud_set tương ứng với các tốc độ baud như sau.

GOWIN VIETNAM

```
always@(posedge clk or posedge reset_n)
if(reset_n)
bps_DR <= 16'd5207;
else begin
case(baud_set)
0:bps_DR <= 16'd5207;
1:bps_DR <= 16'd2603;
2:bps_DR <= 16'd1301;
3:bps_DR <= 16'd867;
4:bps_DR <= 16'd433;
default:bps_DR <= 16'd5207;
endcase
end
always@(posedge clk or posedge reset_n)
if(reset_n)
div_cnt <= 16'd0;
else if(uart_state) begin
if(div_cnt == bps_DR)
div_cnt <= 16'd0;
else div_cnt <= div_cnt + 1'b1;
end
else
div_cnt <= 16'd0;
// bps_clk gen
always@(posedge clk or posedge reset_n)
if(reset) bps_clk <= 1'b0;
else if(div_cnt == 16'd1)
bps_clk <= 1'b1;
else bps_clk <= 1'b0;
```

Bảng 6-5 – Baud rate setting

Code	Baud Rate
000	9600 bps
001	19200 bps
010	38400 bps

6.2.2 Thiết kế modun đầu ra dữ liệu

Bằng cách đếm đồng hồ tốc độ baud, xác định trạng thái vòng lặp gửi dữ liệu, mã như sau

Để các mô-đun có thể kiểm soát hoặc gọi các mô-đun khác, một tín hiệu kết thúc truyền byte được

```
//bps counter
always@(posedge clk or posedge reset_n)
if(reset_n) bps_cnt <= 4'd0;
else if(bps_cnt == 4'd11)
                bps_cnt <= 4'd0;
else if(bps_clk)
        bps_cnt <= bps_cnt + 1'b1;
else bps_cnt <= bps_cnt;
```

phát sinh. Sau khi một bit dữ liệu được truyền xong, tín hiệu tx_done sẽ xuất ra một xung cao của đồng hồ

```
always@(posedge clk or posedge reset_n)
if(reset_n)
        tx_done <=1'b0;
else if(bps_cnt==4'd11)
        tx_done <= 1'b1;
else
        tx_done <=0;
```

Tạo tín hiệu trạng thái truyền dữ liệu, tức là khi đang truyền bình thường, tín hiệu uart_state ở mức cao, còn trong các trường hợp khác thì ở mức thấp. Cấu trúc mạch được thực hiện ở đây cũng có thứ tự ưu tiên, nhưng về bản chất thì khác với ngôn ngữ C. Trong Hình 6-10, MUX2_1 và MUX2_2 chính là cấu trúc mạch được thiết kế để thực hiện if—else if—else.

Do cổng nối tiếp là một bộ thu phát không đồng bộ, nên để đảm bảo dữ liệu gửi đi ổn định khi đồng hồ đến, cần phải lưu trữ dữ liệu đầu vào

```
always@(posedge clk or posedge reset_n)
if(reset_n)
        data_byte_reg<=8'd0;
else if(send_en)
        uart_state <= 1'b1;
else if (bps_counter==4'd11)
        uart_state <=0;
else
        uart_state=uart_state;
```

```
always@(posedge clk or posedge reset_n)
if(send_en)
        data_byte_reg<=data_byte;
else
        data_byte_reg<=data_byte_reg;
```

6.2.3 Mô-đun điều khiển trạng thái truyền dữ liệu

Trong sơ đồ cấu trúc mô-đun 6-10 còn có một bộ chuyển mạch đa kênh chọn một, có tác dụng xác định trạng thái truyền dữ liệu dựa trên giá trị của bps_cnt. Như biểu đồ thời gian 6-2 đã chỉ ra, tại các giá trị đếm đồng hồ baud rate khác nhau, sẽ có dữ liệu truyền tương ứng

Lúc này viết code đã hoàn thành, chúng ta tiến hành phân tích và tổng hợp thiết kế cho đến khi không còn lỗi.

```
always@(posedge clk or posedge reset_n)
  if(reset_n)
    uart_tx <= 1'b1;
  else begin
    case(bps_cnt)
      0: uart_tx <= 1'b1;
      1: uart_tx <= START_BIT;
      2: uart_tx <= data_byte_reg[0];
      3: uart_tx <= data_byte_reg[1];
      4: uart_tx <= data_byte_reg[2];
      5: uart_tx <= data_byte_reg[3];
      6: uart_tx <= data_byte_reg[4];
      7: uart_tx <= data_byte_reg[5];
      8: uart_tx <= data_byte_reg[6];
      9: uart_tx <= data_byte_reg[7];
      10: uart_tx <= STOP_BIT;
      default: uart_tx <= 1'b1;
    endcase
  end
```

6.3 Tạo testbench và mô phỏng

Nội dung phân tích từ sơ đồ nguyên lý được tạo ra ở trên và sơ đồ cấu trúc mô-đun mà chúng tôi thiết kế cơ bản là nhất quán, nhưng điều này không có nghĩa là chức năng của mô-đun thiết kế có thể đạt yêu cầu của chúng tôi. Điều này cần phải thực hiện mô phỏng chức năng của mô-đun thiết kế để xác nhận tính chính xác của chức năng mô-đun. Đầu tiên, thiết kế tệp testbench mô phỏng, trong tệp mô phỏng này tạo ra xung nhịp clk có chu kỳ 20ns làm xung nhịp làm việc của mô-đun cổng nối tiếp, sau đó tạo ra tín hiệu reset và tín hiệu enable, cùng với dữ liệu sẽ được truyền. Tại đây, tất cả sự thay đổi dữ liệu được lệch với xung nhịp hệ thống 1ns, nhằm để có thể nhìn rõ hơn mối quan hệ thời gian giữa dữ liệu đầu vào và đầu ra với xung nhịp.

```
initial clk = 1;
always #(`CLK_PERIOD/2) clk = ~clk;
initial begin
  reset_n = 1'b0;
  data_byte = 8'd0;
  send_en = 1'd0;
  baud_set = 3'd4;
    #(`CLK_PERIOD*500 + 1);
  reset_n = 1'b1;
  #(`CLK_PERIOD*50);
    // Send first byte
  data_byte = 8'haa;
  send_en = 1'd1;
  #`CLK_PERIOD;
  send_en = 1'd0;
    @(posedge tx_done);
  #(`CLK_PERIOD*5000);
    // Send second byte
  data_byte = 8'h55;
  send_en = 1'd1;
  #`CLK_PERIOD;
  send_en = 1'd0;
    @(posedge tx_done);
  #(`CLK_PERIOD*5000);
  $stop;
      End
```

Sau đó, bật chức năng Tạo Modelsim và lấy dạng sóng file được hiển thị trong Hình 6-11.

Có thể thấy rằng uart_tx tín hiệu đầu ra là 1 trước khi reset và kích hoạt active và tín hiệu đầu ra chỉ bắt đầu gửi một nhóm dât UART sau khi kết thúc reset và bật, và khi dữ liệu được gửi được 0xaa, tín hiệu đầu ra nối tiếp là 1, 0 (bit bắt đầu), 01010101b (LSB), 1 (bit dừng) lần lượt; Khi dữ liệu được gửi được 0x55, tín hiệu đầu ra là 1, 0 (bit bắt đầu), 10101010b (LSB), 1 (bit dừng). Đồng thời, trong quá trình gửi data uart_state là 1 khi nó ở trạng thái gửi, và sau khi mỗi byte data được gửi, một xung chu kỳ clk được tạo ra tx_done, phù hợp với kỳ vọng thiết kế và mô phỏng vượt qua.

Hình 6-11 – Dạng sóng mô phỏng truyền dữ liệu 1 byte

6.4 Module Kiểm tra UART

Trước khi chúng ta thực hiện xác minh trên board cho mô-đun bộ truyền UART một byte, chúng ta cần viết một mô-đun kiểm tra để kiểm soát thời điểm gửi một byte gửi data và nội dung gửi.

```
module uart_tx_test(
  input clk,
   input reset_n,
   output uart_tx,
  output led
);
  parameter MCNT = 49_999_999;   //Định thời gian 1 giây
  reg [7:0] data_byte;
  reg send_en;
  // Thiết kế định bộ thời gửi dữ liệu đều đặn
  reg [25:0]cnt;  // định nghĩa thanh ghi bộ đếm
  wire tx_done;
  // tiến trình bộ đếm
  always@(posedge clk or negedge reset_n)
  if(!reset_n)
    cnt <= 25'd0;
  else if(cnt == MCNT)
    cnt <= 25'd0;
  else
    cnt <= cnt + 1'b1;
  always@(posedge clk or negedge reset_n)
  if(!reset_n)
    send_en <= 1'b0;
  else if(cnt == MCNT)
    send_en <= 1'b1;
  else
    send_en <= 1'b0;
  always@(posedge clk or negedge reset_n)
 if(!reset_n)
    data_byte <= 8'b0;
  else if(tx_done)
    data_byte <= data_byte + 1'b1;
  else
    data_byte <= data_byte;
    uart_byte_tx uart_byte_tx(
    .clk(clk),
    .reset_n(reset_n),

    .data_byte(data_byte),
    .send_en(send_en),
    .baud_set(3'd0),
          .uart_tx(uart_tx),
    .tx_done(tx_done),
    .uart_state(led)
  );
Endmodule
```

Ở đây ý tưởng thiết kế của chúng tôi là: thông qua bộ đếm, thời gian đếm là 1S, khi hết thời gian đếm, tín hiệu send_en được tạo ra, khi gửi xong, sau khi tx_done được tạo ra, dữ liệu cần được gửi data_byte tự cộng 1, để chúng ta có thể quan sát thấy rằng cổng nối tiếp gửi một byte dữ liệu samỗi 1S và dữ liệu tăng dần, baud_set được đặt thành 3'd0, tốc độ truyền tương ứng là 9600 và cổng nối tiếp gửi thử nghiệm Mã cho mô-đun trông như thế này:

6.5 Xác thực chương trình trên phần cứng

Việc xác minh cấp hội đồng của thí nghiệm này chủ yếu xác minh các mục tiêu sau:

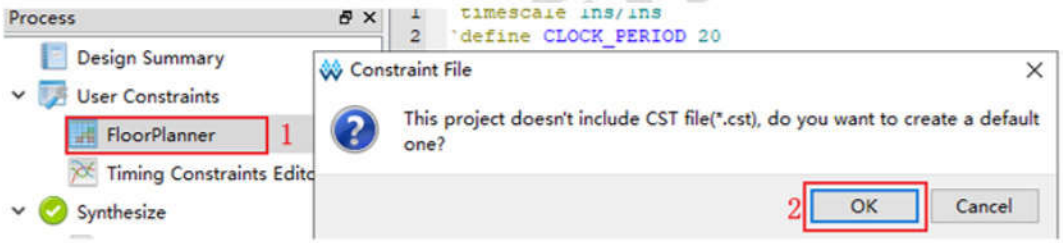
1. Liệu tệp FS được tạo có thể được tải xuống bảng phát triển một cách chính xác hay không.
2. Mở trợ lý gỡ lỗi cổng nối tiếp trên máy tính và bạn có thể thấy rằng dữ liệu nhận được mỗi 1S bắt đầu tăng từ 00.

Phần cứng cần thiết cho hệ thống:

1. Bảng phát triển ACG525.
2. Trình tải xuống Gowin EDA và cáp tải xuống.
3. Nguồn điện 12V.
4. Một cáp Type-C.
5. Một PC với đầy đủ các chức năng phát triển và điều kiện phần cứng đáp ứng các yêu cầu thử nghiệm.

6.5.1 Thêm ràng buộc I/O

Trong phần mềm Gowin, chọn khung **Process** nhấp vào **FloorPlanner** tại file .cst mới, như Hình 6-12 bên dưới



Hình 6-12– Tạo file ràng buộc I/O mới

Trong cửa sổ **FloorPlanner**, nhấp vào **I/O Constraints** để vào cửa sổ I/O。 gán chân theo sơ đồ, như Hình 6-13 bên dưới

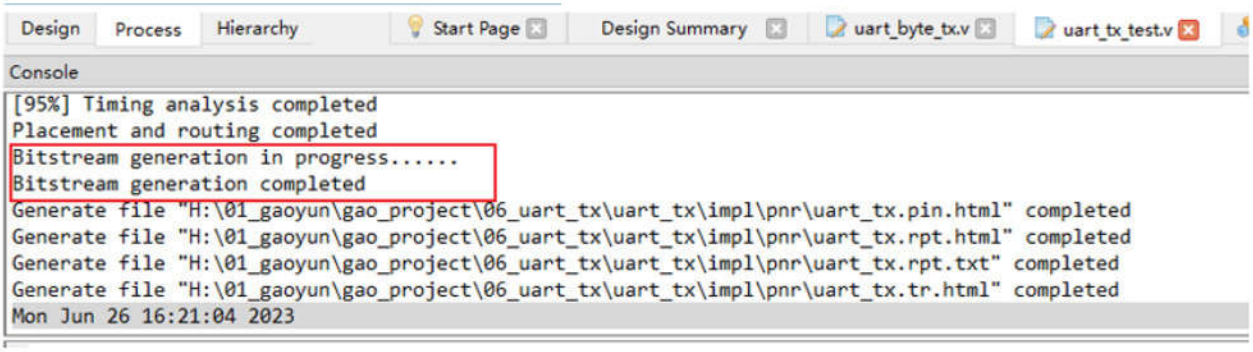
	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type
1	clk	input		T9	4	False	LVCMOS33
2	led	output		V12	4	False	LVCMOS33
3	reset_n	input		B16	1	False	LVCMOS33
4	uart_tx	output		V8	5	False	LVCMOS33

Hình 6-13– Gán chân

Điều này hoàn thành việc bổ sung ràng buộc pin. Tuy nhiên, tại thời điểm này, nội dung ràng buộc được lưu trong bộ nhớ và chưa được ghi vào tệp, hãy nhấp vào nút **Save** trên thanh công cụ hoặc trực tiếp Ctrl+S.

6.5.2 Place & Route

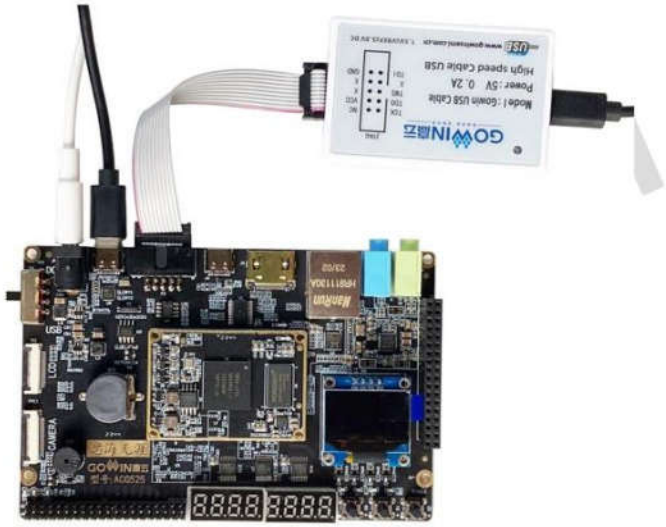
Nhấp vào **Process** chọn **Place & Route** rồi **run**, sau khi Place & Route thành công sẽ tạo ra file fs và console nắc như Hình 6-14 bên dưới,



Hình 6-14 – Tạo thành công file fs

6.5.3 Kết nối phần cứng

Lần lượt kết nối mạch nạp, nguồn điện và cáp nối tiếp với bo mạch phát triển và các kết nối phần cứng tổng thể được hiển thị trong Hình 5-26 bên dưới. Nó có thể được cấp nguồn bằng nguồn điện hoặc **USB** và người dùng có thể chọn làm như vậy.



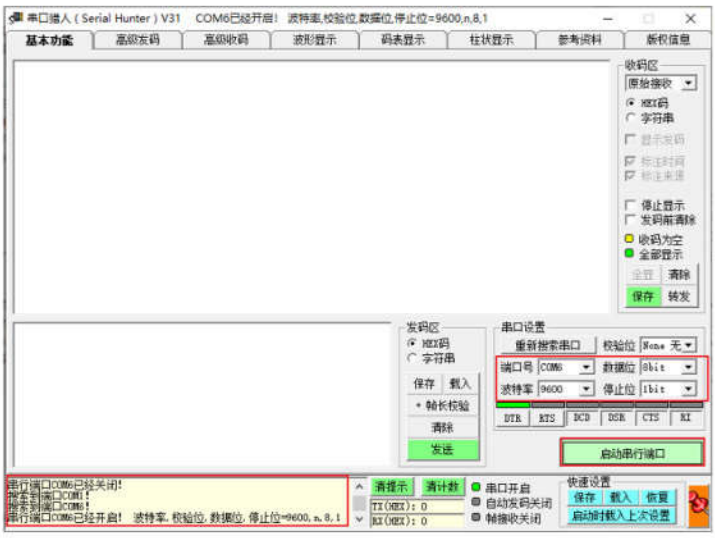
Hình 6-15 Kết nối phần cứng

6.5.4 Cấu Hình Port kết nối cho bộ debugger

Mở phần mềm cổng nối tiếp trên PC, sử dụng trình săn cổng nối tiếp tại đây, đặt khẩu hiệu nối tiếp tương ứng của bo mạch phát triển trên phần mềm cổng nối tiếp, bạn có thể mở trình quản lý thiết bị và kiểm tra số cổng (bo mạch phát triển cần được bật nguồn), như trong Hình 6-16 bên dưới.

Hình 6-16 – Chọn port tương thích

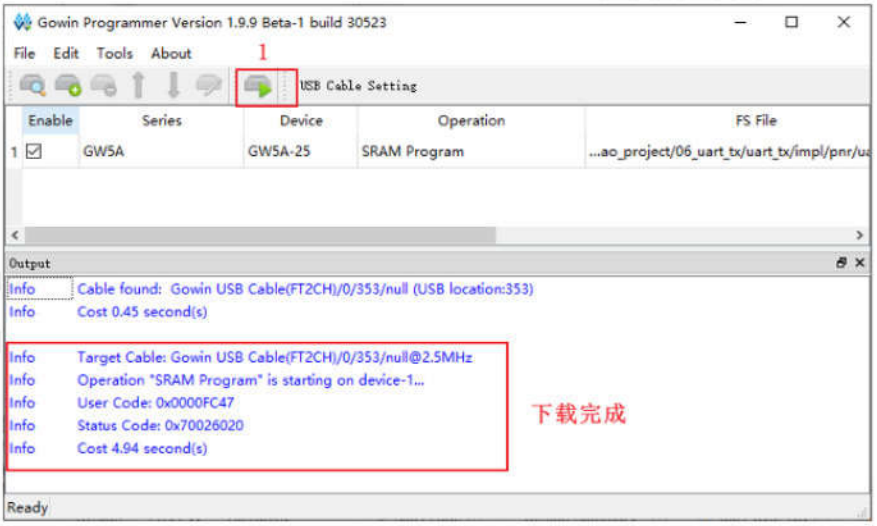
Sau đó đặt tốc độ truyền thành 9600, bit dữ liệu 8bit, bit dừng 1bit, sau đó nhấp để khởi động cổng nối tiếp, bạn có thể thấy tình huống kết nối cụ thể trong cửa sổ thông tin in bên dưới phần mềm cổng nối tiếp, như trong Hình 6-17 bên dưới



Hình 6-17 – Giao diện phần mềm Serial Hunter cài đặt Port COM

6.5.5 Tải Fle fs

Nhấp chuột vào **Program Device**, vào cửa sổ **programmer**, phần mềm bật lên cửa sổ Cable Setting, nhấp vào Save, sau cùng nhấp vào tải xuống file fs tải thành công sẽ hiển thị cửa sổ như Hình 6-18.



Hình 6-18 – Tải file fs thành công

Sau khi chương trình hoàn thành, sau khi chạy một lúc, bạn có thể thấy rằng trợ lý cổng nối tiếp nhận dữ liệu tăng dần từ 00, như trong Hình 6-19 bên dưới, có thể thấy rằng kỳ vọng thiết kế được đáp ứng, dữ liệu nhận và gửi là nhất quán.

Hình 6-19 – Cửa sổ Hunter nhận dữ liệu truyền nối tiếp

6.6 Tổng kết chương

Trong chương này, chúng ta tìm hiểu cách phân loại và nguyên tắc của UART, giới thiệu định dạng dữ liệu của giao thức UART và ý nghĩa của các thông số liên quan, đồng thời thiết kế mạch phần cứng của cổng nối tiếp, đồng thời điều khiển dữ liệu và tín hiệu điều khiển được gửi bởi cổng nối tiếp bằng cách lập trình tệp thử nghiệm trong quá trình gỡ lỗi cấp bảng, để nhận ra chức năng FPGA gửi dữ liệu được hiển thị trên máy tính nối tiếp.

GOWIN VIETNAM

Chương 7: Thiết kế module nhận dữ liệu qua cổng nối tiếp

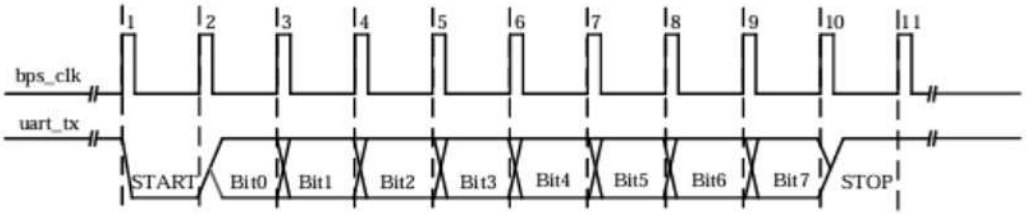
Mã nguồn bài thí nghiệm	|----Bài thí nghiệm số 2 |----ch7_uart_rx

Dẫn nhập chương

Chương này chúng ta sẽ học phần thiết kế tiếp nhận dữ liệu UART. Để đối phó với ảnh hưởng của nhiễu điện từ mạnh trong quá trình sử dụng thực tế, phần này đề xuất một phương pháp thiết kế mô-đun nhận dữ liệu qua cổng nối tiếp cải tiến.

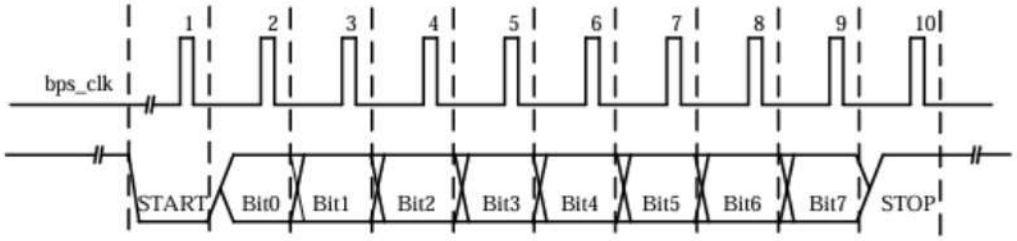
7.1 Phân tích nguyên lý nhận data qua UART

Trong phần trước, chúng ta đã học về thiết kế và thực hiện mô-đun gửi dữ liệu qua cổng nối tiếp. Đồ thị thời gian khi đầu gửi UART gửi một byte dữ liệu như Hình 7-1 dưới đây



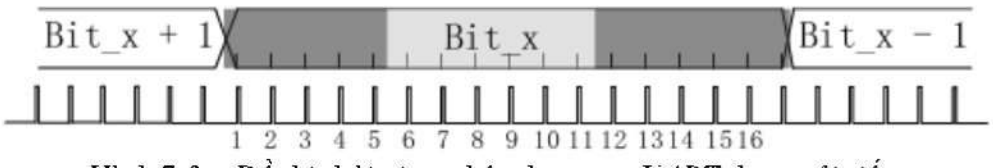
Hình 7-1– Đồ thị thời gian gửi data qua UART

Bài giảng này giới thiệu về thiết kế và thực hiện mô-đun nhận dữ liệu qua cổng nối tiếp. Khi lấy mẫu từng bit trên đường dữ liệu, thông thường, người ta cho rằng điểm giữa của mỗi bit dữ liệu là ổn định nhất. Do đó, trong các ứng dụng thông thường, mức điện áp tại thời điểm giữa được coi là mức điện áp của bit đó, như Hình 7-2 dưới đây



Hình 7-2 – Đồ thị thời gian nhận data qua UART

Tuy nhiên, trong ứng dụng công nghiệp thực tế, thường có nhiễu điện từ rất mạnh tại hiện trường, việc chỉ lấy mẫu một lần để xác định trạng thái điện áp của dữ liệu là không đáng tin cậy. Có khả năng cao là tín hiệu bị nhiễu sẽ được thu thập, dẫn đến kết quả sai. Do đó, ở đây đề xuất sơ đồ cách tiếp nhận dữ liệu single-bit cải tiến, sử dụng phương pháp lấy mẫu nhiều lần để xác định trạng thái theo xác suất, như Hình 7-3 dưới đây.



Hình 7-3 – Đồ thị thời gian nhận data qua UART được cải tiến

Trong hình trên, mỗi bit dữ liệu được chia thành 16 đoạn nhỏ. Đối với bit_x, trong khoảng thời gian vừa xảy ra thay đổi và sắp xảy ra thay đổi, dữ liệu rất có thể không ổn định (hai đoạn được đánh dấu màu xám đậm). Khi thu thập dữ liệu trong hai khoảng thời gian này, rất có khả năng sẽ nhận được kết quả sai, vì vậy mức điện áp trong hai khoảng thời gian này được coi là không hợp lệ và sẽ bị bỏ qua. Trong khi đó, khoảng thời gian ở giữa (được đánh dấu màu xám nhạt) thì dữ liệu tương đối ổn

định và thường đại diện cho kết quả chính xác. Đây chính là phương pháp đo lường ở giữa đã đề cập trước đó, nhưng cũng không loại trừ khả năng đoạn dữ liệu này bị ảnh hưởng bởi nhiễu điện từ mạnh, dẫn đến mức điện áp sai. Do đó, đối với đoạn điện áp này, sẽ tiến hành lấy mẫu nhiều lần và tính xác suất của mức điện cao và thấp. Trong 6 lần thu thập kết quả, sẽ lấy mức điện xuất hiện nhiều lần hơn làm kết quả mẫu. Ví dụ, nếu kết quả lấy mẫu 6 lần là 1/1/1/1/0/1, thì mức điện được chọn là 1; nếu là 0/0/1/0/0/0, thì mức điện được chọn là 0. Khi kết quả lấy mẫu 6 lần mà 1 và 0 mỗi loại chiếm một nửa (mỗi loại 3 lần), thì có thể kết luận rằng môi trường đường truyền hiện tại rất xấu, dữ liệu không đáng tin cậy và sẽ không được xử lý.

7.2 Thiết kế avaf triển khai module nhận data qua UART

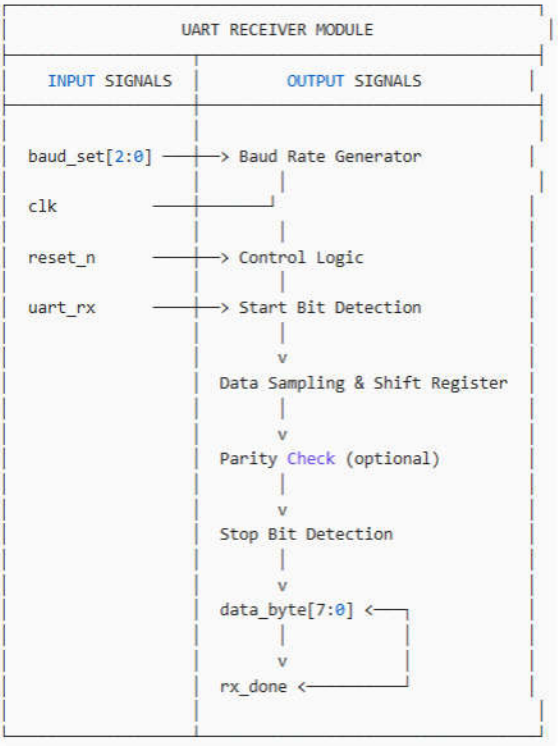
7.2.1 Sơ đồ khối module nhận data Uart

Dựa trên nguyên lý trên, sơ đồ khối tổng thể module nhận data được thể hiện như Hình 7-4 bên dưới

Hình 7-4 – Sơ đồ khối module nhận data qua UART

Bảng 7-1 – Mô tả danh sách tín hiệu của module

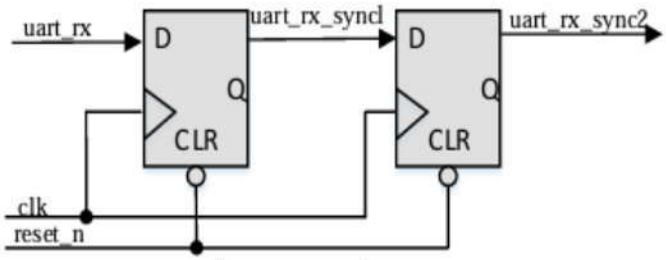
Tên tín hiệu	I/O	Mô tả chức năng
Clk	I	Xung clk của module 50Mhz
reset_n	I	Tín hiệu reset mức thấp
uart_rx	I	Khối nhận data
baud_set	I	Khối chọn tốc độ truyền
data_byte	O	Đầu ra dữ liệu song song



GOWIN VIETNAM

7.2.2 Thiết kế đồng bộ tín hiệu UART đơn BIT

Tín hiệu uart_rx nhận từ cổng nối tiếp ở đây là một tín hiệu không đồng bộ so với tín hiệu bên trong FPGA. Nếu không xử lý và sử dụng trực tiếp, dễ xảy ra vi phạm thời gian dẫn đến trạng thái bất ổn. Do đó, cần phải đồng bộ tín hiệu vào miền đồng hồ của FPGA trước khi có thể sử dụng cho các mô-đun tiếp theo. Phương pháp đồng bộ phổ biến là sử dụng hai cấp bộ nhớ, tức là sử dụng bộ nhớ để đồng bộ tín hiệu với đồng hồ hệ thống bằng cách đánh hai nhịp. Mạch tham khảo như Hình 7-5 dưới đây. Trong đó, uart_rx là tín hiệu đầu vào cổng nối tiếp không đồng bộ, uart_rx_sync2 là tín hiệu đã được đồng bộ.



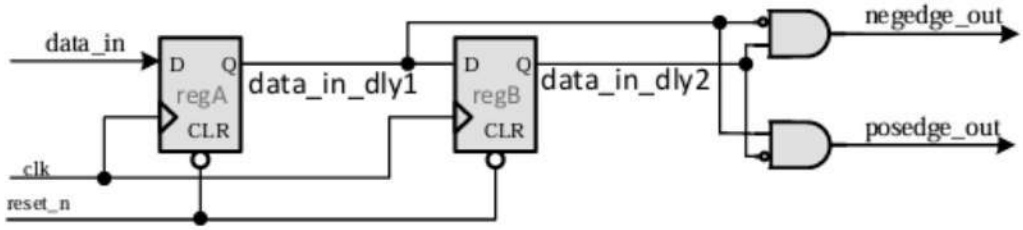
Hình 7-5 – Sơ đồ module đồng bộ tín hiệu đơn bit

```
always@(posedge clk or posedge reset_n)
if(reset_n)begin
uart_rx_sync1 <= 1'b1;
uart_rx_sync2 <= 1'b1;
end
else begin
uart_rx_sync1 <= uart_rx;
uart_rx_sync2 <= uart_rx_sync1;
end
```

Trong một số thiết kế tốc độ cao, cũng có thiết kế sử dụng ba cấp bộ nhớ để đồng bộ tín hiệu đơn Bit. Phương pháp này chỉ nhằm mục đích nâng cao thời gian trung bình giữa các lần hỏng (Mean Time Between Failure, MTBF), mạch của nó là kết nối một cấp bộ nhớ sau hai cấp bộ nhớ.

7.2.3 Thiết kế phát hiện cạnh

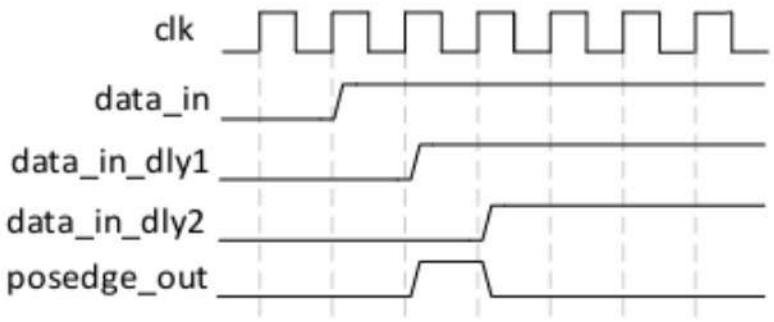
Từ sơ đồ thời gian nhận dữ liệu qua cổng nối tiếp cho thấy, tín hiệu bắt đầu nhận dữ liệu là khi dữ liệu tuần tự chuyển từ mức cao không hoạt động sang mức thấp, tức là mức điện áp chuyển từ cao xuống thấp, điều này yêu cầu phải phát hiện cạnh xuống. Nguyên lý của việc này là sử dụng các thanh ghi dưới sự điều khiển của tín hiệu đồng hồ, trạng thái đầu vào sẽ là trạng thái đầu ra của thời điểm tiếp theo, như được minh họa trong Hình 7-6 dưới đây.



Hình 7-6 – Sơ đồ nguyên lý phát hiện cạnh

Trong quá trình hoạt động, có thể giả định rằng data_in thay đổi từ 0 sang 1, tức là có 1 cạnh tăng và sơ đồ như Hình 7-7 bên dưới

GOWIN VIETNAM

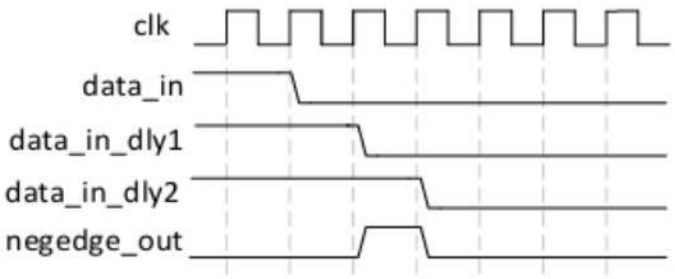


Hình 7-7 – Dạng sóng hoạt động khi phát hiện cạnh tăng

Tín hiệu đầu ra của Flipflop đầu tiên thanh ghi RegA là data_in_dly1 chính là tín hiệu data_in trễ đi 1 chu kỳ xung clock.

Tín hiệu đầu ra của Flipflop thứ 2, thanh ghi RegB là data_in_dly2 chính là tín hiệu data_in trễ đi 2 chu kỳ và data_in_dly1 trễ đi 1 chu kỳ xung clock. Cạnh lên có thể phát hiện ra bằng cách xét tương quan logic của (data_in_dly1 & ! data_in_dly2).

Tương tự đối với cạnh xuống, tín hiệu data_in sẽ chuyển từ 0 sang 1, sơ đồ thời gian như Hình 7-8 bên dưới



Hình 7-8 – Dạng sóng phát hiện cạnh xuống

Cạnh xuống có thể được phát hiện bằng cách thực hiện một phép toán logic tổ hợp trên hai đầu ra thanh ghi (!data_in_dly1 & data_in_dly2).

Thông qua phương pháp Delays như trên ta có thể nhận ra rằng khi có cạnh lên thì ngõ ra posedge_out sẽ lên mức cao trong 1 chu kỳ xung đồng và khi có cạnh xuống thì ngõ ra negedge_out sẽ lên mức cao trong 1 chu kỳ xung clock. Chú ý, ký hiệu ‘’!” được sử dụng ở trên là phép toán logic phủ định

Sau khi thực hiện phép toán logic phủ định trên 4'b0110, kết quả là 4'b0000; trong khi đó, “~” là phép toán đảo bit, áp dụng trên 4'b0110 sẽ cho kết quả 4'b1001. Đối với dữ liệu đơn bit, kết quả khi sử dụng “!” và “~” là giống nhau. Mô-đun nhận dữ liệu nối tiếp cần phát hiện cạnh xuống để nhận biết bit bắt đầu của tín hiệu nhận dữ liệu nối tiếp. Để phát hiện cạnh xuống của dữ liệu đầu vào nối tiếp đã được đồng bộ, verilog code cụ thể như sau.

```
always@(posedge clk or posedge reset)
if(reset_n)begin
uart_rx_reg1 <= 1'b0;
uart_rx_reg2 <= 1'b0;
end
else begin
uart_rx_reg1 <= uart_rx_sync2;
uart_rx_reg2 <= uart_rx_reg1;
```

```
end
assign
uart_rx_nedge = !uart_rx_reg1 & uart_rx_reg2;
uart_rx_posedgge=uart_rx1&!uart_rx_reg2;
```

7.2.4 Thiết kế module tạo xung clk cho khối

Mô-đun nhận dữ liệu nối tiếp là một trong những thành phần chính, bao gồm mô-đun tạo xung nhịp tốc độ baud. Tại đây, dựa trên phương pháp lấy mẫu quá mức đã đề cập trong phần nguyên lý của chương này, tức là tần số lấy mẫu thực tế gấp 16 lần tốc độ baud, ta có thể xác định mối quan hệ giữa giá trị đếm và tốc độ baud như được thể hiện trong bảng 7-2 về tính toán xung nhịp lấy mẫu, trong đó chu kỳ xung nhịp hệ thống là System_clk_period, ở đây là 20ns.

Bảng 7-2 – Tính toán tần số xung clk

Baud rate	Baud period	Clock count calculation	System_clk_period = 20ns
9600	104167ns	104167/System_clk_period	325-1
19200	52083ns	52083/System_clk_period	163-1
38400	26041ns	26041/ System_clk_period	81-1
57600	17361ns	17361/ System_clk_period	54-1
115200	8680ns	8680/ System_clk_period	27-1

Một lần nữa, một bộ chọn được sử dụng ở đây để khớp các tốc độ truyền khác nhau với các giá trị đếm phân chia xung clk. Verilog code thiết kế được hiển thị bên dưới.

```
always@(posedge clk or posedge reset_n)
if(reset_n)
bps_DR <= 16'd324;
else begin
case(baud_set)
0:bps_DR <= 16'd324;
1:bps_DR <= 16'd162;
2:bps_DR <= 16'd80;
3:bps_DR <= 16'd53;
4:bps_DR <= 16'd26;
default:bps_DR <= 16'd324;
endcase
end
```

Bây giờ tạo ra một đồng hồ mẫu, gấp 16 lần đồng hồ tốc độ truyền.

```
//counter
always@(posedge clk or posedge reset)
if(reset)
   div_cnt <= 16'd0;
else if(uart_state)begin
   if(div_cnt == bps_DR)
      div_cnt <= 16'd0;
   else
```

GOWIN VIETNAM

```
        div_cnt <= div_cnt + 1'b1;
    end
    else
      div_cnt <= 16'd0;

    // bps_clk gen
    always@(posedge clk or posedge reset)
    if(reset)
      bps_clk <= 1'b0;
    else if(div_cnt == 16'd1)
      bps_clk <= 1'b1;
    else
      bps_clk <= 1'b0;
```

Bộ đếm xung nhịp lấy mẫu, điều kiện để làm sạch bộ đếm là bps_cnt == 8'd159, đại diện cho việc nhận một byte đã hoàn tất. (bps_cnt == 8'd12 && (START_BIT > 2)) là điều kiện để kiểm tra xem có lỗi trong việc phát hiện bit bắt đầu hay không, điều này sẽ được giải thích chi tiết ở phần sau

```
    //bps counter
    always@(posedge clk or posedge reset)
    if(reset)
      bps_cnt <= 8'd0;
    else if(bps_cnt == 8'd159 | (bps_cnt == 8'd12 && (START_BIT > 2)))
      bps_cnt <= 8'd0;
    else if(bps_clk)
      bps_cnt <= bps_cnt + 1'b1;
    else
      bps_cnt <= bps_cnt;

    always@(posedge clk or posedge reset)
    if(reset)
      rx_done <= 1'b0;
    else if(bps_cnt == 8'd159)
      rx_done <= 1'b1;
    else
    rx_done <= 1'b0;
```

7.2.5 Thiết kế module nhận data

Lấy ví dụ về bit bắt đầu trong Hình 7-3, các giá trị bps_cnt tương ứng với các khoảng thời gian lấy mẫu ở giữa lần lượt là 6, 7, 8, 9, 10, 11. Tại những thời điểm này, dữ liệu của bit hiện tại được cộng dồn trực tiếp. Sau đó, giá trị bps_cnt đầu tiên của khoảng thời gian lấy mẫu dữ liệu tiếp theo sẽ là giá trị bps_cnt đầu tiên của khoảng thời gian lấy mẫu dữ liệu trước đó cộng thêm 16. Do đó, giá trị bps_cnt của khoảng thời gian lấy mẫu của bit dữ liệu đầu tiên Bit0 ngay sau bit bắt đầu lần lượt là 22, 23, 24, 25, 26, 27, và tại những thời điểm này, dữ liệu của bit hiện tại cũng được cộng dồn trực tiếp.

GOWIN VIETNAM

Tiếp tục như vậy, ta có thể nhận được giá trị bps_cnt tương ứng với các khoảng thời gian lấy mẫu của các bit khác.

Bây giờ, giải thích lý do tại sao một trong những điều kiện làm sạch ở trên là (bps_cnt == 8'd12 && (START_BIT > 2)). Trong trường hợp lý tưởng (bit bắt đầu thực sự), khi giá trị đếm bps_cnt là 12, giá trị tính toán của START_BIT nên là 0. Tuy nhiên, trong thực tế, có thể xuất hiện một tín hiệu nhiễu, không phải tín hiệu bắt đầu thực sự, cũng dẫn đến sự xuất hiện của cạnh xuống. Nếu không có sự kiểm tra, mà trực tiếp coi đó là tín hiệu bắt đầu và vào trạng thái nhận, thì chắc chắn sẽ nhận được dữ liệu sai, và có thể bỏ lỡ việc nhận dữ liệu đúng.

Để tăng cường khả năng chống nhiễu, tại đây đã áp dụng một cách tiếp cận như sau: khi có cạnh xuống xuất hiện trên đường dữ liệu, trước tiên giả định rằng đó là tín hiệu bắt đầu, sau đó tiến hành lấy mẫu trong khoảng thời gian giữa. Nếu tổng kết quả của 6 lần lấy mẫu lớn hơn 2, tức là trong 6 lần lấy mẫu có ít nhất một nửa trạng thái là mức cao, thì rõ ràng điều này không phù hợp với yêu cầu về mức thấp liên tục của tín hiệu bắt đầu thực sự. Lúc này, cạnh xuống vừa đến sẽ được coi là tín hiệu nhiễu và không được xem là tín hiệu bắt đầu.

```
//Phần reset
Khi tín hiệu reset tích cực (high), tất cả các thanh ghi được khởi tạo về 0.
START_BIT, data_byte_pre[0:7], và STOP_BIT đều là các thanh ghi 3-bit (vì được khởi tạo bằng 3'd0).
always@(posedge clk or posedge reset_n)
if(reset_n)begin
START_BIT <= 3'd0;
data_byte_pre[0] <= 3'd0;
data_byte_pre[1] <= 3'd0;
data_byte_pre[2] <= 3'd0;
data_byte_pre[3] <= 3'd0;
data_byte_pre[4] <= 3'd0;
data_byte_pre[5] <= 3'd0;
data_byte_pre[6] <= 3'd0;
data_byte_pre[7] <= 3'd0;
STOP_BIT <= 3'd0;
end
else if(bps_clk)begin
case(bps_cnt)
0:begin // Reset các thanh ghi khi bắt đầu một frame mới
START_BIT <= 3'd0;
data_byte_pre[0] <= 3'd0;
data_byte_pre[1] <= 3'd0;
data_byte_pre[2] <= 3'd0;
data_byte_pre[3] <= 3'd0;
 data_byte_pre[4] <= 3'd0;
        data_byte_pre[5] <= 3'd0;
        data_byte_pre[6] <= 3'd0;
        data_byte_pre[7] <= 3'd0;
```

GOWIN VIETNAM

```
            STOP_BIT <= 3'd0;
        end
```

//Mỗi bit UART được lấy mẫu **6 lần** (ví dụ: bit start được lấy mẫu ở các giá trị bps_cnt = 6,7,8,9,10,11).

Giá trị uart_rx_sync2 (tín hiệu UART đã đồng bộ hóa) được cộng dồn vào thanh ghi tương ứng. Sau 6 lần lấy mẫu, giá trị của thanh ghi sẽ nằm trong khoảng 0-6. Nếu giá trị >= 4, bit đó được coi là '1'; ngược lại là '0'

```
        6 ,7 ,8 ,9 ,10,11:START_BIT <= START_BIT + uart_rx_sync2;
        22,23,24,25,26,27:data_byte_pre[0]<=data_byte_pre[0]+uart_rx_sync2;
        38,39,40,41,42,43:data_byte_pre[1] <= data_byte_pre[1] + uart_rx_sync2;
        54,55,56,57,58,59:data_byte_pre[2] <= data_byte_pre[2] + uart_rx_sync2;
        70,71,72,73,74,75:data_byte_pre[3] <= data_byte_pre[3] + uart_rx_sync2;
        86,87,88,89,90,91:data_byte_pre[4] <= data_byte_pre[4] + uart_rx_sync2;
        102,103,104,105,106,107:data_byte_pre[5]<=data_byte_pre[5]+uart_rx_sync2;
        118,119,120,121,122,123:data_byte_pre[6]<=data_byte_pre[6]+uart_rx_sync2;
        134,135,136,137,138,139:data_byte_pre[7]<=data_byte_pre[7]+uart_rx_sync2;
        150,151,152,153,154,155:STOP_BIT <= STOP_BIT + uart_rx_sync2;
```

- default: //Khi ɬps_cnt không rơi vào các khoảng lấy mẫu, các thanh ghi giữ nguyên giá trị

```
        begin
            START_BIT <= START_BIT;
            data_byte_pre[0] <= data_byte_pre[0];
            data_byte_pre[1] <= data_byte_pre[1];
            data_byte_pre[2] <= data_byte_pre[2];
            data_byte_pre[3] <= data_byte_pre[3];
            data_byte_pre[4] <= data_byte_pre[4];
            data_byte_pre[5] <= data_byte_pre[5];
            data_byte_pre[6] <= data_byte_pre[6];
            data_byte_pre[7] <= data_byte_pre[7];
            STOP_BIT <= STOP_BIT;
        end
    endcase
end
```

7.2.6 Module xác định trạng thái dữ liệu

Trong phần nguyên lý đã giới thiệu, để lấy mẫu một bit dữ liệu, cần thực hiện 6 lần lấy mẫu, sau đó chọn dữ liệu xuất hiện nhiều lần nhất làm kết quả lấy mẫu. Điều này có nghĩa là chỉ những dữ liệu xuất hiện hơn 3 lần trong 6 lần lấy mẫu mới được coi là dữ liệu hợp lệ cuối cùng.

Để thực hiện điều này, có thể sử dụng dữ liệu nhận được data_byte_pre[n] kết hợp với bộ so sánh giá trị để xác định, hoặc có thể trực tiếp gán nó bằng dữ liệu bit cao nhất hiện tại. Dưới đây là ví dụ:

Khi data_byte_pre[n] lần lượt là các giá trị nhị phân 011B/010B/100B/101B, các giá trị này ở định dạng thập phân tương ứng là 3d/2d/4d/5d.

Có thể thấy rằng các giá trị lớn hơn hoặc bằng 4d là 100B/101B. Khi bit cao nhất là 1, tức là giá trị tổng cộng của dữ liệu lúc này lớn hơn hoặc bằng 4d, có thể khẳng định giá trị thực của dữ liệu là

GOWIN VIETNAM

1; khi bit cao nhất là 0, tức là giá trị tổng cộng của dữ liệu lúc này nhỏ hơn hoặc bằng 3d, có thể khẳng định giá trị thực của dữ liệu là 0. Vì vậy, chỉ cần kiểm tra bit cao nhất là đủ.

```
always@(posedge clk or posedge reset_n)
if(reset_n)
    data_byte <= 8'd0; //reset về 0
else if(bps_cnt == 8'd159)begin //chốt dữ liệu khi kết thúc 1 frame
    data_byte[0] <= data_byte_pre[0][2]; //lấy bit MSB
    data_byte[1] <= data_byte_pre[1][2];
    data_byte[2] <= data_byte_pre[2][2];
    data_byte[3] <= data_byte_pre[3][2];
    data_byte[4] <= data_byte_pre[4][2];
    data_byte[5] <= data_byte_pre[5][2];
    data_byte[6] <= data_byte_pre[6][2];
    data_byte[7] <= data_byte_pre[7][2];
end
```

7.3 Thiết kế testbench và mô phỏng

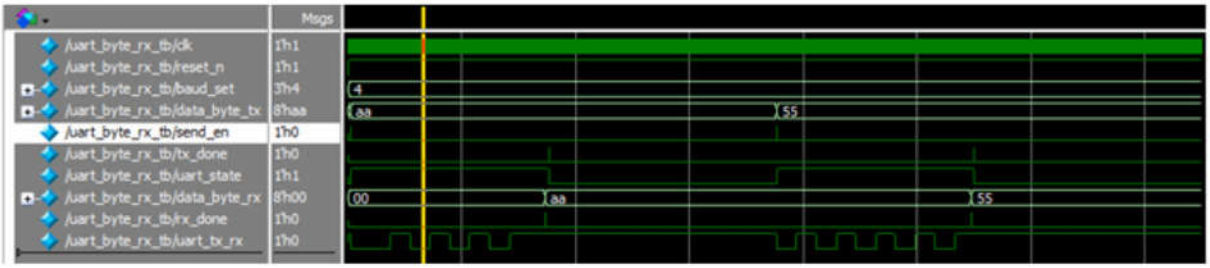
Thiết kế hoàn tất, chúng ta thực hiện mô phỏng chức năng. Trong tệp testbench sau, đầu vào dữ liệu chúng ta sử dụng module truyền của chương trước vì vậy tệp mô phỏng chỉ cần sửa đổi thông tin cổng, khởi tạo mô đun này và kết nối đầu ra module uart_tx với uart_rx trên module nhận trong tệp mô phỏng của chương trước. Một số sửa đổi file testbench như sau:

```
wire uart_tx_rx;

uart_byte_tx uart_byte_tx(
    .clk(clk),
    .reset_n(reset_n),

    .data_byte(data_byte_tx),
    .send_en(send_en),
    .baud_set(baud_set),

    .uart_tx(uart_tx_rx),
    .tx_done(tx_done),
    .uart_state(uart_state)
);

uart_byte_rx uart_byte_rx(
    .clk(clk),
    .reset_n(reset_n),

    .baud_set(baud_set),
    .uart_rx(uart_tx_rx),

    .data_byte(data_byte_rx),
```

GOWIN VIETNAM

```
    .rx_done(rx_done)
);
```

Sau khi thiết kế xong tệp testbench mô phỏng, nhấp vào phần mềm Modelsim, tạo một dự án mới để thực hiện mô phỏng chức năng, bạn có thể thấy tệp sóng như Hình 7-9 bên dưới. Mô-đun gửi qua cổng nối tiếp uart_tx đã gửi 0xaa, 0x55, mô-đun nhận qua cổng nối tiếp uart_rx đã nhận dữ liệu tuần tự và chuyển đổi thành đầu ra song song, dữ liệu đầu ra là 0xaa, 0x55, hoàn toàn khớp với dữ liệu mà mô-đun gửi qua cổng nối tiếp đã gửi.



Hình 7-9 – Dạng sóng mô phỏng nhận đơn bit

7.4 Module kiểm tra nhận data qua UART

Tạo mới một tập tin **uart_rx_test.v** và đặt nó làm mô-đun thiết kế top (mô-đun chính). Khởi tạo (instance) hai mô-đun **uart_byte_rx và uart_byte_tx** vào trong mô-đun này. Mỗi khi UART nhận được một byte dữ liệu, tín hiệu **rx_done** sẽ được tạo ra. Lúc này, gán tín hiệu **rx_done** cho **send_en**, và đưa dữ liệu nhận được từ mô-đun nhận UART truyền ngược trở lại qua mô-đun gửi UART. Mã Verilog cho mô-đun kiểm tra nhận UART như sau:

```
module uart_rx_test(
input  clk,
input  reset_n,
input  uart_rx,
output uart_tx
);
reg [7:0] data_byte_tx;
wire [7:0] data_byte_rx;
reg send_en;
wire tx_done;
wire rx_done;
always@(posedge clk or negedge reset_n)
if(!reset_n) begin
data_byte_tx <= 8'd0;
send_en <= 1'd0;
end
else if(rx_done) begin
data_byte_tx <= data_byte_rx;
send_en <= rx_done;
end
else begin
data_byte_tx <= data_byte_tx;
```

GOWIN VIETNAM

```
    send_en <= 1'd0;
    end
     uart_byte_tx uart_byte_tx(
          .clk(clk),
          .reset_n(reset_n),

          .data_byte(data_byte_tx),
          .send_en(send_en),
          .baud_set(3'd0),

          .uart_tx(uart_tx),
          .tx_done(tx_done),
          .uart_state( )
       );
       uart_byte_rx uart_byte_rx(
          .clk(clk),
          .reset_n(reset_n),

          .baud_set(3'd0),
          .uart_rx(uart_rx),

          .data_byte(data_byte_rx),
          .rx_done(rx_done)
       );
     Endmodule
```

GOWIN VIETNAM

7.5 Xác thực chương trình trên phần cứng

Việc xác minh cấp hội đồng của thí nghiệm này chủ yếu xác minh các mục tiêu sau:

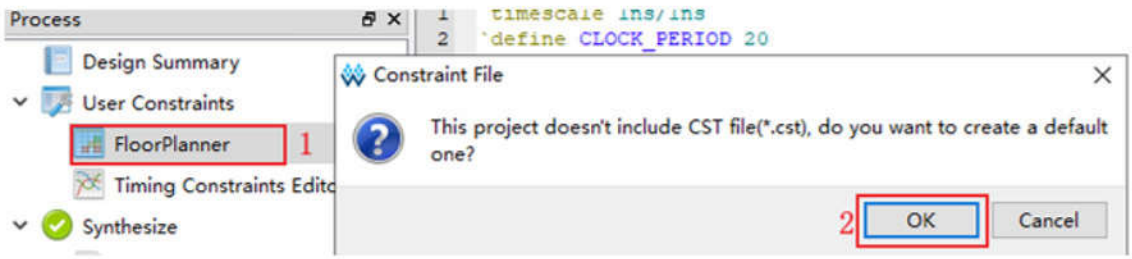
1. Liệu tệp FS được tạo có thể được tải xuống bảng phát triển một cách chính xác hay không.
2. Mở trợ lý gỡ lỗi cổng nối tiếp trên máy tính và bạn có thể thấy rằng dữ liệu nhận được mỗi 1S bắt đầu tăng từ 00.

Phần cứng cần thiết cho hệ thống:

1. Bảng phát triển ACG525.
2. Trình tải xuống Gowin EDA và cáp tải xuống.
3. Nguồn điện 12V.
4. Một cáp Type-C.
5. Một PC với đầy đủ các chức năng phát triển và điều kiện phần cứng đáp ứng các yêu cầu thử nghiệm.

7.5.1 Thêm ràng buộc I/O

Trong phần mềm Gowin, chọn khung **Process** nhấp vào **FloorPlanner** tại file .cst mới, như Hình 7-10 bên dưới



Hình 7-10 – Tạo file rang buộc I/O mới

Trong cửa sổ **FloorPlanner**, nhấp vào **I/O Constraints** để vào cửa sổ I/O。 gán chân theo sơ đồ, như Hình 7-11 bên dưới

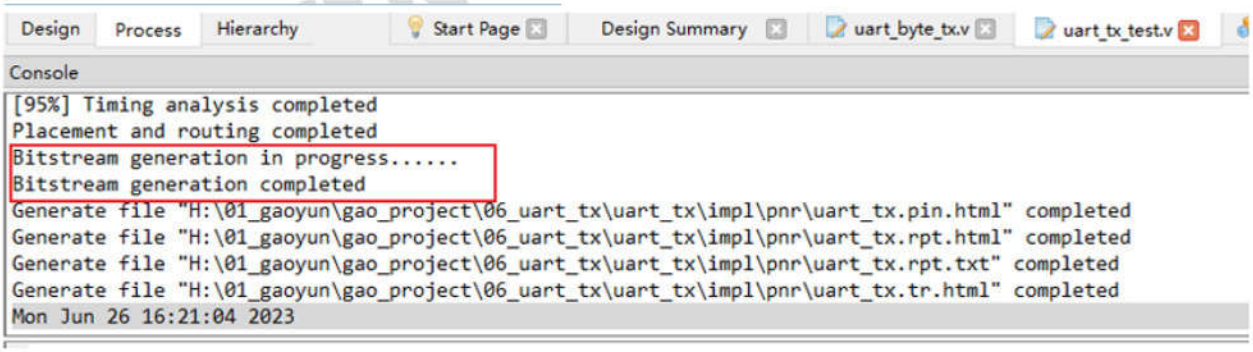
Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type
clk	input		T9	4	False	LVCMOS33
reset_n	input		N9	5	False	LVCMOS33
uart_rx	input		U8	5	False	LVCMOS33
uart_tx	output		V8	5	False	LVCMOS33

Hình 7-11– Gán chân

Điều này hoàn thành việc bổ sung ràng buộc pin. Tuy nhiên, tại thời điểm này, nội dung ràng buộc được lưu trong bộ nhớ và chưa được ghi vào tệp, hãy nhấp vào nút **Save** trên thanh công cụ hoặc trực tiếp Ctrl+S.

7.5.1 Place & Route

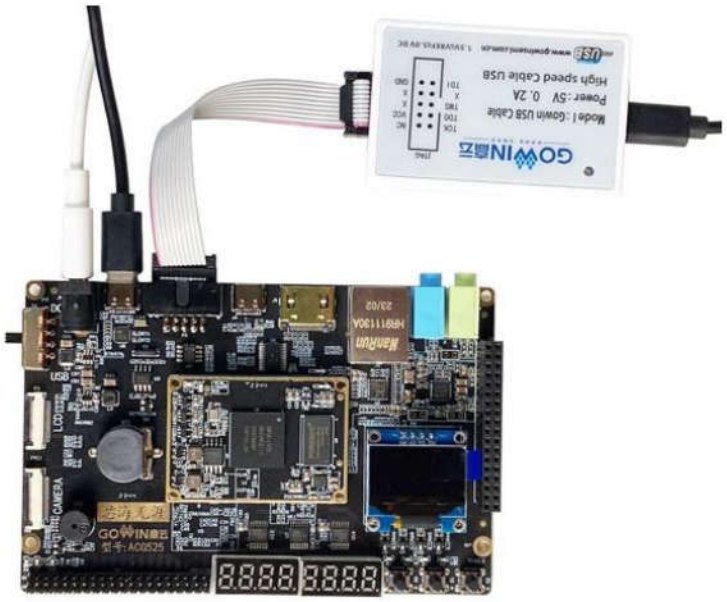
Nhấp vào **Process** chọn **Place & Route** rồi **run**, sau khi Place & Route thành công sẽ tạo ra file fs và console nắc như Hình 7-12 bên dưới,



Hình 7-12 – Tạo thành công file fs

7.5.2 Kết nối phần cứng

Lần lượt kết nối mạch nạp, nguồn điện và cáp nối tiếp với bo mạch phát triển và các kết nối phần cứng tổng thể được hiển thị trong Hình 7-13 bên dưới. Nó có thể được cấp nguồn bằng nguồn điện hoặc **USB** và người dùng có thể chọn làm như vậy.



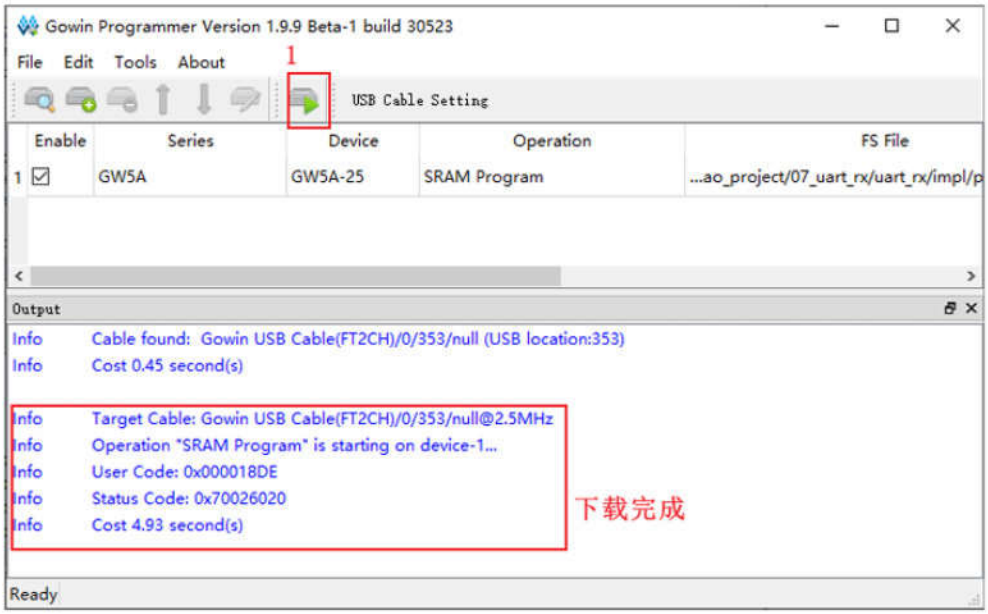
Hình 7-13 – Kết nối phần cứng

7.5.3 Cấu Hình debuger port nối tiếp

Trên máy tính, mở một phần mềm giao tiếp cổng UART — ở đây sử dụng phần mềm "串口猎人" (Serial Port Hunter). Trong phần mềm UART, thiết lập cổng COM tương ứng với bo mạch phát triển, tốc độ baud (baud rate) là 9600, dữ liệu 8 bit, 1 bit dừng. Sau đó, nhấn nút "Khởi động cổng nối tiếp". Lúc này, trong cửa sổ hiển thị thông tin ở phía dưới phần mềm UART, bạn có thể thấy tình trạng kết nối cụ thể, như Hình 7-14 dưới đây.

Hình 7-14 – Cài đặt cổng nhận nối tiếp cho công cụ Serial Port Hunter

7.5.4 Tải file fs



Hình 7-15 – Tải file fs thành công

Tải chương trình xong, nhập vào khu vực mã, sau đó nhấp vào gửi và bạn có thể nhận lại A trong vùng nhận mã, điều này cho thấy module truyền nhận hoạt động bình thường.

7.6 Tổng kết chương

Chương này đã tìm hiểu về nguyên lý liên quan đến việc nhận dữ liệu qua giao tiếp UART. Trong quá trình thiết kế, nhằm giải quyết các vấn đề như nhiễu điện từ mạnh thường gặp trong môi trường công nghiệp, đã đề xuất một phương pháp nhận dữ liệu cải tiến dựa trên trọng số. Thông qua kiểm tra trên bo mạch, kết hợp với mô-đun gửi dữ liệu UART, đã tiến hành thử nghiệm chức năng vòng lặp UART (loopback) để xác minh rằng mô-đun nhận UART hoạt động bình thường. Cần lưu ý rằng, thí nghiệm trong chương này chỉ hỗ trợ thử nghiệm vòng lặp UART với **dữ liệu một byte**. Nếu gửi nhiều byte cùng lúc, mô-đun gửi có thể chưa hoàn tất việc truyền thì mô-đun nhận đã nhận thêm dữ liệu mới, dẫn đến việc dữ liệu nhận được không khớp với dữ liệu đã gửi, gây ra hiện tượng **mất dữ liệu**.

GOWIN VIETNAM

Chương 8: ĐIỀU KHIỂN LED QUA UART

Mã nguồn dự án	Bài mẫu 02 | ch8_uart_rx_ctrl_led

Dẫn nhập chương

Trong thiết kế FPGA, chúng ta thường sử dụng cổng nối tiếp để thực hiện việc truyền và nhận dữ liệu. Tùy thuộc vào loại dữ liệu được gửi, cổng nối tiếp có thể được sử dụng cho các tình huống như truyền Hình ảnh, điều khiển, truy cập đọc/ghi, v.v. Phần này sẽ dựa trên mô-đun nhận cổng nối tiếp đã được thiết kế ở các chương trước, hướng dẫn mọi người thực hiện thiết kế điều khiển đèn LED qua cổng nối tiếp

Thiết kế cho phép người dùng thông qua cổng nối tiếp, chỉ định chế độ bật tắt của đèn LED cũng như giá trị thời gian thay đổi của mỗi trạng thái. Chế độ bật tắt được cấu thành từ 8 trạng thái, đèn LED sẽ lặp đi lặp lại 8 trạng thái này để đạt được mục đích điều khiển đèn LED qua cổng nối tiếp.

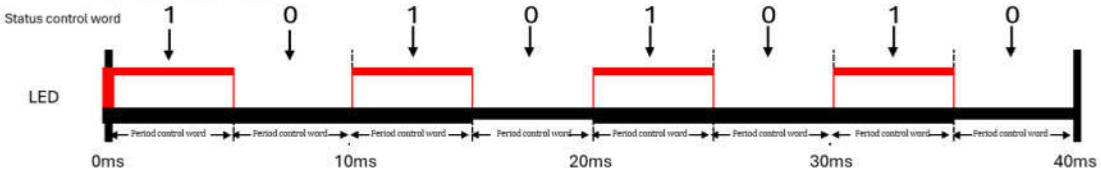
8.1 Phân tích thiết kế

Từ mục đích thiết kế, chúng ta có thể thấy rằng thiết kế cần phải điều khiển độ sáng và thời gian thay đổi trạng thái của đèn LED thông qua cổng nối tiếp. Hai yếu tố này thuộc về các chức năng khác nhau, tương ứng với hai control word khác nhau. Để tiện gọi, chúng ta sẽ gọi chúng lần lượt là từ điều khiển trạng thái (status control word) và từ điều khiển chu kỳ(period control word).

Trong thiết kế, LED thay đổi theo chu kỳ 8 trạng thái, liên tục lặp lại. Vậy, chúng ta thiết lập status control word là 8 bit, mỗi bit tương ứng với một trạng thái trong 8 trạng thái của LED.

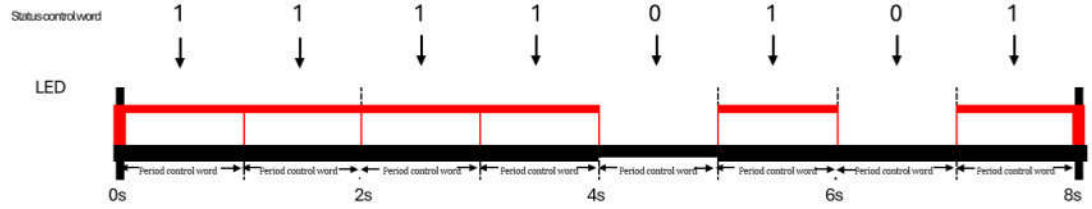
Từ điều khiển chu kỳ (Period control word) kiểm soát thời gian thay đổi của đèn LED. Nếu thời gian thay đổi của đèn LED quá ngắn, tốc độ thay đổi quá nhanh, thì mắt người sẽ không thể quan sát được quá trình thay đổi. Vì vậy, chúng ta có thể sử dụng một dữ liệu lớn hơn, 32 bit, để đại diện cho từ điều khiển chu kỳ(Period control word).

Do đó, trong thiết kế này, để cổng nối tiếp điều khiển đèn LED thực hiện các chức năng cần thiết, chúng ta cần gửi 5 byte từ điều khiển. Ví dụ, nếu lúc này chúng ta muốn điều khiển đèn LED theo quy luật bật tắt liên tục, mỗi 5ms thay đổi một lần, như Hình 8-1 đã chỉ ra:



Hình 8-1– Điều khiển LED (5ms)

Vậy chúng ta chỉ cần đặt giá trị của status control word là 0xAA (8'b10101010), sau đó sử dụng period control word để tạo ra tín hiệu đánh dấu chu kỳ 5ms. Mỗi khi tạo ra một tín hiệu đánh dấu, chúng ta sẽ lấy một bit từ status control word, dựa vào giá trị của bit tương ứng để điều khiển mức cao/thấp của IO được chỉ định, từ đó điều khiển đèn LED bật tắt. Cứ như vậy lặp lại, thực hiện chu trình bật tắt của đèn LED mỗi 5ms. Tương tự, để điều khiển đèn LED theo quy luật sáng sáng sáng sáng tắt sáng tắt sáng, mỗi 1 giây thay đổi một lần, như Hình 8-2 đã chỉ ra



Hình 8-2 – Điều khiển LED tắt sáng（1S）

Vậy chúng ta chỉ cần đặt giá trị của từ điều khiển trạng thái là 0xF5, thông qua từ điều khiển chu kỳ để tạo ra tín hiệu dấu hiệu chu kỳ 1 giây.

8.2 Định nghĩa khung giao thức truyền UART

Sau khi xác định kích thước dữ liệu của từ điều khiển là 5 byte, để đảm bảo rằng dữ liệu nhận được là dữ liệu hợp lệ và cũng để thuận tiện trong việc xác định chức năng của từng byte trong dữ liệu nhận, chúng ta có thể định nghĩa một khung giao thức truyền qua cổng nối tiếp, như Hình 8-3 dưới đây:

55	A5	time_set [31:24]	time_set [23:16]	time_set [15:8]	time_set [7:0]	ctrl [7:0]	F0

Hình 8-3 – Khung giao thức truyền UART tùy chỉnh

Khung giao thức này được chúng tôi tự định nghĩa, bao gồm đầu khung (0x55 0xA5) + 5 byte dữ liệu + đuôi khung (0xF0). Người dùng trên máy chủ gửi dữ liệu theo khung giao thức truyền tải này, và FPGA sẽ đánh giá và giải nén dữ liệu dựa trên giao thức này sau khi nhận được. Chỉ khi dữ liệu nhiều byte liên tiếp mà cổng nối tiếp nhận được đáp ứng giao thức này thì mới được coi là dữ liệu hợp lệ. Sau đó, **time_set** và ctrl được sử dụng lần lượt làm period control word và status control wordđiều khiển LED.

Bằng cách này, chúng ta có thể hiệu quả tránh được các tình huống mà từ điều khiển bị truyền sai, bị mất hoặc bị thừa do sự cố bất ngờ trong quá trình truyền nhưng vẫn được coi là lệnh hợp lệ.

8.3 Nguyên lý và ý tưởng điều khiển LED qua UART

Sau khi xác định khung giao thức truyền qua UART, chúng ta có thể bắt đầu công việc thiết kế.

Đối với thiết kế này, để điều khiển chế độ sáng và tắt của LED cũng như thời gian thay đổi trạng thái, trước tiên cần có khả năng nhận dữ liệu qua UART. Trong các bài học trước, chúng ta đã giới thiệu và thực hiện chi tiết thiết kế logic nhận dữ liệu qua cổng nối tiếp, và trong thiết kế lần này, chúng ta có thể sử dụng trực tiếp nó.

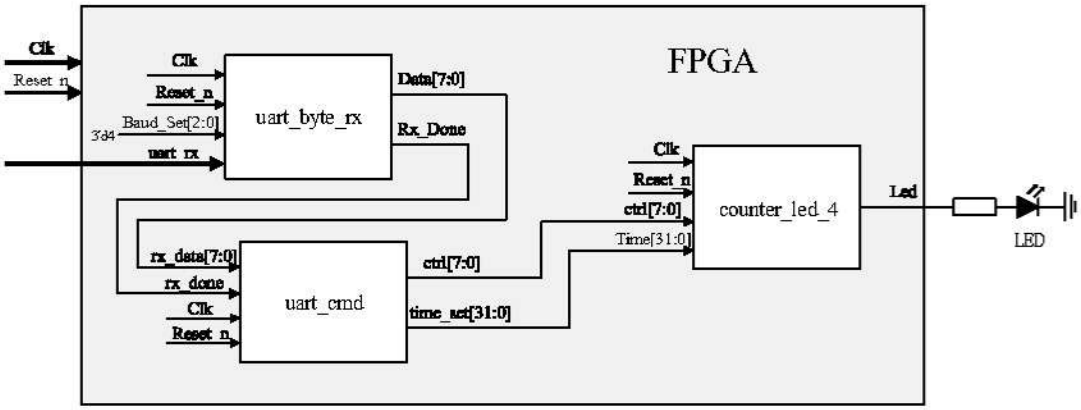
Mô-đun nhận dữ liệu qua cổng nối tiếp chỉ có thể truyền một byte dữ liệu mỗi lần, trong khi theo khung giao thức cổng nối tiếp mà chúng ta đã định nghĩa, một lệnh hợp lệ ít nhất phải chứa 8 byte. Do đó, cần có một mô-đun chuyển đổi lệnh qua cổng nối tiếp, để lưu trữ 8 byte dữ liệu từ đầu ra của mô-đun nhận dữ liệu và dựa trên dữ liệu đã lưu để xác định xem lệnh hiện tại có đúng hay không, sau đó trích xuất status control word và period contrl word từ lệnh đúng và xuất ra.

Control dord xuất ra cuối cùng sẽ được sử dụng để điều khiển trạng thái sáng và tắt của LED cũng như chu kỳ. Để thực hiện chức năng này, chúng ta có thể định nghĩa một bộ đếm và một tín hiệu cờ có độ rộng 3 bit. Sử dụng Period control word làm giá trị tối đa của bộ đếm, khi bộ đếm đạt giá trị tối đa, tín hiệu cờ sẽ tăng lên 1 và bộ đếm sẽ bắt đầu đếm lại. Theo cách này, chúng ta có thể điều khiển tần suất thay đổi của tín hiệu cờ thông qua Period control word. Do tín hiệu cờ có độ rộng 3 bit, với giá trị đếm tăng dần, tín hiệu cờ sẽ thay đổi tuần hoàn trong khoảng từ 0 đến 7 với tần suất cố định. Chúng ta chỉ cần dựa vào giá trị của status control word tương ứng với các bit khác nhau, để điều khiển IO xuất ra mức cao/thấp, từ đó LED sẽ thay đổi trạng thái theo tần suất cố định.

Dựa trên phân tích trên, để thực hiện việc điều khiển LED qua cổng nối tiếp và tần suất của nó, chúng ta có thể chia thành ba phần:

1. Nhận data nối tiếp từ PC và chuyển đổi thành nhiều byte data đơn lẻ để xuất ra.
2. Lưu trữ dữ liệu byte đơn lẻ thành 8 byte, xác định xem dữ liệu có phải là khung hợp lệ hay không, và trích xuất control word đúng để xuất ra.
3. Dựa vào từ điều khiển, sử dụng bộ đếm và tín hiệu trạng thái để điều khiển LED với tần suất nhất định, 8 trạng thái theo chu kỳ, thay đổi tuần hoàn.

Do đó, sơ đồ khối hệ thống của thiết kế lần này được thể hiện trong Hình 8-4:



Hình 8-4 – Sơ đồ cấu trúc thiết kế

Trong đó, **uart_byte_rx** là mô-đun nhận dữ liệu nối tiếp một byte đã được thiết kế ở các chương trước; **uart_cmd** là mô-đun chuyển đổi lệnh qua cổng nối tiếp, mô-đun này lưu trữ dữ liệu một byte để nhận được 8 byte dữ liệu liên tục. Mỗi khi nhận được một byte dữ liệu mới, nó sẽ kiểm tra cấu trúc khung của 8 byte dữ liệu liên tục đó. Nếu nội dung dữ liệu phù hợp với cấu trúc khung lệnh, nó sẽ trích xuất từ điều khiển chu kỳ **(time_set[31:0])** và từ điều khiển trạng thái **(ctrl[7:0])**; **counter_led_4** là mô-đun điều khiển đếm, dựa trên từ điều khiển đầu vào, điều khiển đèn LED sáng và tắt với tần suất và chế độ nhất định.

Dựa trên cấu trúc thiết kế này, chúng ta có thể bắt đầu thiết kế mô-đun cho lần này.

8.3.1 Thiết kế module chuẩn đổi lệnh qua UART (uart_cmd)

Module truyền data qua UART mỗi lần truyền 1 byte data. Theo khung giao thức truyền uart mà chúng ta tự định nghĩa, để xác định data có phải dùng để điều khiển LED hay không, cần tối thiểu 8 byte. Do đó chúng ta cần lưu trữ data, code như sau:

```
input Clk;
input [7:0]rx_data;

reg [7:0] data_str [7:0]; always@(posedge Clk) if(rx_done)begin
    data_str[7] <= #1 rx_data; data_str[6] <= #1 data_str[7];
    data_str[5] <= #1 data_str[6]; data_str[4] <= #1 data_str[5];
    data_str[3] <= #1 data_str[4]; data_str[2] <= #1 data_str[3];
    data_str[1] <= #1 data_str[2]; data_str[0] <= #1 data_str[1];
end
```

Ở đây, phương pháp lưu trữ mà chúng tôi sử dụng là bộ đệm dịch chuyển. Lợi ích lớn nhất của phương pháp này là dữ liệu được lưu trữ sẽ thay đổi liên tục và phù hợp với độ dài khung giao thức truyền tải nối tiếp. Thông qua phương pháp này, chúng ta chỉ cần thực hiện kiểm tra cấu trúc khung một lần cho 8 byte dữ liệu trong data_str khi nhận được byte dữ liệu mới, mà không cần kiểm tra tất cả dữ liệu đã nhận.

GOWIN VIETNAM

Khi dữ liệu trong data_str đáp ứng truyền tải UART mà ta đã định nghĩa, chúng ta sẽ trích xuất từ điều khiển tương ứng để xuất ra. Mã nguồn như sau:

```
input Reset_n; output reg[7:0]ctrl;
output reg[31:0]time_set;

reg r_rx_done; always@(posedge Clk)
```

```
    r_rx_done <= rx_done;

always@(posedge Clk or negedge Reset_n)
if(!Reset_n) begin
    ctrl <= #1 0;
    time_set <= #1 0;
end else if(r_rx_done)begin
    if((data_str[0]==8'h55)&&(data_str[1]==8'hA5)&&(data_str[7]==8'hF0))begin
        time_set[31:24] <= #1 data_str[2];
        time_set[23:16] <= #1 data_str[3];
        time_set[15:8] <= #1 data_str[4];
        time_set[7:0] <= #1 data_str[5];
        ctrl <= #1 data_str[6];
    end
end
```

GOWIN VIETNAM

Ở đây do dữ liệu bộ đệm ở vị trí cao nhất và dữ liệu đầu vào có một chu kỳ đồng hồ trễ, nên cần phải chờ một nhịp sau khi tín hiệu rx_done có hiệu lực để làm điều kiện kiểm tra dữ liệu trong bộ đệm.

Khi tín hiệu r_rx_done có hiệu lực, nó đại diện cho việc mô-đun đã nhận được một byte dữ liệu mới, lúc này sẽ tiến hành kiểm tra cấu trúc khung của dữ liệu trong data_str. Khi dữ liệu phù hợp với cấu trúc khung, nó sẽ được coi là lệnh hợp lệ, và sẽ được trích xuất từ các bit tương ứng để kết hợp thành từ điều khiển chu kỳ và từ điều khiển trạng thái, sau đó xuất ra cho mô-đun điều khiển đếm.

Thông qua việc lưu trữ dữ liệu nhận được trong 8 byte và kiểm tra cấu trúc khung, thiết kế có tính năng chống lỗi rất mạnh cho các lệnh do người dùng phát đi. Ngay cả khi người dùng gửi đi quá nhiều, thiếu hoặc gửi sai một phần dữ liệu trong quá trình phát lệnh, thiết kế vẫn có thể phân tích và đánh giá dữ liệu nhận được dựa trên cấu trúc khung, từ đó tìm ra lệnh đúng.

8.3.2 Thiết kế bộ đếm điều khiển LED（counter_led_4）

Mô-đun điều khiển đếm cần phải dựa vào Period control word để đếm và phát sinh tín hiệu đánh dấu, đồng thời dựa vào status control word để điều khiển các IO chỉ định phát ra mức cao/thấp, điều khiển LED sáng/tắt.

Tạo tín hiệu đánh dấu rất đơn giản, cần định nghĩa một bộ đếm. Khi giá trị của bộ đếm đạt **Period control word**, thì cho tăng tín hiệu đánh dấu một lần, code như sau:

```
input Clk; input Reset_n;
input [31:0]Time; reg[31:0]counter;
always@(posedge Clk or negedge Reset_n) if(!Reset_n)
counter <= #1 0;
```

```
 else if(counter >= Time - 1)
counter <= #1 0;
 else
 counter <= #1 counter + 1'b1;

 reg [2:0]counter2;
 always@(posedge Clk or negedge
Reset_n) if(!Reset_n)
 counter2 <= #1 0;
 else if(counter >= Time - 1)
counter2 <= #1 counter2 +
1'b1;
```

Ở đây, **counter2** 3 bit biểu diễn được giá trị 0:7, dùng làm th đánh dấu. Khi vượt quá 7 thì nó sẽ **tự động tràn và về 0**. Còn status control word dùng để điều khiển LED sáng/tắt cũng vừa vặn **8 bit**. Vì vậy,ta có thể sử dụng **câu lệnh case**, dựa vào giá trị của counter2, để điều khiển trạng thái sáng/tắt của LED. Mã nguồn như sau:

```
always@(posedge Clk or negedge Reset_n) if(!Reset_n)
Led <= #1 0;
case(counter2) 0:Led <= #1 Ctrl[0];
1:Led <= #1 Ctrl[1];
2:Led <= #1 Ctrl[2];
3:Led <= #1 Ctrl[3];
4:Led <= #1 Ctrl[4];
5:Led <= #1 Ctrl[5];
6:Led <= #1 Ctrl[6];
7:Led <= #1 Ctrl[7];
      default:Led <= #1 Led; endcase
```

Mỗi khi bộ đếm đạt đến giá trị đặt trước, giá trị của counter2 sẽ tự động tăng lên một, dẫn đến **sự thay đổi trong việc điều khiển LED**. Khi giá trị của counter2 vượt quá 7, nó sẽ **tràn và trở lại 0**. Nhờ đó, **LED có thể thay đổi trạng thái theo chu kỳ**, lặp lại liên tục giữa 8 trạng thái đầu vào với một **tần số nhất định**

GOWIN VIETNAM

8.3.3 Tạo top module

Sau khi đã hoàn thành thiết kế từng module con, ta cần tạo Top level module để **kết nối tuần tự lại với nhau**, nhằm đạt được chức năng mong muốn. Trong thiết kế này, chúng ta chỉ cần thực hiện việc **khai báo (instance) các module con trong top-level và kết nối các cổng tương ứng**. Mã nguồn đầy đủ như sau:

Đến đây, chúng ta đã hoàn thành thiết kế điều khiển đèn LED sáng/tắt thông qua giao tiếp UART.

```
module uart_rx_ctrl_led( Clk,
Reset_n, Led, uart_rx
);
```

```
    input Clk; input Reset_n; output Led; input uart_rx;
    wire [7:0]ctrl;
    wire [31:0]time_set; wire [7:0]rx_data; wire rx_done;
    parameter Baud_Set = 3'd4; counter_led_4 counter_led(
        .Clk(Clk),
        .Reset_n(Reset_n),
        .Ctrl(ctrl),
        .Time(time_set),
        .Led(Led)
    );
                    uart_cmd uart_cmd(
                        .Clk(Clk),
        .Reset_n(Reset_n),
        .rx_data(rx_data),
        .rx_done(rx_done),
        .ctrl(ctrl),
        .time_set(time_set)
    );
    uart_byte_rx uart_byte_rx(
        .Clk(Clk),
        .Reset_n(Reset_n),
        .Baud_Set(Baud_Set),
        .uart_rx(uart_rx),
        .Data(rx_data),
        .Rx_Done(rx_done)
    );
endmodule
```

GOWIN VIETNAM

Để kiểm chứng xem thiết kế có đáp ứng được chức năng như mong đợi hay không, bước tiếp theo chúng ta cần **tiến hành mô phỏng (simulation)** để kiểm tra, quan sát **dạng sóng thời gian (waveform timing)** có chính xác hay không.

8.3.4 Tạo testbench và kiểm tra mô phỏng

Trong **tập tin testbench**, ta cần mô phỏng hoạt động của **giao tiếp UART**, để cấp đầu vào cho thiết kế, kiểm tra xem module có thể **nhận dạng chính xác từ điều khiển** không, trong quá trình mô phỏng, ta sẽ **chèn thêm một số nhiễu** đầu vào UART. Dưới đây là **mã đầy đủ của tập tin testbench** trong lần kiểm thử này:

```
`timescale 1ns / 1ps
module uart_rx_ctrl_led_tb();
    reg Clk;
    reg Reset_n; wire Led; reg uart_rx;
    wire [31:0] delay_time;

    uart_rx_ctrl_led uart_rx_ctrl_led(
        .Clk(Clk),
        .Reset_n(Reset_n),
        .Led(Led),
        .uart_rx(uart_rx)
    );
    parameter Baud_Set = 3'd4;

    assign delay_time = (Baud_Set == 3'd0) ? 20'd104166:
                        (Baud_Set  ==  3'd1)  ?
                        20'd52083: (Baud_Set ==
                        3'd2)    ?   20'd26041:
                        (Baud_Set  ==  3'd3)  ?
                        20'd17361:
                                              20'd8680;
    initial Clk =
    1; always#10
    Clk = ~Clk;
    initial
       begin
       Reset_n
       = 0;
       uart_rx = 1;
       #201;
       Reset_n = 1;
       #200;
       uart_tx_byte(8'h55);
       #(delay_time*10);
       uart_tx_byte(8'ha5);
       #(delay_time*10);
       uart_tx_byte(8'h55);
       #(delay_time*10);
       uart_tx_byte(8'ha5);
       #(delays_time*10);
       uart_tx_byte(8'h00);
```

GOWIN VIETNAM

```
  (delay_time*10); uart_tx_byte(8'h00); #(delay_time*10); uart_tx_byte(8'hc3); #(delay_time*10);
uart_tx_byte(8'h50); #(delay_time*10); uart_tx_byte(8'hAA); #(delay_time*10);
uart_tx_byte(8'hf0); #(delay_time*10);
  uart_tx_byte(8'h55);
#(delay_time*10);
uart_tx_byte(8'ha5);
#(delay_time*10);
uart_tx_byte(8'h9a);
#(delay_time*10);
uart_tx_byte(8'h78);
#(delay_time*10);
uart_tx_byte(8'h56);
#(delay_time*10);
uart_tx_byte(8'h34);
#(delay_time*10);
uart_tx_byte(8'h12);
#(delay_time*10);
uart_tx_byte(8'hf1);
#(delay_time*10); #15000000;
  $stop; end
  task uart_tx_byte; input
[7:0]tx_data; begin
  uart_rx = 1;
  #20;
  uart_rx = 0; #delay_time;
  uart_rx = tx_data[0]; #delay_time;
  uart_rx = tx_data[1]; #delay_time;
  uart_rx=tx_data[2]; #delay_time;
uart_rx=tx_data[3]; #delay_time;
uart_rx=tx_data[4];  #delay_time;
```

GOWIN VIETNAM

```
            uart_rx = tx_data[5];
            #delay_time;
               uart_rx = tx_data[6];
               #delay_time;
               uart_rx = tx_data[7];
               #delay_time;
               uart_rx = 1;
               #delay_time;
         end
      endtask
endmodule
```

Xét đến việc ở các mức **baud rate khác nhau**, khi mô phỏng tín hiệu UART thì **khoảng cách giữa các bit và giữa các byte cũng sẽ khác nhau**, nên trong thiết kế đã sử dụng lệnh assign để **tự động tính toán giá trị trễ dựa trên giá trị baud_set do người dùng thiết lập**.

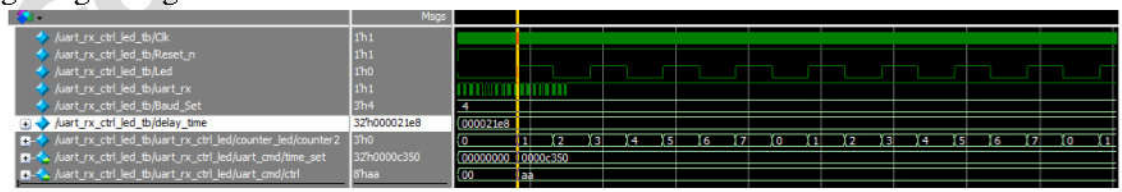
Trong quá trình kích thích, **baud rate được đặt là 115200**, do đó **thời gian truyền một bit** qua UART sẽ là: \frac{1}{115200} \times 10^9 = 8680 \text{ ns}],

Để **giảm lặp lại trong mã mô phỏng**, chúng tôi sử dụng **task để hiện thực quá trình mô phỏng đầu ra của UART**.

Nhờ việc gọi task, quá trình **UART gửi lệnh đến mô-đun cần kiểm thử** có thể được mô phỏng một cách dễ dàng và tự động.

Có thể thấy rằng, trong tập tin kích thích, ngoài việc gửi **lệnh hợp lệ**, chúng tôi còn **chèn thêm một số dữ liệu nhiễu**. **Lệnh hợp lệ** ở đây là dãy: 0x55_a5_00_00_c3_50_aa_f0, trong đó có nghĩa là: bộ đếm sẽ đếm đến **50000** thì sẽ **chuyển trạng thái điều khiển LED một lần**, tức khoảng **1ms** thay đổi mức điện áp điều khiển LED. Giá trị điều khiển LED là 10101010 (tức 0xAA), nghĩa là LED sẽ thay đổi theo chu kỳ: **sáng – tắt – sáng – tắt – sáng – tắt – sáng – tắt**.

Sau khi hoàn thành testbench, **tiến hành mô phỏng**, để **waveform** và kiểm tra xem hệ thống có hoạt động đúng không.



Hình 8-5 dạng sóng mô phỏng

Để dễ quan sát, ta thêm tín hiệu time_set và ctrl từ mô-đun uart_cmd, cùng với tín hiệu counter2 từ mô-đun counter_led_4. Có thể thấy, sau khi dữ liệu được **nhận UART**, module uart_cmd đã **nhận diện chính xác lệnh** và thu **dữ liệu chính xác**: time_set là 0xc350, ctrl là 0xaa. Mức điều khiển LED thay đổi theo quy tắc **cao – thấp – cao – thấp – cao – thấp – cao – thấp** khi tín hiệu counter2 thay đổi.

Sau khi counter2 vượt quá 7 và bị tràn, nó sẽ **quay lại 0** và bắt đầu **tăng dần từ 0**, LED tiếp tục thay đổi theo chu kỳ như trước mà không bị gián đoạn.

Có thể thấy, **dạng sóng mô phỏng** hoàn toàn **phù hợp với dự đoán**, điều này cho thấy thiết kế hoạt động bình thường và có thể **hoàn thành đầy đủ chức năng** như yêu cầu. Đến đây, chúng ta chỉ còn **một bước cuối cùng** là tiến hành **xác thực trên phần cứng** để kiểm tra hiệu quả thực tế của thiết kế

8.4 Kiểm chứng trên hardware

Phần này sẽ tiến hành **gán chân** cho thiết kế, và dựa trên tệp **fs** được tạo ra từ dự án đã phân bổ chân, sẽ thực hiện **nạp chương trình vào board ACG525** để tiến hành kiểm tra phần cứng.

Thông qua phần mềm **debug serial trên máy tính**, chúng ta sẽ **gửi lệnh** để điều khiển trạng thái **sáng/tắt** và **tần số chớp tắt** của LED.

Cuối cùng, qua việc quan sát kết quả **xác thực phần cứng** trên bảng và so sánh với **kết quả dự đoán**, chúng ta sẽ xác nhận tính chính xác và hiệu quả của thiết kế.

8.4.1 Ràng buộc I/O

Tiến hành tổng hợp code rồi đến Floorplan để gán chân theo thông tin bên dưới

Bảng 8-1– Bảng biểu gán chân

Signial Name	Pin NO.	Signial Name	Pin NO.
CLK	T9	uart_rx	U8
Reset_n	B16	led	D14

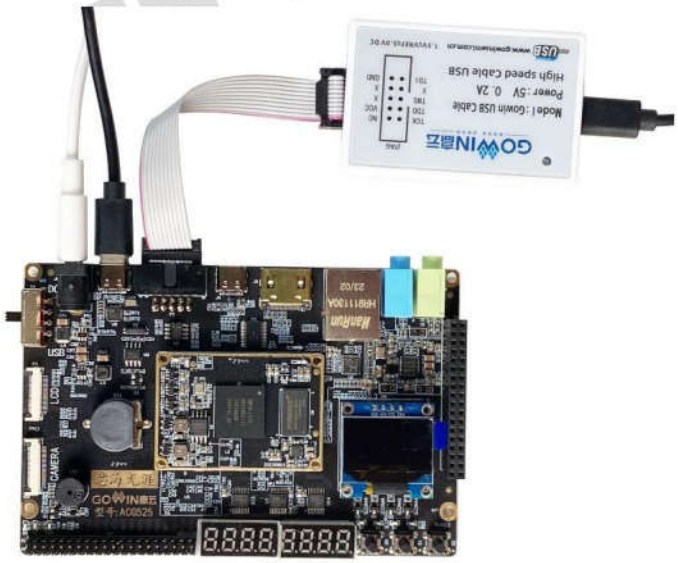
Sau khi **hoàn tất việc cấu Hình**, chúng ta sẽ **ràng buộc các chân** tương ứng với **LVCMOS33**. Sau đó, có thể **tạo tệp fs**. Khi tệp fs đã được tạo xong, chúng ta sẽ **kết nối phần cứng** và chuẩn bị **nạp chương trình (programming)** để kiểm tra.

8.4.2 Hệ thống phần cứng

1. Bo mạch ACG525
2. Mạch nạp x1
3. Cáp USBx1
4. Cáp nguồnx1

8.4.3 Kết nối phần cứng

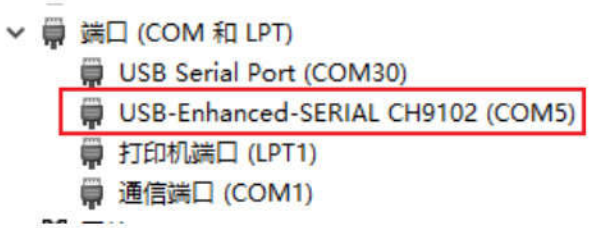
Kết nối cáp tải xuống và cáp nguồn của bo mạch phát triển, sử dụng một đầu của cáp cổng nối tiếp để kết nối cổng UART trên bo mạch phát triển và một đầu với cổng USB của máy tính, như trong Hình 8-6 bên dưới.



Hình 8-6 – Kết nối phần cứng

Sau khi kết nối xong, hãy **đảm bảo công tắc nguồn của bảng phát triển** được đặt ở phía **ON**. Tiếp theo, **nạp tệp fs** đã được tạo vào bảng phát triển.

Sau khi quá trình **nạp chương trình hoàn tất**, mở **Trình quản lý thiết bị (Device Manager)** để **kiểm tra cổng serial**. Tên cổng serial trong **Trình quản lý thiết bị** sẽ hi thị như trong Hình 8-7:



Hình 8-7– Truy vấn cổng COM

Trong trường hợp này, **cổng serial trên máy tính của tác giả là COM5**, tuy nhiên khi người dùng kiểm tra, cần **dựa vào cổng thực tế trên máy tính của mình**.

Tiếp theo, **mở phần mềm Serial Hunter**, kết nối với **cổng COM vừa tra được**, và **thiết lập baud rate là 115200**.

Sau đó, dùng **Serial Hunter để gửi lệnh điều khiển**; **cách gửi lệnh** được minh họa trong Hình 8-8:

Hình 8-8 – Mã từ Serial Hunter

Có thể thấy rằng, **lệnh hợp lệ** trong số các dữ liệu được gửi là:

55 A5 02 FA F0 80 AA F0 Tức là: **từ điều khiển chu kỳ** là 0x02FAF080 (tương đương **50_000_000**, tức **1 giây**), và **từ điều khiển trạng thái** là 0xAA (10101010).

Điều này nghĩa là **LED sẽ bật – tắt – bật – tắt – ... theo chu kỳ, cứ mỗi 1 giây thay đổi một lần**, và **sau mỗi 8 lần thay đổi thì lặp lại**.

Sau khi **gửi lệnh cùng với một số dữ liệu nhiễu** xuống FPGA, **LED0 bắt đầu nhấp nháy đúng theo quy luật dự kiến**. Tiếp theo, tác giả tiếp tục thử nghiệm bằng cách **gửi nhiều lần lệnh**, và thậm chí **gửi sai, gửi thiếu hoặc gửi thừa một phần dữ liệu**. Để tránh ảnh hưởng từ lệnh lần trước, **mỗi lần gửi lệnh mới đều thực hiện reset chương trình** trước đó.

Các **lệnh liên quan** và **hiện tượng thực tế quan sát được trên bo** như sau:

1. **Gửi lệnh:** 18 65 55 A5 02 FA F0 80 F0 F0 23 12
2. → Trên **bo mạch phát triển, LED0 sáng trong 4 giây rồi tắt trong 4 giây**, lặp lại liên tục theo chu kỳ.
3. **Gửi lệnh:** 55 A5 02 FA 80 A7 F0 → **LED không có phản ứng** trên bo mạch

4. Gửi lệnh 55 A5 02 FA F0 80 F0 A0, **LED cũng không có phản ứng** trên bo mạch.

Trong thao tác số 2, **lệnh bị thiếu phần dữ liệu** — mặc dù có **phát hiện đúng phần đầu khung lệnh (frame header)** là 55 A5, nhưng **không có đủ dữ liệu cho phần đuôi khung lệnh (frame tail)**, nên **lệnh này bị xác định là không hợp lệ**. Do đó, **LED không được điều khiển** — hiện tượng này **hoàn toàn khớp với phản ứng thực tế trên bo mạch**.

Trong thao tác 3, lệnh được gửi không chính xác, mặc dù phát hiện đúng tiêu đề khung 55 A5, nhưng giá trị ở cuối khung là với chúng ta **giá trị được định nghĩa không trùng khớp**, nên **lệnh bị xác định là lệnh không hợp lệ, LED không được điều khiển**.

Hiện tượng thực tế quan sát được trên bo mạch **phù hợp hoàn toàn với thao tác đã thực hiện**.

Qua **4 lần kiểm tra gửi lệnh, LED đều hoạt động đúng theo kỳ vọng**.

Đến đây, việc xác minh ở cấp độ phần cứng đã **thành công**, thiết kế đã **hoàn toàn thực hiện đúng chức năng như mong đợi trên bo phát triển ACG525**, và **không có lỗi xảy ra** trong toàn bộ quá trình.

8.5 Tổng kết chương

Trong chương này, chúng ta đã cùng nhau **hoàn thành thiết kế điều khiển đèn LED thông qua giao tiếp UART (cổng nối tiếp)**.

Thiết kế sử dụng **lệnh được gửi qua UART**, với **giao thức truyền thông tự định nghĩa** để đảm bảo **độ chính xác của lệnh** và **xác định đúng các từ điều khiển tương ứng**.

Cuối cùng, **các từ điều khiển này được truyền đến mô-đun điều khiển LED thông qua bộ đệm**, từ đó **thực hiện việc điều khiển trạng thái sáng/tắt và tần số chớp tắt của LED**.

Các mô-đun được thiết kế trong chương này **không quá phức tạp**, với mục đích chính là giúp người đọc **nắm được ý tưởng cơ bản về cách sử dụng UART để thực hiện chức năng điều khiển**.

Giao tiếp UART có **rất nhiều ứng dụng trong thực tế**, người dùng có thể **phát triển tư duy** và **thử sức với các thiết kế phức tạp và thú vị hơn** dựa trên nền tảng đã học được từ chương này.

GOWIN VIETNAM

Chương 9: Thiết kế và kiểm chứng module truyền nhận UART nhiều byte

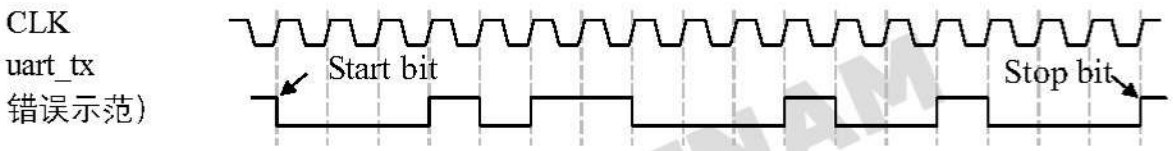
Mã nguồn dự án	02_设计实例	ch9_uart_loopback

Dẫn nhập chương

Trong các chương trước, chúng ta đã hoàn thành thiết kế mô-đun truyền và nhận một byte. Tuy nhiên, 2 mô-đun này có ứng dụng tương đối hạn chế. Trong hầu hết các trường hợp, UART thường được sử dụng cho việc truyền và nhận nhiều byte liên tục. Do đó, chương này sẽ dựa trên mô-đun truyền & nhận data một byte đã có, hướng dẫn mọi người thiết kế mô-đun truyền data nhiều byte và mô-đun nhận data nhiều byte, và xác minh thiết kế qua mô phỏng và thử nghiệm trên bảng mạch.

9.1 Nguyên lý và thiết kế mô-đun truyền data UART nhiều byte.

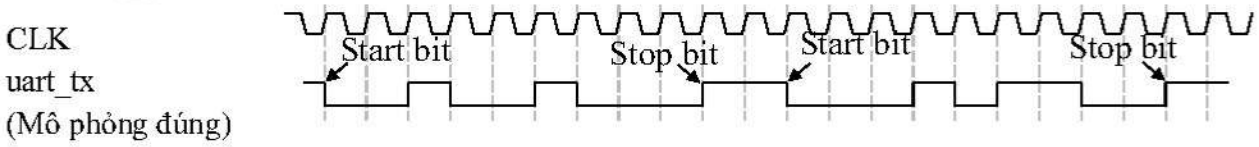
Để thực hiện việc truyền data nhiều byte qua UART, lấy cấu Hình 8N1 phổ biến làm ví dụ (8 bit data, không kiểm tra chẵn lẻ,1 bit stop), nhiều bạn sẽ có một hiểu lầm, cho rằng chỉ cần đưa data vào giữa start bit và stop bit. Ví dụ, nếu tôi muốn gửi data 16 bit 0x1234 qua UART, theo cách suy nghĩ trên, tôi sẽ mô phỏng UART xuất ra 0_0010_1100_0100_1000_1, như Hình 9-1 đã chỉ ra:



Hình 9-1– Truyền 0x1234 (Biểu thì hiểu nhầm)

Sau khi data này được UART đích nhận, nó sẽ được phân tích theo dạng 8N1 để trích xuất các bit data. Mỗi lần phân tích cần nhận 10 bit data, nếu phân tích đúng, sẽ trích xuất được 8 bit hợp lệ ở giữa. Để phân tích ra được 16 bit 0x1234, cần phải phân tích 2 lần, tức cần 20 bi. Trong khi đó, khi mô phỏng UART, chúng ta chỉ gửi 18 bit, vì vậy bất kể cuối cùng phân tích ra được gì, cũng sẽ không phải là 0x1234 mà chúng ta mong đợi nhận được.

Truyền đúng nên được chia thành hai lần: lần 1 truyền 0_0100_1000_1, lần 2 truyền 0_0010_1100_1, như Hình 9-2 đã chỉ ra:



Hình 9-2 – Truyền đúng 0x1234

Cổng UART sau khi nhận được data sẽ phân tích data theo dạng 8N1, chia thành 2 lần để nhận được 8'b0001_0010 (0x12) và 8'b0011_0100 (0x34). Khi ghép nối hai data 8 bit này lại, ta sẽ nhận được 0x1234.

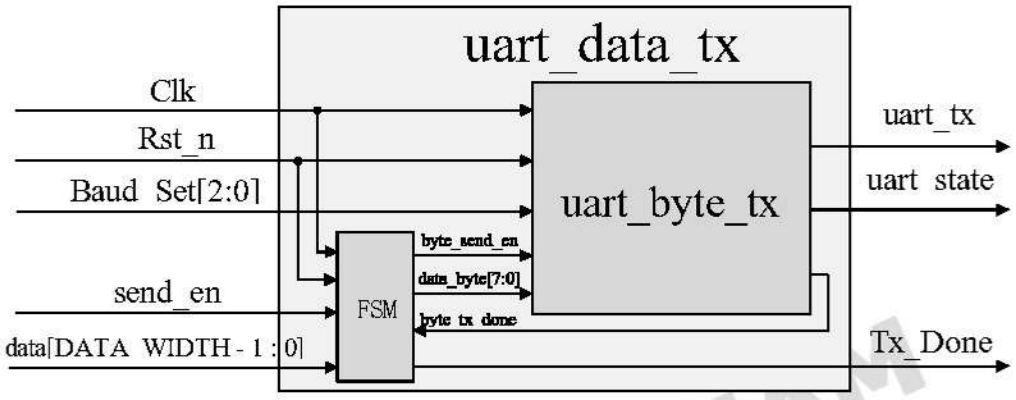
Do đó, việc truyền data nhiều byte qua UART có thể được xem như quá trình truyền nhiều data đơn byte liên tiếp. Trong chương trước, chúng ta đã học và hoàn thành thiết kế mô-đun truyền data đơn byte. Trong chương này, chúng ta chỉ cần tách data cần gửi thành nhiều đơn byte, sau đó gọi mô-đun truyền data đơn byte để gửi từng cái một, từ đó có thể thực hiện việc truyền data nhiều byte qua UART.

9.1.1 Nguyên lý truyền data nhiều byte qua UART

Để đảm bảo tính liên tục của data, ta có thể sử dùng tín hiệu hoàn thành truyền đơn byte (byte_tx_done) từ module truyền đơn byte làm tín hiệu điều kiện.Khi tín hiệu này active có nghĩa một byte data đã được gửi hoàn tất,lúc này cần xuất byte tiếp theo cho module truyền data đơn byte.

Trong một lần truyền data nhiều byte, lượng data cần gửi là xác định, cổng UART không thể gửi mãi được. Do đó, sau mỗi lần gửi hoàn tất một byte, chúng ta cần kiểm tra xem việc gửi đã hoàn thành hay chưa. Dựa vào tín hiệu byte_tx_done, chúng ta tính toán số byte đã gửi, sau đó so sánh với bề rộng data đã nhận. Nếu việc gửi chưa hoàn tất, sẽ tiếp tục gửi byte tiếp theo, khi xác định gửi đã hoàn tất, tín hiệu hoàn thành truyền data sẽ được active, kết thúc quá trình truyền.

Dựa vào đó, chúng ta có thể thiết kế một máy trạng thái để tạo ra data đơn byte, kiểm tra xem việc gửi từng byte có hoàn thành hay không, và xác định xem toàn bộ quá trình gửi đã hoàn tất chưa. Do đó, cấu trúc của mô-đun truyền dữ liệu qua cổng nối tiếp được thiết kế như Hình 9-3:



Hình 9-3 – Sơ đồ khối truyền nhận UART nhiều byte

9.1.2 Thiết kế module truyền dữ liệu nhiều byte qua UART

Sau khi xác định được chức năng và cấu trúc tổng thể của thiết kế, bước tiếp theo là viết code. Đầu tiên là việc khởi tạo mô-đun truyền dữ liệu đơn byte, mã như sau:

```
input Clk; input Rst_n;
input [2:0]Baud_Set; output uart_tx; output uart_state;
reg [7:0] data_byte; reg byte_send_en; wire byte_tx_done;
uart_byte_tx uart_byte_tx(
    .Clk(Clk),
    .Rst_n(Rst_n),
    .data_byte(data_byte),
    .send_en(byte_send_en),
    .Baud_Set(Baud_Set),
    .uart_tx(uart_tx),
    .Tx_Done(byte_tx_done),
    .uart_state(uart_state)
);
```

Để thiết kế có thể phù hợp với nhiều tình huống ứng dụng hơn, chúng ta có thể sử dụng **parameter** để định nghĩa bề rộng data đầu vào của mô-đun truyền data nhiều byte qua UART. **parameter** được sử dụng để định nghĩa hằng số bên trong hoặc bên ngoài mô-đun, lợi ích lớn nhất của cách này là có thể truyền tham số (ghi đè) khi khởi tạo mô-đun. Bằng cách này, chúng ta có thể sử dụng **parameter** để thiết lập bề rộng data đầu vào của mô-đun truyền dữ liệu nhiều byte là 8, mã như sau:

```
parameter DATA_WIDTH = 8;
```

Như vậy, trong trường hợp mặc định, mô-đun chỉ thực hiện việc truyền dữ liệu đơn byte. Tuy nhiên,

nếu chúng ta muốn mô-đun gửi nhiều byte cùng một lúc, ví dụ như 8 byte, chúng ta chỉ cần định nghĩa bề rộng dữ liệu đầu vào là 64 thông qua parameter ở cấp độ trên, và tham số sẽ được truyền đến mô-đun con để ghi đè giá trị hằng số.

Đối với mô-đun truyền dữ liệu nhiều byte qua UART, khi tín hiệu **send_en** đầu vào có hiệu lực, điều đó cho thấy quá trình truyền bắt đầu. FSM sẽ tạo ra dữ liệu gửi một byte và tín hiệu cho phép gửi cho mô-đun truyền dữ liệu đơn byte. Sau đó, máy trạng thái cần chờ tín hiệu byte_tx_done để xác nhận rằng byte dữ liệu hiện tại đã được gửi hoàn tất. Đồng thời, FSM cần đếm dựa trên tín hiệu này để xác định xem quá trình truyền đã hoàn tất hay chưa. Nếu chưa hoàn tất, nó sẽ tiếp tục gửi dữ liệu và tín hiệu cho phép cho byte tiếp theo. Nếu quá trình truyền đã hoàn tất, máy trạng thái sẽ phát sinh tín hiệu hoàn thành gửi **tx_done**, đánh dấu kết thúc quá trình truyền dữ liệu nhiều byte này. Tóm lại, FSM cần ít nhất bốn trạng thái để thực hiện các chức năng trên. Bốn trạng thái này lần lượt dùng để chờ tín hiệu cho phép gửi, khởi động việc gửi dữ liệu đơn byte, chờ hoàn thành việc gửi dữ liệu đơn byte, và kiểm tra xem tất cả dữ liệu đã được truyền hoàn tất hay chưa. Mã định nghĩa cho bốn trạng thái tương ứng như sau:

```
Localparam S0 = 0;  //chờ tín hiệu yêu cầu gửi send_en
localparam S1 = 1; //cho phép gửi và gửi 1 byte
localparam S2 = 2; //等待单字节数据发送完成
localparam S3 = 3; //检查所有数据是否发送完成
```

Tiếp theo, chúng ta sẽ triển khai chức năng cho bốn trạng thái, bắt đầu với trạng thái S0. Trạng thái này cần chờ tín hiệu yêu cầu gửi send_en. Khi send_en active, mô-đun sẽ nhận dữ liệu đầu vào và chuyển trạng thái sang S1. Verilog code tương ứng như sau:

```
S0:
    begin
        data_byte <= 0;
        cnt <= 0;
        Tx_Done <= 0;
        if(send_en)begin
            state <= S1;
            data_r <= data;//data_r chuỗi dữ liệu cần gửi
        end
        else begin
            state <= S0;
            data_r <= data_r;
        end
    end
```

Tiếp theo là trạng thái S1. Trạng thái này cần tạo ra tín hiệu cho phép gửi dữ liệu đơn byte và dữ liệu đơn byte, sau đó chuyển trạng thái sang S2. Để phù hợp với các yêu cầu khác nhau về thứ tự byte trong các tình huống ứng dụng khác nhau, chúng ta có thể sử dụng **parameter** để định nghĩa một hằng số **MSB_FIRST** để kiểm soát thứ tự byte (**big-endian** hoặc **little-endian**). Verilog code như sau:

```
parameter MSB_FIRST = 1;// 1: big-endian, 0 little-endian
```

GOWIN VIETNAM

Verilog code trạng thái S1

```
S1:// tạo tín hiệu cho phép gửi và gửi, data_byte dữ liệu hiện tại
    begin
        byte_send_en <= 1;
        if(MSB_FIRST == 1)//chọn byte cao trước
        begin
            data_byte <= data_r[DATA_WIDTH-1:DATA_WIDTH - 8];
            data_r <= data_r << 8; //dịch trái data_r 8 bit
        end
        else begin
            data_byte <= data_r[7:0]; //gán 8 bit thấp cho data_byte
            data_r <= data_r >> 8; //dịch phải data_r 8 bit
        end
        state <= S2;
    end
```

Tiếp theo là trạng thái S2. Ở đây, cần chờ tín hiệu byte_tx_done từ mô-đun truyền dữ liệu đơn byte. Khi tín hiệu này có hiệu lực, điều đó có nghĩa là dữ liệu đơn byte đã được gửi hoàn tất. Chúng ta có thể dựa vào tín hiệu này để đếm số lượng byte đã gửi và chuyển trạng thái sang S3. Verilog code như sau:

Cuối cùng là trạng thái S3. ở đây, chúng ta sẽ so sánh số lượng byte đã gửi với DATA WIDTH để phát sinh tín hiệu **Tx_Done**. Khi số lượng byte đã gửi ≥ bề rộng dữ liệu đầu vào, điều đó có nghĩa là tất cả dữ liệu đã được gửi hoàn tất, và chúng ta active tín hiệu **Tx_Done** và chuyển trạng thái trở lại S0 chờ lần truyền tiếp. Nếu số lượng dữ liệu đã gửi < **DATA WIDTH**, nghĩa là quá trình truyền vẫn chưa kết thúc, chúng ta sẽ chuyển trạng thái trở lại S1 để chuẩn bị truyền byte tiếp theo. Verilog code như sau:

GOWIN VIETNAM

```
S2: begin
        byte_send_en <= 0; // hạ xuống inactive sau khi active S1
        if(byte_tx_done)begin
            state <= S3;
            cnt <= cnt + a9'd8;
        end
        else
            state <= S2;
    end
```

```
S3: if(cnt >= DATA_WIDTH)begin state <= S0;
        cnt <= 0;
        Tx_Done <= 1;
    end
    else begin
        state <= S1;
        Tx_Done <= 0;
    end
```

Chúng ta sẽ tạo một file testbench mô phỏng để xác minh hoạt động của mô-đun truyền dữ liệu đa

byte. Tệp kích thích này sẽ bao gồm việc khởi tạo tín hiệu, cung cấp dữ liệu đầu vào, và kiểm tra các tín hiệu đầu ra như Tx_Done. Dưới đây là một ví dụ về cách tạo tệp kích thích mô phỏng cho mô-đun này.

9.1.3 Tạo file testbench và mô phỏng

Việc tạo Testbench mô phỏng tương đối đơn giản, chỉ cần thiết lập đúng **DATA WIDTH** và giá trị của kiểu dữ liệu (big-endian/ little-endian) khi khởi tạo mô-đun truyền dữ liệu đa byte, đồng thời tạo ra dữ liệu đầu vào và tín hiệu cho phép truyền trong tệp kích thích. Dưới đây là code hoàn chỉnh của Testbench mô phỏng lần này.

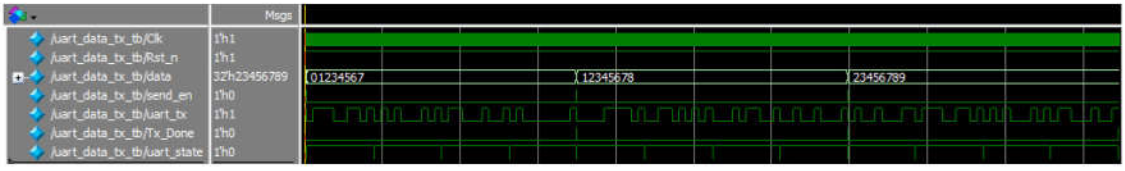
```
module uart_data_tx_tb();

    parameter DATA_WIDTH = 32; parameter MSB_FIRST = 0;

    reg Clk; reg Rst_n;
      reg [DATA_WIDTH - 1 : 0]data; reg send_en;
      wire uart_tx; wire Tx_Done; wire uart_state;

      uart_data_tx #(
          .DATA_WIDTH(DATA_WIDTH),
          .MSB_FIRST(MSB_FIRST)
      )
      uart_data_tx(
          .Clk(Clk),
          .Rst_n(Rst_n),
          .data(data),
          .send_en(send_en),
          .Baud_Set(3'd4),
          .uart_tx(uart_tx),
          .Tx_Done(Tx_Done),
          .uart_state(uart_state)
      );
      initial Clk = 1; always #10 Clk = !Clk; initial begin
          Rst_n = 0;
          data = 0;
          send_en = 0;
          #201;
          Rst_n = 1;
```

GOWIN VIETNAM

```
    #2000;
    data = 32'h01234567;
    send_en = 1;
    #20;
    send_en = 0;
    #20;
    @(posedge Tx_Done); #1;
    data = 32'h12345678;
    send_en = 1;
    #20;
    send_en = 0;
    #20;
    @(posedge Tx_Done); #1;
    data = 32'h23456789;
    send_en = 1; #20;
    send_en = 0;
    #20;
    @(posedge Tx_Done); #1;
    #2000;
    $stop;
    end endmodule
```

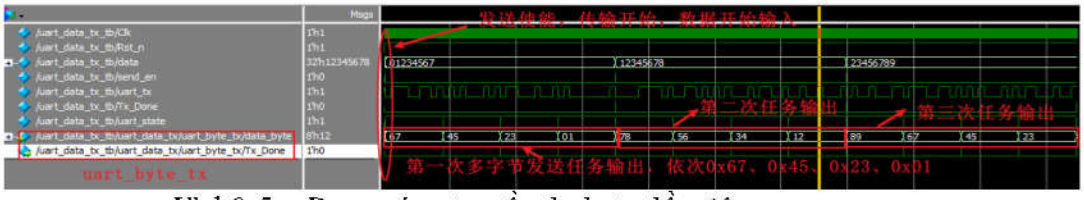
GOWIN VIETNAM

Trong tệp kích thích, bề rộng dữ liệu đầu vào được thiết lập là 32, đầu ra theo thứ tự byte little-endian, và đã nhập ba dữ liệu đã biết. Do đó, lấy dữ liệu đầu vào lần đầu tiên làm ví dụ, trong lần truyền dữ liệu đa byte đầu tiên, dữ liệu đầu ra sẽ lần lượt là 0x67, 0x45, 0x23, 0x01, tổng cộng 32 byte dữ liệu. Hai lần truyền tiếp theo cũng sẽ cho kết quả tương tự. So sánh sóng mô phỏng với kết luận, chúng ta sẽ biết được thiết kế có đúng hay không. Sóng mô phỏng lần này được thể hiện trong Hình 9-4.



Hình 9-4 – Dạng sóng mô phỏng truyền đa byte

Có thể thấy ba lần dữ liệu đều được nhận thành công, và cổng nối UART đã hoàn thành ba nhiệm vụ truyền đa byte. Để thuận tiện cho việc xem dữ liệu cụ thể mà UART đã gửi, ở đây đã thêm tín hiệu data_byte và Tx_Done của mô-đun uart_byte_tx để quan sát. Thời gian sóng hoàn chỉnh như Hình 9-5 được thể hiện.



Hình 9-5 – Dạng sóng truyền đa byte đầu tiên

Có thể thấy, sau khi tín hiệu **send_en** có hiệu lực, mô-đun truyền dữ liệu đa byte bắt đầu hoạt động, giá trị đầu ra phù hợp với kết quả mong đợi của lần truyền dữ liệu đa byte đầu tiên. Kết quả của hai lần truyền tiếp theo cũng phù hợp với mong đợi, ở đây tạm thời không liệt kê chi tiết thời gian, thiết kế lần này thành công

9.2 Nguyên lý và thiết kế module nhận đa byte

Mô-đun nhận dữ liệu cổng nối tiếp khi hoạt động sẽ nhận dữ liệu nối tiếp liên tục và xuất dữ liệu hợp lệ theo từng byte. Cái gọi là nhận dữ liệu đa byte là khi mô-đun nhận dữ liệu cổng nối tiếp nhận được dữ liệu, không xuất ra dữ liệu từng byte nữa, mà sẽ ghép nối và kết hợp dữ liệu thành một dữ liệu đa byte sau khi xuất ra.

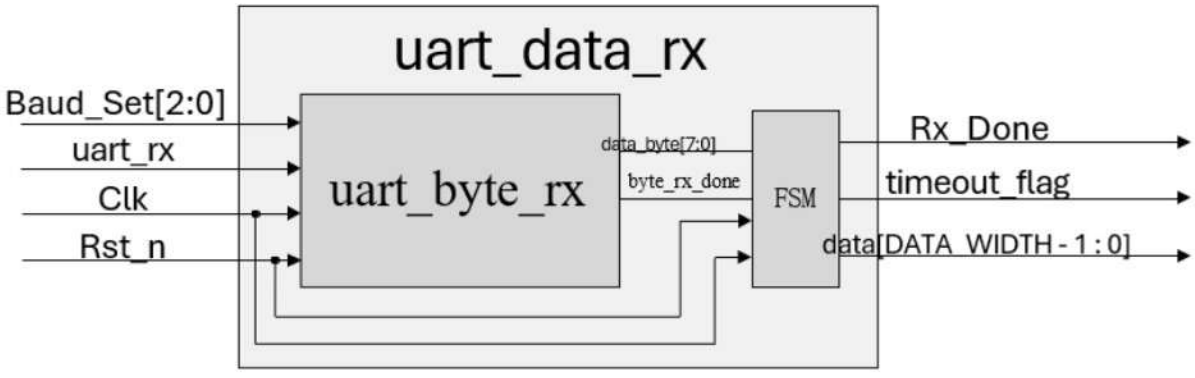
Kết hợp với mô-đun truyền dữ liệu đa byte mà chúng ta đã thiết kế trước đó, để thực hiện thiết kế mô-đun nhận dữ liệu đa byte, chúng ta có thể khởi tạo mô-đun nhận dữ liệu cổng nối tiếp từng byte, sau đó thiết kế một máy trạng thái để ghép nối dữ liệu đầu ra của mô-đun nhận dữ liệu từng byte. Đồng thời, dựa trên tín hiệu hoàn thành nhận dữ liệu do mô-đun nhận dữ liệu từng byte tạo ra, đếm lượng dữ liệu hiện tại đã nhận (đã ghép nối). Khi lượng dữ liệu ghép nối đạt đến bề rộng dữ liệu chỉ định, sẽ xuất dữ liệu.

9.2.1 Nguyên lý nhận đa byte UART

Tất nhiên, chỉ như vậy là không đủ. Nếu trong quá trình nhận dữ liệu cổng UART, do một số nhiễu bên ngoài, dẫn đến một phần dữ liệu bị thay đổi và không thể nhận diện, thì dữ liệu được nhận diện thành công sau đó sẽ thay thế vị trí của những dữ liệu này, cuối cùng ghép nối thành dữ liệu sai.。

Ví dụ, giả sử trong 1 lần nhận dữ liệu, UART đáng lẽ có thể nhận đúng 0x98, 0x76, 0x54, 0x32. Nhưng trong quá trình truyền, do nhiễu bên ngoài, một vài bit của 0x76 đã bị thay đổi và không được UART nhận diện, cuối cùng dữ liệu nhận được là 0x98, 0x54, 0x32. Nếu những dữ liệu này được đưa trực tiếp cho FSM để ghép nối, dữ liệu đầu ra chắc chắn sẽ sai.

Để ngăn chặn tình huống này, chúng ta cần thực hiện việc đếm thời gian cho mỗi lần nhận, ngay khi thời gian nhận vượt quá giới hạn, sẽ phát sinh tín hiệu cờ thời gian nhận, đồng thời thông qua máy trạng thái kết thúc quá trình nhận dữ liệu cổng nối tiếp này. Qua phân tích trên, chúng ta có thể đưa ra cấu trúc hệ thống thiết kế như Hình 9-6:



Hình 9-6 – Sơ đồ khối module nhận UART đa byte

9.2.2 Thiết module nhận đa byte qua UART

Sau khi đã xác định được cấu trúc tổng thể của thiết kế, viết verilog code. Trước tiên là **khởi tạo mô-đun nhận UART đơn byte**, với đoạn mã như sau:

```
input Clk;
input Rst_n;
input uart_rx;
wire [7:0] data_byte;
wire byte_rx_done;
uart_byte_rx uart_byte_rx(
    .Clk(Clk),
    .Rst_n(Rst_n),
    .baud_set(Baud_Set),
    .uart_rx(uart_rx),
    .data_byte(data_byte),// byte vừa được nhận từ mô-đun UART 1 byte
    .Rx_Done(byte_rx_done)// cờ báo hiệu hoàn tất nhận dữ liệu
);
```

Để thiết kế có thể thích ứng với nhiều tình huống ứng dụng hơn, chúng ta có thể sử dụng **tham số (parameter)** để định nghĩa độ rộng bit đầu ra của mô-đun nhận UART nhiều byte là **16 bit**.Bằng cách này, khi gọi (sử dụng) mô-đun, ta có thể dễ dàng **thay đổi độ rộng đầu ra** bằng cách chỉnh sửa từ cấp trên (top-level). Code như sau:

```
parameter DATA_WIDTH = 16;
```

Tiếp theo, chúng ta cần xem xét **cách tạo tín hiệu báo quá thời gian khi nhận dữ liệu UART**.Trong thực tế, UART có thể hoạt động ở **nhiều tốc độ baud khác nhau**, và tương ứng, **tần số truyền dữ liệu** cũng sẽ khác nhau. Vậy thì **làm sao để xác định rằng việc nhận dữ liệu đã bị "quá thời gian"?**

Ở đây, chúng ta tham khảo **định nghĩa về timeout trong giao thức Modbus**.

Trong Modbus, quy định rằng **khoảng cách giữa hai bản tin** phải lớn hơn **3.5 lần thời gian truyền 1 ký tự**. Tức là: **nếu thời gian truyền vượt quá 3.5 ký tự**, thì xem như là một **bản tin mới bắt đầu**. Dựa trên nguyên tắc đó, trong thiết kế này, **chúng ta coi việc nhận UART bị quá thời gian nếu thời gian giữa hai byte nhận liên tiếp vượt quá 3.5 ký tự**.

Lấy ví dụ với tốc độ baud **115200 bps**, tức 1s truyền được 115200 bits, ta tính được:

- Thời gian truyền 1 byte là: (1/115200) x 10^6 x (1 + 8 + 1) = 86.805 \ **µs**] (gồm 1 start bit, 8 data bit, 1 stop bit).
- Như vậy: **3.5 ký tự = 86.805 × 3.5 = 300.8175 µs** → Nếu UART không nhận được dữ liệu nào trong **300.8175µs**, thì được coi là **quá thời gian nhận**.

Trong thiết kế này, chúng ta dùng **clock 50 MHz** (chu kỳ = 20ns). Thời gian nhận 1 byte sẽ được đo bằng **số lần clock giữa 2 tín hiệu kết thúc byte (byte_rx_done) liên tiếp**.

Tính ra, khi bộ đếm đạt: 300.8175×10^3/20−1=15190 → thì có thể coi là **đã quá thời gian**.

Khi quá thời gian, ta sẽ tạo ra **tín hiệu timeout** (timeout flag).

Bộ đếm này sẽ bắt đầu đếm **mỗi khi nhận 1 byte mới**, và được **reset về 0 khi kết thúc nhận 1 byte**.

Cụ thể, ta tận dụng:

- Tín hiệu **mức thấp bắt đầu truyền** (!uart_rx)
- Và tín hiệu **byte_rx_done** từ mô-đun nhận 1 byte UART

Nhờ đó, ta xác định chính xác thời điểm bắt đầu & kết thúc của từng byte→active timeout.

```
parameter TIMEOUT = 15190;//115200 sẽlà 15190,9600 sẽlà 182292

reg [31:0]timeout_cnt;
always@(posedge Clk or negedge Rst_n)
if(!Rst_n)
    timeout_flag <= 1'd0;
else if(timeout_cnt >= TIMEOUT)
    timeout_flag <= 1'd1;
else if(state == S0)
    timeout_flag <= 1'd0;

reg to_state;// dùng để theo dõi trạng thái của việc nhận dữ liệu
always@(posedge Clk or negedge Rst_n)
if(!Rst_n)
    to_state <= 0;
else if(!uart_rx)
    to_state <= 1;
else if(byte_rx_done)
    to_state <= 0;
always@(posedge Clk or negedge Rst_n)
if(!Rst_n)
    timeout_cnt <= 32'd0;
else if(to_state)begin
    if(byte_rx_done)
        timeout_cnt <= 32'd0;
    else if(timeout_cnt >= TIMEOUT)
    timeout_cnt <= TIMEOUT;//nếu timeout_cnt=TIMEOUT giữ nguyên=TIMEOUT
    else
        timeout_cnt <= timeout_cnt + 1'd1;
end
```

GOWIN VIETNAM

Sau khi thiết kế xong **tín hiệu quá thời gian nhận dữ liệu**, bước tiếp theo là thiết kế **máy trạng thái (FSM – Finite State Machine)**. Kết hợp với phần phân tích nguyên lý và sơ đồ cấu trúc trước đó, ta có thể thấy rằng **quy trình hoạt động của FSM trong thiết kế lần này như sau**:

1. **Chờ tín hiệu hoàn tất nhận dữ liệu từ mô-đun nhận 1 byte**, lưu vào bộ đệm byte đầu tiên và bắt đầu đếm số byte đã nhận.
2. **Bắt đầu bộ đếm timeout**. Nếu **timeout xảy ra trước khi byte tiếp theo đến**, thì tạo tín hiệu **Rx_Done**, kết thúc quá trình nhận. Nếu **chưa timeout**, thì ghép thêm byte mới vào dữ liệu đã có và tiếp tục đếm số byte đã nhận.
3. **So sánh số byte đã nhận với độ rộng dữ liệu đầu ra**. Nếu đã đủ, tạo tín hiệu **Rx_Done**, kết thúc nhận. Nếu chưa đủ, **quay lại bước 2** để tiếp tục nhận.

Theo đó，ta chỉ cần 3 trạng thái để thực hiện chức năng cho FSM, code như sau:

```
localparam S0 = 0;  //chờ tín hiệu hoàn thành nhận 1 byte và nhận byte
localparam S1 = 1;  //Xác định xem có quá thời gian không?
localparam S2 = 2;  //Kiểm tra hoàn thành truyền dữ liệu nhiều byte
```

Thực hiện **trạng thái S0,S1,S2**. Các ứng dụng khác nhau, có thể có yêu cầu khác nhau về **thứ tự byte của dữ liệu** (byte order), ta sử dụng **parameter** định nghĩa 1 hằng số **MSB_FIRST**, điều khiển chọn **big-endian** hay **little-endian**. Code như sau:

```
S0, S1
    begin
        Rx_Done <= 0;
        data_r <= 0;
        if(DATA_WIDTH == 8)begin
            data <= data_byte;
            Rx_Done <= byte_rx_done;
        end
        else if(byte_rx_done)begin state <= S1;
            cnt <= cnt + 9'd8; if(MSB_FIRST == 1)
                data_r <= {data_r[DATA_WIDTH - 1 - 8 : 0], data_byte};
            else
                data_r <= {data_byte, data_r[DATA_WIDTH - 1 : 8]};
        end
    end
    if(timeout_flag)begin state <= 0;
        Rx_Done <= 1;
    end
    else if(byte_rx_done)begin state <= S2;
        cnt <= cnt + 9'd8; if(MSB_FIRST == 1)
            data_r <= {data_r[DATA_WIDTH - 1 - 8 : 0], data_byte};
        else
            data_r <= {data_byte, data_r[DATA_WIDTH - 1 : 8]};
    end
```

```
S2:
    if(cnt >= DATA_WIDTH)begin state
        <= S0;
        cnt <= 0;
        data <= data_r;
        Rx_Done <= 1;
    end
    else begin
        state <= S1;
        Rx_Done <= 0;
    end
```

GOWIN VIETNAM

Tạo Testbench mô phỏng

Để thuận tiện cho việc kiểm tra, trong testbench, ta có thể khởi tạo mô-đun gửi chuỗi nhiều byte đã thiết kế trước đó, nhằm tạo ra tín hiệu đầu vào nối tiếp cần thiết để kiểm tra mô-đun nhận chuỗi nhiều byte. Code của tệp kích thích mô phỏng lần này như sau:

```
module uart_data_rx_tb();

    parameter DATA_WIDTH = 32;
    parameter MSB_FIRST = 0;
    reg Clk;
    reg Rst_n;

    reg [DATA_WIDTH - 1 : 0]data;
    wire [DATA_WIDTH - 1 : 0]rx_data;
    reg send_en;
    wire uart_tx;
    wire uart_rx;
    wire Tx_Done;
    wire uart_state;
    wire Rx_Done;
    wire  timeout_flag;

    ssign uart_rx = uart_tx;

    uart_data_tx
    #(
        .DATA_WIDTH(DATA_WIDTH),
        .MSB_FIRST(MSB_FIRST)
    )
    uart_data_tx(
        .Clk(Clk),
        .Rst_n(Rst_n),
        .data(data),
        .send_en(send_en),
        .Baud_Set(3'd4),
        .uart_tx(uart_tx),
        .Tx_Done(Tx_Done),
        .uart_state(uart_state)
    );
```

GOWIN VIETNAM

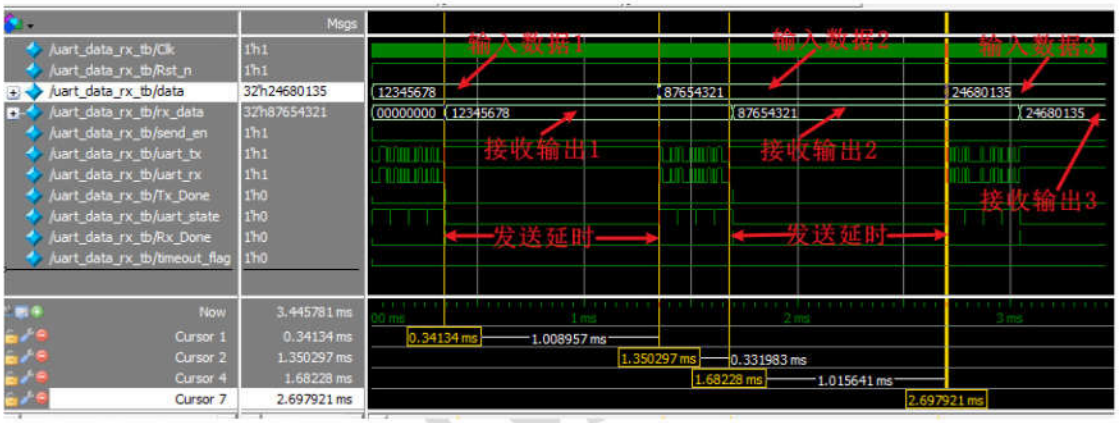
```
        uart_data_rx #(
            .DATA_WIDTH(DATA_WIDTH),
            .MSB_FIRST(MSB_FIRST)
        )
        uart_data_rx(
            .Clk(Clk),
            .Rst_n(Rst_n),
            .uart_rx(uart_rx),
            .data(rx_data),
            .Rx_Done(Rx_Done),
            .timeout_flag(timeout_flag),
            .Baud_Set(3'd4)
        );
        initial Clk = 1; always #10 Clk = !Clk;
        initial begin
            Rst_n = 0;
            data = 0;
            send_en = 0;
            #201;
            Rst_n = 1;
            #2000;
            data = 32'h12345678;
            send_en = 1;
            #20;
            send_en = 0;
            #20;
        @(posedge Tx_Done);
            #1;
            #1000000;
            data = 32'h87654321;
            send_en = 1;
            #20;
            send_en = 0;
            #20;
            @(posedge Tx_Done); #1;
            #1000000;
            data = 32'h24680135;
            send_en = 1;
            #20;
            send_en = 0;
            #20;
    @(posedge Tx_Done);
        #1;
        #400000;
        $stop;
    End
endmodule
```

GOWIN VIETNAM

Trong tệp kích thích, tốc độ baud được thiết lập là 115200. Để xác minh xem mô-đun có thể xác định chính xác việc truyền có bị quá thời gian hay không, khoảng cách giữa hai lần nhập dữ liệu được đặt là 1000.001us.

Đồng thời, trong tệp kích thích, DATA_WIDTH được thiết lập là 32, và thứ tự byte được đặt là little-endian. Do là thứ tự byte nhỏ, nên mô-đun gửi chuỗi nhiều byte sẽ gửi byte thấp trước rồi đến byte cao. Lấy ví dụ lần gửi dữ liệu đầu tiên, mô-đun gửi chuỗi nhiều byte sẽ lần lượt gửi dữ liệu 0x78, 0x56, 0x34, 0x12. Vì mô-đun nhận chuỗi nhiều byte cũng sử dụng thứ tự byte nhỏ, nên các byte dữ liệu khi được mô-đun nhận chuỗi nhiều byte tiếp nhận và ghép lại, byte nhận trước sẽ được coi là byte cao nhất. Do đó, dữ liệu đầu ra của mô-đun nhận chuỗi nhiều byte sẽ là 32'h12345678。

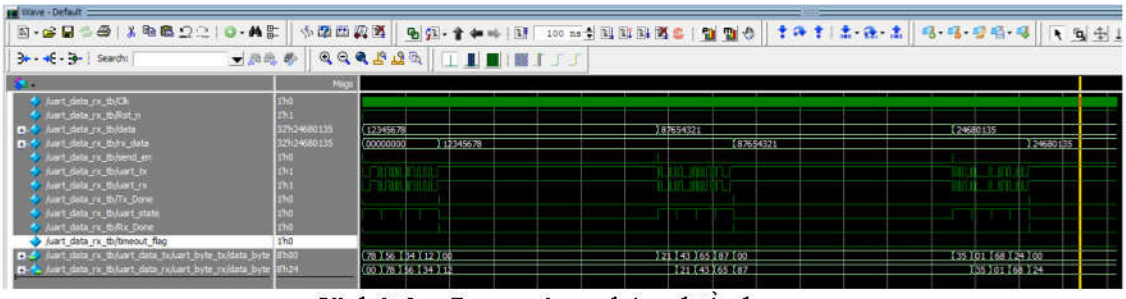
Quan sát dạng sóng mô phỏng và kiểm tra xem có nhất quán với suy luận hay không, chúng ta có thể biết thiết kế có chính xác hay không. Dạng sóng tổng thể của thiết kế mô phỏng lần này như Hình 9-7.



Hình 9-7– Dạng sóng tổng thể

Có thể thấy, ngay cả khi bị trễ 1000.001us, tín hiệu **timeout_flag** cũng không được đặt cao, và dữ liệu đầu ra của mô-đun nhận chuỗi nhiều byte phù hợp với dự kiến của chúng ta.

Tiếp theo, chúng ta sẽ quan sát quá trình truyền dữ liệu ở giữa, xem thứ tự byte có hoạt động bình thường hay không. Để thuận tiện cho việc quan sát, trong mô phỏng đã thêm tín hiệu data_byte (dữ liệu byte đầu ra) của **uart_byte_tx** và tín hiệu **data_byte** (dữ liệu byte đầu vào) của uart_byte_rx, thời gian sóng như Hình 9-8.



Hình 9-8 – Dạng sóng nhận nhiều byte

Có thể thấy, dạng sóng dữ liệu phù hợp với những gì chúng ta đã suy luận, cho thấy chức năng thứ tự byte cũng hoạt động bình thường. Đến đây, thiết kế mô-đun nhận chuỗi nhiều byte đã hoàn thành.

9.3 Tạo top module và thử nghiệm trên phần cứng

Mặc dù chúng ta đã thông qua mô phỏng để kiểm tra và xác minh tính khả thi của chức năng hai mô-đun, nhưng cần lưu ý rằng mô phỏng cuối cùng chỉ là điều kiện lý tưởng và luôn có sự khác biệt so

với tình huống thực tế. Do đó, thiết kế vẫn cần phải thực hiện kiểm tra trên bảng mạch để xác minh tính toàn vẹn của chức năng.

Để thuận tiện cho việc thử nghiệm, chúng ta sẽ tạo top module code mới, khởi tạo mô-đun nhận và gửi chuỗi nhiều byte, kết nối đầu ra của mô-đun nhận với đầu vào của mô-đun gửi. Như vậy, dữ liệu mà mô-đun nhận nhận được sẽ được gửi đi qua mô-đun gửi, và thông qua phần mềm điều chỉnh chuỗi trên máy tính, chúng ta có thể thực hiện một vòng lặp chuỗi đơn giản. Bằng cách kiểm tra xem dữ liệu trong vòng lặp có nhất quán hay không, chúng ta sẽ biết được chức năng của mô-đun có được thực hiện hay không.

```
module uart_loopback( Clk,
    Rst_n, uart_rx,
    led, uart_tx
 );
    parameter DATA_WIDTH = 32; parameter MSB_FIRST = 0;
    input Clk; input Rst_n; input uart_rx;
    output [2:0]led; output uart_tx;
    wire [DATA_WIDTH-1:0]data; wire Rx_Done;
    wire [7:0]data_byte;
    uart_data_rx #(
        .DATA_WIDTH(DATA_WIDTH),
        .MSB_FIRST(MSB_FIRST)
    )
    uart_data_rx(
        .Clk(Clk),
        .Rst_n(Rst_n),
        .uart_rx(uart_rx),

        .data(data),
        .Rx_Done(Rx_Done),
        .timeout_flag(led[0]), .Baud_Set(3'd4)
    );
    uart_data_tx #(
        .DATA_WIDTH(DATA_WIDTH),
        .MSB_FIRST(MSB_FIRST)
                                                )uart_data_tx(
                                                    .Clk(Clk),
        .Rst_n(Rst_n),

        .data(data),
        .send_en(Rx_Done),
        .Baud_Set(3'd4),

        .uart_tx(uart_tx),
        .Tx_Done(led[1]),
        .uart_state(led[2])
    );
endmodule
```

GOWIN VIETNAM

Trong Top Module, ta set DATA_WIDTH là 32, baudrate 115200, tín hiệu timeout, tín hiệu hoàn thành

gửi, và tín hiệu trạng thái chuỗi được kết nối với đèn LED để hiển thị.

Sau khi thêm top module thử nghiệm và tổng hợp không có lỗi, cần thêm phân bổ chân và ràng buộc cho thiết kế. Bảng phân bổ chân cho thiết kế lần này được trình bày trong bảng 9-1.

Signial Name	Pin NO.	Signal Name	Pin NO.
CLK	T9	led[2]	B9
Rst_n	B16	led[1]	C14
uart_rx	U8	led[0]	D14
uart_tx	V8		

Bảng 9-1– Bảng gán chân

Ràng buộc mức chân là LVCMOS 33, sau khi thiết lập xong, biên dịch thiết kế và tạo tệp fs, tiếp theo chuẩn bị thực hiện xác minh cấp bảng mạch cho thiết kế.

9.4 Kiểm chứng trên bo mạch

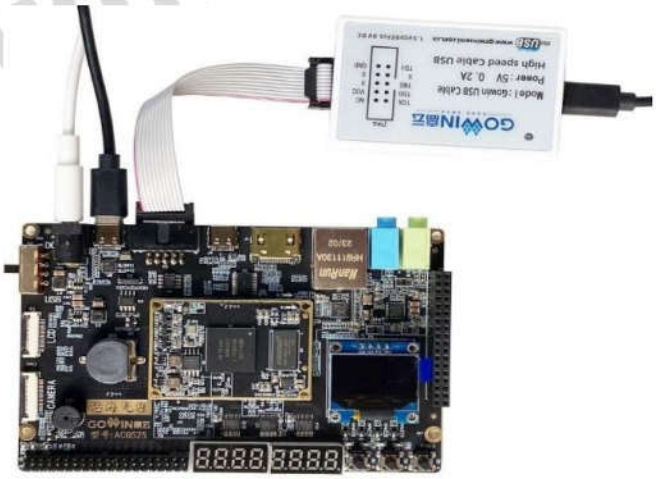
Trong phần này, sẽ sử dụng mô-đun nhận và gửi chuỗi nhiều byte đã thiết kế trên bảng phát triển ACG525, thông qua cách vòng lặp chuỗi để phản hồi dữ liệu từ máy tính. Bằng cách so sánh dữ liệu trong vòng lặp với dữ liệu đã gửi, chúng ta sẽ xác minh mô-đun thiết kế.

9.4.1 Phần cứng yêu cầu của hệ thống

1. ACG525 Development Board x1
2. Mạch nạp Gowin x1
3. Mạch USB Cable x1
4. Adapter nguồn x15

9.4.2 Kết nối phần cứng

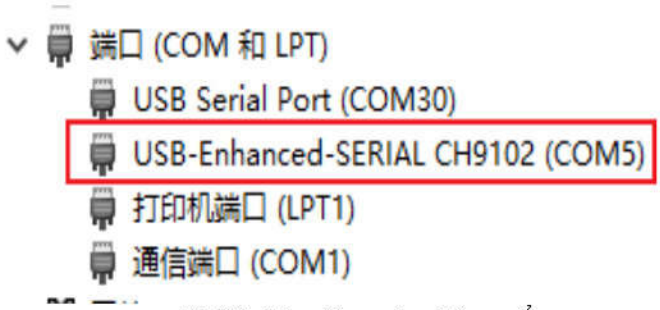
Kết nối dây tải xuống và dây nguồn của bảng phát triển, sử dụng dây chuỗi một đầu kết nối với giao diện UART trên bảng phát triển, đầu còn lại kết nối với cổng USB của PC



Hình 9-9 – Kết nối phần cứng

Sau khi kết nối hoàn tất, đảm bảo công tắc nguồn của bảng phát triển được bật sang phía ON, tiếp theo sẽ ghi tệp fs đã tạo vào bảng phát triển.

Sau khi ghi xong, mở Trình quản lý thiết bị, kiểm tra số cổng chuỗi, tên cổng chuỗi trong Trình quản lý thiết bị như Hình 9-10.



Hình 9-10 – Tra cứu đúng cổng

Ở đây, số cổng PC của tác giả là COM5, người dùng cần dựa vào số cổng trên máy tính của mình khi tra cứu. Sau đó, mở phần mềm Chuỗi Hunter, kết nối với cổng vừa tra cứu, thiết lập tốc độ baud 115200. Tiếp theo, nhập dữ liệu 4 byte và nhấn gửi để bắt đầu vòng lặp, như kết quả thí nghiệm trong Hình 9-11.

Hình 9-11 – Kiểm tra kết quả

Trong thí nghiệm này, tác giả đã nhiều lần gửi dữ liệu với tổng cộng 76 byte. Kết quả cho thấy phần mềm đã nhận lại dữ liệu vòng lặp với độ dài đúng 76 byte và nội dung nhận được hoàn toàn trùng khớp với dữ liệu đã gửi. Điều này chứng tỏ chức năng thu phát của giao tiếp nối tiếp hoạt động bình thường, xác nhận rằng thiết kế mô-đun thu phát nhiều byte qua UART đã thành công.

9.5 Tổng kết chương

Chương này đã hướng dẫn mọi người hoàn thành thiết kế mô-đun nhận chuỗi nhiều byte và mô-đun gửi chuỗi nhiều byte, đồng thời thông qua thử nghiệm mô phỏng và xác minh cấp bảng mạch để chứng minh tính khả thi của thiết kế. Để thuận tiện sử dụng, một số tín hiệu trong mô-đun được định nghĩa bằng tham số parameter, người dùng có thể truyền tham số từ lớp bao bọc cấp cao và điều chỉnh giá trị các tham số này theo nhu cầu thiết kế của mình.

Chương 10: Thiết kế FSM (Finite State Machine)

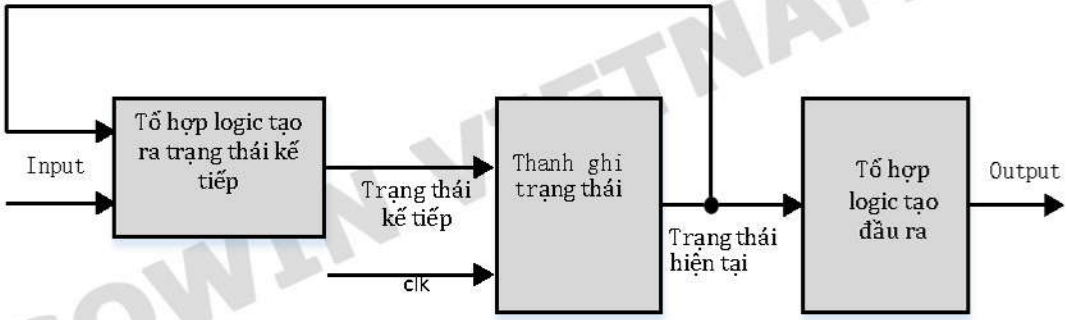
Mã nguồn project	| - 02_设计实例 |	ch10_fsm_hello

Dẫn nhập chương

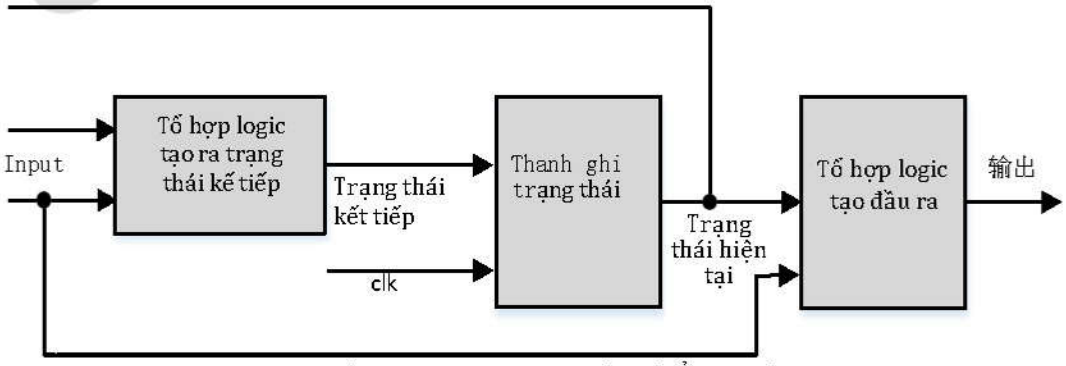
Máy trạng thái, tên đầy đủ là **Máy trạng thái hữu hạn** (Finite State Machine – FSM), là một mô Hình toán học dùng để mô tả một số hữu hạn trạng thái cùng với các chuyển đổi giữa các trạng thái đó và các hành động tương ứng. Trong phần này, chúng ta sẽ tìm hiểu các khái niệm liên quan đến máy trạng thái và sử dụng máy trạng thái để thực hiện việc phát hiện chuỗi ký tự cụ thể. Đồng thời, ta cũng sẽ tìm hiểu sự khác biệt giữa máy trạng thái một đoạn, hai đoạn và ba đoạn, cũng như ưu nhược điểm của từng loại.

10.1 Nguyên lý hoạt động của FSM

Máy trạng thái được chia thành máy trạng thái hữu hạn kiểu Moore và máy trạng thái hữu hạn kiểu Mealy. Đầu ra của máy trạng thái Moore chỉ được xác định bởi trạng thái hiện tại (không phụ thuộc trực tiếp vào đầu vào). Đầu ra của máy trạng thái hữu hạn Mealy không chỉ liên quan đến trạng thái hiện tại mà còn liên quan đến đầu vào của nó, đây là điểm khác biệt với máy trạng thái hữu hạn Moore. Sơ đồ nguyên lý của chúng lần lượt như Hình 10-1 máy trạng thái kiểu Moore và Hình 10-2 máy trạng thái kiểu Mealy.



Hình 10-1– Máy trạng thái kiểu Moore



Hình 10-2 – Máy trạng thái kiểu Mealy

Máy trạng thái có thể thực hiện chuyển đổi trạng thái theo các tín hiệu điều khiển dựa trên trình tự trạng thái đã được thiết lập trước. Do đó, nảy sinh vấn đề làm thế nào để **mã hóa trạng thái một cách hiệu quả**. Phương pháp mã hóa đơn giản nhất là **sử dụng mã nhị phân trực tiếp**. Ngoài ra còn có thể sử dụng **mã Gray** hoặc **mã one-hot (mã một nóng)**. Giả sử có **8 trạng thái từ A đến H**, các phương pháp mã hóa khác nhau tương ứng được trình bày trong **bảng 10-1** dưới đây.

Bảng 10-1 Định dạng mã hóa cho các trạng thái khác nhau

Trạng thái	Định dạng mã hóa		
	Nhị phân	Mã one-hot	Mã Gray
A	3`b000	8`b0000_0000	4`b0000
B	3`b001	8`b0000_0010	4`b0001
C	3`b010	8`b0000_0100	4`b0011
D	3`b011	8`b0000_1000	4`b0010
E	3`b100	8`b0001_0000	4`b0110
F	3`b101	8`b0010_0000	4`b0111
G	3`b110	8`b0100_0000	4`b0101
H	3`b111	8`b1000_0000	4`b0100

Mã one-hot (mã một nóng): mỗi trạng thái sử dụng một thanh ghi riêng, khi so sánh trạng thái chỉ cần kiểm tra một bit, do đó so với các phương pháp mã hóa khác thì mạch giải mã đơn giản hơn.;

Mã Gray: số lượng thanh ghi cần thiết giống với mã nhị phân, mạch giải mã phức tạp hơn, nhưng hai trạng thái liền kề chỉ thay đổi một bit. Mã Gray thường được dùng trong các ứng dụng chuyển đổi nhiều bit giữa các miền xung nhịp không đồng bộ, chẳng hạn như FIFO bất đồng bộ.

Mã nhị phân: là phương pháp mã hóa phổ biến nhất, sử dụng ít thanh ghi hơn nhưng mạch giải mã phức tạp hơn.

Theo khuyến nghị của các nhà sản xuất FPGA, việc lựa chọn phương pháp mã hóa nào cần căn cứ vào **độ phức tạp của máy trạng thái, loại thiết bị** và **yêu cầu phục hồi từ trạng thái bất hợp lệ**.

- Sử dụng mã nhị phân sẽ tiêu tốn ít thanh ghi hơn so với mã one-hot.
- Với mã nhị phân, chỉ cần 7 thanh ghi là có thể thực hiện một máy trạng thái với 100 trạng thái, trong khi mã one-hot sẽ cần tới 100 thanh ghi.

-Tuy nhiên, ngược lại, mặc dù mã one-hot sử dụng nhiều thanh ghi hơn, nhưng **logic tổ hợp** của nó lại **đơn giản hơn**.

Thông thường:

- Đối với **CPLD** (thiết bị có tài nguyên logic tổ hợp phong phú), **nên dùng mã nhị phân**.
- Đối với **FPGA** (thiết bị có nhiều tài nguyên logic tuần tự hơn), **nên dùng mã one-hot**.
- Các phương pháp mô tả máy trạng thái:
- **Mô tả một đoạn**: toàn bộ máy trạng thái được viết trong một khối always, vừa mô tả việc chuyển trạng thái, vừa mô tả đầu vào và đầu ra.
- **Mô tả hai đoạn**: dùng hai khối always để mô tả máy trạng thái. Một khối always mô tả việc chuyển trạng thái bằng logic tuần tự đồng bộ (theo xung nhịp), khối always còn lại sử dụng logic tổ hợp để xác định điều kiện chuyển trạng thái, đồng thời mô tả quy luật chuyển trạng thái và đầu ra.
- **Mô tả ba đoạn**: dựa trên phương pháp hai đoạn, thêm một khối always riêng biệt chỉ dùng để mô tả đầu ra trạng thái (có thể dùng mạch tổ hợp).
- Một khối **always** sử dụng logic tuần tự đồng bộ để mô tả quá trình chuyển trạng thái.
- Một khối **always** sử dụng logic tổ hợp để xác định điều kiện chuyển trạng thái và mô tả quy luật chuyển trạng thái.
- Khối **always** còn lại dùng để mô tả đầu ra của trạng thái, và phần đầu ra này có thể được tạo ra bằng mạch tổ hợp hoặc mạch tuần tự.

Có thể thấy rằng, **sự khác biệt giữa máy trạng thái hữu hạn hai đoạn và một đoạn** nằm ở chỗ:

GOWIN VIETNAM

Phương pháp hai đoạn **tách rời phần tuần tự (chuyển trạng thái)** và **phần tổ hợp (xác định điều kiện chuyển trạng thái và sinh đầu ra)** thành **hai khối always riêng biệt**, do đó được gọi là **máy trạng thái hai đoạn**.

Nếu tiếp tục **tách riêng phần xác định điều kiện chuyển trạng thái và phần sinh đầu ra** trong phần tổ hợp thành hai khối always riêng, thì đó chính là **máy trạng thái ba đoạn**.

Trong **máy trạng thái hai đoạn**, khi **logic tổ hợp đặc biệt phức tạp**, cần chú ý **thêm một bộ chốt (flip-flop)** ở đầu ra để **loại bỏ nhiễu đột biến (glitch)** do logic tổ hợp gây ra.

Còn với **máy trạng thái ba đoạn**, vấn đề này **không xảy ra**, vì **khối always thứ ba dùng để mô tả đầu ra sẽ tự động sinh ra bộ chốt**, giúp ổn định tín hiệu đầu ra.

Thực tế, **các thiết bị phần cứng hiện nay không còn quá khắt khe về tiêu thụ tài nguyên** như trước đây. Vì vậy, **việc sử dụng máy trạng thái hai đoạn hoặc ba đoạn, kết hợp với đầu ra có chốt (output registered)** là cách được khuyến nghị khi mô tả máy trạng thái hữu hạn (FSM).

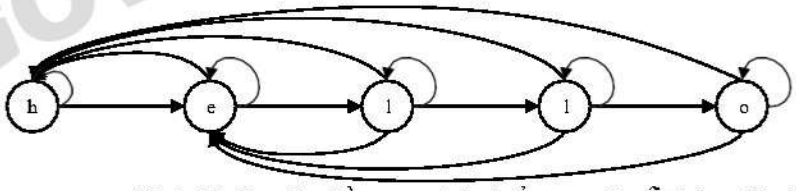
Khi viết máy trạng thái, cũng cần lưu ý những điểm quan trọng sau đây:

1. Để **tránh sinh ra latch (chốt không mong muốn)**, cần **liệt kê đầy đủ tất cả các hành động đầu ra tương ứng với từng trạng thái**, hoặc **sử dụng mệnh đề default** để định nghĩa hành vi cho các trạng thái chưa được xác định.
2. Khi định nghĩa các trạng thái, **khuyến khích sử dụng localparam hoặc parameter để đặt tên trạng thái**, điều này giúp **mã dễ đọc, rõ ràng và 3. ít sai sót hơn**, đồng thời thuận tiện cho việc sửa đổi sau này.

Ngoài ra, trong trường hợp **reset hoặc trạng thái bị chạy sai (runaway)**, cần đảm bảo có thể **quay lại trạng thái khởi tạo hoặc trạng thái định trước**. Vì vậy, trong thiết kế cần có **mạch reset đồng bộ hoặc không đồng bộ** để đảm bảo **máy trạng thái có trạng thái khởi đầu sau khi cấp nguồn**.

10.2 Khiển khai FSM phát hiện chuỗi

Để thực hiện việc phát hiện chuỗi ký tự "hello", trước tiên cần vẽ sơ đồ chuyển trạng thái, như được minh họa trong **Hình 10-3 dưới đây**



Hình 10-3 – Sơ đồ trạng thái kiểm tra chuỗi ký tự "hello"

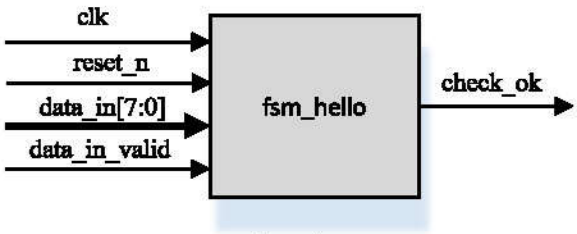
Từ sơ đồ trên có thể thấy rằng: nếu trạng thái hiện tại **không thỏa mãn điều kiện chuyển tiếp**, thì **máy trạng thái sẽ quay lại trạng thái khởi đầu h**, hoặc quay lại **trạng thái e** (trong trường hợp không đủ điều kiện chuyển tiếp nhưng ký tự đầu vào là 'h').

Mỗi trạng thái đều có hướng chuyển tiếp xác định và **đầu ra chỉ phụ thuộc vào trạng thái hiện tại**, vì vậy đây là một **máy trạng thái kiểu Moore**.

Hãy **tạo một project mới có tên là hello**, sau đó trong project này **thêm một tệp nguồn mới (source file)** và **lưu với tên hello.v**.

Sơ đồ khối mô tả **mô-đun phát hiện chuỗi "hello"** thể hiện trong **Hình 10-4:**

GOWIN VIETNAM



Hình 10-4 – Sơ đồ khối module phát hiện chuỗi

Bảng 10-2 – Mô tả chức năng các port của module FSM

Tên Port	I/O	Mô tả chức năng port
clk	I	Xung clock cho module，tần số 50MHz
reset_n	I	Tín hiệu đặt lại/rết module
data_in	I	Dữ liệu vào rộng 8bit
data_in_valid	I	Cờ hợp lệ cho phép nhập, mức 1 có dữ liệu vào, ngược lại mức 0
check_ok	O	Kiểm tra chuỗi "hello" ok thì xuất tín hiệu cao trong 1 chu kỳ xung clk

Trước tiên, sử dụng mã one-hot để xác định từng trạng thái.

```
localparam
    CHECK_h  = 5'b0_0001, CHECK_e  = 5'b0_0010, CHECK_l1  = 5'b0_0100,
    CHECK_l2  = 5'b0_1000,
    CHECK_o   = 5'b1_0000;
```

Sau đó，thiết kế trạng thái ban đầu:

```
always@(posedge clk or posedge reset) if(reset)begin
    check_ok <= 1'b0; state <= CHECK_h;
end
else begin
    case(state)
    //phần này của mã bị bỏ qua và sẽ được mô tả sau default:state <= CHECK_h;
    endcase
end
```

Theo Hình 10-3, bắt đầu viết phần chuyển trạng thái. Khi ở trạng thái ban đầu CHECK_h, nếu phát hiện có 'h' vào, thì sẽ chuyển sang trạng thái CHECK_e để bắt đầu kiểm tra ký tự 'e', nếu không sẽ giữ lại trạng thái CHECK_h và chờ ký tự 'h' được nhập.

```
CHECK_h:
    begin
        check_ok <= 1'b0; if(data_valid && data_in == "h")
        state <= CHECK_e; else
        state <= CHECK_h;
    end
```

GOWIN VIETNAM

Khi ở trạng thái CHECK_e, nếu phát hiện có 'e' vào, thì sẽ chuyển sang trạng thái CHECK_l1 để bắt đầu kiểm tra 'l' đầu tiên; nếu phát hiện có ký tự 'h' vào, thì vẫn giữ lại trạng thái CHECK_e và chờ 'e' vào; nếu phát hiện có ký tự khác vào, sẽ quay lại trạng thái ban đầu chờ vòng kiểm tra mới; nếu không, sẽ luôn ở trạng thái CHECK_e để chờ ký tự vào

```
CHECK_e:
    if(data_valid && data_in == "e") state <= CHECK_l1;
    else if(data_valid && data_in == "h") state <= CHECK_e;
    else if(data_valid) state <= CHECK_h;
    else
        state <= CHECK_e;
```

Khi ở trạng thái CHECK_l1, nếu phát hiện có ký tự 'l' được nhập, thì sẽ chuyển sang trạng thái CHECK_l2 để bắt đầu kiểm tra ký tự thứ hai 'l'; nếu phát hiện có ký tự 'h' được nhập, thì sẽ chuyển sang trạng thái CHECK_e để chờ ký tự 'e' được nhập; nếu phát hiện có ký tự khác được nhập, sẽ quay lại trạng thái ban đầu để chờ một vòng kiểm tra mới; nếu không, sẽ luôn ở trạng thái CHECK_l1 để chờ ký tự được nhập.

```
CHECK_l1:
if(data_valid && data_in == "l") state <= CHECK_l2;
else if(data_valid && data_in == "h") state <= CHECK_e;
else if(data_valid) state <= CHECK_h;
else
    state <= CHECK_l1;
```

GOWIN VIETNAM

Khi ở trạng thái CHECK_l2, nếu phát hiện có ký tự 'l' nhập vào, thì sẽ chuyển sang trạng thái CHECK_o để bắt đầu kiểm tra ký tự 'o'; nếu phát hiện có ký tự 'h' nhập vào, thì sẽ chuyển đến trạng thái CHECK_e để chờ ký tự 'e' nhập vào; nếu phát hiện có ký tự khác nhập vào, thì sẽ quay lại trạng thái ban đầu để chờ một vòng kiểm tra mới; nếu không, sẽ tiếp tục ở trạng thái CHECK_l2 chờ nhập ký tự.

```
CHECK_l2:
if(data_valid && data_in == "l") state <= CHECK_o;
else if(data_valid && data_in == "h") state <= CHECK_e;
else if(data_valid)
    state <= CHECK_h;
     else
    state <= CHECK_l2;
```

Khi ở trạng thái CHECK_o, nếu phát hiện có ký tự 'o' vào, thì đã hoàn thành một lần kiểm tra chuỗi "hello", quay lại ban đầu để chờ vòng kiểm tra mới, đồng thời điều khiển check_ok lên mức cao; nếu phát hiện có 'h' vào, thì sẽ chuyển đến trạng thái CHECK_e để chờ ký tự 'e' vào; nếu phát hiện có ký tự khác vào, thì sẽ quay lại ban đầu để chờ một vòng kiểm tra mới; nếu không, sẽ tiếp tục ở trạng thái CHECK_o chờ nhập ký tự.

```
CHECK_o:
    if(data_valid && data_in == "h") state <= CHECK_e;
    else if(data_valid)begin state <= CHECK_h; if(data_in == "o")
            check_ok <= 1'b1; else
            check_ok <= 1'b0;
    end else
        state <= CHECK_o;
```

Phân tích và tổng hợp thiết kế cho đến khi không có lỗi.

10.3 Thiết kế Testbench và mô phỏng

Để kiểm tra mô phỏng, chúng ta hãy viết 1 tệp testbench，tạo tệp Verilog tên là fsm_hello_tb.v. Ngoài tạo clk và reset thì mô phỏng còn tạo ra các tình huống nhập ký tự khác nhau. Trong thiết kế Testbench, việc tạo một ký tự duy nhất được thiết kế dưới dạng một tác vụ và việc tạo các chuỗi khác nhau được tạo ra bằng cách gọi tác vụ để xác minh tính đúng đắn của thiết kế và mã cụ thể như sau:

```
`timescale 1ns/1ns
`define CLK_PERIOD 20 module fsm_hello_tb;

    reg clk;
    reg reset_n;

    reg data_valid; reg [7:0]data_in;

    wire check_ok; fsm_hello fsm_hello(
        .clk(clk),
            .reset_n(reset_n),
            .data_valid(data_valid),
            .data_in(data_in),
            .check_ok(check_ok)
        );

        initial clk = 1; always#(`CLK_PERIOD/2) clk = ~clk;

        initial begin
            reset_n = 0;
            data_valid = 0;
            data_in = 0; #(`CLK_PERIOD*20);
            reset_n = 1;
          #(`CLK_PERIOD*20 + 1);
            repeat(2) begin gen_char("I"); #(`CLK_PERIOD);
                gen_char("A"); #(`CLK_PERIOD);
```

GOWIN VIETNAM

```
  gen_char("h");
#(`CLK_PERIOD);
  gen_char("e");
#(`CLK_PERIOD);
  gen_char("l"); #(`CLK_PERIOD);
  gen_char("l"); #(`CLK_PERIOD);
  gen_char("l"); #(`CLK_PERIOD);
  gen_char("h");
#(`CLK_PERIOD);
  gen_char("e");
#(`CLK_PERIOD);
  gen_char("l"); #(`CLK_PERIOD);
  gen_char("l"); #(`CLK_PERIOD);
  gen_char("o");
#(`CLK_PERIOD);
  gen_char("e");
#(`CLK_PERIOD);
  gen_char("h");
#(`CLK_PERIOD);
  gen_char("h");

  #(`CLK_PERIOD);
  gen_char("o"); #(`CLK_PERIOD);
  end
  #200;
  $stop;
  end
  task gen_char;
  input[7:0]char; begin
  data_in = char; data_valid = 1'b1;
#(`CLK_PERIOD);
  data_valid = 1'b0;
  end endtask
    endmodule
```

GOWIN VIETNAM

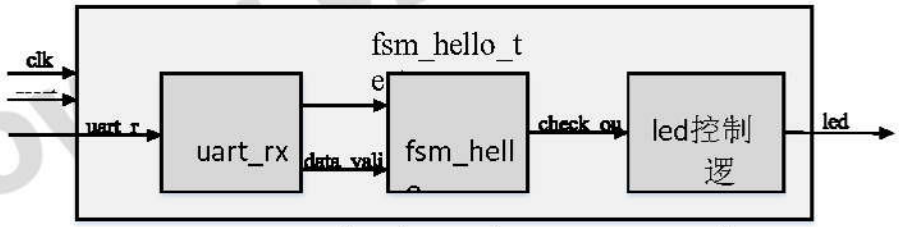
Mở phần mềm Modelsim tạo dự án mới tên"fsm_hello", thêm tập tin "fsm_hello.v'' và "fsm_hello_tb.v". Sau đó, chạy cấu Hình mô phỏng và chạy mô phỏng , và bạn có thể thấy dạng sóng file được hiển thị trong Hình 10-5 bên dưới, Có thể thấy rằng trạng thái không thay đổi cho đến khi reset được đặt ở mức cao. Sau khi đặt lại hợp lệ, ký tự chỉ được nhập nếu nó được nhập data_valid ở mức cao, trạng thái sẽ được chuyển đổi theo thiết kế, khi chuỗi "hello" được phát hienj chính xác checkok sẽ active 1 chu kỳ clk.

Hình 10-5– Dạng sóng mô phỏng

Đến đây, một thiết kế máy trạng thái đơn giản đã được hoàn thành. Ý tưởng thiết kế máy trạng thái sẽ thường xuyên được sử dụng trong các chương tiếp theo. Ở đây, chúng tôi sẽ không giải thích thêm về máy trạng thái hai giai đoạn và ba giai đoạn, mà sẽ cung cấp các giải thích chi tiết trong các ví dụ cụ thể.

10.4 Kiểm tra chương trình phát hiện chuỗi trên board mạch

Thông qua việc học về máy trạng thái kiểm tra chuỗi ở trên, kết hợp với mô-đun nhận chuỗi đã học ở các chương trước, thiết kế một hệ thống kiểm tra chuỗi nhận qua cổng nối tiếp. Sơ đồ khối cấu trúc của toàn bộ hệ thống như Hình 10-6 dưới đây. Chức năng của hệ thống là FPGA nhận dữ liệu chuỗi do PC gửi qua cổng nối tiếp, sau đó thông qua mô-đun kiểm tra chuỗi để kiểm tra chuỗi, mỗi khi nhận đầy đủ chuỗi "hello" sẽ cho phép LED lật trạng thái.



Hình 10-6 – Sơ đồ khối hệ thống phát hiện chuỗi

Thiết kế Top module fsm_hello_test, verilog code như bên dưới,

```
module fsm_hello_test(
    clk,
    reset_n,

    uart_rx,
    led
);
    assign reset=~reset_n;
    input clk;    //system clk 50Mhz
    input reset_n;    //chân reset

    input uart_rx;    //tín hiệu nối tiếp

    output led;  //LED hiển thị
    reg led;
```

```
wire [7:0]data; //UART nhận dữ liệu và nhập vào dạng ký
tự


  wire data_valid;  //1byte nhận xong lên 1

  wire check_ok;     //1 chuỗi đúng lên 1

//nhận dữ liệu cổng nối tiếp làm đầu vào để phát hiện chuỗi
uart_byte_rx uart_byte_rx(
      .clk(clk),
      .reset_n(reset_n),
      .baud_set(3'd0),
      .uart_rx(uart_rx),

      .data_byte(data),
      .rx_done(data_valid)
  );
```

```
        .clk(clk),
        .reset_n(reset_n),
        .data_valid(data_valid),
        .data_in(data),
        .check_ok(check_ok)
    );
    always@(posedge clk or posedge reset) if(reset)
        led <= 1'b0; else if(check_ok)
        led <= ~led; else
        led <= led;
endmodule
```

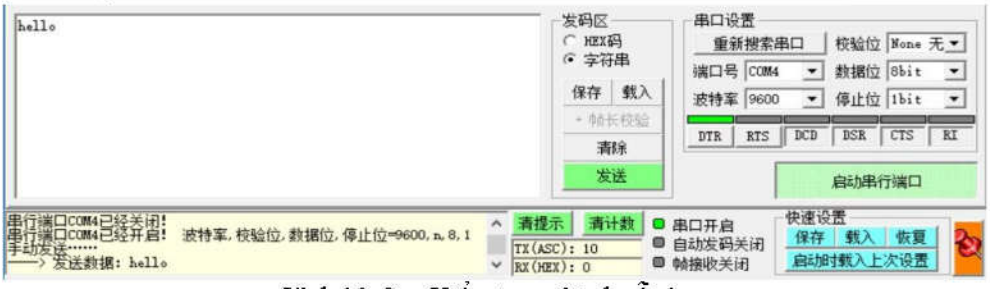
GOWIN VIETNAM

Mô phỏng của Top mô-đun có thể được thực hiện tương tự như mô-đun nhận chuỗi, ở đây sẽ không giải thích thêm về thiết kế mã mô phỏng của Top mô-đun, độc giả tự mình thực hiện xác minh mô phỏng để kiểm tra tính chính xác của chức năng thiết kế. Khi đã xác định chức năng thiết kế là chính xác, tiến hành ràng buộc các chân vào/ra của top mô-đun cao, led được liên kết với chân LED0 trên bảng mạch. Tình Hình ràng buộc chân cụ thể như Hình 10-7 dưới đây.

	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type	Drive
1	clk	input		T9	4	False	LVCMOS33	OFF
2	led	output		D14	1	False	LVCMOS33	8
3	reset_n	input		B16	1	False	LVCMOS33	OFF
4	uart_rx	input		U8	5	False	LVCMOS33	OFF

Hình 10-7– Ràng buộc I/O

Sau khi hoàn thành việc ràng buộc chân, tiến hành biên dịch toàn bộ dự án và tạo tệp fs, sau đó tải tệp fs lên bo mạch phát triển. Khi tệp fs được tải lên FPGA, có thể thấy rằng LED0 của chân đã được liên kết là tắt. Kết nối máy tính với UART của FPGA qua dây dữ liệu, mở một phần mềm cổng nối tiếp trên máy tính (lấy ví dụ là Serial Hunter), chọn cổng nối tiếp tương ứng với FPGA, thiết lập tốc độ baud của UART là 9600, và định dạng dữ liệu gửi là chuỗi. Gửi một vài nhóm dữ liệu chuỗi trên phần mềm cổng nối tiếp và quan sát sự thay đổi của LED0 trên bo mạch. Dưới đây là cửa sổ phần mềm UART gửi chuỗi 3 lần.



Hình 10-8 – Kiểm tra gửi chuỗi 1

Hình 10-9 – Kiểm tra gửi chuỗi 2

Hình 10-10 – Kiểm tra gửi chuỗi 3

(1) Khi gửi chuỗi "hello", LED0 trên bo mạch từ tắt chuyển sang sáng, kết quả kiểm tra đúng.

(2) Khi gửi chuỗi "helllo", LED0 trên bo mạch không thay đổi, kết quả kiểm tra đúng.

(3) Khi gửi chuỗi "lhoolehello", LED0 trên bo mạch từ sáng chuyển sang tắt, kết quả kiểm tra đúng.

10.5 Tổng kết chương

Chương này giúp độc giả hiểu nguyên lý làm việc của máy trạng thái cũng như cách thực hiện máy trạng thái kiểm tra chuỗi. Độc giả có thể gửi các dữ liệu chuỗi khác nhau từ máy tính để thử nghiệm, quan sát sự thay đổi của LED0 trên bo mạch. Khi gửi chuỗi có đầy đủ chuỗi "hello", LED0 sẽ đảo ngược một lần (sáng chuyển thành tắt hoặc tắt chuyển thành sáng). Qua việc thử nghiệm trên bo mạch, có thể xác minh tính chính xác của thiết kế, toàn bộ thiết kế hệ thống đạt được hiệu quả như mong đợi.

Chương 11: Thiết kế và xác thực chống rung cho nút nhấn độc lập

Mã code	|- thư mục 02 |	ch11_key_filter

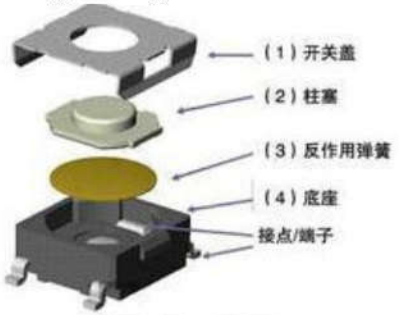
Dẫn nhập chương

Nút bấm, như một thành phần cơ bản trong giao tiếp giữa người và máy, được sử dụng rộng rãi trong thiết kế điện tử, từ việc khởi động lại hệ thống đến việc điều khiển các cài đặt. Hiện nay, có rất nhiều loại nút bấm, phổ biến như nút bấm đa hướng, nút bấm tự khóa, nút bấm màng mỏng, v.v.

Phần này sẽ giới thiệu phương pháp đồng bộ tín hiệu bất đồng bộ dữ liệu đơn Bit và phát hiện cạnh vào thiết kế khử nhiễu nút bấm thường dùng trong FPGA. Đồng thời, trong tệp kích thích thử nghiệm mô phỏng, sẽ đưa vào hàm sinh số ngẫu nhiên và mô Hình kích thích. Cuối cùng, sẽ có một ví dụ thực tế để minh họa cách thiết kế theo mô-đun.

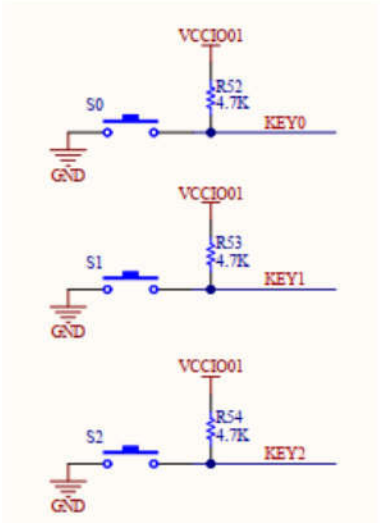
11.1 Cấu trúc vật lý và thiết kế mạch nút bấm

Sơ đồ phần cứng của nút nhất thông thường như Hình 11-1 dưới đây:



Hình 11-1– Sơ đồ cấu trúc phần cứng nút nhấn

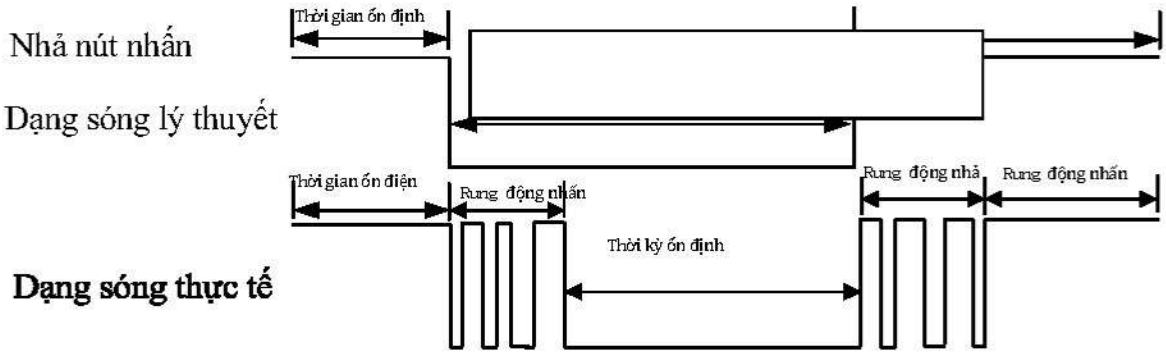
Bảng phát triển ACG525 sử dụng nút bấm hai chân, nằm ở góc dưới bên phải của board, sơ đồ nguyên lý như Hình dưới đây. Từ sơ đồ nguyên lý có thể thấy, khi nút bấm không được ấn xuống, chân IO ở mức cao, khi nút bấm được ấn xuống thì trở thành mức thấp, do đó hệ thống có thể xác định trạng thái của nút bấm bằng cách kiểm tra mức điện áp của chân IO.



Hình 11-2 – Sơ đồ nguyên lý nút nhấn

Trong sơ đồ cấu trúc nút bấm, có thể thấy nút bấm có một lò xo phản hồi, vì vậy khi ấn xuống hoặc thả ra sẽ tạo ra sự rung động vật lý bổ sung, và sự rung động này sẽ gây ra sự dao động điện áp. Trong quá trình nút bấm từ trạng thái ấn xuống đến thả ra, sự thay đổi

điện áp như Hình 11-3 dưới đây:

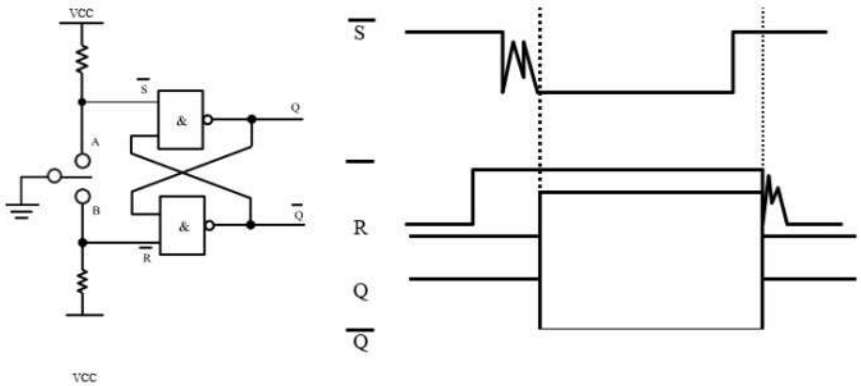


Hình 11-3 – Sự thay đổi mức độ của phím từ nhấn sang nhả

Trong Hình trên, số lần và khoảng thời gian tạo ra rung động đều là không thể dự đoán, vì vậy cần phải sử dụng bộ lọc để loại bỏ ảnh hưởng có thể của rung động đối với các thiết bị bên ngoài khác. Thông thường, tổng thời gian rung động sẽ kéo dài trong vòng 20ms. Rung động này có thể được loại bỏ thông qua mạch phần cứng hoặc thiết kế logic, hoặc cũng có thể được thực hiện bằng phần mềm. Trong đó, việc loại bỏ rung động bằng mạch phần cứng phù hợp với những trường hợp có số lượng nút bấm ít.

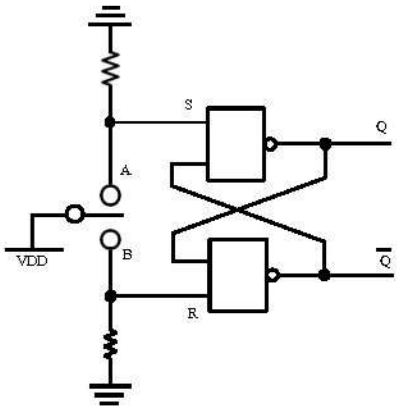
11.2 Mạch phần cứng thực hiện khử rung cho nút nhấn

Sử dụng sơ đồ mạch của bộ kích hoạt RS dựa trên cổng NAND để loại bỏ rung động, như Hình 11-4 dưới đây. Giả sử trạng thái ban đầu của công tắc kết nối với B, lúc này $\overline{\{S\}}$=1, $\overline{\{R\}}$=0, bộ kích hoạt sẽ đặt về 0 tức là Q=0; khi B chuyển sang A, có một khoảng thời gian mà công tắc không tiếp xúc với A cũng như B, lúc này $\overline{\{S\}}$=1 $\overline{\{R\}}$=1 tức là Q=0; khi chuyển sang A trong khoảnh khắc, có $\overline{\{R\}}$=1, $\overline{\{S\}}$=0, bộ kích hoạt giữ Q=0, bộ kích hoạt sẽ đặt về 1 tức là Q=1. Lúc này, ngay cả khi A xuất hiện rung động tức là $\overline{\{S\}}$=1, trạng thái của bộ kích hoạt vẫn giữ nguyên tức là Q=1. Toàn bộ quá trình được thể hiện trong dạng sóng như Hình $\overline{\{Q\}}$. Tương tự, có thể phân tích sự thay đổi trạng thái của bộ kích hoạt khi công tắc chuyển từ A sang B.



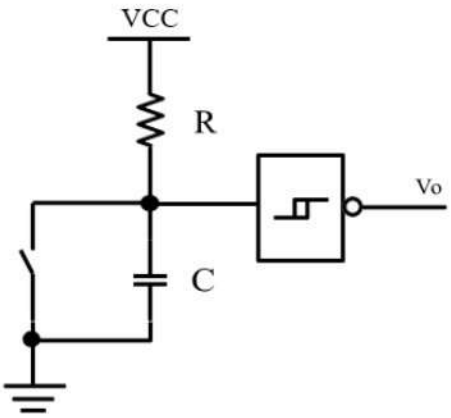
Hình 11-4 – Mạch khử nhiễu RS dựa trên cổng NAND và phân tích dạn sóng

Có thể sử dụng bộ RS flip-flop được cấu thành từ cổng AND và NOT để thực hiện chức năng. Như Hình 11-5, có thể đối chiếu với quá trình phân tích trên để tự phân tích trạng thái hoạt động của mạch

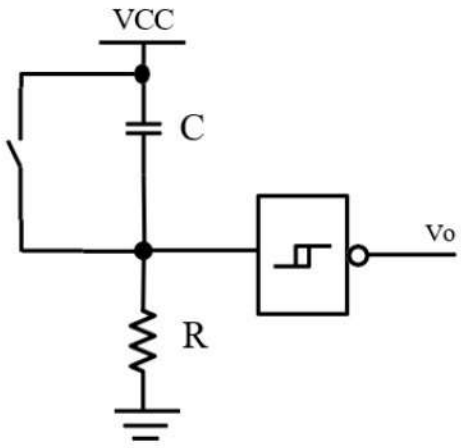


Hình 11-5 – Sơ dồ mạch khử nhiễu RS dựa vào NOR

Có thể thấy rằng, hai cách kết nối trên chỉ phù hợp với các loại công tắc đơn cực, không phù hợp với các nút bấm hai chân hoặc bốn chân hiện nay. Đối với loại nút bấm này, hiện tại thường sử dụng mạch RC như Hình 11-6 và Hình 11-7

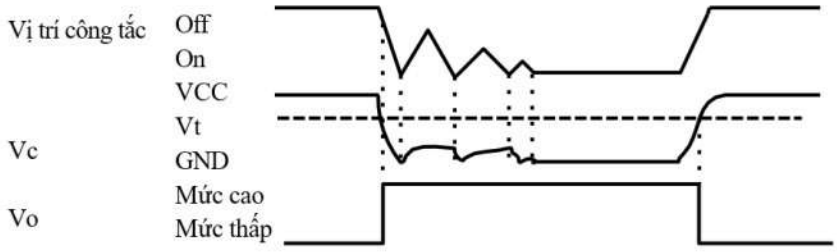


Hình 11-6 – Đầu ra mức cao



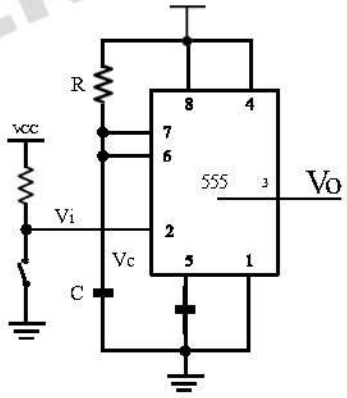
Hình 11-7– Đầu ra mức thấp

Nguyên lý của nó là sử dụng tụ điện và điện trở để tích phân dạng sóng, sau đó thông qua bộ kích hoạt Schmitt (bộ so sánh) để định Hình dạng sóng và từ đó có được dạng sóng đầu ra lý tưởng. Đối với mạch đầu ra mức cao, khi công tắc mở, tụ điện được sạc với Vc=5V, đầu ra Vo là 0; khi công tắc đóng, tụ điện xả xuống Vc=0, đầu ra Vo là 5V; khi nút bấm bị rung, nếu hằng số thời gian sạc RC nhỏ, tụ điện sẽ liên tục thực hiện quá trình sạc và xả, sạc đến 5V và xả xuống 0V. Nếu hằng số thời gian sạc RC được đặt đủ lớn, khi công tắc rung từ trạng thái mở sang đóng, tụ điện không sạc đến mức điện áp ngưỡng của bộ kích hoạt Schmitt, thì đầu ra Vo giữ nguyên ở 5V. Khi công tắc mở, tụ điện sẽ sạc lại đến 5V, vượt quá điện áp ngưỡng, thì đầu ra sẽ thay đổi thành mức thấp. Toàn bộ quá trình như Hình 11-8 cho thấy, có sự rung lắc khi đóng hoặc thả đầu ra chỉ thay đổi một lần.

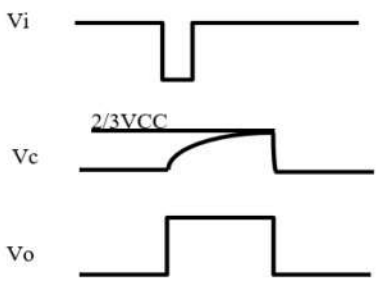


Hình 11-8 – Mạch RC với đầu ra điện áp cao

Cũng có thể sử dụng bộ hẹn giờ 555 để tạo thành bộ kích hoạt đơn ổn định, cũng có thể loại bỏ sự rung lắc của nút bấm. Đầu ra Vo thường ở mức thấp, khi có xung Vi giảm xuống, nó sẽ đảo ngược thành mức cao. Lúc này, nguồn điện sẽ sạc cho tụ C qua R, khi điện áp trên tụ C đạt VCC*2/3, transistor xả trong bộ hẹn giờ 555 sẽ dẫn, tụ C sẽ xả nhanh chóng, đầu ra sẽ đảo ngược thành mức thấp. Trong quá trình điện áp tụ từ 0 tăng lên 2/3VCC, đầu ra luôn ở mức cao, ngay cả khi có sự rung lắc, đầu ra cũng sẽ không thay đổi. Bằng cách lựa chọn hợp lý thời gian tạm ổn Td, có thể đạt được hiệu quả chống rung.



Hình 11-9 – Mạch khử rung dựa trên mạch kích hoạt trạng thái đơn



Hình 11-10 – Phân tích dạng sóng của mạch khử rung dùng IC555

11.2 Cài đặt bộ máy trạng thái để lọc phím nhấn

11.3 Thiết kế giao diện mô đun

Đối với mô đun khử rung nút bấm đơn, giao diện của nó như Hình 1-11 dưới đây, mô tả chức năng khai báo giao diện như trong bản 11-1:

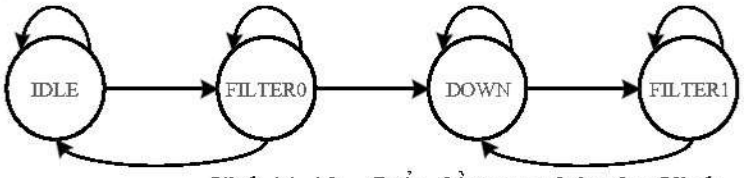
Hình 11-11– Giao diện mô đun lọc nhiễu phím bấm

Bảng 11-1 Mô tả chức năng giao diện mô đun khử nhiễu phím bấm

Tên giao tiếp	I/O	Mô tả chức năng
clk	I	Xung clk 50M
reset_n	I	Chân reset module
key_in	I	Tín hiệu phím nhấn vào
key_flag	O	Cờ báo chuyển trạng thái nút nhấn
key_state	O	Trạng thái nút nhấn
reset_n	I	Chân reset module

11.4 Thiết kế chuyển trạng thái và điều kiện của FSM

Đối với thiết kế FPGA, thường sử dụng FSM để xử lý chống rung cho nút bấm. Trong Hình dưới đây, có thể thấy rằng nếu phân tích mã trạng thái theo khái niệm FSM, ta sẽ có các trạng thái sau: trạng thái nhàn rỗi IDLE khi không nhấn, trạng thái lọc rung FILTER0 khi nhấn, trạng thái ổn định DOWN khi nhấn và trạng thái lọc rung FILTER1 khi thả. Mã độc lập của chúng lần lượt là 4'b0001, 4'b0010, 4'b0100 và 4'b1000. Biểu đồ chuyển trạng thái của chúng như Hình 11-12 cho thấy, có thể thấy đây là FSM kiểu Mealy.



Hình 11-12 – Biểu đồ trạng thái như Hình

Điều kiện chuyển trạng thái như bảng 11-2 dưới đây.

Bảng 11-2 – Điều kiện chuyển trạng thái chống rung nút bấm

Trạng thái hiện tại	Trang thái tiếp theo	Điều kiện chuyển đổi trạng thái
IDLE	FILTER0	Phát hiện cạnh giảm của nút bấm, bắt đầu quá trình chống rung khi nhấn;
FILTER0	IDLE	Trong quá trình chống rung khi nhấn, nếu phát hiện cạnh tăng và chưa vượt quá 20ms, thì coi như nút bấm vẫn đang rung, vẫn chưa ổn định nên quay lại trạng thái ban đầu;
FILTER0	DOWN	Phát hiện cạnh giảm thì bắt đầu đếm thời gian, nếu > 20ms thì coi như đã nhấn thực sự;

GOWIN VIETNAM

DOWN	FILTER1	Sau khi trạng thái nhấn ổn định, nếu phát hiện cạnh tăng bắt đầu chống rung khi thả:
FILTER1	DOWN	Trong quá trình phục hồi chống rung khi thả nút, phát hiện cạnh tăng thì bắt đầu đếm time, nếu phát hiện cạnh giảm và < 20ms, coi như vẫn đang rung, tiếp tục chống rung;
FILTER1	IDLE	Phát hiện cạnh tăng thì bắt đầu đếm thời gian, > 20ms thì coi như đã thả hoàn tất.

11.5 Thiết kế phát hiện cạnh tín hiệu nhập phím

Từ góc độ thiết kế máy trạng thái, cần phải phát hiện cạnh của tín hiệu đầu vào nút bấm key_in. Về kiến thức lý thuyết liên quan đến việc phát hiện cạnh, có thể tham khảo trong mô-đun nhận dữ liệu nối tiếp, ở đây không lặp lại giải thích. Cụ thể, mã phát hiện cạnh tín hiệu dữ liệu nút bấm như sau:

```
always@(posedge clk or posedge reset) if(reset)begin
    key_in_sync1 <= 1'b0; key_in_sync2 <= 1'b0;
end
```

```
else begin
    key_in_sync1 <= key_in; key_in_sync2 <= key_in_sync1;
end
//Sử dụng bộ kích D để lưu tạng thái mức tín hiệu đầu vào bên ngoài (đã đồng bộ với miền đồng hồ hệ thống) tại hai cạnh lên của xung đồng hồ liền kề
always@(posedge clk or posedge reset) if(reset)begin
    key_in_reg1 <= 1'b0; key_in_reg2 <= 1'b0;
end
else begin
    key_in_reg1 <= key_in_sync2; key_in_reg2 <= key_in_reg1;
end

//tạo tín hiệu phát hiện cạnh
assign key_in_nedge = !key_in_reg1 & key_in_reg2; assign key_in_pedge =
key_in_reg1 & (!key_in_reg2);
```

11.5.1 Thiết kế bộ đếm

Trong bảng chuyển trạng thái cần có module counter 20ms và module cho phép đếm. Ở đây có thể gộp lại thành 1 module . Khuyến nghị 1 always chỉ nên thao tác 1 tín hiệu.

```
always@(posedge clk or posedge reset) if(reset)
    cnt <= 20'd0; else if(en_cnt)
    cnt <= cnt + 1'b1; else
    cnt <= 20'd0;

always@(posedge clk or posedge reset) if(reset)
    cnt_full <= 1'b0;
else if(cnt == 20'd999_999) cnt_full <= 1'b1;
else
    cnt_full <= 1'b0;
```

GOWIN VIETNAM

11.5.2 Thiết kế Máy trạng thái (FSM)

Bây giờ ta thiết kế FSM, trước tiên dùng tham số hóa cục bộ để định nghĩa trạng thái.

```
localparam
    IDLE= 4'b0001, FILTER0= 4'b0010, DOWN= 4'b0100, FILTER1= 4'b1000;
```

Do số trạng thái điều kiện đánh giá khá ít, ta sẽ mô tả FSM 1 đoạn. Khi reset, counter sẽ về 0, trạng thái về IDLE, key_flag và key_state quay về trạng thái ban đầu.

```
always@(posedge clk or posedge reset) if(reset)begin
    en_cnt <= 1'b0; state <= IDLE; key_flag <= 1'b0; key_state <=
    1'b1;
end
else begin
    case(state)
        default:begin
            state <= IDLE; en_cnt <= 1'b0; key_flag <= 1'b0; key_state
            <= 1'b1;
        end endcase
end
```

Khi trạng thái là IDLE & chưa nhấn，nếu phát hiện cạnh xuống sẽ chuyển sang lọc rung khi nhấn FILTER0 và kích hoạt counter, nếu không tiếp tục giữ trạng thái IDLE.

```
IDLE :begin
    key_flag <= 1'b0; if(key_in_nedge)begin
        state <= FILTER0; en_cnt <= 1'b1;
    end else
        state <= IDLE;
end
```

Khi ở FILTER0, nếu trong vòng 20ms có cạnh lên, thì coi như vẫn đang trong rung khi nhấn, trạng thái về IDLE và counter đặt về 0. Trong khi nhấn, sau lần rung cuối cùng, sẽ không có cạnh lên xuất hiện, bộ đếm tiếp tục đếm, khi đếm đầy sẽ đặt key_flag thành 1, key_state thành 0, trạng thái chuyển sang ổn định khi nhấn DOWN và đặt counter về 0. Vậy, có thể xác định trạng thái nút bấm bằng cách kiểm tra key_flag && !key_state

```
FILTER0:begin
    if(cnt_full)begin
        key_flag <= 1'b1; key_state <= 1'b0;
```

```
        en_cnt <= 1'b0; state <= DOWN;
    end
    else if(key_in_pedge)begin state <= IDLE;
        en_cnt <= 1'b0;
    end else
        state <= FILTER0;
end
```

GOWIN VIETNAM

Sau khi vào TT ổn định DOWM，đặt key_flag về 0。Nếu phát hiện cạnh lên thì chuyển vào TT lọc rung khi thả FILTER1，nếu không giữ nguyên TT hiện tại.

```
FILTER1:begin
    if(cnt_full)begin key_flag <= 1'b1; key_state <= 1'b1; state <=
        IDLE; en_cnt <= 1'b0;
    end
    else if(key_in_nedge)begin en_cnt <= 1'b0;
        state <= DOWN;
    end else
        state <= FILTER1;
end
```

Khi ở FILTER1, nếu bộ đếm 20ms chưa hết thời gian mà phát hiện cạnh giảm, thì coi như đang trong quá trình rung thả, trạng thái quay về DOWN đặt bộ đếm về 0. Trong quá trình thả, sau lần rung cuối cùng sẽ không có cạnh xuống, do đó bộ đém tiếp tục đếm đến khi đầy, key_flag&key_state lên 1, TT chuyển sang IDLE đặt counter về 0, chờ lần nhấn nút tiếp theo.

Nếu viết lại dạng FSM 2 đoạn sẽ như sau, phần chuyển trạng thái sẽ được bỏ qua

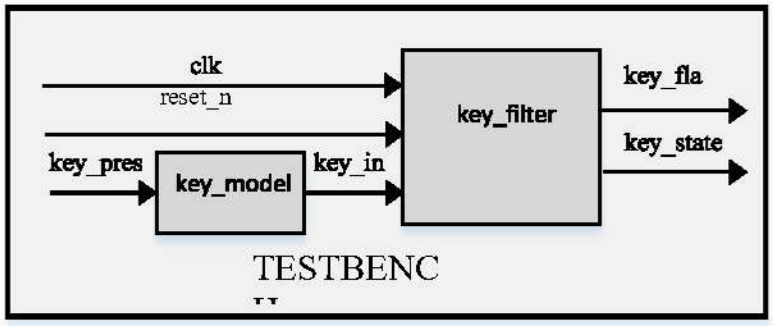
```
always@(posedge clk or posedge reset) if(reset)
    curr_state <= IDLE;
else
```

```
    curr_state <= next_state;

always@(*)
begin
    case(curr_state)
    //phần này giống FSM 1 đoạn
    endcase
end
```

GOWIN VIETNAM

11.6 Thiết kế testbench và mô phỏng

Trong tệp testbench, ngoài việc tạo ra tín hiệu clk và reset bình, còn mô phỏng quá trình từ khi nhấn nút đến khi thả. Mô phỏng này sử dụng cách thiết kế mô Hình mô phỏng riêng cho nút bấm, cấu trúc bên trong của toàn bộ tệp kích thích sẽ như Hình 11-13 dưới đây:



Hình 11-13 – Cấu trúc bên trong tệp mô phỏng

11.6.1 Thiết kế mô Hình nút nhấn key_model

Chức năng của mô Hình nút bấm là mô phỏng tín hiệu đầu ra trong quá trình nhấn nút thực tế, bao gồm cả rung động trong quá trình nhấn. Do thời gian rung trong quá trình nhấn nút thực tế là không xác định, ở đây sử dụng hàm sinh số ngẫu nhiên $random trong Verilog để tạo ra rung động.

Hàm hệ thống $random có thể sinh ra một số nguyên ngẫu nhiên 32bit có dấu. Cách sử dụng thông thường là $random % b, trong đó b > 0. Như vậy sẽ tạo ra một số ngẫu nhiên trong khoảng (-b + 1):(b - 1). Nếu chỉ cần số ngẫu nhiên dương, có thể sử dụng {$random} % b để tạo ra. Trong phần này, cần tạo ra rung động khi nhấn và thả nút trong vòng 20ms, thực tế nên tạo ra số ngẫu nhiên trong khoảng 25'd20_000_000, nhưng để tiết kiệm thời gian mô phỏng, chỉ tạo ra một số ngẫu nhiên 16 bit, tức là từ 0 đến 16'd65535.

```
DOWN:begin
        key_flag <= 1'b0; if(key_in_pedge)begin
            state <= FILTER1; en_cnt <= 1'b1;
        end else
            state <= DOWN;
end
```

11.6.2 Sử dụng task

Nhiệm vụtask là cú pháp thường cùng tương test bench, cú pháp như sau:

```
task <tên nhiệm vụ>;
    <câu lệnh khai báo cổng và kiểu dữ liệu>
    <câu 1>
    <câu 2>
    <câu n>
endtask
```

cú pháp của task như sau:

```
<tên task> ( Cổng1,cổng 2,...cổng n) ;
```

Quá trình nhấn và thả nút được thiết kế dưới dạng task, ở đây thực hiện năm mươi lần rung động từ 0 đến 65535ns khi nhấn, sau đó gán giá trị cố định 0 cho key_in và trì hoãn 25ms (lớn hơn 20ms thì được coi là ổn định). Đồng thời, cũng thực hiện rung động sau khi thả, gán giá trị cố định 1 cho key_in và trì hoãn 25ms.

```
task key_gen; begin
key_out = 1'b1; repeat(50)begin
    myrand = {$random}%65536;//0~65535; #myrand key_out = ~key_out;
end
key_out = 0;
#25000000;

repeat(50)begin
    myrand = {$random}%65536;//0~65535; #myrand key_out = ~key_out;
end
key_out = 1;
#25000000;
end endtask
```

GOWIN VIETNAM

Trên cơ sở đó, để thực hiện mô Hình nút bấm key_model có thể được điều khiển thông qua key_press bên ngoài, tức là khi cung cấp cho key_press một xung, key_model sẽ xuất ra một tín hiệu nhấn nút. Như vậy, trong mô-đun key_model, sự xuất hiện của cạnh lên của key_press sẽ được sử dụng làm điều kiện để gọi task ở trên, mã cụ thể như sau:

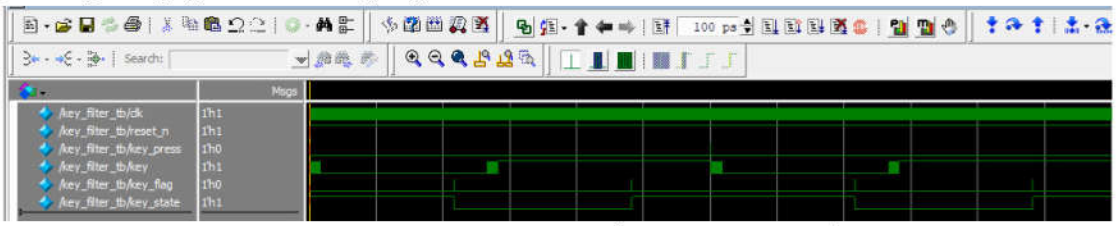
```
initial begin key_out = 1'b1; while(1)
    begin
        @(posedge key_press); key_gen;
    end
end
```

Sau khi thiết kế xong mô Hình nút bấm key_model, việc thiết kế toàn bộ testbench mô phỏng trở nên tương đối đơn giản. Thêm và tạo mới tệp key_filter_tb.v, trong đó thực hiện hóa mô-đun khử rung nút bấm và mô Hình nút bấm, đồng thời tạo ra tín hiệu kích thích đồng hồ và reset, mã cụ thể như sau. Mô phỏng đã tạo ra hai lần nhấn và thả nút.

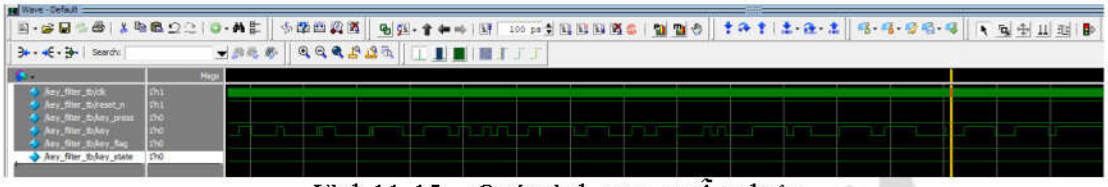
```
`timescale 1ns/1ns
`define CLK_PERIOD 20 module key_filter_tb;

    reg  clk;
    reg reset_n; reg key_press; wire key;

    wire key_flag; wire key_state;

    key_model key_model_inst(
        .key_press(key_press),
        .key_out(key)
    );
    key_filter key_filter_inst(
        .clk(clk),
        .reset_n(reset_n),

        .key_in(key),
        .key_flag(key_flag),
        .key_state(key_state)
    );
    initial clk= 1; always#(`CLK_PERIOD/2) clk = ~clk;
    initial begin
        reset_n = 1'b0; key_press = 1'b0; #(`CLK_PERIOD*10);
        reset_n = 1'b1; #(`CLK_PERIOD*10 + 1);

        key_press = 1'b1; #(`CLK_PERIOD);
        key_press = 1'b0; #60000000;
        key_press = 1'b1; #(`CLK_PERIOD);
        key_press = 1'b0;
        #60000000;
        $stop;
    end
endmodule
```

GOWIN VIETNAM

Sau khi thiết kế xong testbench mô phỏng, mở Modelsime và tạo một dự án mới để tiến hành mô phỏng, có thể thu được các dạng sóng mô phỏng như trong Hình 11-14 và Hình 11-15 bên dưới. Khi nhấn và thả nút ổn định, sẽ tạo ra một tín hiệu key_flag có mức cao trong một chu kỳ đồng hồ, và key_state cũng sẽ thay đổi bình thường. Ở đây, phóng to quá trình rung động được tạo ra, có thể thấy mỗi thời gian rung đều khác nhau, như vậy đã thành công trong việc mô phỏng quá trình rung ngẫu nhiên了随机抖动过程。



Hình 11-14 – Dạng sóng mô phỏng chống rung nhút nhấn



Hình 11-15 – Quá trình rung ngẫu nhiên

Từ phân tích các dạng sóng trên, có thể thấy chức năng khử rung nút bấm độc lập được thiết kế trong chương này hoạt động bình thường.

11.7 Tổng kết chương

Chương trình này đóng vai trò nối tiếp giữa các chương học, vì các chương sau thường sử dụng cách trích dẫn trực tiếp từ chương trình này, nên ở đây chương trình không thực hiện ràng buộc chân chỉ sử dụng công cụ mô phỏng để đạt được các dạng sóng chính xác là đủ để hoàn thành nhiệm vụ thiết kế của chương này.

Trong quá trình thiết kế, chương này đã ôn tập lại việc đồng bộ hóa tín hiệu bất đồng bộ đơn bit cũng như thiết kế liên quan đến phát hiện cạnh và máy trạng thái. Trong quá trình mô phỏng, đã giới thiệu nhiệm vụ (task) và hàm ngẫu nhiên ($random) và sử dụng mô Hình mô phỏng để đơn giản hóa việc viết tệp kích thích. Việc xác minh cấp bảng cụ thể sẽ được giới thiệu chi tiết trong chương tiếp theo.

GOWIN VIETNAM

Chương 12: Module hóa thiết kế bộ đếm lên & xuống

工程源码	|- 02_设计实例 | ch12_key_led

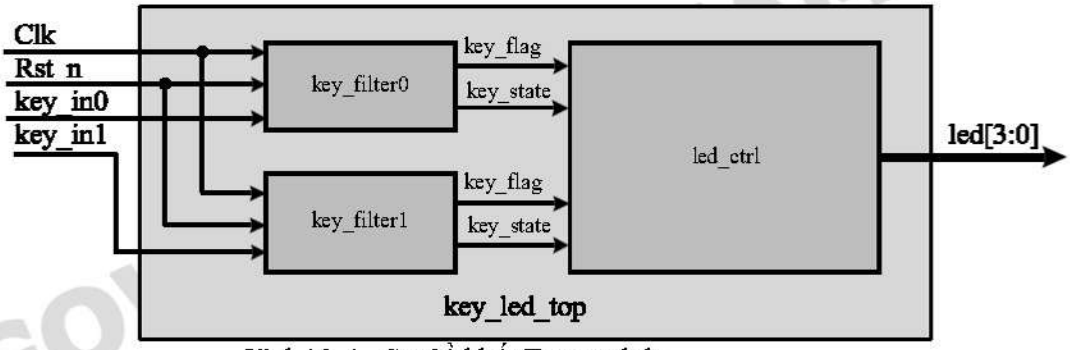
Dẫn nhập chương

Trong quá trình thiết kế dự án tương đối lớn, nội dung thiết kế thường không chỉ được viết trong một tệp thiết kế mà sẽ được phân chia thành các tệp con khác nhau cho các chức năng khác nhau, cuối cùng được gọi lại trong tệp cấp cao nhất.

Trong chương trước, đã thiết kế và xác minh chức năng khử rung cho nút bấm độc lập, ở đây dựa trên mô-đun khử rung nút bấm trước đó để thực hiện một bộ đếm cộng trừ, mỗi lần nhấn nút 0, 4 LED sẽ hiển thị trạng thái theo định dạng cộng nhị phân, tăng 1; mỗi lần nhấn nút 1, 4 LED sẽ hiển thị trạng thái theo định dạng cộng nhị phân, giảm 1, từ đó học cách thiết kế mô-đun. Trong chương năm, sẽ sử dụng các mô-đun độc lập đã viết trong chương bốn để kết hợp thiết kế một số dự án có tính ứng dụng cao hơn, từ đó hiểu rõ hơn về những lợi ích của phương pháp thiết kế mô-đun.

12.1 Phân hóa chức năng của module

Để thực hiện điều khiển đèn LED bằng 2 phím theo cách cộng nhị phân cộng/trừ, có thể chia khối module key_led_top thành 2 module 1 khử rung & 1 điều khiển LED， với các kết nối giữa các module như Hình 12-1



Hình 12-1– Sơ đồ khối Topmodule

12.1.1 Thiết kế chức năng module

Ở đây, **key_filter** đã được viết hoàn thành trong các chương trước, chỉ cần gọi nó trong **key_led_top**. Ta sẽ viết **led_ctrl**, như sau:

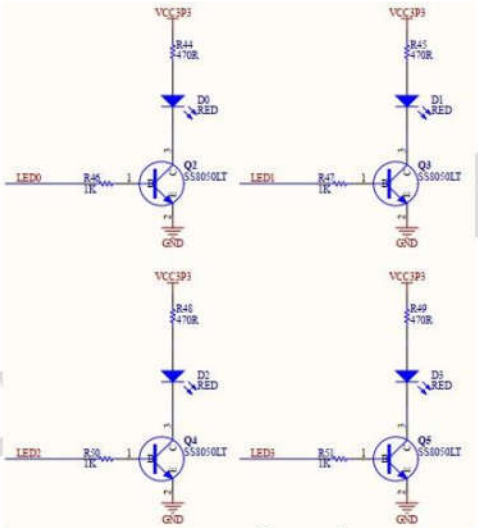
```
input Clk; input Rst_n;
input key_flag0,key_flag1; input key_state0,key_state1;
output [3:0]led;
```

Mô-đun này dựa vào trạng thái hai nút bấm để thực hiện việc cộng trừ của bộ đếm, và từ phần trước có thể thấy khi key_flag && key_state lên 1, tức là nút đã nhấn.

```
reg [3:0]led_r;
always@(posedge Clk or negedge Rst_n) if(!Rst_n)
    led_r <= 4'b0000;
else if(key_flag0 && !key_state0) led_r <= led_r + 1'b1;
else if(key_flag1 && !key_state1) led_r <= led_r - 1'b1;
else
    led_r <= led_r;
```

Giá trị khởi tạo của **led_r** trong bộ đếm là 4'b0000. Ở đây, khi nhấn phím 0, bộ đếm sẽ tăng thêm 1, giá trị của bộ đếm trở thành 4'b0001. Dựa vào sơ đồ mạch đèn LED trên bo mạch phát triển, ta biết rằng đèn LED sáng khi ở mức cao. Do đó, nếu xuất trực tiếp giá trị của led_r ra các đèn LED, ta sẽ thấy hiệu ứng sáng/tắt của các đèn LED là: tắt – tắt – tắt – sáng.

```
assign led = led_r;
```



Hình 12-2 – Mạch điều khiển đèn LED

Như thế, các module đọc lập đã được hoàng thành. Ta bắt đầu thiết kế Top module. Trong tệp key_led_top mới tạo, hãy khởi tạo 3 modules và thiết lập tín hiệu wire.

```
module key_led_top(Clk,Rst_n,key_in0,key_in1,led);
```

```
    input Clk; input Rst_n; input key_in0; input key_in1;

    output [3:0]led;

    wire key_flag0,key_flag1; wire key_state0,key_state1;

    key_filter key_filter0(
        .Clk(Clk),
        .Rst_n(Rst_n),
        .key_in(key_in0),
        .key_flag(key_flag0),
        .key_state(key_state0)
    );

    key_filter key_filter1(
        .Clk(Clk),
        .Rst_n(Rst_n),
        .key_in(key_in1),
        .key_flag(key_flag1),
        .key_state(key_state1)
    );

    led_ctrl led_ctrl0(
        .Clk(Clk),
        .Rst_n(Rst_n),
        .key_flag0(key_flag0),
        .key_flag1(key_flag1),
        .key_state0(key_state0),
        .key_state1(key_state1),
        .led(led)
    );
endmodule
```

GOWIN VIETNAM

Tiến hành phân tích tổng hợp đến khi hết lỗi

12.2 Tạo testbench và mô phỏng

Để kiểm tra mô phỏng, cần viết tệp testbench. Vì trong mô phỏng này có gọi đến hai phím nhấn, nên cần phải chỉnh sửa mô Hình mô phỏng phím nhấn đã thiết kế ở bài trước, thêm vào tín hiệu cho phép press, tức là mỗi khi có cạnh lên của press thì sẽ thực hiện một lần xuất tín hiệu key. Phần nhiệm vụ task press_key thì giữ nguyên, không thay đổi.

Nếu không chỉnh sửa, cả hai mô Hình mô phỏng phím nhấn sẽ thực thi đồng thời, dẫn đến lỗi.

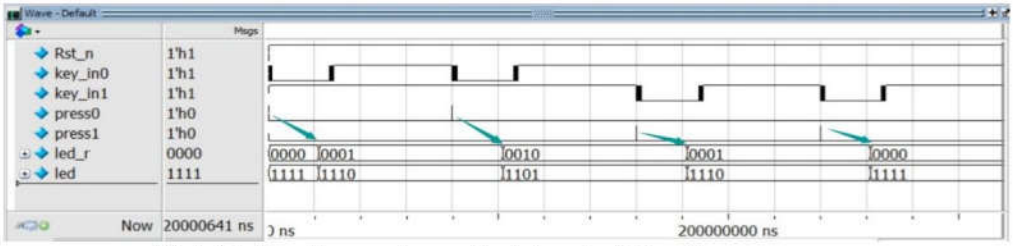
Hãy tạo mới tệp key_led_tb.v, nhập nội dung sau vào và lưu vào thư mục testbench. Sau đó tiến hành phân tích và tổng hợp lại cho đến khi không còn lỗi hay cảnh báo. Nội dung dưới đây không chỉ tạo tín hiệu xung nhịp (clock) và tín hiệu reset, mà còn mô phỏng quá trình nhấn và thả phím 0 hai lần, và phím 1 hai lần. Như vậy, toàn bộ đoạn mã sẽ như sau.

```
`timescale 1ns/1ns

`define clk_period 20 module key_led_tb;
    reg Clk; reg Rst_n;
    wire key_in0; wire key_in1;
    reg press0,press1; wire [3:0]led;
    key_led_top key_led_top(
        .Clk(Clk),
        .Rst_n(Rst_n),
        .key_in0(key_in0),
        .key_in1(key_in1),
        .led(led)
    );

    key_model key_model0(
        .press(press0),
        .key(key_in0)
    );
    key_model key_model1(
        .press(press1),
        .key(key_in1)
    );
    initial Clk= 1; always#(`clk_period/2) Clk = ~Clk;
    initial begin
        Rst_n = 1'b0; press0 = 0;
        press1 = 0;
```

GOWIN VIETNAM

```
        #(`clk_period*10) Rst_n = 1'b1; #(`clk_period*10 + 1);

        press0 = 1; #(`clk_period*3) press0 = 0;

        #80_000_000;

        press0 = 1; #(`clk_period*3) press0 = 0;

        #80_000_000;

        press1 = 1; #(`clk_period*3) press1 = 0;

        #80_000_000;

        press1 = 1; #(`clk_period*3) press1 = 0;

        #80_a000_000;
        $stop;
    End

endmodule
```

Sau khi thiết lập xong tập lệnh mô phỏng, tiến hành mô phỏng chức năng, ta có thể quan sát được tệp dạng sóng như Hình 12-3. Mỗi lần nhấn phím 0, bộ đếm led_r sẽ tăng thêm 1; còn khi nhấn phím 1, bộ đếm led_r sẽ giảm đi 1



Hình 12-3 – Dạng sóng mô phỏng 3 chức năng

Đến đây，việc xác minh chức năng mô phỏng đã hoàn thành.

12.3 Xác thực trên hardware

Giai đoạn xác thực trên bo của thí nghiệm này chủ yếu xác mình các mục tiêu sau：

1. Có thể ghi và tải chương trình đúng cách hay không?
2. Trên bảng mạch ACG525，sau khi bấm phím tương ứng，LED có sáng tắt theo ý đồ hay không? Hệ thống phần cứng như sau:

Bảng mạch ACG525

Cáp nguồn

Có 1 PC và mạch nạp.

12.3.1 Ràng buộc I/O

Nhấp chuột vào FloorPlanner rồi thực hiện các ràng buộc I/O，các ràng buộc cụ thể như sau

Bảng 12-1– Bảng mô tả ràng buộcI/O

	Tính hiệu chức năng	Chân tương ứng của bo mạch ACG525
Chân cơ bản	clk	T9
	reset_n	C15
	key_in0	B16
	key_in1	A15
	led[0]	D14
	led[1]	C14
	led[2]	B9
	led[3]	A9

Hoàn thành gán chân，như Hình 12-4 dưới đây

I/O Constraints

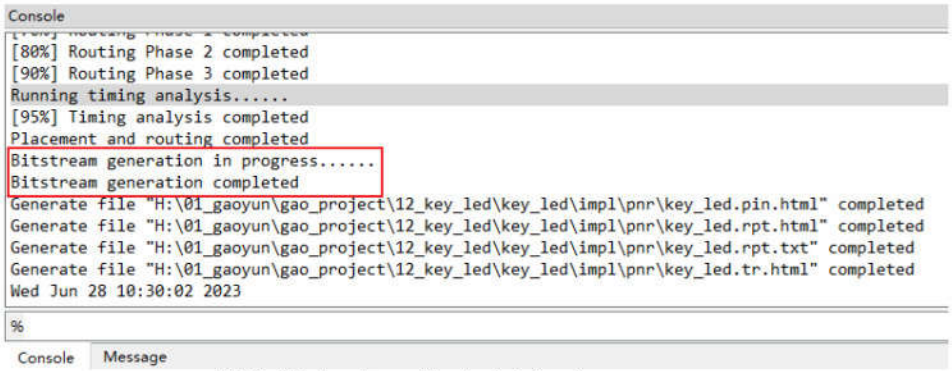
	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type	Drive
1	Clk	input		T9	4	False	LVCMOS33	OFF
2	Rst_n	input		C15	1	False	LVCMOS33	OFF
3	key_in0	input		B16	1	False	LVCMOS33	OFF
4	key_in1	input		A15	1	False	LVCMOS33	OFF
5	led[0]	output		D14	1	False	LVCMOS33	8
6	led[1]	output		C14	1	False	LVCMOS33	8
7	led[2]	output		B9	0	False	LVCMOS33	8
8	led[3]	output		A9	0	False	LVCMOS33	8

Hình 12-4 – Hình ảnh hiệu ứng sau khi gán chân

GOWIN VIETNAM

12.3.2 Place & Route

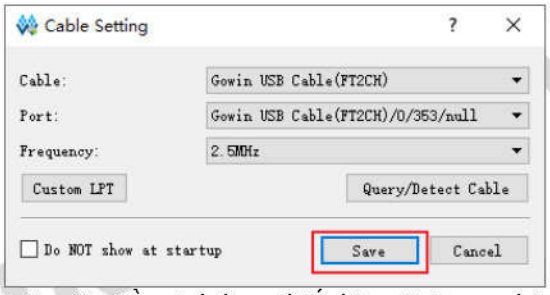
Nhấp chuột vào"Place & Route", hết lỗi, tạo file fs thành công như Hình 12-5 bên dưới:



Hình 12-5 – Tạo file fs thành công

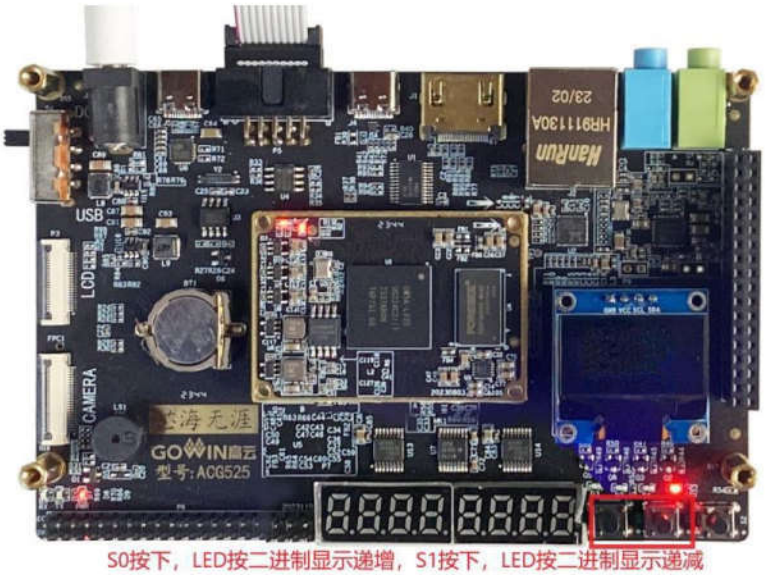
12.3.3 Tải file fs và quan sát hiện tượng

Nhấp chuột "Program Device" vào giao diện tải xuống, giao diện "Cable Setting" xuất hiện, như Hình 12-6 dưới đây, nhấp Save.



Hình 12-6 – Sơ đồ minh họa thiết bị mới được phát hiện

Sau đó nhấp vào tải file fs xuống , hoàn thành Programming, tiến hành bài test như Hình 12-7 bến dưới xem kết quả có phù hợp như thiết kế.



Hình 12-7– Nhìn hiện tượng trên bảng mạch

Thấy hiện tượng tương ứng với mô tả trước đó, tức là thiết kế không có sai sót. Đến đây, thông qua mô phỏng và xác minh cấp bảng đã chứng minh rằng thiết kế bộ đếm cộng trừ là chính xác.

12.4 Thuyết minh các vấn đề thường gặp

Khi nút bấm được nhấn, bộ đếm tăng giảm, phải xử lý thông qua khử rung nút bấm, nếu không do đặc tính cơ học của nút bấm sẽ gây ra sự thay đổi mức điện không xác định và số lần nhảy.

12.5 Tổng kết chương

Chương này đã thực hiện thiết kế cộng trừ thông qua phương pháp thiết kế mô-đun và đã tiến hành mô phỏng hành vi cũng như xác minh cấp bảng. Phương pháp thiết kế mô-đun được sử dụng rất rộng rãi trong các chương tiếp theo, đặc biệt trong các thí nghiệm tổng hợp, sẽ càng cảm nhận rõ ràng hơn những lợi ích của phương pháp thiết kế này. Đề nghị độc giả theo dõi nội dung thí nghiệm này để thực hiện đầy đủ toàn bộ thí nghiệm.

GOWIN VIETNAM

Chương 13: Thiết kế xác minh điều khiển Led 7 đoạn

Mã nguồn project	02_设计实例 |----ch13_hex8_hc595

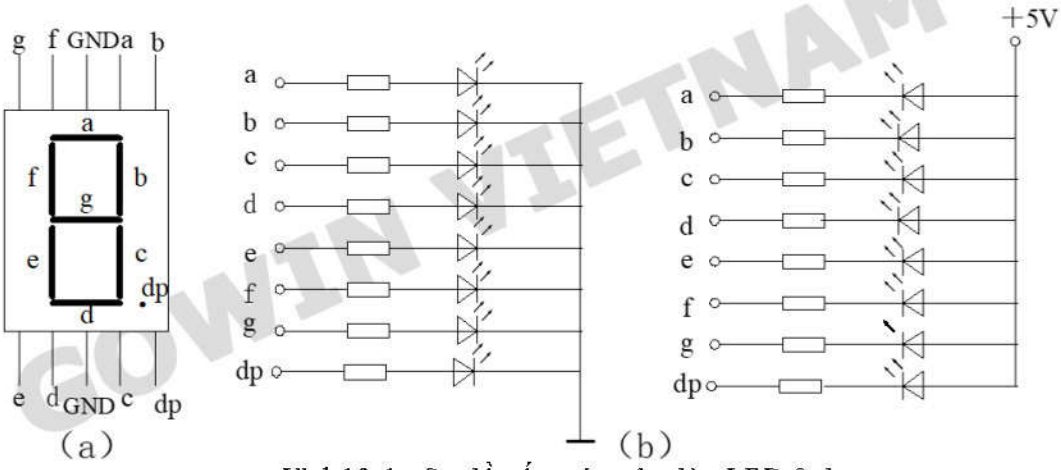
Dẫn nhập chương

Các thiết bị hiển thị thường được sử dụng trong hệ thống điện tử gồm màn Hình số, LCD và màn Hình VGA. Trong đó, màn Hình số có thể chia thành hiển thị đoạn (7 đoạn, Hình chữ H, v.v.) và hiển thị ma trận (8*8*, *16*16, v.v.), ứng dụng của LCD có thể phân thành LCD ký tự (1602, 12864, v.v.) và màn Hình LCD màu thật. Màn Hình VGA thường là màn Hình máy tính hiện nay. Bo mạch phát triển ACG525 cung cấp giao tiếp với cả 3 thiết bị trên.

Chương này sẽ thực hiện điều khiển màn Hình số (Led 7 đoạn) bằng FPGA để hiển thị động và trích xuất cấu trúc mạch thực hiện. Bắt đầu từ cấu trúc mạch để viết mã, mô phỏng để xác minh thiết kế. Nội dung chính của chương không còn là mã, mà là cấu trúc mạch. Khi cấu trúc mạch đã xác định, việc viết mã chỉ là quy trình thi công theo bản vẽ. Đây cũng là cách suy nghĩ phần cứng ngày càng gần gũi với bản chất thiết kế FPGA.

13.1 Nguyên lý điều khiển màn Hình số

Cấu trúc của 8 đoạn LED được mô tả như Hình 13-1 dưới đây:



Hình 13-1– Sơ đồ cấu trúc của đèn LED 8 đoạn

Từ Hình trên có thể thấy màn Hình số có hai cấu trúc: chung cực âm và chung cực dương. Sự khác biệt giữa hai loại này nằm ở việc đầu chung được kết nối với đất hay điện áp cao. Đối với màn Hình số chung cực âm, cần cung cấp điện áp cao cho các đoạn tương ứng để làm sáng, trong khi đối với màn Hình số chung cực dương, cần cung cấp điện áp thấp để làm sáng. Trên bo mạch phát triển, màn Hình số sử dụng là chung cực dương. Đồng thời, để hiển thị số hoặc ký tự, cần phải mã hóa và giải mã số hoặc ký tự đó. Ở đây không xem xét dấu thập phân, tức là đơn giản hóa thành màn Hình số 7 đoạn, định dạng mã hóa và giải mã như bảng dưới đây:

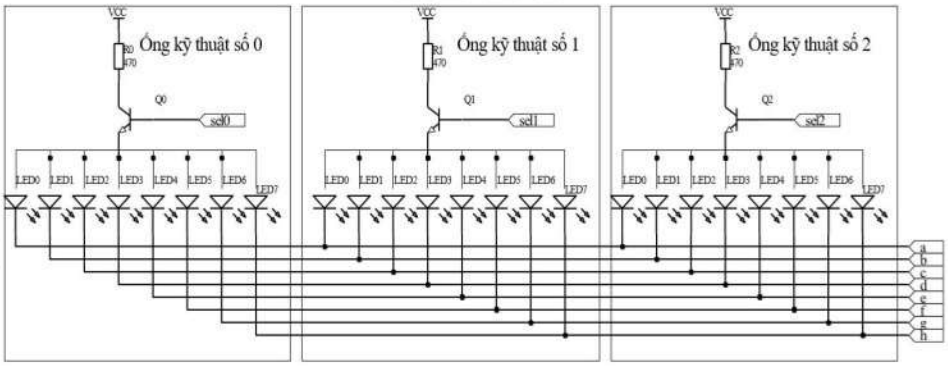
Bảng 13-1– Bảng mã hóa và giải mã số và ký tự

待显示内容 Data_disp	段码（二进制格式）								段码（十六进制格式）
	a	b	c	d	e	f	g	h	
0	0	0	0	0	0	0	1	1	8`hc0
1	1	0	0	1	1	1	1	1	8`hf9
2	0	0	1	0	0	1	0	1	8`ha4

3	0	0	0	0	1	1	0	1	8'hb0
4	1	0	0	1	1	0	0	1	8'h99
5	0	1	0	0	1	0	0	1	8'h92
6	0	1	0	0	0	0	0	1	8'h82
7	0	0	0	1	1	1	1	1	8'hf8
8	0	0	0	0	0	0	0	1	8'h80
9	0	0	0	0	1	0	0	1	8'h90
a	0	0	0	1	0	0	0	1	8'h88
b	1	1	0	0	0	0	0	1	8'h83
c	0	1	1	0	0	0	1	1	8'hc6
d	1	0	0	0	0	1	0	1	8'ha1
e	0	1	1	0	0	0	0	1	8'h86
f	0	1	1	1	0	0	0	1	8'h8e

Màn hình số đoạn có hai chế độ hoạt động: chế độ hiển thị tĩnh và hiển thị động. Đặc điểm của hiển thị tĩnh là mỗi màn Hình số cần phải kết nối một đường dữ liệu 8 bit để duy trì mã Hình hiển thị. Khi mã Hình được gửi vào một lần, Hình hiển thị có thể giữ nguyên cho đến khi mã Hình mới được gửi vào. Phương pháp này do mỗi màn Hình số đều cần đường dữ liệu riêng biệt nên mạch phần cứng khá phức tạp, chi phí cao và ít được sử dụng.

Để tiết kiệm IO và chi phí, thường áp dụng cấu trúc mạch như Hình 13-2 dưới đây, như vậy 3 màn Hình số kết nối với nhau sẽ giảm được 7*2 I/O so với chế độ tĩnh.

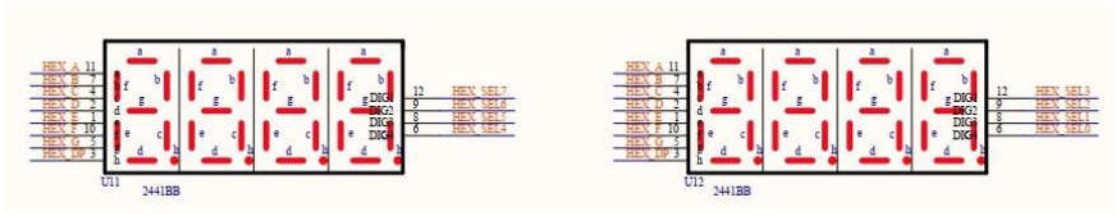


Hình 13-2 – Mạch điều khiển 3 LED 7 đoạn dung phương pháp quét (multiplexing)

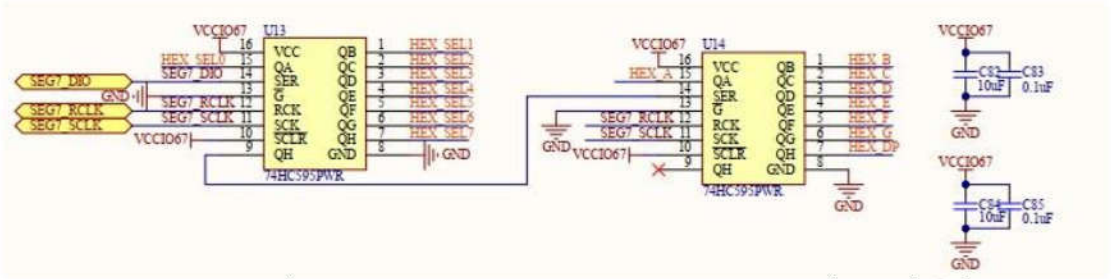
Như vậy, đã thực hiện được một chế độ hiển thị khác, đó là hiển thị động. Đặc điểm của hiển thị động là tất cả các đường chọn đoạn của màn Hình số được nối song song với nhau, và đường chọn vị trí sẽ kiểm soát màn Hình số nào đang hoạt động. Màn Hình số được chọn sẽ sử dụng phương pháp quét động để hiển thị. Phương pháp quét động có nghĩa là lần lượt gửi mã Hình và đường chọn tương ứng đến từng màn Hình số, nhờ vào hiệu ứng lưu ánh sáng của đèn LED và tác dụng lưu ảnh của mắt người, khiến cho cảm giác như tất cả các màn Hình số đều đang hiển thị cùng một lúc.

Giả sử thời gian quét được đặt là 1 giây, ba màn Hình số sẽ được chia thành 3 giây. Trong giây đầu tiên, đường dữ liệu sel là b100, lúc này màn Hình số 0 được chọn, a=0, LED0 của màn Hình số 0 sẽ sáng; trong giây thứ hai, đường dữ liệu sel là b010, lúc này màn Hình số 1 được chọn, b=0, LED1 của màn Hình số 1 sẽ sáng; trong giây thứ ba, đường dữ liệu sel là `b001, lúc này màn Hình số 2 được chọn, c=0, LED2 của màn Hình số 2 sẽ sáng. Hiệu ứng lúc này sẽ là LED0 của màn Hình số 0 sáng trong một giây, sau đó LED1 của màn Hình số 1 sáng trong một giây, cuối cùng là LED2 của màn Hình số 2 sáng trong một giây, và sau đó lại lặp lại.

Nếu sử dụng thời gian làm mới 1ms, nhờ vào hiệu ứng lưu ánh sáng của màn Hình số và tác dụng lưu ảnh của mắt người, LED0 của màn Hình số 0, LED1 của màn Hình số 1 và LED2 của màn Hình số 2 sẽ "cùng lúc" sáng mà không có cảm giác nhấp nháy.



Hình 13-3 – Sơ đồ nguyên lý của 2 khối 4 LED 7 đoạn nối tiếp

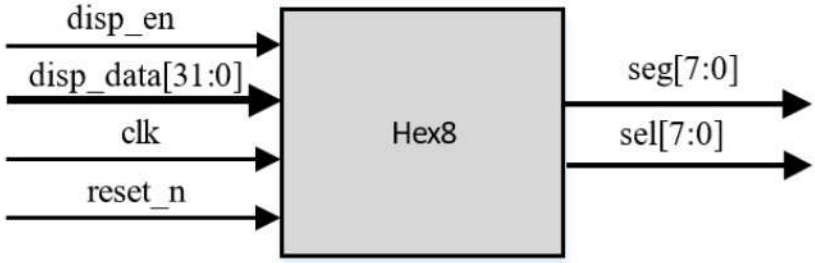


Hình 13-4 – 7 Sơ đồ nguyên lý của 8 module LED 7 đoạn điều khiển bằng 74HC595

13.2 Thiết kế điều khiển quét động cho màn Hình số

13.2.1 Sơ đồ khối mô đun và phân chia chức năng

Từ phân tích ở trên, có thể đưa ra biểu đồ khối đầu vào và đầu ra như Hình 13-5 dưới đây, danh sách các giao diện như bảng dưới đây:



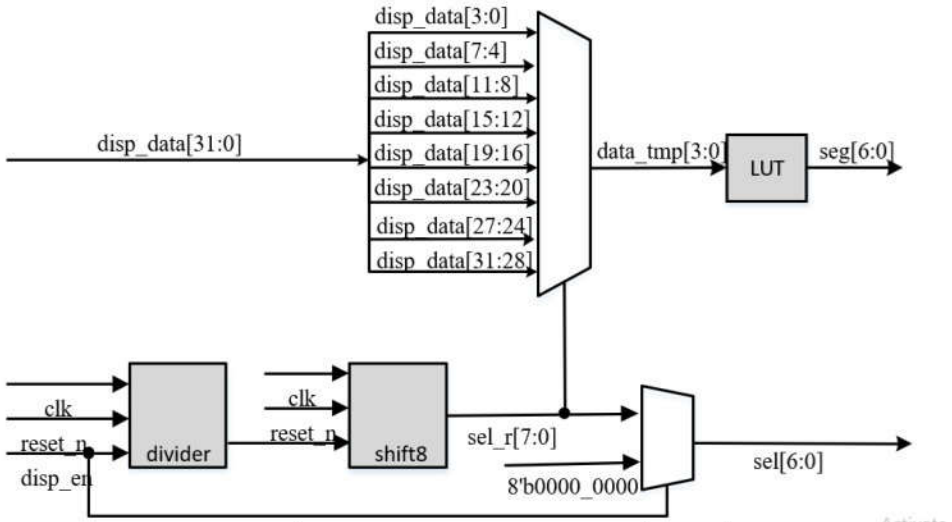
Hình 13-5 – Sơ đồ khối module điều khiển Led 7 đoạn

Bảng 13-2 – Danh sách các giao tiếp

Tên tín hiệu	I/O	Mô tả chức năng
clk	I	Xung clk， 50MHz
reset_n	I	Tín hiệu reset
disp_en	I	Tín hiệu kích hoạt màn Hình số, 1 để kích hoạt, 0 để tắt
disp_data[31:0]	I	8 data chờ hiển thị trên Led 7 đoạn, bốn Led tạo thành một mã BCD
sel[7:0]	O	Chọn vị trí Led7 đoạn, chọn màn Hình số hiện tại cần hiển thị
seg[7:0]	O	Chọn đoạn Led, nội dung hiện tại cần hiển thị

Dựa trên phân tích trên, trước tiên cần có một đồng hồ điều khiển với chu kỳ 1ms, do đó cần một mạch chia tần số; khi thực hiện chọn vị trí Led7đoạn, cần một mạch dịch vòng; sau khi chọn vị trí, cần có bộ chọn để mở dữ liệu đầu vào. Để thực hiện chức năng bảng mã và giải mã của Led7đoạn, cần có một bộ giải mã.

Sơ đồ mạch logic của mô-đun điều khiển màn Hình số có thể được đơn giản hóa như Hình 13-6 dưới đây, trong đó chức năng của từng phần được trình bày trong bảng 13-3.



Hình 13-6 – Sơ đồ module của mạch logic điều khiển Led7đoạn

Bảng 13-3 – Mô tả chức năng module con

Tên	Mô tả chức năng
divder	Chia tần số định thì 1ms (1khz)
shift8	Thanh ghi dịch vòng 8 vị trí
MUX8	Lựa chọn dữ liệu nhập
MUX2	Chọn kích hoạt
LUT	Bọ giải mã

13.2.2 Thiết kế module Clk quét

Từ clk hệ thống 50M, chia tần số để có clk quét 1KHz, giá trị tổng của bộ đếm (cnt) là 25000d, do bắt đầu đếm từ 0 nên cnt max 24999, và độ rộng bit của cnt định nghĩa là 15 bit.

```
always@(posedge clk or posedge reset) if(reset)
    divider_cnt <= 15'd0; else if(!en)
    divider_cnt <= 15'd0; else if(divider_cnt == 24999)
    divider_cnt <= 15'd0; else
    divider_cnt <= divider_cnt + 1'b1;

//Module chương trình tạo clk 1khz
always@(posedge clk or posedge reset) if(reset)
    clk_1K <= 1'b0;
else if(divider_cnt == 24999) clk_1K <= ~clk_1K;
else
    clk_1K <= clk_1K;
```

13.2.3 Thiết kế module chọn vị trí Led7đoạn

Tiếp theo, viết một reg dịch vòng 8 bit, ở đây sử dụng reg dịch vòng để tạo sự thay đổi từ

0000_0001b → 1000_0000b, từ đó thực hiện việc chọn vị trí Led7đoạn, tức là chọn một Led7đoạn trong mỗi chu kỳ đồng hồ quét. Giá trị đầu ra của reg dịch và mối quan hệ tương ứng với việc chọn Led7đoạn được trình bày trong bảng dưới đây, trong đó sel7 là bit cao.

Bảng 13-4 – Mối quan hệ giữa thanh ghi dịch và Led7đoạn

sel0	sel1	sel2	sel3	sel4	sel5	sel6	sel7	Led được chọn
1	0	0	0	0	0	0	0	Led7đoạn 0
0	1	0	0	0	0	0	0	Led7đoạn 1
0	0	1	0	0	0	0	0	Led7đoạn 2
……								
0	0	0	0	0	0	0	1	Led7đoạn 7

Code như sau:

```
always@(posedge clk_1K or posedge reset) if(reset)
    sel_r <= 8'b0000_0001;
else if(sel_r == 8'b1000_0000)
    sel_r <= 8'b0000_0001;
else
    sel_r <=  sel_r << 1;
```

13.2.4 Thiết kế hiển thị dữ liệu cho Led 7 đoạn

```
reg [3:0]data_tmp;//lưu trữ data
    always@(*)
        case(sel_r)
            8'b0000_0001:data_tmp = disp_data[3:0];
            8'b0000_0010:data_tmp = disp_data[7:4];
            8'b0000_0100:data_tmp = disp_data[11:8];
            8'b0000_1000:data_tmp = disp_data[15:12];
            8'b0001_0000:data_tmp = disp_data[19:16];
            8'b0010_0000:data_tmp = disp_data[23:20];
            8'b0100_0000:data_tmp = disp_data[27:24];
            8'b1000_0000:data_tmp = disp_data[31:28];
            default:data_tmp = 4'b0000;
        endcase
```

Sử dụng bộ chuyển mạch 8 chọn 1, đầu vào chọn là Led7đoạn được chọn, tức là đầu ra của reg dịch vòng, sử dụng bộ chuyển mạch để truyền dữ liệu cần hiển thị đến Led tương ứng. Thiết kế giải mã dữ liệu hiển thị

Như đã nói ở trên, để Led7đoạn hiển thị số/ ký tự, cần thực hiện mã hóa và giải mã cho số /ký tự đó. Ở đây, sử dụng một bảng tra cứu 4 đầu vào để thực hiện giải mã hiển thị 8 bit.

```
always@(*)
    case(data_tmp)
        4'h0:seg = 7'b1000000;
        4'h1:seg = 7'b1111001;
        4'h2:seg = 7'b0100100;
        4'h3:seg = 7'b0110000;
        4'h4:seg = 7'b0011001;
        4'h5:seg = 7'b0010010;
        4'h6:seg = 7'b0000010;
        4'h7:seg = 7'b1111000;
        4'h8:seg = 7'b0000000;
        4'h9:seg  =  7'b0010000;  4'ha:seg  =  7'b0001000;  4'hb:seg  =
        7'b0000011; 4'hc:seg = 7'b1000110; 4'hd:seg = 7'b0100001;
        4'he:seg = 7'b0000110;
        4'hf:seg = 7'b0001110; endcase
```

13.2.5 Thiết kế tín hiệu kích hoạt cho module

Ý tưởng thiết kế mô-đun tạo mỗi mô-đun độc lập, thiết kế cổng của chúng phải thuận tiện cho việc gọi và điều khiển sau này. Dựa trên ý tưởng này, cần thêm tín hiệu kích hoạt. Đối với mô-đun kích hoạt, có thể sử dụng một bộ chuyển mạch 2 chọn 1 để thực hiện.

```
assign sel = (en)?sel_r:8'b0000_0000;
```

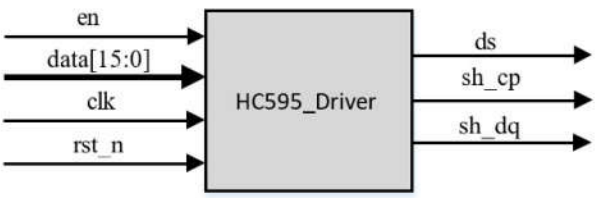
13.3 Thiết kế Module điều khiển 74HC595

Trong thiết kế Led7đoạn trên bo mạch ACG525, đã sử dụng chip 74HC595. Chip này có chức năng là thanh ghi dịch, thông qua phương pháp dịch, giúp tiết kiệm chân FPGA. FPGA chỉ cần xuất ra 3 chân để đạt được mục đích gửi dữ liệu đến Led7đoạn, so với phương pháp chọn đoạn và chọn vị trí truyền thống, điều này tiết kiệm đáng kể tài nguyên thiết kế IO. Phương pháp sử dụng cụ thể và các thông số kỹ thuật của 74HC595 như sau: trong tài liệu dữ liệu có thể thấy, tần số hoạt động của chip 74HC595 khác nhau dưới các nhiệt độ và điện áp hoạt động khác nhau, được trình bày trong bảng 13-5. Do chip trên bo ACG525 sử dụng nguồn 3.3V, nên trong thiết kế tần số hoạt động, trực tiếp sử dụng đồng hồ đã chia tần số 50M để làm đồng hồ hoạt động, giúp chip hoạt động ở tần số 12.5M trong trạng thái 3.3V.

Bảng 13-5 – Giới hạn tần số thiết kế của IC 74HC595

SYMBOL	PARAMETER	Vcc(V)	MIN	TYP	MAX	UNIT
$fmax$	Maximum clock frequency SH_CP and ST_CP	2.0	4.8	-	-	MHz
		4.5	24	-	-	MHz
		6.0	28	-	-	MHz
T_{amb} = -40to + 125°C						

Tín hiệu vào/ra của module điều khiển 74HC595 như Hình 13-7,



Hình 13-7– Sơ đồ mô tả giao diện của module điều khiển 74HC595

Bảng 13-6 74HC595 Mô tả chức năng của các I/O

Tên tín hiệu	I/O	Mô tả chức năng
clk	I	Xung clock 50M
rst_n	I	Tính hiệu đặt lại
en	I	Tính hiệu kích hoạt 1 ennable，0 tắt
data[15:0]	I	Dữ liệu SEG và SEL của 8 Led7đoạn
ds	O	Đầu ra dữ liệu nối tiếp
sh_cp	O	Đầu ra clk của thanh ghi dịch
st_cp	O	Đầu ra clk của thanh ghi lưu trữ

Trong thiết kế mô-đun điều khiển 74HC595, trước tiên cần thiết kế việc tạo ra đồng hồ làm việc, thực hiện chia tần số 50M xuống còn 2.

```
parameter CNT_MAX = 2;
reg [15:0]r_data; always@(posedge clk) if(s_en)
    r_data <= data;
reg [7:0]divider_cnt;//分频计数器;
always@(posedge clk or posedge reset) if(reset)
    divider_cnt <= 0;
else if(divider_cnt == CNT_MAX - 1'b1) divider_cnt <= 0;
else
    divider_cnt <= divider_cnt + 1'b1;
wire sck_plus;
assign sck_plus = (divider_cnt == CNT_MAX - 1'b1);
```

Đếm sck_pluse để thực hiện việc nhập data nối tiếp qua bảng tra cứu, cũng như tạo ra clk dịch sh_cp và clk st_cp .

```
                                                reg [5:0]SHCP_EDGE_CNT;

always@(posedge clk or posedge reset) if(reset)
                                                    SHCP_EDGE_CNT <= 0;
else if(sck_plus)begin if(SHCP_EDGE_CNT == 6'd32)
        SHCP_EDGE_CNT <= 0;
    else
        SHCP_EDGE_CNT <= SHCP_EDGE_CNT + 1'b1;
end else
    SHCP_EDGE_CNT <= SHCP_EDGE_CNT;
```

Bảng tra cứu thực hiện đầu ra trạng thái.

```
always@(posedge clk or posedge reset) if(reset)begin
```

13.4 Mô đun hiển thị số giả lập thử nghiệm

Dưới đây tạo ra tín hiệu reset và tín hiệu kích hoạt, khởi tạo dữ liệu cần hiển thị và chuyển đổi, lần lượt hiển thị trên Led7đoạn "12345678", "87654321" và "89abcdef".

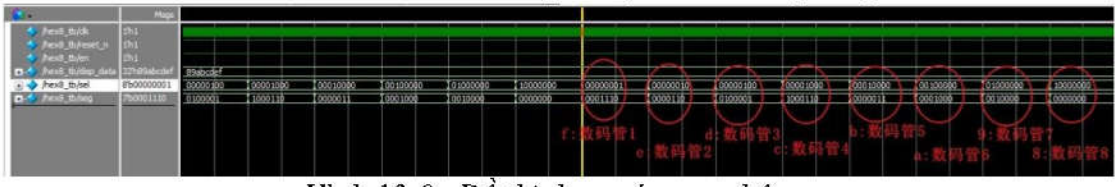
```
initial begin
    reset_n = 1'b0;
    //en = 1;
    disp_data = 32'h12345678; #(`clk_period*20); reset_n = 1;
    #(`clk_period*20); #20000000;
    disp_data = 32'h87654321; #20000000;
    disp_data = 32'h89abcdef; #20000000;
    $stop;
end
```

```
    st_cp <= 1'b0; ds <= 1'b0; sh_cp <= 1'd0;
    end
    else begin
    case(SHCP_EDGE_CNT)
    0: begin sh_cp <= 0; st_cp <= 1'd0;ds <= r_data[15];end 1: begin sh_cp <= 1;
  st_cp <= 1'd0;end
    2: begin sh_cp <= 0; ds <= r_data[14];end 3: begin sh_cp <= 1; end
    4: begin sh_cp <= 0; ds <= r_data[13];end 5: begin sh_cp <= 1; end
    6: begin sh_cp <= 0; ds <= r_data[12];end 7: begin sh_cp <= 1; end
    8: begin sh_cp <= 0; ds <= r_data[11];end 9: begin sh_cp <= 1; end
    10: begin sh_cp <= 0; ds <= r_data[10];end 11: begin sh_cp <= 1; end
    12: begin sh_cp <= 0; ds <= r_data[9];end 13: begin sh_cp <= 1; end
    14: begin sh_cp <= 0; ds <= r_data[8];end 15: begin sh_cp <= 1; end
    16: begin sh_cp <= 0; ds <= r_data[7];end 17: begin sh_cp <= 1; end
    18: begin sh_cp <= 0; ds <= r_data[6];end 19: begin sh_cp <= 1; end
    20: begin sh_cp <= 0; ds <= r_data[5];end 21: begin sh_cp <= 1; end
    22: begin sh_cp <= 0; ds <= r_data[4];end 23: begin sh_cp <= 1; end
    24: begin sh_cp <= 0; ds <= r_data[3];end 25: begin sh_cp <= 1; end
    26: begin sh_cp <= 0; ds <= r_data[2];end 27: begin sh_cp <= 1; end
    28: begin sh_cp <= 0; ds <= r_data[1];end 29: begin sh_cp <= 1; end
    30: begin sh_cp <= 0; ds <= r_data[0];end 31: begin sh_cp <= 1; end
    32: st_cp <= 1'd1; default:
    begin
    st_cp <= 1'b0; ds <= 1'b0; sh_cp <= 1'd0;
    end
endcase end
```

Sau khi thiết kế xong mạch điều khiển màn Led7đoạn và các tệp giả lập tương ứng, mở Modelsim và tạo một dự án giả lập mới để thực hiện giả lập chức năng. Có thể thấy Hình ảnh sóng thử nghiệm toàn cục như Hình 13-8, phóng to sóng cục bộ như Hình 13-9 (chuyển sang hiển thị nhị phân để quan sát tốt hơn). Có thể nhận thấy rằng trước khi tín hiệu reset được đặt cao, Led7đoạn đều hiển thị 0. Sau khi kết thúc reset, led7đoạn mới bắt đầu hiển thị bình thường, và khi dữ liệu

cần hiển thị là 89ABCDEFh (MSB), led7đoạn từ 1 đến 8 lần lượt được chọn và hiển thị là FEDCBA98h (LSB). Vậy, giả lập đã thành công.

Hình 13-8 – Toàn cảnh dạng sóng kiểm tra mô phỏng



Hình 13-9– Đồ thị dạng sóng cục bộ

13.5 Xác thực trên bo mạch phần cứng

Thêm và tạo tệp đỉnh hex_data.v, đặt nó là top module, và viết mô-đun điều khiển led7đoạn vào topmodule để led7đoạn hiển thị "01234567", mã như sau:

```
module hex_top(
    clk, reset_n, sh_cp, st_cp, ds
);

    input clk;  //50M input reset_n;

    output sh_cp; output st_cp; output ds;

    wire [31:0]disp_data;
    wire [7:0] sel;//chọn vị trí của Led7đoạn)
    wire [6:0] seg;//Chọn đoạn trên Led để hiển thị nội dung]
    assign disp_data = 32'h01234567; hc595_driver hc595_driver(
        .clk(clk),
        .reset_n(reset_n),
        .data({1'd0,seg,sel}),
        .s_en(1'b1),
        .sh_cp(sh_cp),
        .st_cp(st_cp),
        .ds(ds)
    );

    hex8 hex8(
        .clk(clk),
        .reset_n(reset_n),
        .en(1'b1),
        .disp_data(disp_data),
        .sel(sel),
        .seg(seg)
    );
endmodule
```

Phân tích và tổng hợp tệp đỉnh cho đến khi không còn lỗi và cảnh báo. Sau đó, nhấp vào FloorPlanner để thực hiện ràng buộc I/O, cụ thể ràng buộc I/O như Hình 13-10.

I/O Constraints

	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type
1	clk	input		T9	4	False	LVCMOS33
2	ds	output		H4	7	False	LVCMOS33
3	reset_n	input		B16	1	False	LVCMOS33
4	sh_cp	output		F4	7	False	LVCMOS33
5	st_cp	output		F3	7	False	LVCMOS33

Hình 13-10 – Thông tin biểu ràng buộc I/O

Tiếp theo, nhấp vào "Place & Route" để thực hiện bố trí và kết nối. Nếu không có lỗi, sẽ tạo ra tệp fs cho thí nghiệm này. Nhấp vào Program Device để tải xuống, sau khi tải xong, màn Hình số sẽ hiển thị như Hình 13-11.

Hình 13-11– Hiển thị dữ liệu trên bo mạch

Từ Hình trên, có thể thấy mỗi số đều có dấu "." phía sau được bật sáng, điều này là do khi thực hiện mô-đun "hc595_driver", bit cao nhất của data là 0. Nếu muốn nó không được bật sáng, chỉ cần đặt bit đó thành 1, lúc này màn Hình số sẽ hiển thị như Hình 13-12

Hình 13-12 – Đèn sáng theo thiết kế mà không có dấu chấm cách

Đến đây chúng ta hoàn thành điều khiển Led hiển thị trạng thái động

13.6 Giải thích các vấn đề thường gặp

1. Trong thí nghiệm này, bo mạch sử dụng chế độ điều khiển ba dây 74HC595 trên bo, việc sử dụng chế độ này có thể tiết kiệm đáng kể tài nguyên của các cổng I/O.

2. Trong trường hợp này, trong mô-đun HEX8, sử dụng bộ đếm chia tần số để tạo ra xung đồng hồ tần số thấp 1KHz từ đồng hồ 50MHz, sau đó sử dụng tín clk đã chia này để điều khiển các flip-flop D. Tín hiệu clk này, do mặc định không đi qua đường global clk, sẽ có độ rung và lệch với global clk. Một nhược điểm khác của tín hiệu clk này là độ trễ đến từng flip-flop D có sự khác biệt lớn, điều này có thể ảnh hưởng đến tính ổn định của hệ thống. Tín hiệu clk này được gọi là đồng hồ điều khiển. Trong phần sau, trong mô-đun truyền dữ liệu qua cổng nối tiếp, chúng tôi sẽ lấy tín hiệu đồng hồ tốc độ baud của cổng nối tiếp làm ví dụ để trình bày thiết kế và cách sử dụng tín hiệu clk chất lượng cao trong hệ thống FPGA (tiết lộ trước, tín hiệu đồng hồ này được gọi là clk kích hoạt).

13.7 Tổng kết chương

Chương này đã giới thiệu các cách điều khiển Led 7 đoạn như: quét tĩnh, quét động và chế độ điều khiển SPI. Nhờ vào lợi thế của thiết kế mô-đun, nếu cần thực hiện điều khiển trực tiếp màn Hình số bằng FPGA, chỉ cần gọi trực tiếp hex8 và điều khiển tệp hc595, vừa đạt được hiệu quả tiết kiệm chân cho bộ dịch chuyển, vừa giữ lại kiến trúc có thể sử dụng song song các lựa chọn đoạn và bit bất cứ lúc nào.

Khuyến nghị độc giả nên theo dõi nội dung thí nghiệm này để thực hiện hoàn chỉnh toàn bộ thí nghiệm.